Computer Architecture

Annotated Study Guide — page by page

This guide follows the lecturer's handwritten notes from page 1 to page 87, in the exact original order. For every content-bearing page you get the original screenshot, a clean transcription of what is written, and a full explanation of every concept, formula, diagram, table, example and annotation — expanded from first principles, with derivations and every variable explained. All 41 past-paper scans (ca1–ca41) are transcribed and solved in full, and each is placed at the point in the notes where its topic appears.

What you get on every page	Why
Screenshot of the original handwritten page	Read the guide next to the notes, or entirely from the guide
Transcription of the page's content	Handwriting made legible; abbreviations expanded
Explanation of every concept, formula and diagram	First-principles teaching, not just a restatement
Past-paper questions on that topic, fully solved	41 scans transcribed and solved, placed in sequence

> **How to use this guide.** Open the handwritten notes and this guide side by side. Read a page of the notes, then read the matching page here: the transcription tells you what the page says, and the explanation tells you what it means. When you reach a past-paper question, attempt it first and then check the full solution.

Contents

Page 1 — Cover page (GoodNotes template) . . . 6

Page 2 — What Computer Architecture is — architecture vs organization, and the von Neumann model . . . 7

Previous-year question 3 — ca3 — Architecture vs organization, Amdahl, caches, virtual memory, FPSQR design, power . . . 8

Page 3 — Levels of cache and the memory hierarchy . . . 11

Page 4 — Levels of parallelism, the memory models (UMA/NUMA/COMA) and RISC vs CISC . . . 13

Page 5 — Parallel computers — definition, need and five classifications . . . 15

Page 6 — SIMD and MIMD organisations (Flynn's taxonomy in pictures) . . . 17

Page 7 — MIMD and shared-memory multiprocessors — classification by memory modules . . . 19

Page 8 — Distributed memory; classification by interconnection network; the bus . . . 21

Page 9 — Multistage (multiple) bus and crossbar topology . . . 23

Page 10 — Mesh and torus topologies . . . 25

Page 11 — Hypercube, tree architectures, and the first look at the vector/array processor . . . 27

Page 12 — Systolic arrays; classification based on the processing element; RISC vs CISC (second pass) . . . 29

Page 13 — Consequences of RISC/CISC for pipelining and CPI; vector processors for DSP . . . 31

Page 14 — Pipelining introduced; the five stages; levels of parallelism revisited . . . 33

Page 15 — IEEE-754 floating-point arithmetic — the worked addition example . . . 35

Page 16 — Superscalar and SIMD; the systolic idea; multicore as the response to the power wall . . . 37

Previous-year question 41 — ca41 — Vector/array features used in GPUs; cache coherence necessity; memory-hierarchy evaluation . . . 38

Page 17 — Systolic arrays in detail; interconnection cost; thermal design and multicore . . . 40

Page 18 — The ISA — the contract between software and hardware; ARM and the RISC architecture . . . 42

Page 19 — Speed-up and Amdahl's Law — the derivation . . . 44

Previous-year question 1 — ca1 — Performance (MIPS, CPU time), pipelining speed-up, loop dependences . . . 45

Page 20 — Amdahl in practice; integer vs floating-point instruction mix . . . 48

Previous-year question 4 — ca4 — Amdahl (enhanced fraction), quad-core, benchmark CPI . . . 49

Previous-year question 33 — ca33 — Three modification versions ranking; Amdahl on a matrix; energy; execution time 50

Page 21 — Amdahl worked past-paper question (enhanced fraction) . . . 53

Page 22 — Amdahl worked past-paper question (continued) . . . 55

Previous-year question 36 — ca36 — Amdahl with a slowdown (FP faster, cache slower); vectorisation percentage . . . 56

Page 23 — Power, voltage and frequency scaling (the power wall) . . . 58

Previous-year question 26 — ca26 — Amdahl (scalar + matrix); energy saved by V/f scaling; AMAT with L2 . . . 59

Page 24 — Cost of an integrated circuit; MTTF, MTTR, MTBF and availability . . . 61

Previous-year question 30 — ca30 — Server-farm MTTF; 22-core Amdahl; pipeline throughput after splitting a stage . 62

Page 25 — MTTF and failure-rate worked examples . . . 65

Page 26 — Reliability past-paper questions (server farm, disk failures, die yield) . . . 67

Page 27 — Availability from MTTF; the bridge to performance metrics (CPU time) . . . 69

Page 28 — CPU time worked example — instruction mix to average CPI . . . 71

Previous-year question 5 — ca5 — CPU time from an instruction mix at 500 MHz . . . 72

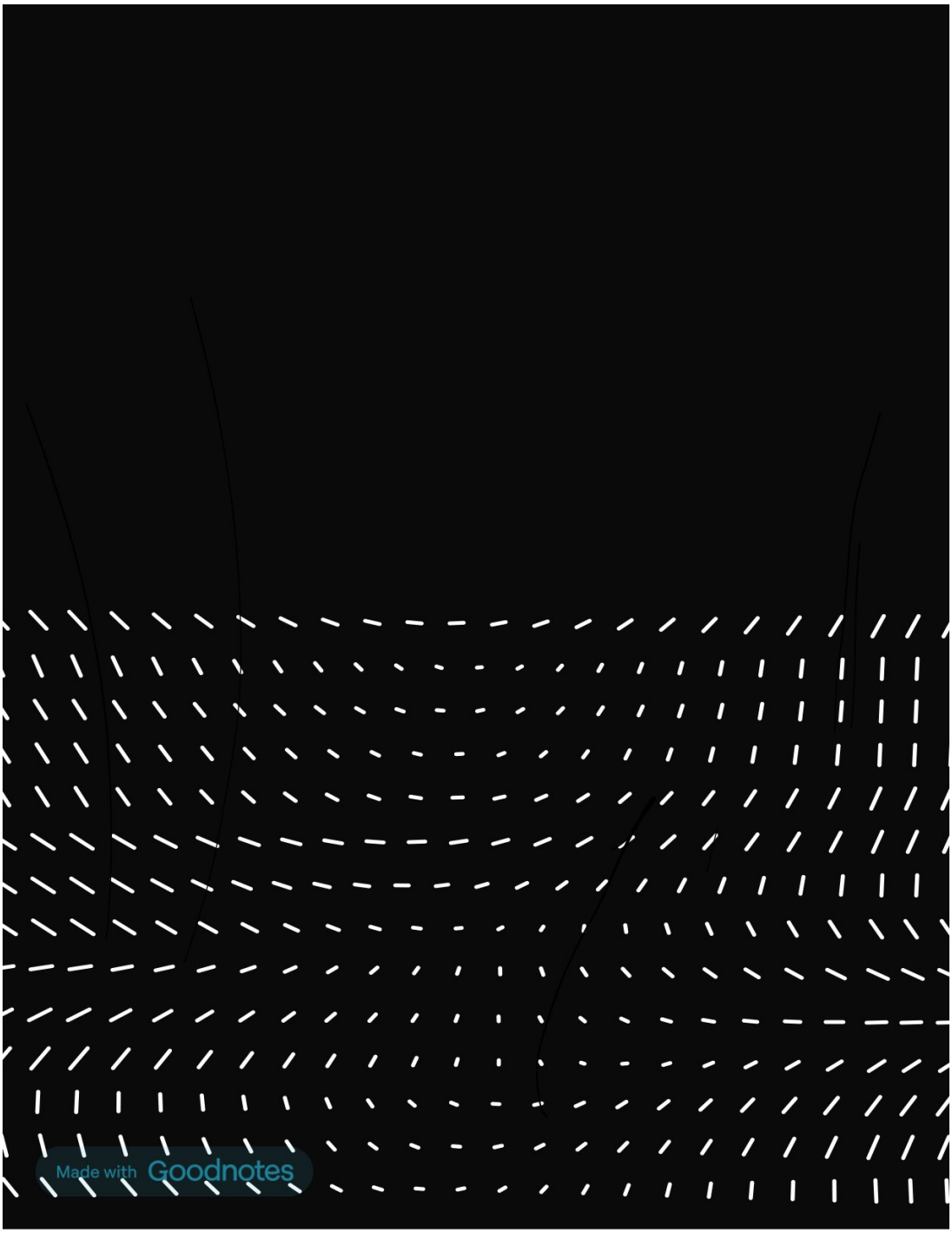
Page 29 — The components of CPU time; a worked "what is the clock rate?" question . . . 73

Page 30 — CPU-time / MIPS past-paper question (instruction classes) . . . 75
Page 31 — CPU-time past-paper question (worked arithmetic) . . . 77
Page 32 — CPU-time past-paper question (continued) . . . 79
Page 33 — Characteristics of pipelining; speed-up; the space–time (timing) diagram . . . 81
Previous-year question 19 — ca19 — Performance (MIPS, CPU time) + pipelining speed-up (repeat of ca1 Q1) . . . 82
Page 34 — The five pipeline stages in detail; instruction fetch and PC update . . . 83
Previous-year question 20 — ca20 — Resource and control hazards in a MIPS fragment (two memories) . . . 84
Page 35 — Pipeline timing with real instruction latencies; the load-use stall . . . 86
Previous-year question 9 — ca9 — 5-stage pipeline flow, branch in pipeline, hazards and nops . . . 87
Previous-year question 10 — ca10 — MIPS pipeline: data dependences, optimal schedule with forwarding, CPI; C-to-MIPS reordering . . . 88
Previous-year question 13 — ca13 — Four RISC fragments (dependences, forwarding/stalls); 5-stage latencies; MIPS schedule and CPI . . . 90
Previous-year question 16 — ca16 — For each sequence: must stall / forwarding / no stall . . . 91
Previous-year question 21 — ca21 — Single-memory structural hazard; pipeline mix; dynamic scheduling with Tomasulo . . . 93
Previous-year question 31 — ca31 — Types of hazards and impact; 5-stage cycle count with forwarding . . . 93
Page 36 — Forwarding, load vs store handling, and the compiler's role in scheduling . . . 95
Page 37 — Branch prediction — the one-bit predictor . . . 97
Previous-year question 17 — ca17 — Pipeline instruction mix: branch-adder use, data-memory use, speed-up . . . 98
Previous-year question 32 — ca32 — Four predictors; 5-stage vs 10-stage clock; speculation; cache optimisation; branch overhead . . . 99
Page 38 — Tournament (hybrid) branch predictor . . . 101
Previous-year question 8 — ca8 — Branch-target-buffer penalty vs fixed penalty; write-miss stall with ownership and invalidates . . . 102
Page 39 — One-bit predictor state machine (worked table) . . . 104
Page 40 — Two-bit saturating counter predictor (state machine) . . . 106
Page 41 — Two-bit / correlating predictor (worked table and scheduling note) . . . 108
Page 42 — Register renaming with a reorder buffer (data dependence resolution) . . . 110
Previous-year question 34 — ca34 — OOO advantages; pipeline phases with a static predictor; Tomasulo and the cycle for F10 . . . 111
Previous-year question 39 — ca39 — Register renaming with T-registers; stalls with a dependence-aware pipeline; Tomasulo and ROB . . . 113
Page 43 — Reservation-station fields and the common data bus (CDB) . . . 116
Page 44 — Tomasulo tables — instruction status and reservation stations . . . 118
Page 45 — Parallelisation, CPI and the basics of branches . . . 120
Page 46 — Data dependences — RAW, WAR and WAW . . . 122
Previous-year question 14 — ca14 — Hazards and reordering; register renaming with P-registers; RAW hazards . . . 123
Previous-year question 22 — ca22 — Loop dependences and renaming (repeat of ca1 Q2a) . . . 124
Page 47 — Tomasulo worked example (loop) with reservation-station tables . . . 126
Previous-year question 12 — ca12 — Tomasulo at cycle 6; vector vs SIMD vs GPU . . . 127
Previous-year question 18 — ca18 — Tomasulo on a loop with multiply latency and cache misses . . . 128
Page 48 — Tomasulo example with a small FP unit and load/store buffers . . . 130

Page 49 — Loop unrolling — the DAXPY example . . . 132
Page 50 — Loop unrolling worked example (3× unrolled DAXPY) . . . 134
Previous-year question 11 — ca11 — DAXPY loop unrolled 3× with fresh registers . . . 135
Page 51 — Unroll until no stalls — the scheduled loop . . . 137
Previous-year question 24 — ca24 — DAXPY unroll-until-no-stalls; schedule; execution time per element . . . 138
Page 52 — Instruction scheduling rules (dynamic and static) . . . 141
Previous-year question 28 — ca28 — Static vs dynamic scheduling; control-hazard techniques; Tomasulo with speculation . . . 142
Page 53 — Precise exceptions and speculation (exceptions in flight) . . . 144
Page 54 — Tomasulo hardware — the full datapath with a reorder buffer and CDB . . . 146
Page 55 — Speculation — issue, execute, read, write and commit . . . 148
Previous-year question 25 — ca25 — ROB contents after two loop iterations; critical-word-first; cache tag bits; blocking/prefetching . . . 149
Previous-year question 40 — ca40 — Tomasulo + ROB/commit; vector vs hybrid (GPU) execution time . . . 150
Page 56 — Tomasulo instruction-status table (worked) . . . 153
Page 57 — Tomasulo — instruction-status and reservation-station tables (worked, cycles 1–10) . . . 155
Page 58 — Tomasulo tables (continued — second worked set) . . . 157
Page 59 — Tomasulo tables (continued — with cycle numbers) . . . 159
Page 60 — Tomasulo tables (continued — final set) . . . 161
Page 61 — Superscalar processors and vector operations . . . 163
Page 62 — Vector/loop example and loop scheduling with loads . . . 165
Page 63 — Loop scheduling with speculation . . . 167
Page 64 — Loop scheduling with speculation (continued) . . . 169
Page 65 — Speculation timing table (issue / execute / read / write / commit) . . . 171
Page 66 — Speculation timing table (continued) . . . 173
Page 67 — Blank / illegible page . . . 175
Page 68 — UMA shared memory and multicore with private caches . . . 176
Page 69 — Cache and coherence block diagram . . . 178
Page 70 — Cache basics — direct mapping (block number mod number of lines) . . . 180
Page 71 — Set-associative cache — organisation and address bits . . . 182
Page 72 — Set-associative cache — worked address example . . . 184
Page 73 — Replacement policies — FIFO, LRU, optimal, random . . . 186
Page 74 — Direct-mapped cache worked example — address field split . . . 188
Previous-year question 38 — ca38 — Cache misses/evictions/non-empty blocks; branch-target-buffer penalty . . . 189
Page 75 — Cache address split — worked example (tag/index/offset) . . . 191
Previous-year question 37 — ca37 — Micro-benchmark cache block sizes; direct-mapped address components . . . 192
Page 76 — Write-through policy; the local miss-rate / average-access-time formula . . . 194
Previous-year question 27 — ca27 — Write-back vs write-through break-even; RAW/WAR/WAW; pipeline phases and branch overhead . . . 195
Page 77 — Write-back and write-allocate policies . . . 198
Page 78 — Write-allocate / write-back combinations (summary) . . . 200

Page 79 — Write-policy summary; AMAT; cache-optimisation checklist 202
Previous-year question 35 — ca35 — Critical-word-first and early restart cycles; merging write buffer 203
Page 80 — Multilevel caches; inclusive/exclusive; local vs global miss rate 205
Previous-year question 15 — ca15 — AMAT with a second-level cache; multicore CPI to beat a single core 206
Page 81 — Prefetching — hardware and software 208
Page 82 — Virtual memory — address translation, the TLB and page faults 210
Page 83 — Summary of instruction-level optimisations (the compiler/hardware toolbox) 212
Page 84 — Cache coherence — the problem, MSI/MESI and snooping vs directory 214
Previous-year question 2 — ca2 — Loop dependences / parallelisation; MSI cache-coherence snapshot 215
Previous-year question 23 — ca23 — Loop parallelisation; vector architecture structure; MSI coherence snapshot . . 216
Previous-year question 29 — ca29 — GPU peak throughput and the memory bound; Intel i7 coherence; MESI 218
Page 85 — MSI example — write-invalidate on a shared bus (snapshot) 220
Previous-year question 6 — ca6 — Three-processor bus coherence (write-through + write-invalidate) trace 221
Page 86 — MSI example (continued) and directory-based protocol 223
Previous-year question 7 — ca7 — Amdahl with communication overhead; cache optimisation; MIPS CPU time; snoopy vs directory 224
Page 87 — MSI example (continued) — write policy for a shared block 227
Appendix A — Previous-year question index (all 41 scans) 229
Appendix B — Source coverage table (all 87 note pages) 230
Appendix C — Verification report 232

Original notes — page 1: Cover page (GoodNotes template)



Original handwritten notes — page 1.

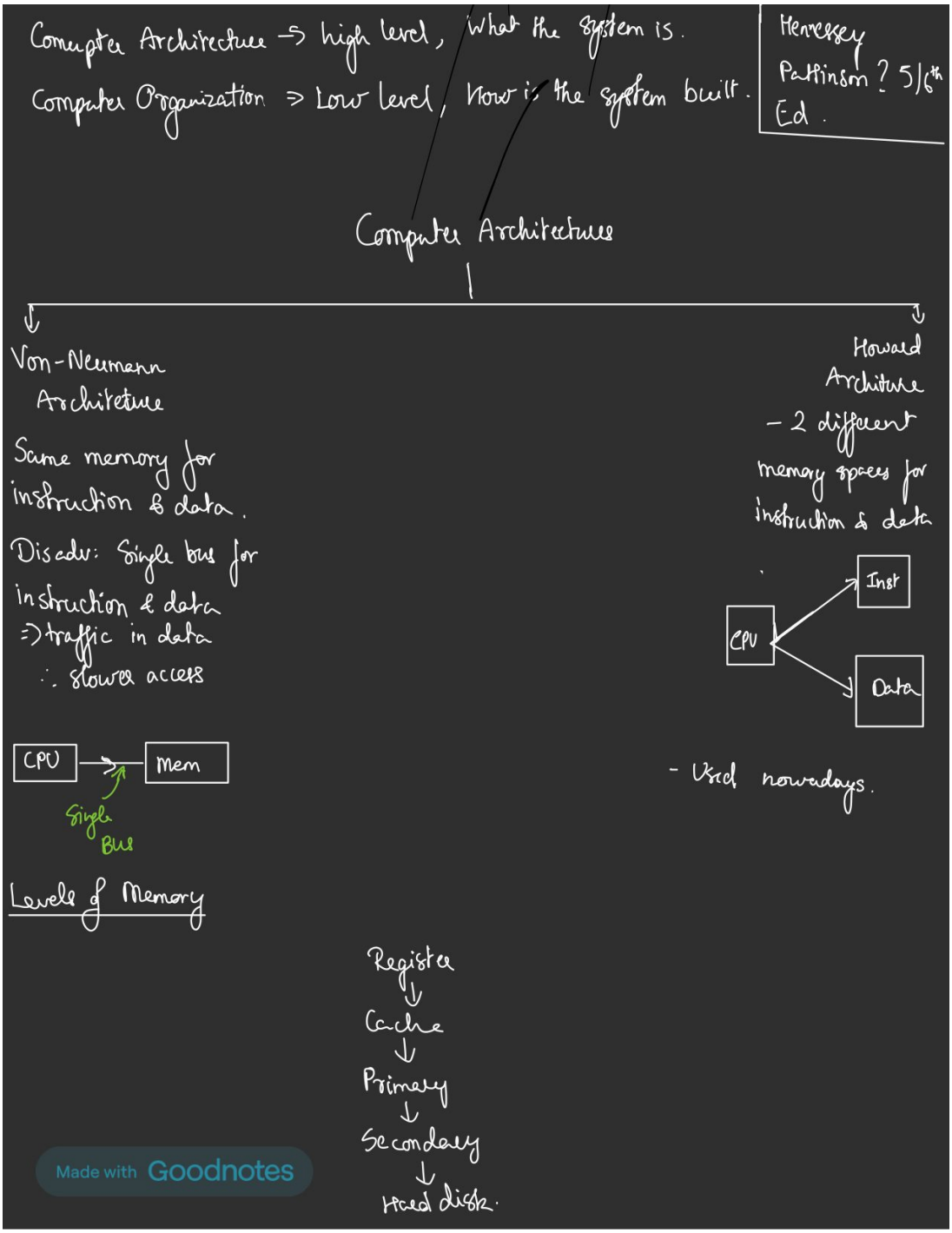
Transcription of the page

```
(Cover sheet produced by the GoodNotes app — a hand-drawn grid.
No lecture content appears on this page.)
```

Explanation

This is the cover sheet generated by the GoodNotes note-taking app; the only marking is the app's decorative grid and the "Made with GoodNotes" watermark at the foot of every page. There is no academic content, so there is nothing to learn here. It is included only so that the page-by-page numbering of this guide matches the original document exactly: page 1 of the notes is page 1 of this guide.

Original notes — page 2: What Computer Architecture is — architecture vs organization…



Original handwritten notes — page 2.

Transcription of the page

```
Computer Architecture  ->  high level: WHAT the system is.
Computer Organization   ->  low  level: HOW the system is built.

Computer Architecture (block diagram):
        Processor  ->  Control
        ALU
        Memory
        Input
        Output
   (stored-program machine)

Von Neumann model:
   Processor  ->  program stored in memory
   Instruction & data in the same memory
   Input  ->  Instruction & data in  ->  Output
Made with GoodNotes
```

Explanation

What the page says. The lecturer opens with the two words that the whole subject is built on, and then draws the von Neumann machine. Both ideas are examined directly, so learn them precisely.

Architecture versus organization

- **Computer Architecture** is the *high-level, logical* view: it answers **what** the system does. It is the set of attributes visible to a programmer — the instruction set, the data types, the addressing modes and the programmer-visible registers. It is fixed for a family (x86, ARM, MIPS) and changes slowly.
- **Computer Organization** is the *low-level, physical* view: it answers **how** the system is built. It covers the data paths, the control signals, the memory technology, the clock and the buses. Two machines can share one architecture but have completely different organizations (and very different speeds and prices).

Attribute	Architecture (what)	Organization (how)
View	High-level / logical	Low-level / physical
Sees	Only the ISA	The whole implementation
Example	“Is there an FP multiply instruction?”	“Is the FP multiplier pipelined?”
Portable?	Program runs on every machine of that family	Changes every technology generation

Exam line. Architecture is programmer-visible and long-lived; organization is transparent to the programmer and changes with technology.

The stored-program (von Neumann) machine

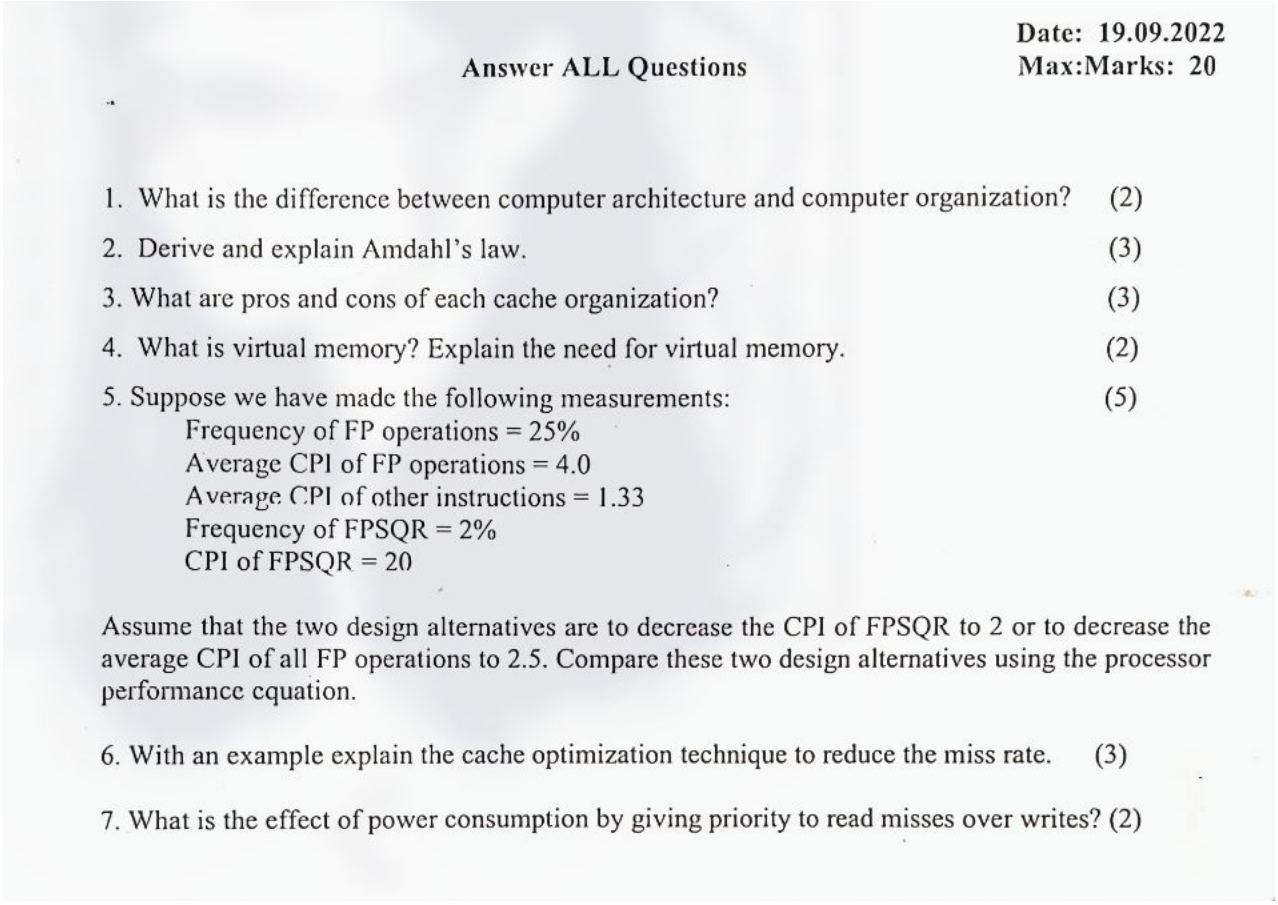
The diagram is the single most important picture in the subject. The machine has five classic units:

- **Control unit** — decides what happens next: it fetches, decodes and sequences instructions.
- **ALU (arithmetic-logic unit)** — performs the arithmetic and logic operations.
- **Memory** — holds *both* the program and the data (this is what “stored-program” means).
- **Input** — brings data in.
- **Output** — sends results out.

```
Stored-program cycle: Fetch → Decode → Execute → (Memory) → Write-back Key idea: instructions and data live in the SAME memory, reached over the SAME path.
```

Inside the processor the programmer sees a small set of registers — the **PC** (program counter, holds the address of the next instruction), the **IR** (instruction register, holds the instruction being executed) and general-purpose registers for operands. Because code and data share one memory, the same fetch path is used for both; this is elegant, but it is also why the memory path becomes the bottleneck (the “von Neumann bottleneck”).

Previous-year question 3 — ca3 — Architecture vs organization, Amdahl, caches, virtual…



Date: 19.09.2022
Max:Marks: 20

Answer ALL Questions

1. What is the difference between computer architecture and computer organization? (2)
2. Derive and explain Amdahl's law. (3)
3. What are pros and cons of each cache organization? (3)
4. What is virtual memory? Explain the need for virtual memory. (2)
5. Suppose we have made the following measurements: (5)
   Frequency of FP operations = 25%
   Average CPI of FP operations = 4.0
   Average CPI of other instructions = 1.33
   Frequency of FPSQR = 2%
   CPI of FPSQR = 20

   Assume that the two design alternatives are to decrease the CPI of FPSQR to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.
6. With an example explain the cache optimization technique to reduce the miss rate. (3)
7. What is the effect of power consumption by giving priority to read misses over writes? (2)

Original past-paper scan ca3.png (re-rendered).

Transcription of the question

```
Date: 19.09.2022     Answer ALL Questions     Max Marks: 20
1. What is the difference between computer architecture and computer organization? (2)
2. Derive and explain Amdahl's law. (3)
3. What are pros and cons of each cache organization? (3)
4. What is virtual memory? Explain the need for virtual memory. (2)
5. Measurements: FP frequency = 25%, average CPI of FP = 4.0, average CPI of other
   instructions = 1.33, FPSQR frequency = 2%, CPI of FPSQR = 20. Two design
   alternatives: (a) reduce the CPI of FPSQR to 2, or (b) reduce the average CPI of
   all FP operations to 2.5. Compare using the processor performance equation. (5)
6. With an example, explain a cache optimisation technique to reduce the miss rate. (3)
7. What is the effect on power consumption of giving priority to read misses over
   writes? (2)
```

Full solution

Q1 — architecture vs organization

Architecture is the programmer-visible, high-level view (what the machine does): the ISA, data types, addressing modes, visible registers. Organization is the low-level implementation (how it is built): datapaths, control, clock, caches. The same architecture can have many organizations.

Q2 — Amdahl's law

```
T_new = T_old[(1 − f) + f/k] ; S = 1/[(1 − f) + f/k] ; S_max = 1/(1 − f) f = enhanced
fraction, k = speed-up of that fraction.
```

Q3 — pros and cons of the cache organisations

Organisation	Pros	Cons
Direct-mapped	Fastest, cheapest, simplest lookup	Conflict misses (one home per block)
Set-associative	Fewer conflicts; good compromise	More hardware; replacement decision
Fully associative	Fewest conflicts (any line)	All tags compared → slow, expensive

Q4 — virtual memory and why we need it

Virtual memory gives each process its own linear address space that is translated to physical addresses by page tables. We need it to (i) run programs larger than physical memory, (ii) isolate and protect processes, and (iii) simplify loading and sharing of code.

Q5 — comparing the two FPSQR designs

```
Original CPI = (FP share × FP CPI) + (other share × other CPI) = 0.25×4.0 + 0.75×1.33 = 1.00 + 0.9975 ≈ 2.00 (a) Improve only FPSQR (2% of instructions) from CPI 20 to 2: CPI_a = 2.00 − 0.02×(20 − 2) = 2.00 − 0.36 = 1.64 → S ≈ 2.00/1.64 = 1.22× (b) Improve ALL FP operations from CPI 4.0 to 2.5 (25% of instructions): CPI_b = 2.00 − 0.25×(4.0 − 2.5) = 2.00 − 0.375 = 1.625 → S ≈ 1.23×
```

> **Conclusion.** Improving *all* FP operations (b) is slightly better than improving only the rarely-used FPSQR (a), because it affects a larger fraction of the program — Amdahl's Law in action.

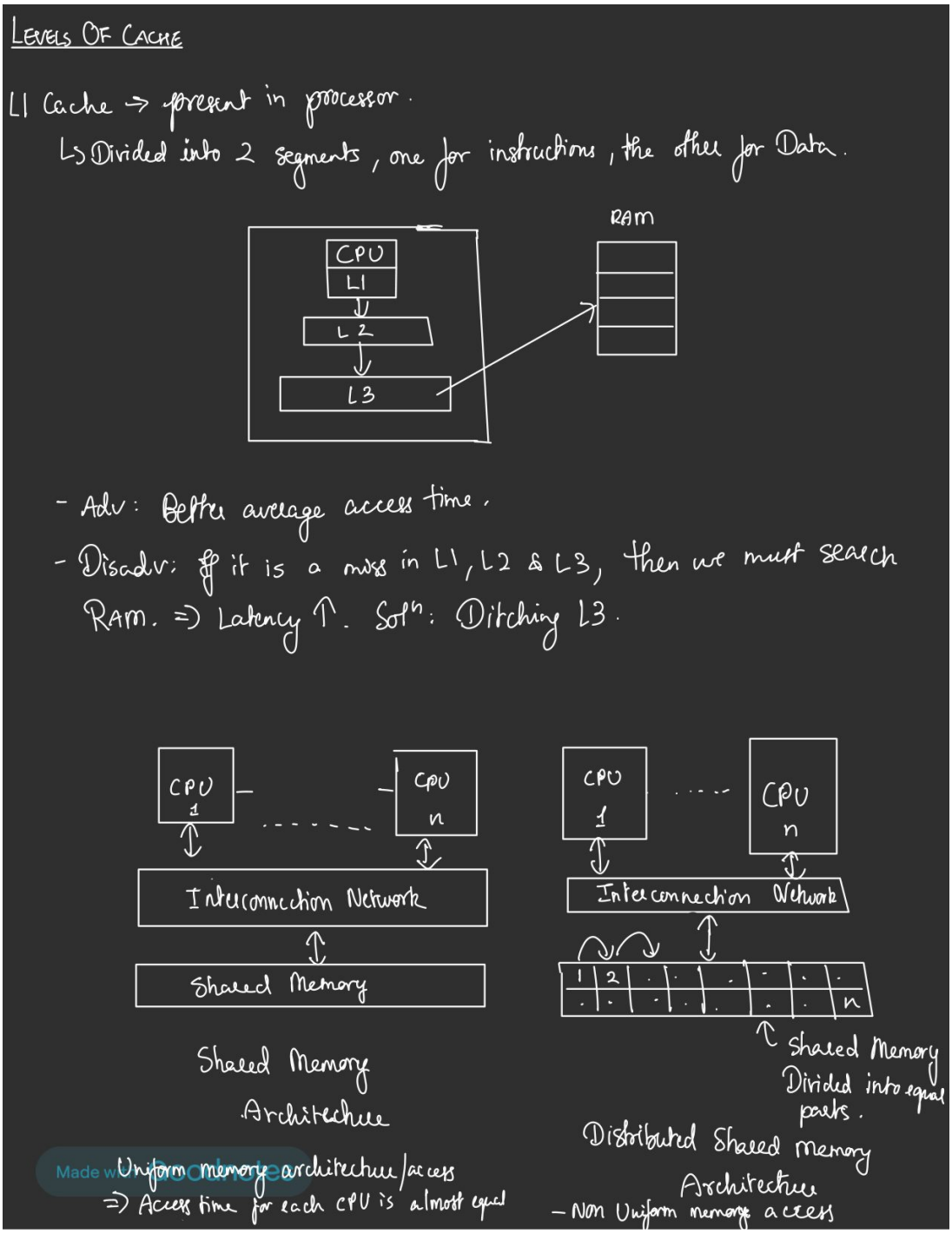
Q6 — reducing the miss rate (example)

Use a larger block or higher associativity, or a better replacement policy. Example: switching from direct-mapped to 2-way set associative removes conflict misses. With a worked AMAT: AMAT = hit time + miss rate × miss penalty.

Q7 — giving read misses priority over writes

Giving read misses priority over write traffic reduces the stall the CPU sees, because reads are on the critical path while writes can be buffered. The trade-off is that write buffers must be large enough to hold the deferred writes; power-wise, deferring writes lets them be batched, slightly reducing memory activity, but the main effect is on performance, not power.

Original notes — page 3: Levels of cache and the memory hierarchy



Original handwritten notes — page 3.

Transcription of the page

```
Levels of Cache

L1 Code cache  — present in the processor.
L1 divided into 2 segments: one for instructions, one for Data.

L2
L3
RAM

Adv: benefits of cache (avg. access time improved).
Disadv: if not in L1, L2, L3, then we must search RAM.

Shared Memory — divided into ...

(Memory hierarchy: registers → L1 → L2 → L3 → main memory → disk)
```

Explanation

What the page says. The processor does not talk to one flat memory. It has a **hierarchy of caches** that gets bigger and slower as you move away from the core. The page draws the levels and states the advantage and the disadvantage.

- **L1 cache** — smallest (tens of KB), fastest (a few cycles), built into the core. It is *split* into an **instruction (code) cache** and a **data cache** so that the fetch unit and the load/store unit can be served in the same cycle without a structural conflict.
- **L2 cache** — larger (hundreds of KB to a few MB), slower (≈10–20 cycles), usually private to a core.
- **L3 cache** — largest on-chip cache (several MB), shared between cores, ≈30–50 cycles.
- **RAM (main memory, DRAM)** — hundreds of cycles; the final on-machine level before disk.

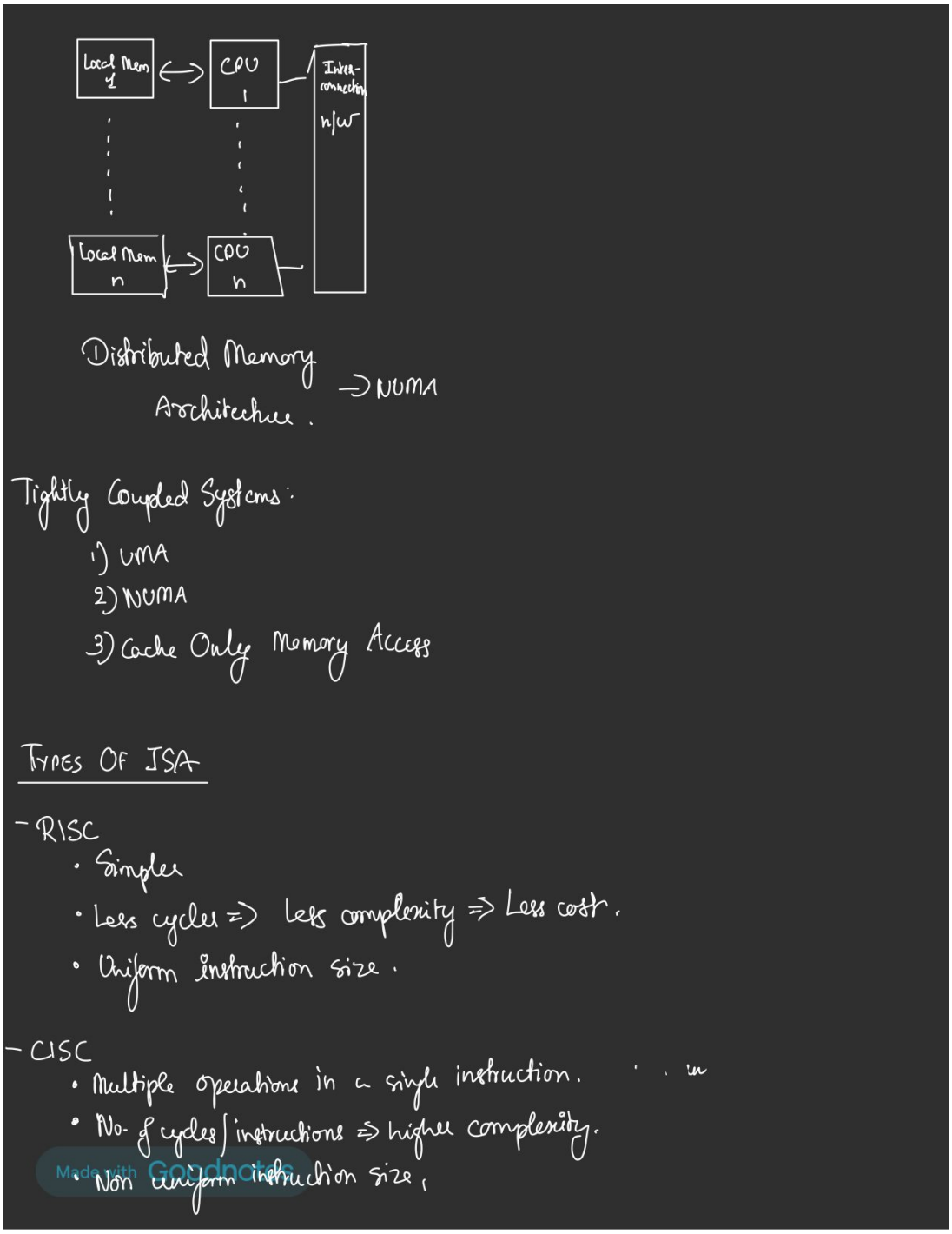
> **Advantage** — locality means most references are served by the small fast levels, so the *average* access time is close to the L1 access time.
> **Disadvantage** — if the data is not in L1, the hardware must search L2, then L3, then RAM before it can proceed; a long miss chain costs hundreds of cycles.

```
Average memory access time (AMAT) over the hierarchy: AMAT = t_L1 + m_L1 · ( t_L2 + m_L2 · ( t_L3 + m_L3 · t_RAM ) ) t = access time of a level, m = miss rate of a level.
```

Reconstructed diagram — the memory hierarchy. Draw it as a pyramid: at the top the registers (fastest, smallest), then L1 (split into I-cache and D-cache), then L2, then L3, then main memory (DRAM), then the disk at the bottom (largest, slowest). Every level is a cache for the level below, and each level down is roughly 5–10× slower and 10× larger.

The note “Shared Memory — divided into …” continues on the next page, where the lecturer classifies multiprocessors by how the memory is arranged (UMA / NUMA / COMA).

Original notes — page 4: Levels of parallelism, the memory models (UMA/NUMA/COMA…



Original handwritten notes — page 4.

Transcription of the page

```
Levels of parallelism:
  Bit-level  <  Instruction-level  <  Data-level  <  Task-level

Memory models:
  1) UMA  — Uniform Memory Access
  2) NUMA — Non-Uniform Memory Access
  3) COMA — Cache-Only Memory Access

RISC:  Less hardware ▫ Less complexity ▫ Less cost.
       Uniform instruction size.
       Load/Store architecture, more registers.
CISC:  Multiple operations in one instruction.
       No. of instructions needed is less.
       Hardware more complex than RISC.
```

Explanation

What the page says. Three ideas are grouped here: where parallelism comes from, how shared memories are organised, and the RISC/CISC contrast. The lecturer returns to all three later, so this is a first pass.

Levels of parallelism (from small grain to large grain)

Level	What is done at once	Example
Bit level	All the bits of one word operated on together	Carry-look-ahead adder
Instruction level	Several independent instructions overlap	Pipelining, superscalar
Data level	The same op on many data items	SIMD / vector / GPU
Task level	Independent programs/threads run together	Multicore, cluster

Going down this list gives more parallelism but also more overhead and more complex programming.

The three shared-memory models

- **UMA — Uniform Memory Access.** All processors reach one central memory with the *same* access time. Simple to program; the shared bus/memory becomes the bottleneck.
- **NUMA — Non-Uniform Memory Access.** A single address space, but each processor has a local memory that is faster than remote memory, so access time depends on *where* the data lives.
- **COMA — Cache-Only Memory Access.** All main memory is treated as a cache; data migrates to the node that is using it.

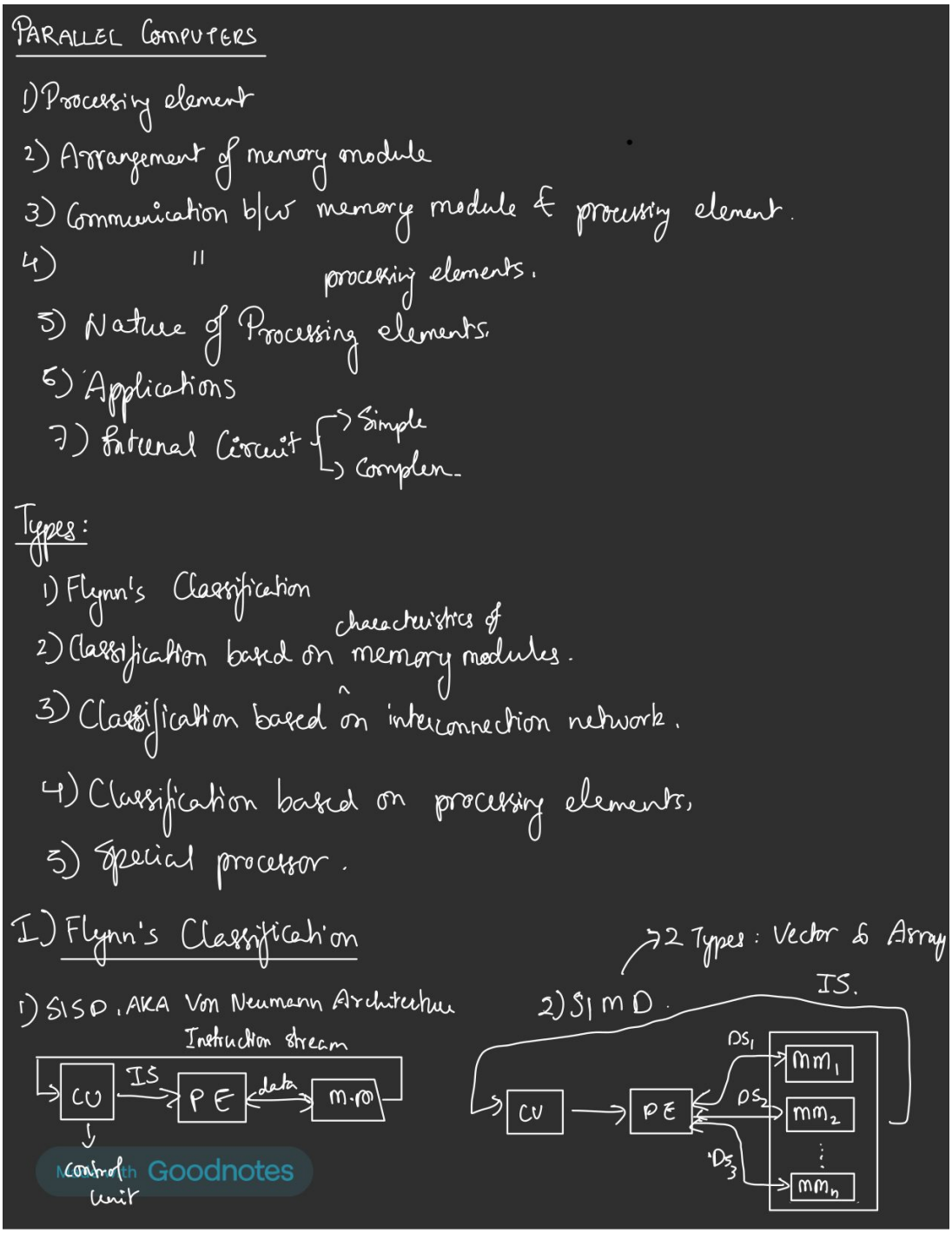
```
UMA: T_access = constant for every processor. NUMA: T_access = f · T_local + (1 − f) · T_remote , T_remote > T_local , f = fraction of accesses served from local memory.
```

RISC versus CISC (first pass)

	RISC	CISC
Instruction size	Uniform (fixed length)	Variable length
Instruction count	More instructions per program	Fewer, richer instructions
Hardware	Simple cheaper, faster to build	Complex costlier
Memory access	Load/Store only; many registers	Operations may act on memory directly
CPI	Close to 1 (easy to pipeline)	Often > 1

Mnemonic. RISC = Reduced Instruction Set Computer (few, simple, uniform instructions). CISC = Complex Instruction Set Computer (many, powerful, variable-length instructions). The notes stress that RISC is *less hardware, less complexity, less cost*, while CISC packs *multiple operations into one instruction*.

Original notes — page 5: Parallel computers — definition, need and five classifications



Original handwritten notes — page 5.

Transcription of the page

```
Parallel Computers

1) Processing elements
2) Communication between memory and processing elements
3) Network of processing elements
4) Applications
5) Based on classification

Classification of parallel computers:
  1) Based on processing elements
  2) Based on interconnection network
  3) Based on memory arrangement
  4) Based on granularity
  5) Flynn's classification
```

Explanation

What the page says. A **parallel computer** is a machine in which several processing elements (PEs) cooperate on one problem. The lecturer lists the ingredients every parallel computer must have and then the five ways of classifying them.

The ingredients of a parallel computer

- **1) Processing elements (PEs)** — the units that actually execute instructions.
- **2) Communication between memory and PEs** — the path that moves operands and results.
- **3) Network of processing elements** — how the PEs are wired to each other.
- **4) Applications** — the software; a parallel machine is only useful if the problem can be split into parallel work.
- **5) Classification** — the ways of naming and comparing the designs.

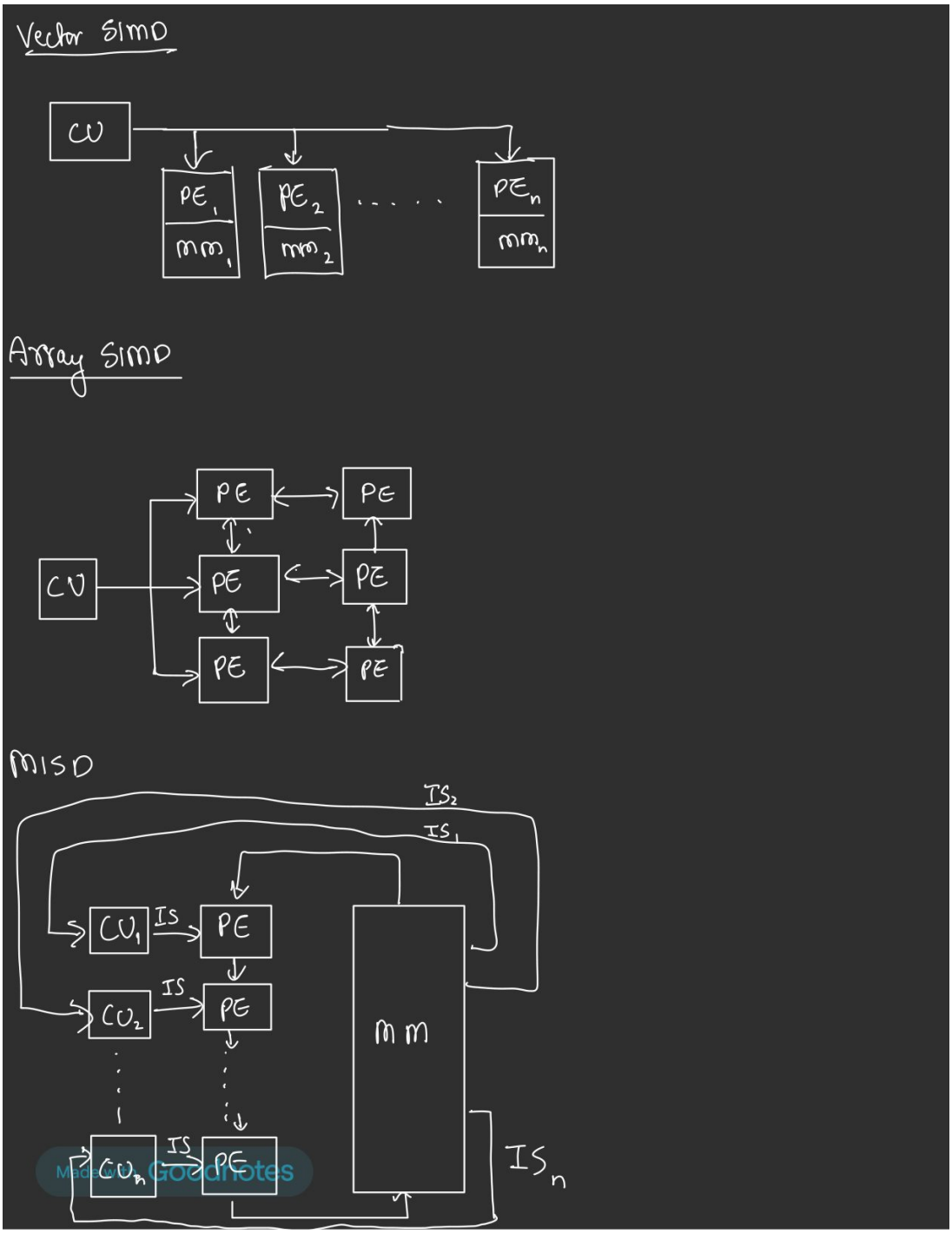
Why we need parallel computers

- The clock frequency can no longer be raised freely — power and heat rise with frequency, so extra performance must come from doing more work at once.
- Many important problems (weather, graphics, simulation, machine learning) are naturally parallel.
- It is often cheaper to use many modest processors than to build one extremely fast processor.

The five classifications (a ready-made exam answer)

- **1) Based on processing elements** — how many PEs and how they are organised.
- **2) Based on the interconnection network** — bus, crossbar, multistage, mesh, torus, hypercube, tree.
- **3) Based on the memory arrangement** — shared, distributed, or hybrid (UMA/NUMA/COMA).
- **4) Based on the granularity of processing** — fine-grained (many tiny tasks) versus coarse-grained (a few large tasks).
- **5) Flynn’s classification** — by instruction and data streams: SISD, SIMD, MISD, MIMD.

Original notes — page 6: SIMD and MIMD organisations (Flynn’s taxonomy in pictures)



Original handwritten notes — page 6.

Transcription of the page

```
Vector  SIMD                          MIMD

   [PE][PE][PE] ...   (one instruction,        [PU] → [PE]
   Mem |  Mem |  Mem    many data)             [PU] → [PE]
                                               [PU] → [PE]
   SIMD                                        (many instructions,
   Single Instruction, Multiple Data           many data)

   MISD  (Single ... theoretical)
   Single Instruction, Multiple Data stream

Worked example:  c[i] = a[i] + b[i]
  for i in 0..n-1: c[i] = a[i] + b[i]   — one instruction over n data.
```

Explanation

What the page says. This is Flynn's taxonomy drawn out. A single control unit can drive one stream of instructions; the number of *data* streams it drives gives the class.

Class	Instruction stream	Data stream	Example
SISD	Single	Single	Ordinary scalar CPU
SIMD	Single	Multiple	Vector unit, GPU, array processor
MISD	Multiple	Single	Theoretical only
MIMD	Multiple	Multiple	Multicore CPU, cluster

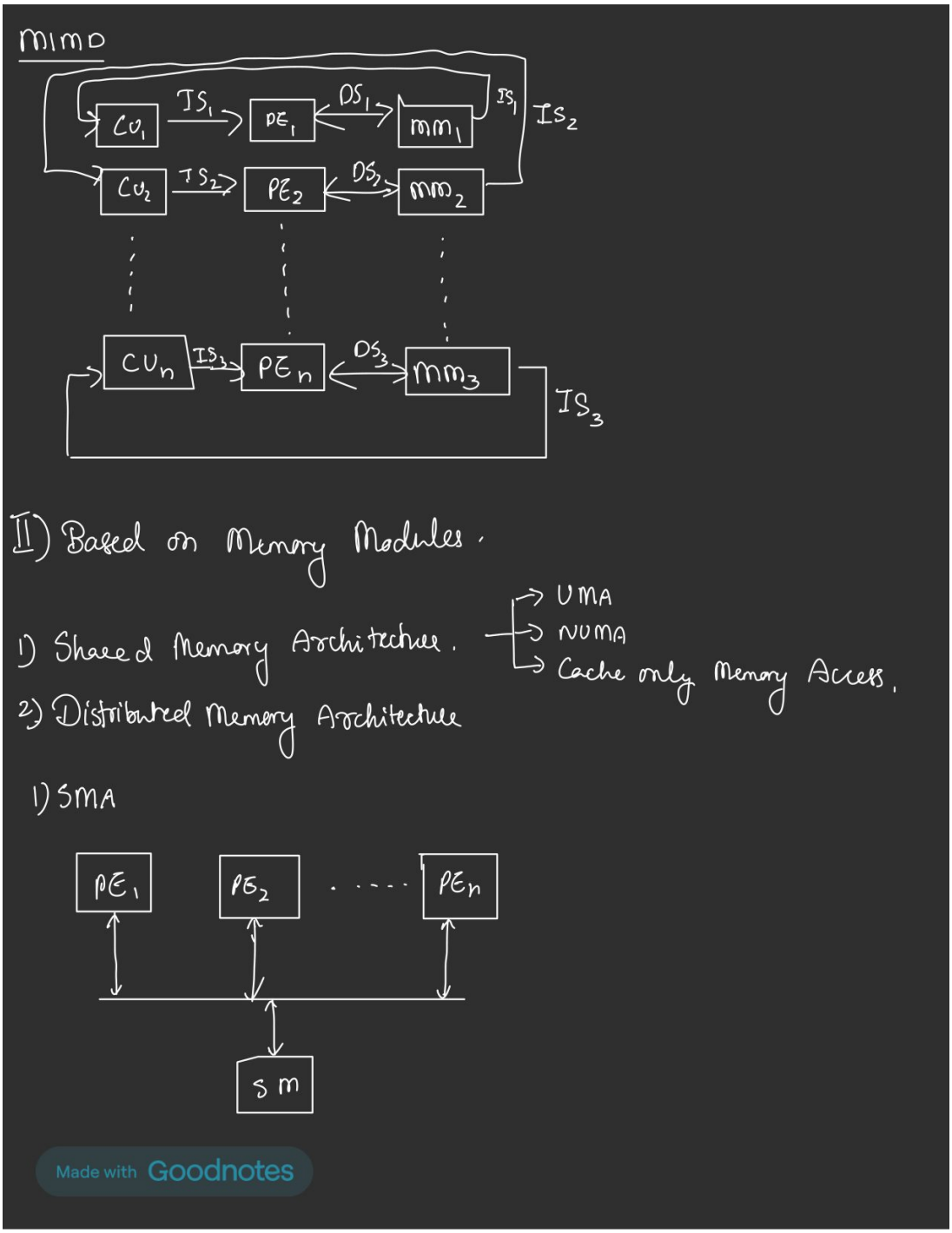
```
SIMD example: c[i] = a[i] + b[i] , i = 0 … n−1 • one ADD instruction is broadcast to n processing elements; • each PE adds its own a[i] and b[i] → c[i] simultaneously; • speed-up ≈ n over the scalar loop (ignoring overhead).
```

Reconstructed diagrams.

- **SIMD:** draw one *control unit* on the left feeding a row of processing elements [PE][PE][PE]…; each PE has its own memory slice. One instruction goes to all of them at once.
- **MIMD:** draw several independent processors [PU|PE] side by side, each with its own instruction stream, connected to shared or distributed memory. They run different programs.

> **Exam point.** SIMD needs the data to be laid out regularly (a vector). MIMD is the general model behind multicore CPUs; it needs no regularity but requires synchronisation and communication between the processors.

Original notes — page 7: MIMD and shared-memory multiprocessors — classification b…



Original handwritten notes — page 7.

Transcription of the page

```
MIMD

   Based on modules:
     1) Shared memory multiprocessors
     2) Distributed memory

Shared Memory:
   a) UMA
   b) NUMA
   c) COMA

(Each processor has its own cache; all caches see one shared main memory.)
```

Explanation

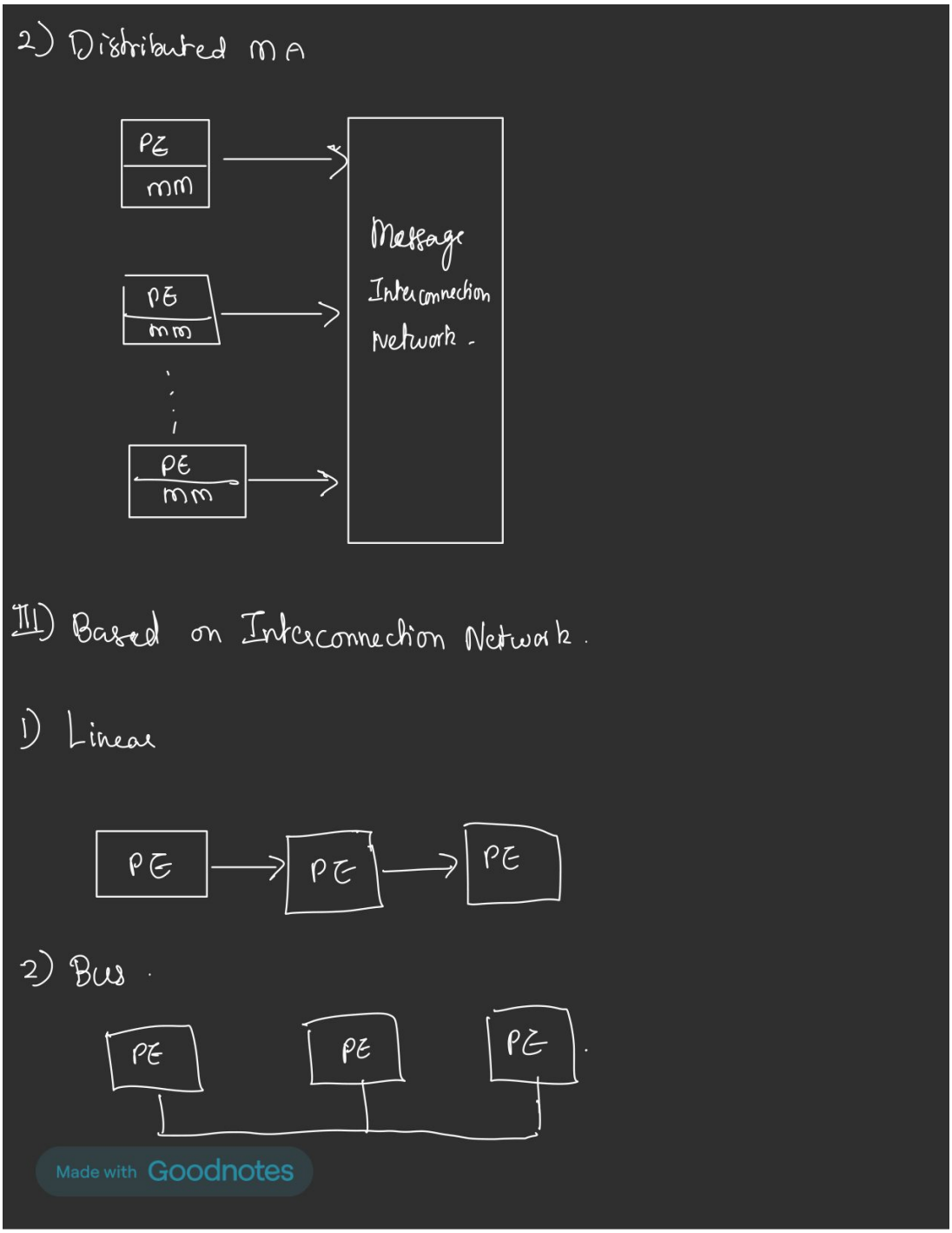
What the page says. MIMD machines split into two families according to where the memory lives: **shared memory** (one address space) and **distributed memory** (each processor has its own memory). The shared-memory family is then split into UMA, NUMA and COMA.

Model	Where the data lives	How processors communicate
Shared memory / UMA	One central memory, uniform access time	Loads and stores to shared RAM
NUMA	One address space, local memory faster	Loads/stores; remote ones are slower
COMA	All memory acts as cache	Data migrates to the node using it
Distributed memory	Private memory per processor	Explicit message passing

Reconstructed diagram — shared-memory multiprocessor. Draw several processors, each with its own private cache, all connected by a common bus to one shared main memory. The private caches are what make it fast (each core keeps its working set locally) and also what create the coherence problem handled later in the notes.

> **Why the cache matters here.** Without caches, every access would hit the single shared memory and the machine would not scale. With caches, most accesses are local — but the same block may now exist in several caches, which is exactly the cache-coherence problem.

Original notes — page 8: Distributed memory; classification by interconnection networ…



Original handwritten notes — page 8.

Transcription of the page

```
2) Distributed Memory

   (each PE has its own local memory; PEs talk over a network)

Based on interconnection network:
   1) Bus
   ...
Bus:  a single shared communication path.
   (All processors attach to one bus; only one transfer at a time.)
```

Explanation

What the page says. The other half of MIMD is **distributed memory**: each processing element owns a private memory, and the PEs exchange data over an **interconnection network**. The first network the lecturer draws is the **bus**.

- **Distributed memory** — each PE has its own memory, so it scales well; the cost is that the programmer must pass messages explicitly.

- **Interconnection network** — the hardware that carries messages between PEs. Networks are classified as *direct* (nodes are also switches: mesh, torus, hypercube, tree) or *indirect* (separate switches: bus, crossbar, multistage).

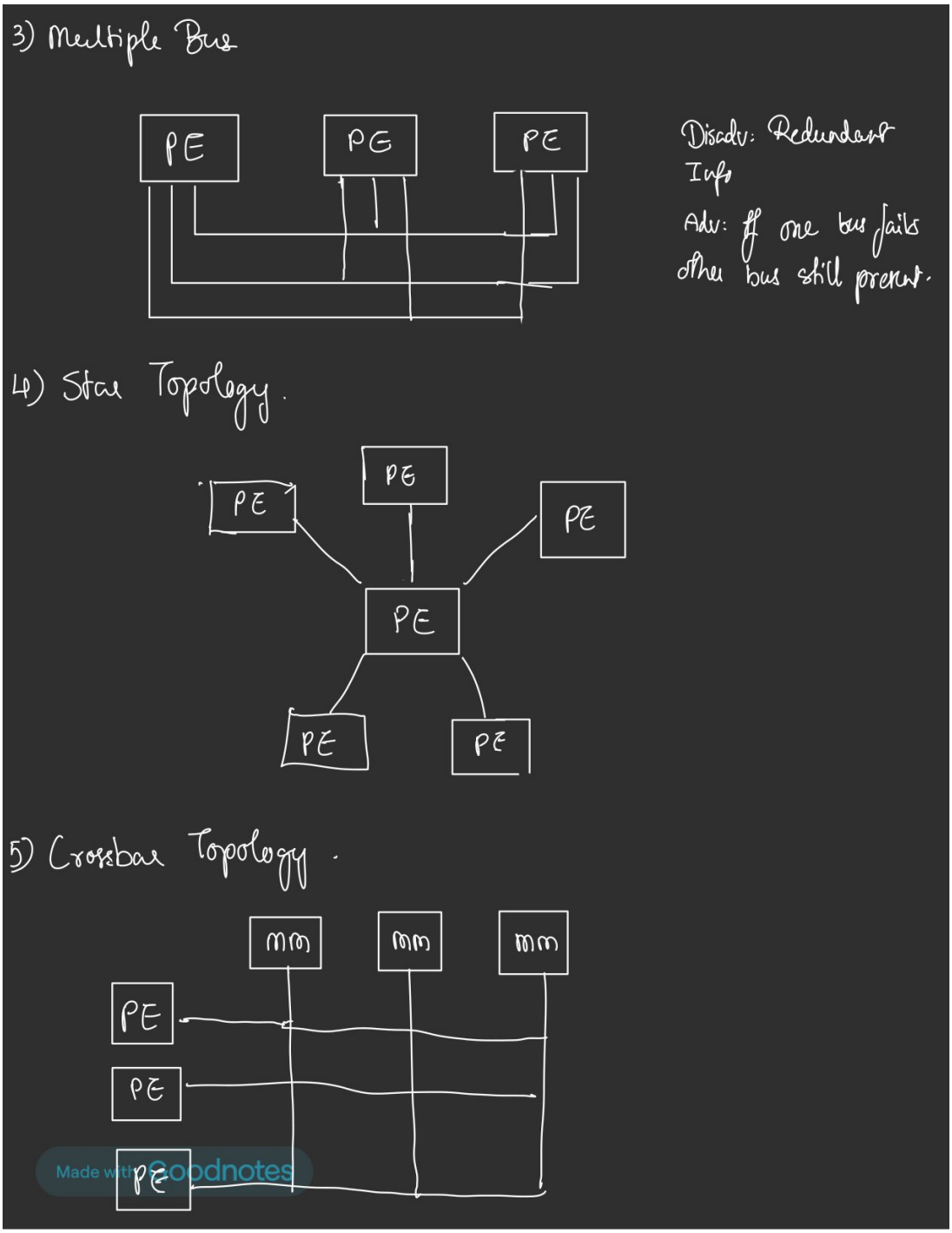
Bus topology

A **bus** is a single shared communication path. It is the simplest and cheapest network, but only one transfer can happen at a time, so as processors are added they fight for the bus and performance collapses.

```
Bus bandwidth: BW = W · f (W = bus width in bytes, f = bus clock) But the whole system can move only BW bytes/s TOTAL, shared by all PEs, so per-processor bandwidth = BW / n → falls as n grows.
```

Reconstructed diagram. Draw a single horizontal line (the bus) with each processor (and the shared memory) hanging off it, like beads on a string. An arrow along the line shows one transfer at a time.

Original notes — page 9: Multistage (multiple) bus and crossbar topology



Original handwritten notes — page 9.

Transcription of the page

```
3) Multistage Bus

   Adv: gives many paths.
   Disadv: cost, complexity.

4) Crossbar Topology

   [P][P][P][P]  (inputs)
    |  |  |  |
   ==crosspoint switches==
    |  |  |  |
   [M][M][M][M]  (outputs)

   Non-blocking: every input can reach every output at once.
```

Explanation

What the page says. To reduce the contention of a single bus, use more than one bus (**multistage**) or a full switching matrix (**crossbar**).

Multistage (multiple) bus

- **Advantage:** several transfers can proceed at once (many paths), so contention falls.
- **Disadvantage:** more hardware higher cost and more complexity.

Crossbar topology

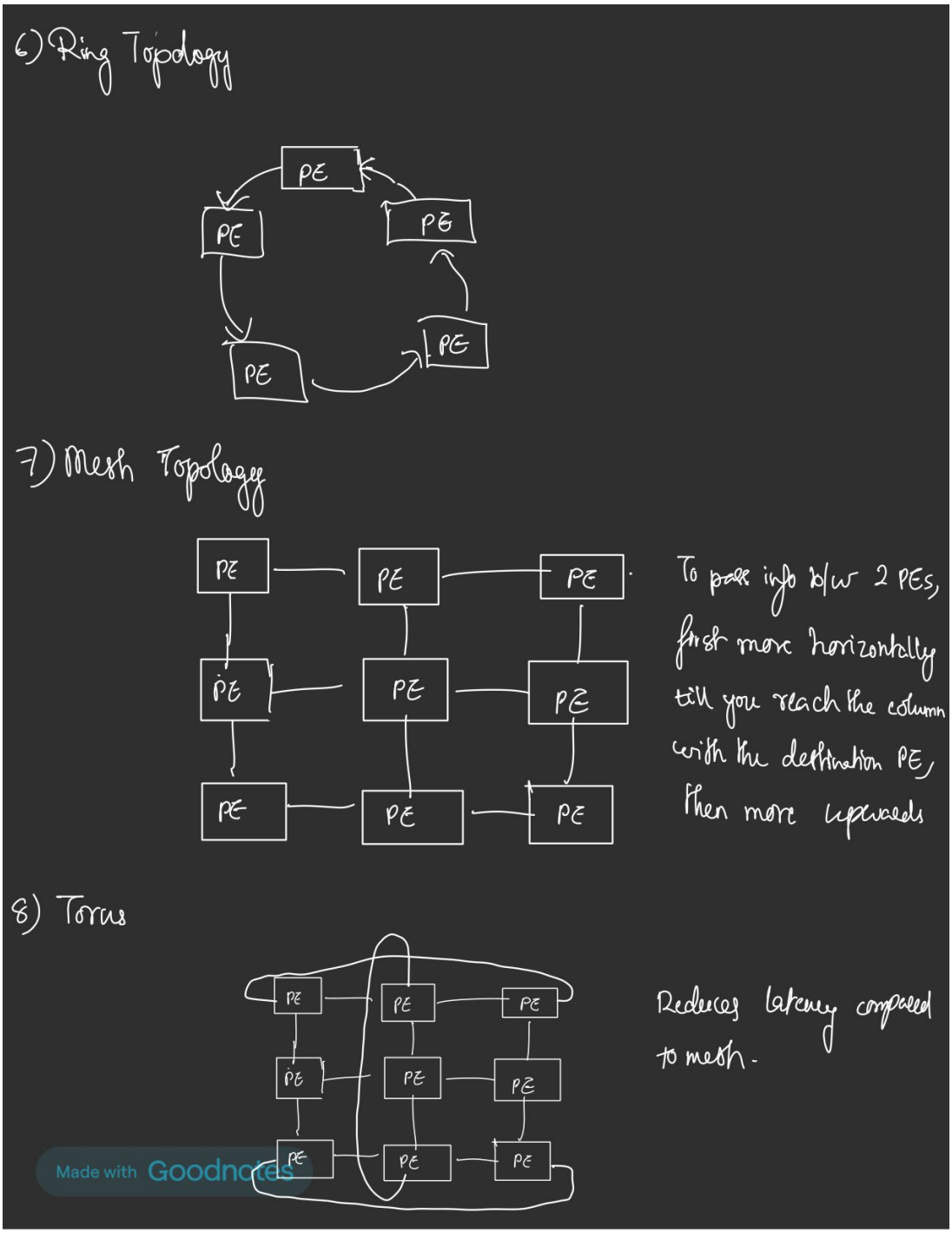
A crossbar is a grid of switches that connects every input (a processor) to every output (a memory module). Any input can be joined to any output at the same time — it is **non-blocking** — as long as no two inputs want the same output.

```
Crossbar with n inputs and n outputs needs n² crosspoint switches. It is non-blocking (all n pairs can transfer at once), but cost grows as O(n²), so it is practical only for small n.
```

Reconstructed diagram. Draw a square grid: processors along the top, memory modules along the left, and a switch at every crossing. Closing one switch connects one processor to one memory module; because the switches are independent, several pairs can be connected at once.

Network	Cost (switches)	Blocking?	Scales to
Bus	O(1)	Blocking (one transfer)	few PEs
Multistage bus	O(n log n)	Some blocking	moderate n
Crossbar	$O(n^2)$	Non-blocking	small n

Original notes — page 10: Mesh and torus topologies



Original handwritten notes — page 10.

Transcription of the page

```
7) Mesh Topology

   [PE]—[PE]—[PE]
    |    |    |
   [PE]—[PE]—[PE]
    |    |    |
   [PE]—[PE]—[PE]

   Simple, good locality. Diameter grows with the side length.
   Routing: each hop moves toward the destination PE.

8) Torus

   Same as mesh but with wrap-around links at the edges.
   Reduces the diameter (and the cost of far hops).
```

Explanation

What the page says. The **mesh** is the simplest direct network: each node is wired only to its immediate neighbours, and the **torus** is a mesh with the edges wrapped around.

Mesh

- Every node connects to its up/down/left/right neighbours (degree 4 for an interior node).
- It is simple to build, has good *locality* (nearby nodes are close), and scales to large machines.
- The disadvantage is that the **diameter** — the longest shortest-path — grows with the side length, so far-apart nodes communicate slowly.
- Routing is dimension-order: move along one dimension until the coordinate matches, then the next.

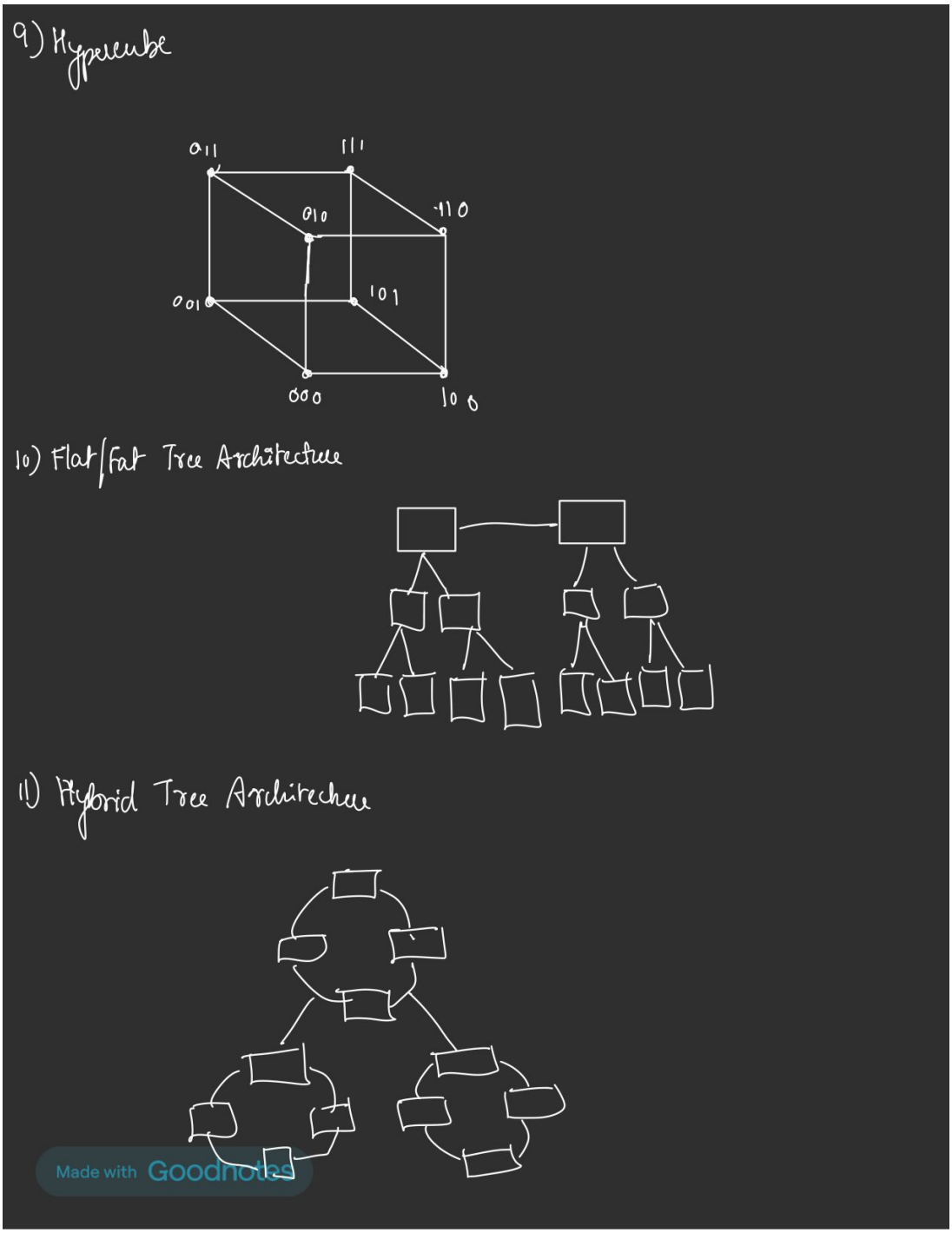
Torus

A torus adds **wrap-around** links between opposite edges. This shortens the longest path (smaller diameter) at the cost of the extra edge wires, so average communication is faster.

```
k × k mesh (n = k² nodes): degree = 4 (interior) diameter = 2(k − 1) = 2(√n − 1) bisection width = k = √n k × k torus (wrap-around): diameter = 2⌊k/2⌋ ≈ √n (about half the mesh diameter)
```

Reconstructed diagram. Draw a 3×3 grid of PE boxes joined by horizontal and vertical lines; for the torus, add lines from the left column to the right column and from the top row to the bottom row (curving around).

Original notes — page 11: Hypercube, tree architectures, and the first look at the vector…



Original handwritten notes — page 11.

Transcription of the page

```
9) Hypercube

   000 — 001
    |      |
   010 — 011     (each node labelled with a d-bit address;
    |      |      neighbours differ in exactly one bit)
   100 — 101
    |      |
   110 — 111

10) Flat / Flat-tree Architectures
11) Hybrid Tree Architecture

(Vector / Array processor introduced here.)
```

Explanation

What the page says. The **hypercube** is a direct network whose diameter is logarithmic, and the page also lists **tree** architectures (flat tree, hybrid tree). The vector/array processor is introduced at the foot of the page and continued later.

Hypercube

- With $n = 2^d$ nodes, each node is labelled by a d-bit address.
- Two nodes are neighbours when their addresses differ in exactly **one bit**.
- Degree = d, diameter = $d = \log_2 n$, and there are $n \cdot d/2$ links.
- It has excellent (logarithmic) diameter, but the wiring cost grows as O(n log n).

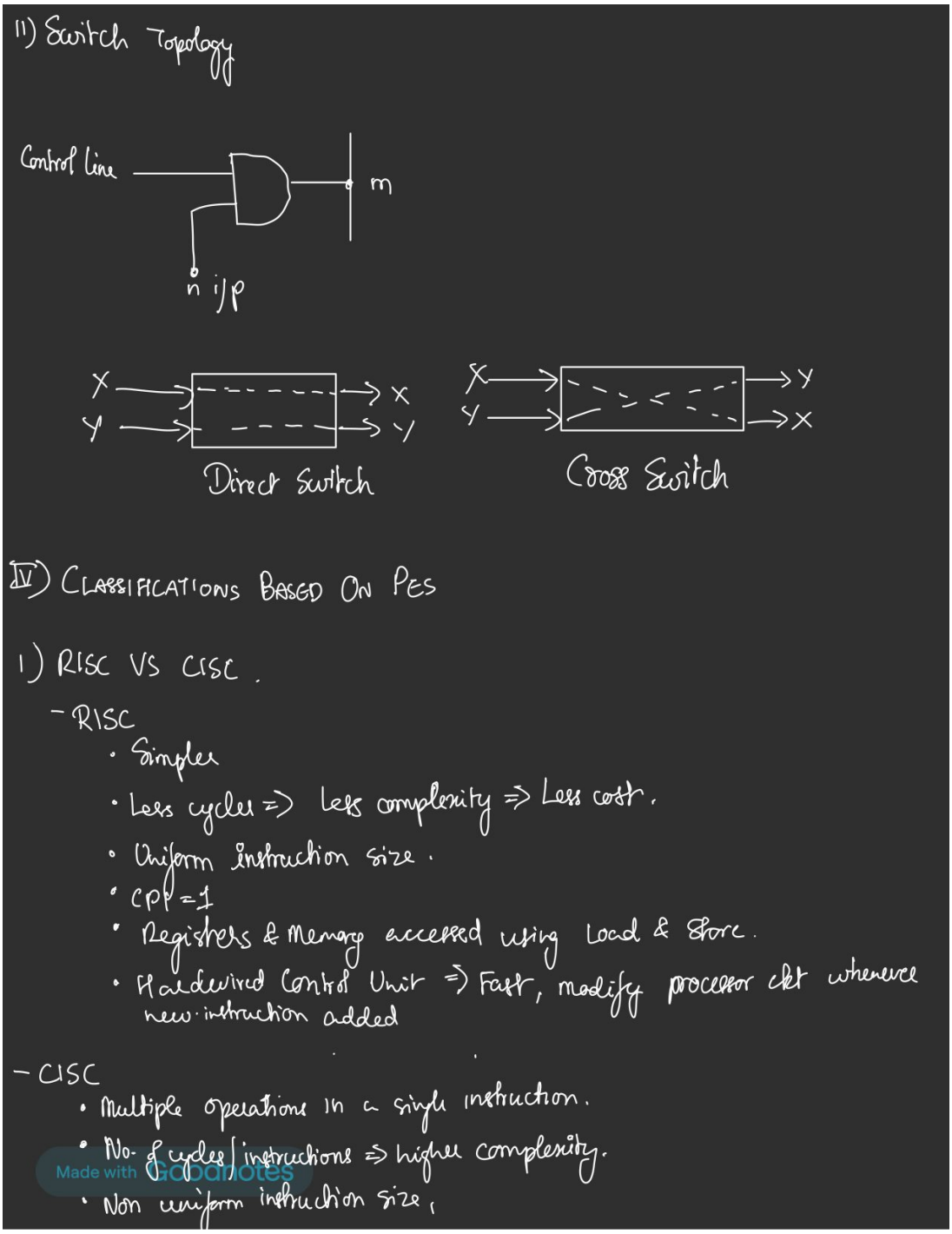
```
Hypercube with n = 2^d nodes: degree = d = log₂n diameter = d = log₂n links = n·d / 2 Routing: XOR the source and destination addresses; flip the set bits one at a time.
```

Tree architectures

A **tree** connects nodes in a hierarchy (a root, its children, and so on). It is cheap (degree 3, diameter 2 $\log_2 n$) and natural for divide-and-conquer algorithms, but the root becomes a bottleneck because all traffic between the two halves must pass through it. The **flat tree** and **hybrid tree** add extra links to relieve the root.

Topology	Degree	Diameter	Bisection width
Mesh (k×k)	4	2(k−1)	k
Torus (k×k)	4	≈ k	2k
Hypercube (2^d)	d	d	n/2
Binary tree	3	2 $\log_2 n$	1

Original notes — page 12: Systolic arrays; classification based on the processing elem…



Original handwritten notes — page 12.

Transcription of the page

```
12) Systolic

Direct Switch / Crossbar Switch

Classification based on the Processing Element (PE):
   1) RISC vs CISC

RISC:
  * Less hardware ▫ Less complexity ▫ Less cost
  * Uniform instruction size
  * Register-to-register (Load/Store) → programmer must move data with Load/Store
  * Pipelining easier, CPI ≈ 1

CISC:
  * Multiple operations in one instruction
  * No. of instructions needed is less
  * Instructions more complex
  * Hardware more complex than RISC
```

Explanation

What the page says. Two more network types (systolic arrays and switch-based networks) are named, then the lecturer returns to the RISC/CISC contrast in more depth.

Systolic arrays

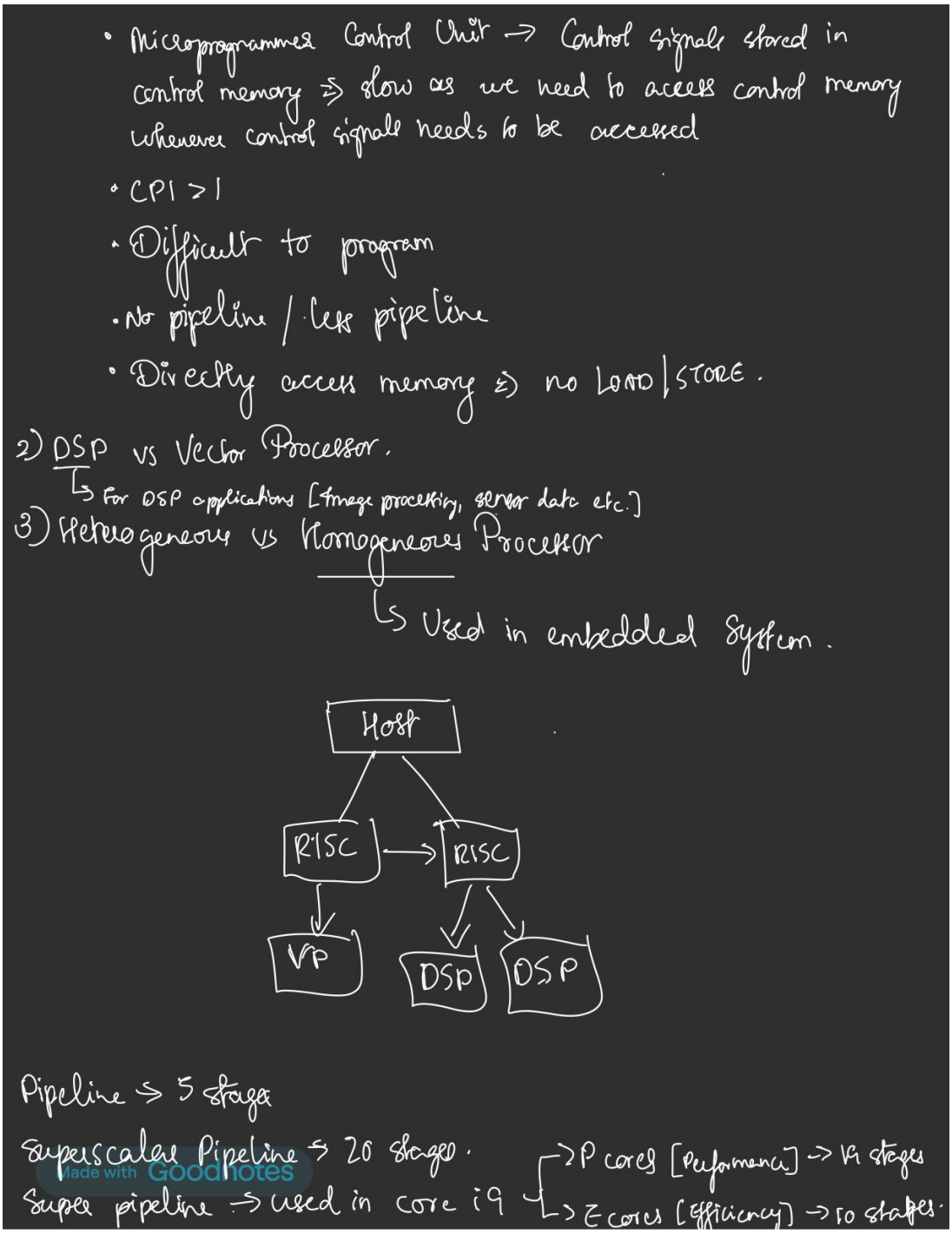
A **systolic array** is a grid of identical, simple processing elements that pass data to their neighbours in lock-step, one “heartbeat” at a time. Each PE does a small multiply-accumulate and forwards the operands. Because only nearest-neighbour wires are needed, the clock can be fast, and a new result emerges every cycle once the array is full. It is ideal for regular computations such as matrix multiplication and convolution (used in DSP and in neural-network accelerators).

RISC versus CISC (second pass)

	RISC	CISC
Instruction size	Uniform (fixed)	Variable
Memory access	Load/Store only (register-to-register)	Operations may act on memory
Instruction count	More instructions	Fewer instructions
Hardware	Simple, cheap, fast to build	Complex, costlier
Pipelining	Easy; CPI ≈ 1	Harder; CPI > 1
Data movement	Programmer/compiler uses Load/Store	Handled inside one instruction

> **The trade-off in one line.** CISC hides work inside a single powerful instruction (fewer instructions, more complex hardware); RISC exposes the work as simple instructions that pipeline beautifully (more instructions, simpler hardware). The compiler does more work for RISC.

Original notes — page 13: Consequences of RISC/CISC for pipelining and CPI; vector p…



Original handwritten notes — page 13.

Transcription of the page

```
Consequences:
  * Register-to-register (Load/Store) ▫ simpler pipeline
  * Uniform instruction size ▫ easier to decode and pipeline
  * CISC: complex instructions ▫ CPI > 1

Pipelining:
   Long pipeline vs short pipeline
   CISC: does all in one instruction, no Load/Store split
   RISC: uses vector processor

Vector processor:
   For DSP applications (image processing, sensor data, etc.)
   Uses in embedded systems

RISC:  Super-scalar pipeline → 10 deep
       Super pipelining → used in core (e.g., Intel) → IPC higher
```

Explanation

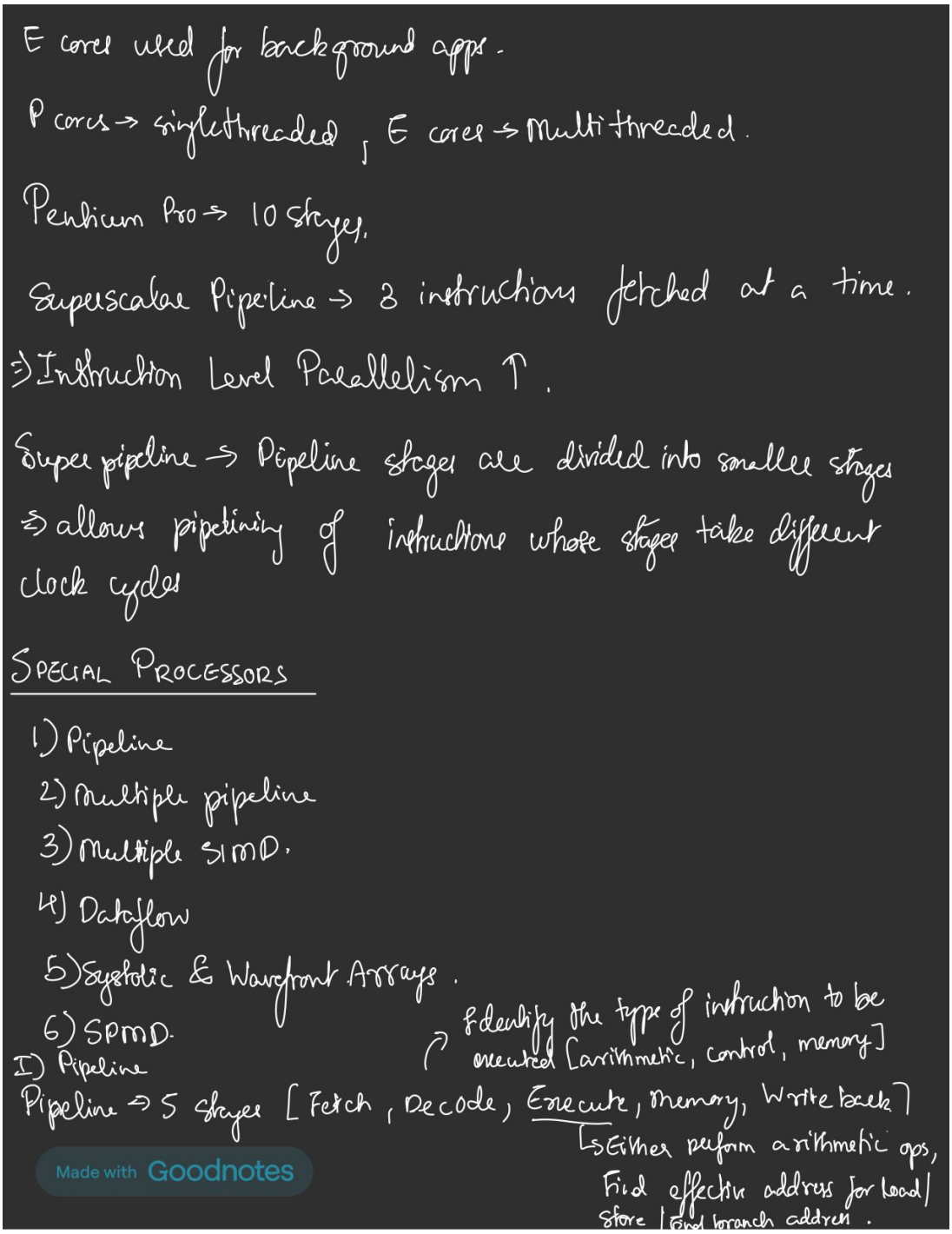
What the page says. The lecturer draws the practical consequences of the ISA choice: RISC's uniform, register-to-register instructions make a clean, deep pipeline possible; CISC's complex instructions make CPI larger than 1. Vector processors are introduced as the tool of choice for DSP (signal/image processing).

- **Uniform instruction size (RISC)** the decoder is simple and the pipeline stages are balanced, so the clock can be short and CPI can approach 1.
- **Load/Store split (RISC)** memory is touched only in dedicated instructions, so the pipeline does not have to handle memory operands in the ALU stage.
- **Complex instructions (CISC)** one instruction may take many cycles CPI > 1.
- **Deeper pipelines** (e.g. superscalar, ~10 stages) raise the clock rate but increase the penalty when a branch is mispredicted.
- **Vector processors** are used for DSP — image processing, sensor data, embedded systems — because the same operation is applied to long arrays of data.

```
IPC = Instructions Per Cycle = 1 / CPI Superscalar / deep pipeline raise IPC; a stall or a mispredict lowers it. Speed-up from pipelining ≈ number of stages (ideal), limited by hazards.
```

This page is the bridge from "what the ISA looks like" to "how we make it fast": the rest of the notes builds the pipeline, then adds branch prediction, dynamic scheduling and speculation to keep CPI close to 1.

Original notes — page 14: Pipelining introduced; the five stages; levels of parallelism r…



Original handwritten notes — page 14.

Transcription of the page

```
Pipeline (instruction-level parallelism):
  Overlap the execution of many instructions.

Levels of parallelism:
  I) Bit-level
  II) Instruction-level
  III) Data-level
  IV) Task-level

Pipeline stages:  Fetch → Decode → Execute → Memory → Write-back
   (also called IF, ID, EX, MEM, WB)

“Pipeline = Fetch, Decode, Execute, Memory, Write-back”
   — used for each instruction; many instructions in flight at once.
```

Explanation

What the page says. Pipelining is introduced as the main form of instruction-level parallelism, and the five classic stages are listed. This is developed in detail on pages 33–36, so here we only fix the vocabulary.

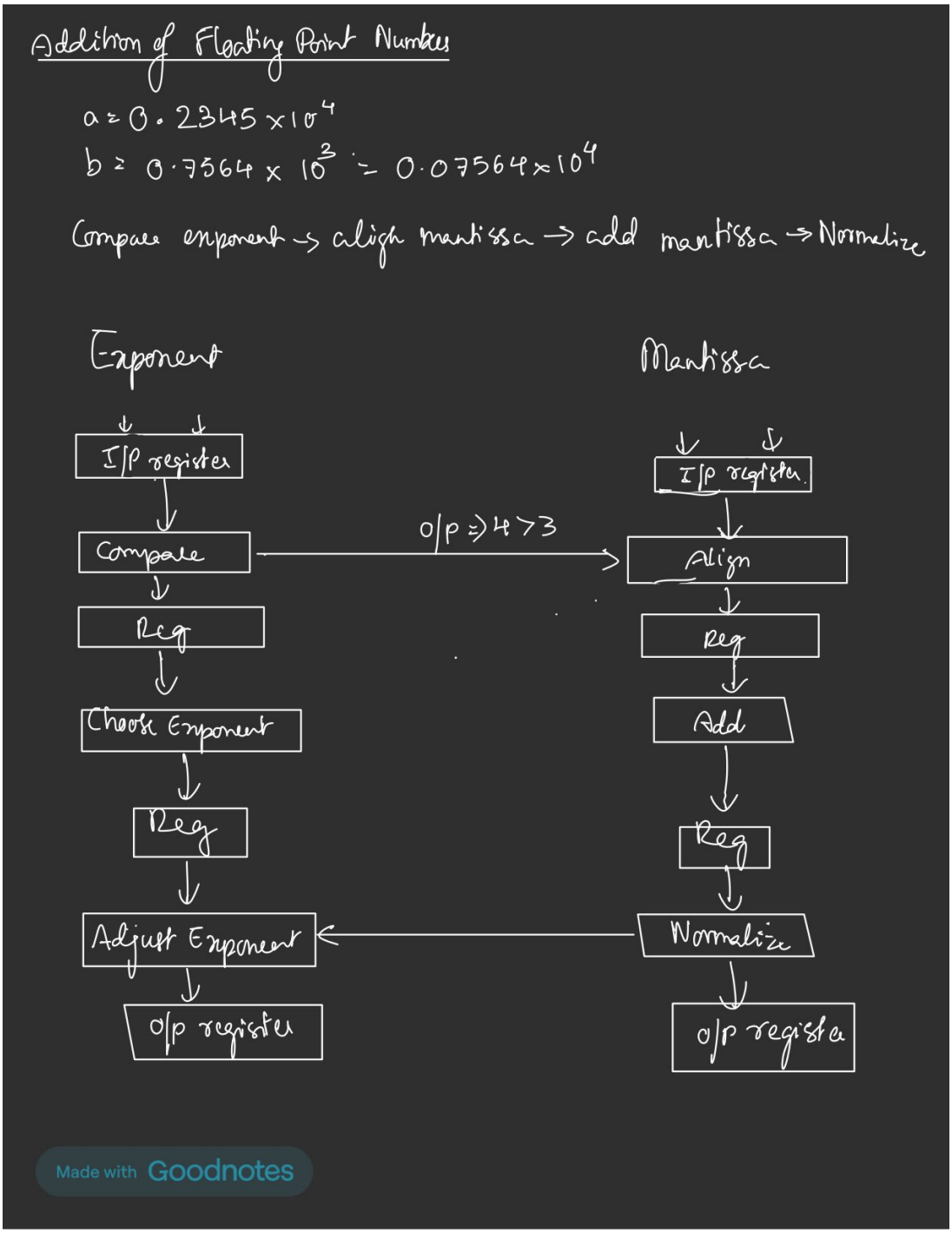
```
The 5 stages (MIPS classic): IF — Instruction Fetch : read the instruction at PC; PC ← PC + 4
ID — Instruction Decode : decode; read the register file EX — Execute : ALU / address
calculation MEM — Memory access : load or store WB — Write Back : write the result into the
register file
```

Why overlap helps. In a non-pipelined machine an instruction occupies the whole datapath for its full latency, so most hardware sits idle. A pipeline puts a different instruction in each stage every cycle, so in steady state one instruction finishes per cycle even though each instruction still takes five cycles.

```
Ideal throughput = 1 instruction per clock (CPI = 1). Speed-up ≈ number of stages = 5 (ideal,
ignoring fill/drain and hazards).
```

> **Remember the two numbers.** *Latency* (time for one instruction) is unchanged at 5 cycles; *throughput* (instructions per cycle) rises to 1. Pipelining improves throughput, not latency.

Original notes — page 15: IEEE-754 floating-point arithmetic — the worked addition ex…



Original handwritten notes — page 15.

Transcription of the page

```
Q:  6.2345 × 10³
 b) 9.3564 × 10⁴  +  0.0964 × 10⁴

Comparing operands (align exponents) → then add mantissas
→ normalize the result.

(Steps of floating-point addition:)
  1) Align the exponents (shift the smaller mantissa right)
  2) Add the mantissas
  3) Normalize the result
  4) Round and re-pack
```

Explanation

What the page says. A worked example of floating-point addition, showing the four steps the hardware performs. Understanding these steps explains why floating-point add is more complex than integer add and why it can take several cycles.

The four steps of floating-point addition

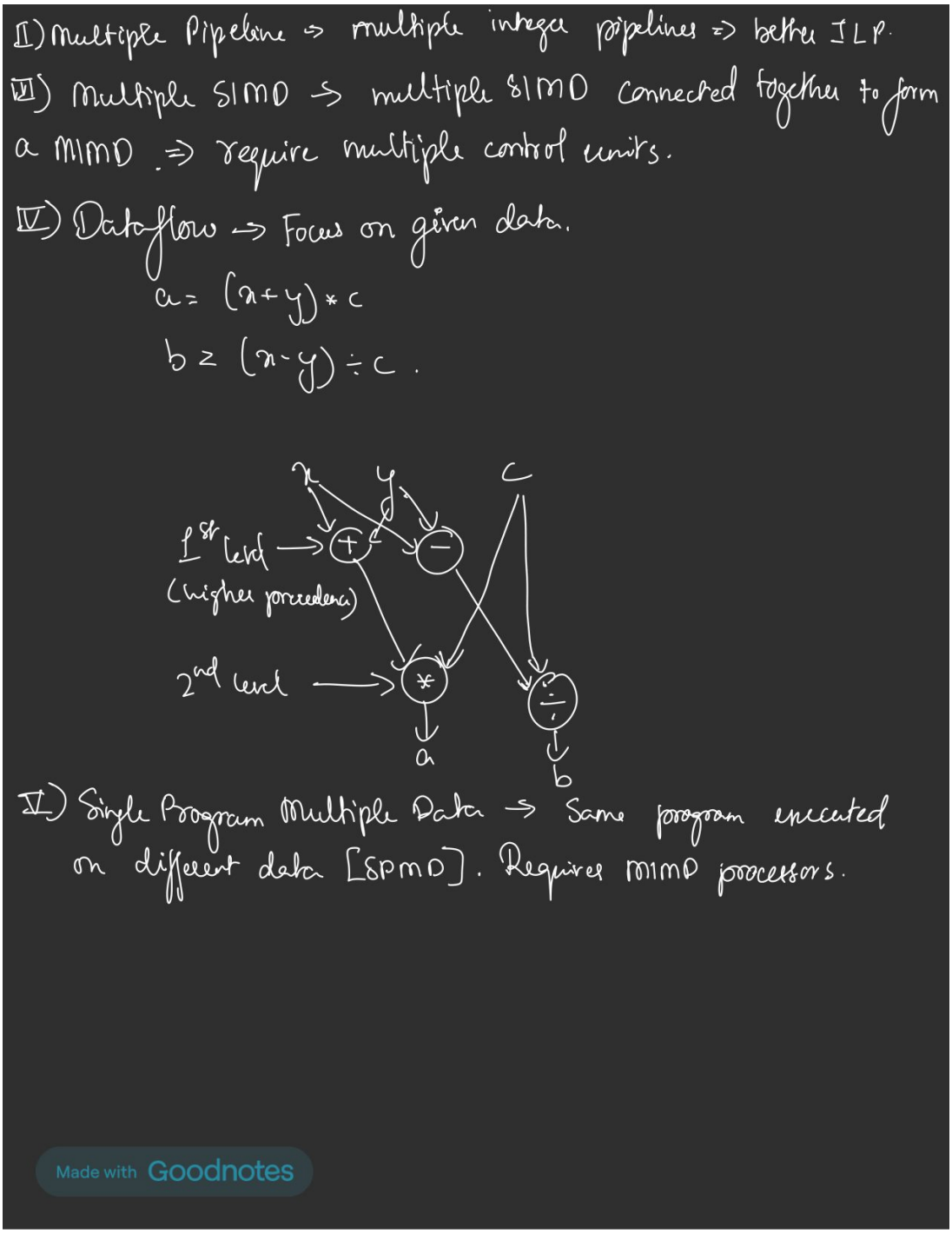
- **1) Align the exponents.** Rewrite both numbers so they share the larger exponent; this means shifting the smaller number's mantissa right (which loses low bits).
- **2) Add the mantissas** (the significands) now that they have the same scale.
- **3) Normalise.** Shift the result so that the leading digit is back in the standard place (1.xxx × 2^e in binary), adjusting the exponent.
- **4) Round and re-pack.** Fit the mantissa back into the available bits and store the sign, exponent and mantissa.

```
Worked example (decimal): 9.3564 × 10⁴ + 0.0964 × 10⁴ exponents already equal (10⁴), so add mantissas: 9.3564 + 0.0964 = 9.4528 result = 9.4528 × 10⁴ (already normalised) If the exponents differed, e.g. 1.2 × 10³ + 3.4 × 10⁴ : rewrite as 0.12 × 10⁴ + 3.4 × 10⁴ = 3.52 × 10⁴
```

> **Why "normalise"?** Floating-point numbers are stored in a canonical form 1.ffff × 2^e (one non-zero digit before the point). After adding, the leading digit may move, so the result must be shifted back into canonical form — otherwise precision is wasted and the encoding is ambiguous.

The notes later connect this to the **IEEE-754** standard (32-bit single and 64-bit double precision: 1 sign bit, 8/11 exponent bits, 23/52 mantissa bits), which defines these fields, the implicit leading 1, and the special values (zero, infinity, NaN).

Original notes — page 16: Superscalar and SIMD; the systolic idea; multicore as the re…



Original handwritten notes — page 16.

Transcription of the page

```
1) Superscalar Pipeline — multiple integer pipelines → higher IPC.
2) Multiple SIMD — multiple SIMD connected together.
   MIMD requires independent control units.

   Example:  (a + b) × c
             (a × b) − c
             a ▫ (b …)

   (Chip → on-chip)
3) Systolic Multiple Code → some comparison operators on-chip {SIMO}.
   Requires MIMD processors.

Thermal design:  multicore uses a big die with many cores and shared L2;
                 larger die needs more transistors → more heat → thermal management.
```

Explanation

What the page says. Three ways to raise throughput beyond a single scalar pipeline, plus the thermal reason the industry moved to multicore.

1) Superscalar

A **superscalar** processor has several parallel execution pipelines, so it can issue more than one instruction per cycle. IPC can rise above 1 (CPI below 1). The hardware must find independent instructions at run time.

2) Multiple SIMD

Several SIMD units are connected together. Because each SIMD unit applies one instruction to many data items, connecting them multiplies the throughput. A MIMD machine, by contrast, needs *independent control units* for each processor.

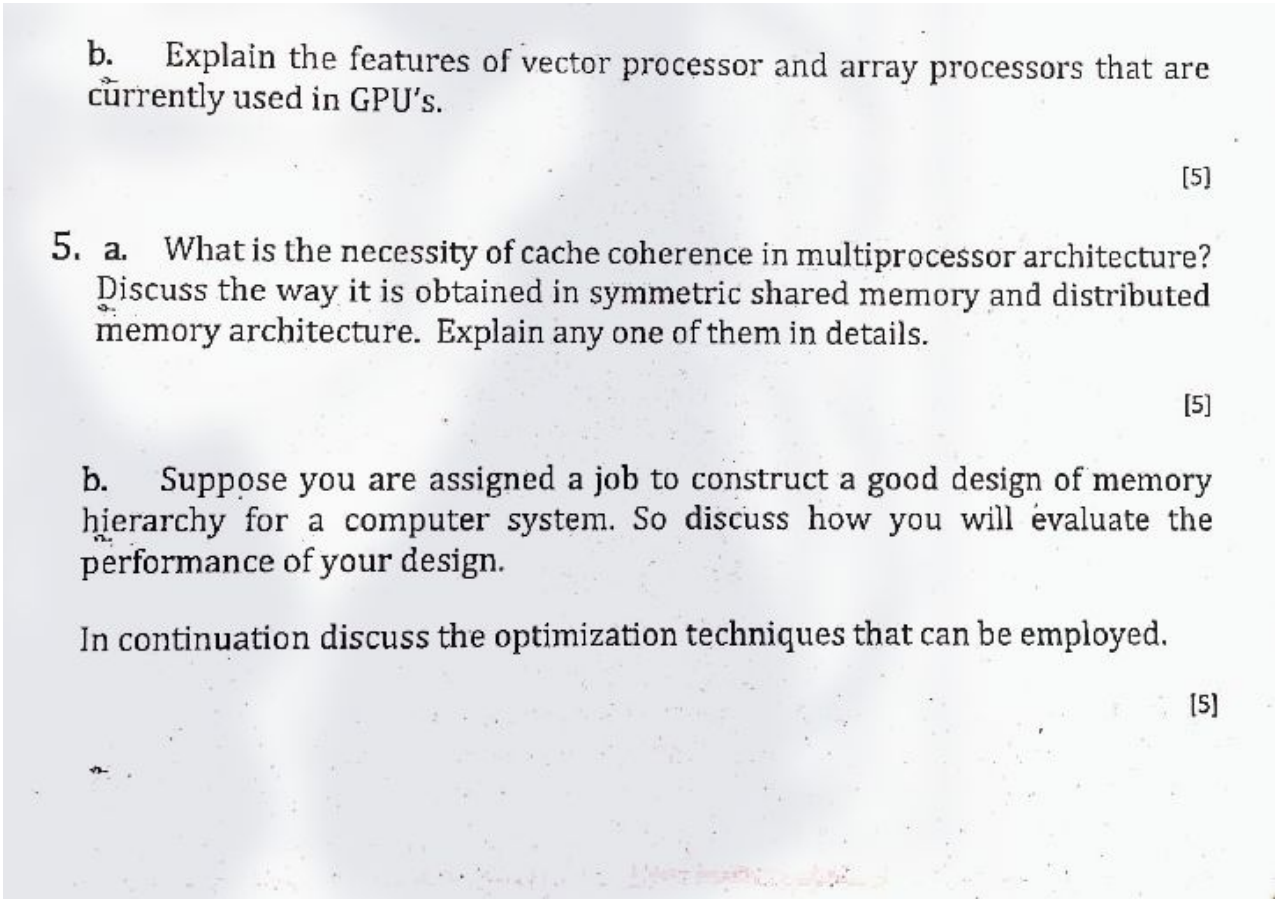
```
Example operations that vectorise well: (a + b) × c , (a × b) − c , a ▫ (b ▫ c) Superscalar width m (ideal): CPI = 1/m , speed-up ≤ m Multiple SIMD (k units of width n): throughput ≈ k · n operations per cycle
```

3) Systolic arrays and the multicore/thermal story

The page notes that systolic-style "multiple code" comparisons can be done on-chip, and that MIMD processors are needed to run independent streams. It then records the crucial design driver: a multicore chip uses a large die with many cores and a shared L2 cache; a larger die means more transistors and therefore **more heat**, so thermal management becomes a first-class constraint.

> **Why multicore?** Clock frequency had to stop rising because power (and heat) grows with frequency. The industry switched to putting more cores on one chip: same clock, more work per cycle overall. This is why the notes repeatedly link parallelism to power and thermal limits.

Previous-year question 41 — ca41 — Vector/array features used in GPUs; cache cohere…



b. Explain the features of vector processor and array processors that are currently used in GPU's.

[5]

5. a. What is the necessity of cache coherence in multiprocessor architecture? Discuss the way it is obtained in symmetric shared memory and distributed memory architecture. Explain any one of them in details.

[5]

b. Suppose you are assigned a job to construct a good design of memory hierarchy for a computer system. So discuss how you will evaluate the performance of your design.

In continuation discuss the optimization techniques that can be employed.

[5]

Original past-paper scan ca41.png (re-rendered).

Transcription of the question

```
(4.) Explain the features of vector processors and array processors that are currently
   used in GPUs. [5]
5a. What is the necessity of cache coherence in multiprocessor architecture? Discuss the
    way it is obtained in symmetric shared-memory and distributed-memory architectures.
    Explain one of them in detail. [5]
5b. Suppose you must construct a good design of the memory hierarchy for a computer
    system. Discuss how you would evaluate the performance of your design and the
    optimisation techniques you could employ. [5]
```

Full solution

4 — vector/array features in GPUs

- **SIMD lanes** — one instruction drives many ALUs (vector lanes).
- **Pipelined vector functional units** — deeply pipelined ALUs process streams of elements.
- **Vector registers** — wide register files hold many elements per thread.
- **Regular data flow** — both assume independent, regular data.
- **Latency hiding by parallelism** — many warps keep the lanes busy (like a long vector).
- **High memory bandwidth** — both are built to stream data.

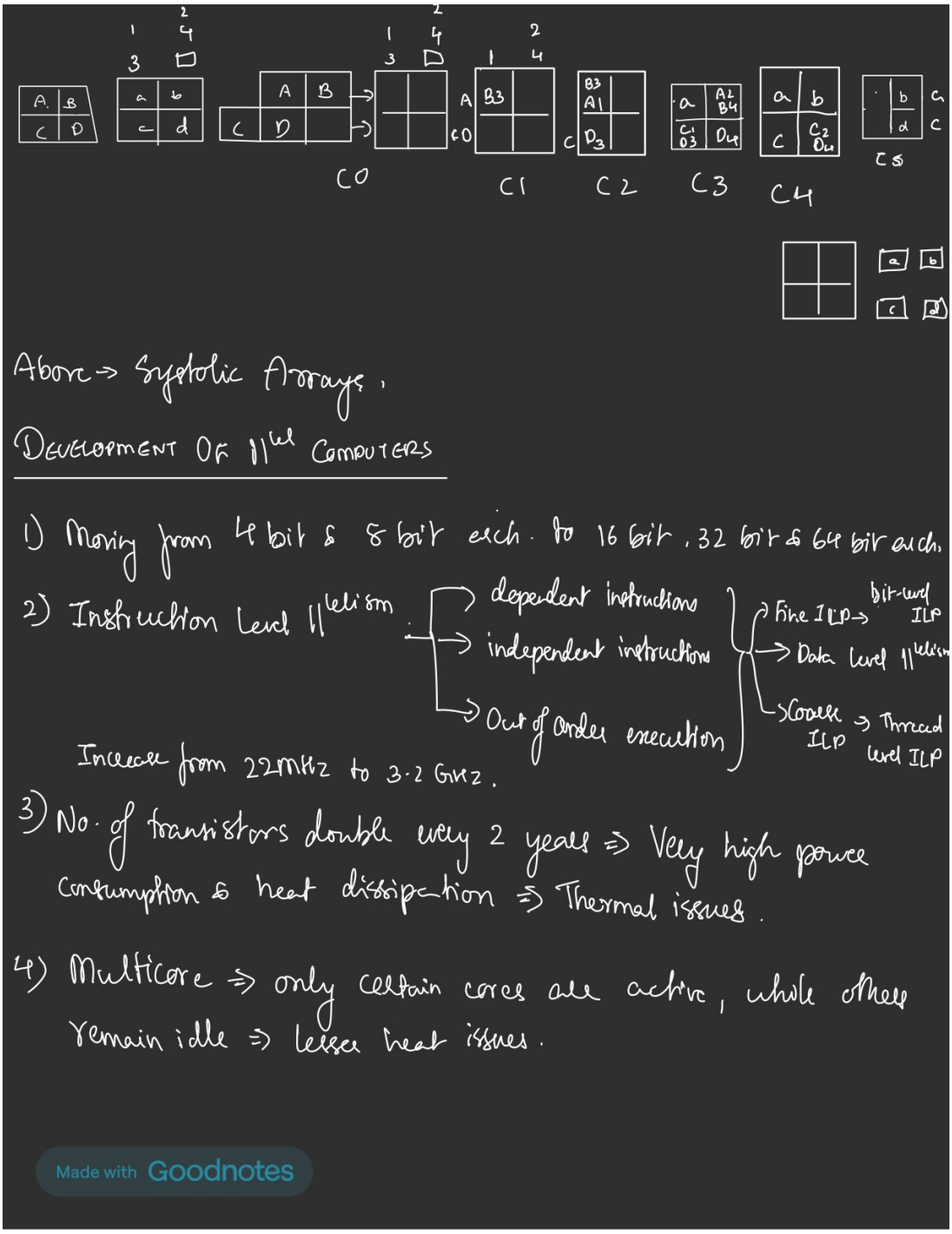
5a — necessity and methods of cache coherence

Coherence is necessary because each core has a private cache, so the same block can exist in several caches; a write by one core must not leave the others stale. In **symmetric shared-memory** systems it is obtained by **snooping** on a broadcast bus; in **distributed-memory** systems by a **directory** that tracks the sharers of each block.

5b — evaluating and optimising the memory hierarchy

```
Evaluate with AMAT = hit time + miss rate × miss penalty and with the three C’s (compulsory,
capacity, conflict) and the CPI. Optimise: reduce miss rate (bigger/higher
associativity/better replacement), reduce miss penalty (multilevel caches,
critical-word-first, non-blocking, prefetching), reduce hit time (smaller/faster cache,
simpler addressing).
```

Original notes — page 17: Systolic arrays in detail; interconnection cost; thermal desig…



Original handwritten notes — page 17.

Transcription of the page

```
Systolic array:
  (a grid of PEs; data flows in, results flow out)
  Development of the systolic array idea.

  Note:
   1) No. of pins is small — data exchange is 16 to 32 bits per line.
   2) Instruction set — each PE has a small, independent instruction set.
   3) No. of transistors is small.
   → Advantages: small pins, high throughput.

Communication network → Thermal management:
   1) Multicore ▫ one big die with many cores + shared L2.
   2) Larger die ▫ more heat.
   (Heat dissipation is the limit.)
```

Explanation

What the page says. The systolic array is analysed (why it is cheap in pins and transistors but fast), and the page closes with the thermal argument for multicore.

Why systolic arrays are efficient

- **Few pins.** Data moves between neighbouring PEs, so only short local wires (16–32 bits) are needed — no long, wide global buses.
- **Simple PEs.** Each element runs a tiny, independent instruction (usually multiply-accumulate), so the control logic is minimal and the design is regular and easy to replicate.
- **Few transistors** per element high density and low power per operation.
- **High throughput.** After the pipeline fills, a new result appears every clock.

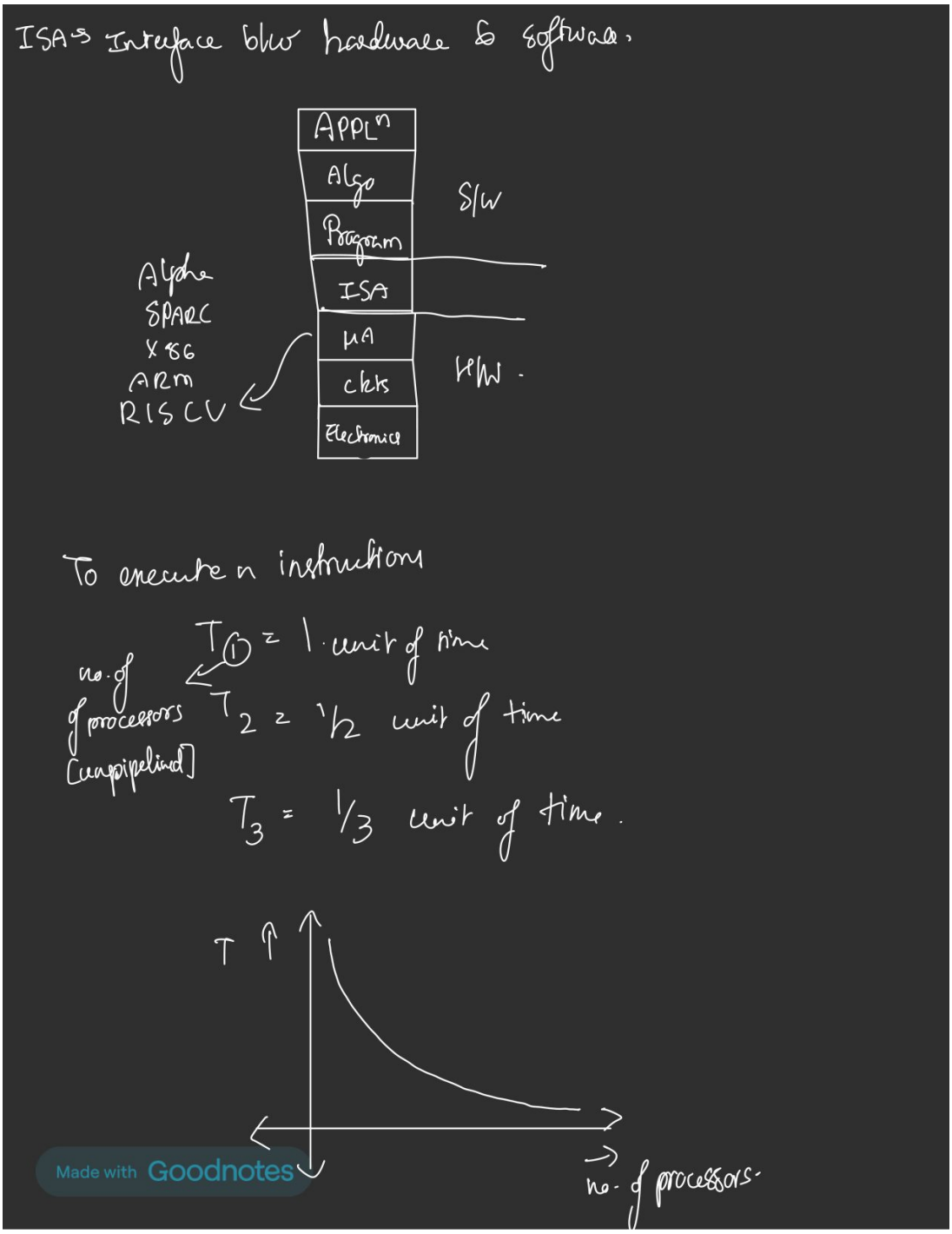
```
Systolic matrix multiply (n×n): Each PE does one multiply-accumulate per beat. Array fills in O(n) beats, then produces a result every beat → throughput ≈ 1 MAC/PE/cycle.
```

The communication-network → thermal argument

- A **multicore** chip is one large die carrying many cores and a shared L2 cache.
- A larger die means **more transistors** switching, hence **more heat** generated per chip.
- Heat must be removed; beyond a limit it cannot be, which is the **thermal/power wall**.

> **Chain of reasoning to remember.** Higher clock more power more heat cannot cool stop raising the clock add more cores instead. Every later topic (cache coherence, multiprocessors) exists because of this chain.

Original notes — page 18: The ISA — the contract between software and hardware; AR…



Original handwritten notes — page 18.

Transcription of the page

```
ISA (Instruction Set Architecture):
  “The interface between hardware and software.”

  [ Software ]  ▫  [ ISA ]  ▫  [ Hardware ]

ARM (example):
  a RISC architecture.

RISC Architecture:
  To increase ILP (instruction-level parallelism)
  T₁ = ?  (number of instructions)
  T₂ = ?  (cycles per instruction)
```

Explanation

What the page says. The **ISA** is the boundary across which software and hardware talk. The lecturer uses ARM (a RISC architecture) as the running example and starts to write the performance quantities.

What the ISA fixes

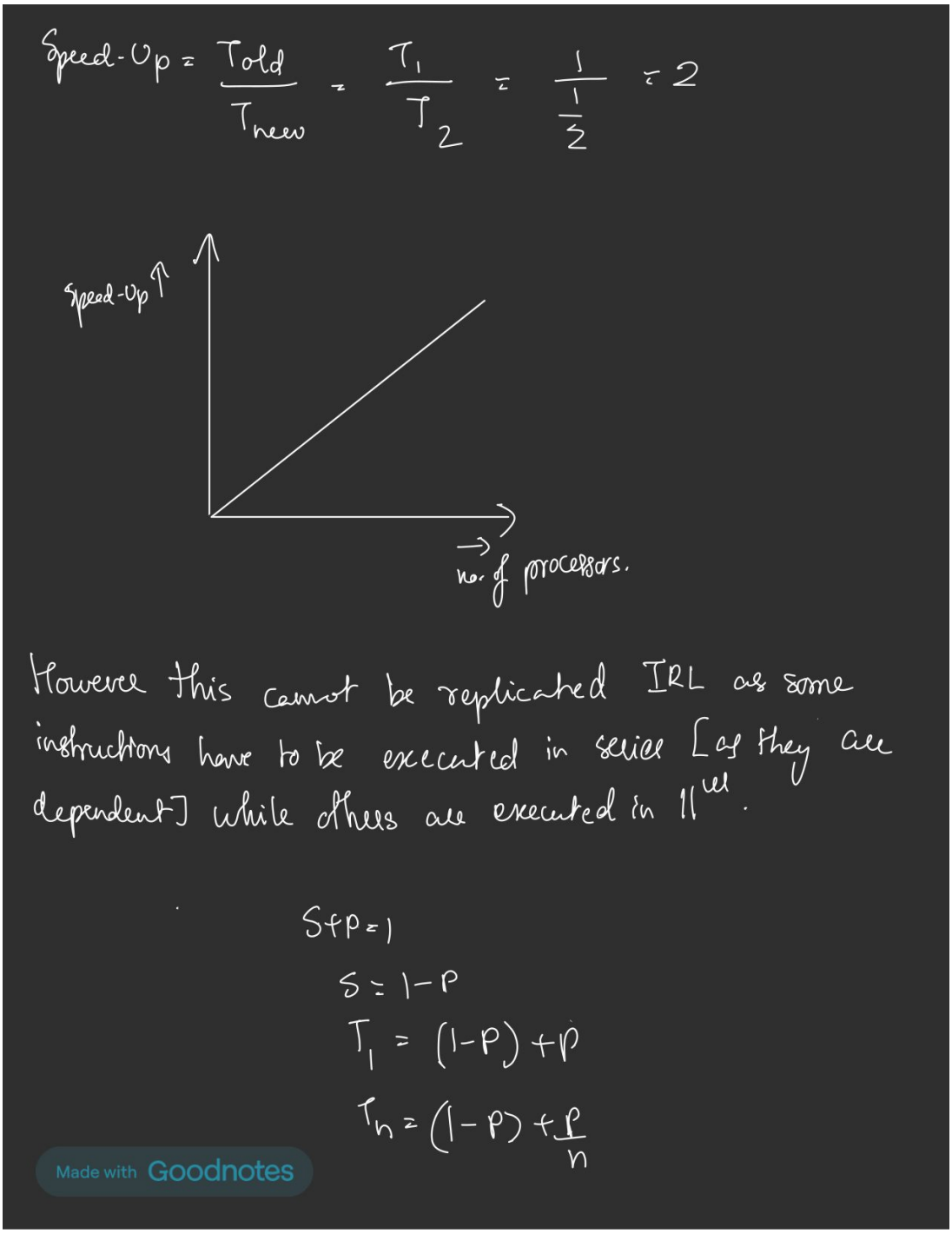
- The **instruction set**: which operations exist (add, load, branch, …).
- The **data types** and their sizes (byte, halfword, word, float).
- The **addressing modes**: how operands are located.
- The **registers** the programmer can see.
- The **instruction encoding** (fixed length for RISC, variable for CISC).

ARM is a well-known **RISC** ISA: fixed 32-bit (or 16-bit Thumb) instructions, a load/store model, and 16 general-purpose registers. The goal of a RISC ISA is to make **instruction-level parallelism (ILP)** easy to exploit — uniform instructions pipeline cleanly.

> The two quantities the lecturer starts to write: T_1 = number of instructions executed T_2 = average cycles per instruction (CPI) Total cycles = $T_1 \times T_2$; time = $T_1 \times T_2 \times$ clock cycle time

This leads directly into the performance equations on the following pages: the whole subject is built on the product of instruction count, CPI and clock period.

Original notes — page 19: Speed-up and Amdahl's Law — the derivation



Original handwritten notes — page 19.

Transcription of the page

```
Speed-up = T_old / T_new = (time before) / (time after)

Amdahl's Law:
  T_new = T_old (1 − f) + T_old · f / k
        = T_old [ (1 − f) + f/k ]

  Speed-up S = T_old / T_new = 1 / [ (1 − f) + f/k ]

  where f = fraction of the work that can be enhanced,
        k = speed-up of the enhanced portion.

As k → ∞:   S_max = 1 / (1 − f)

Hawkiness: cannot be replaced – all instructions must be executed
in some sequence (they are dependent), while others can be done in parallel.
```

Explanation

What the page says. Amdahl's Law is derived from scratch and the limit is stated. This is one of the most examined topics, so the derivation must be reproducible.

Deriving Amdahl's Law

Split the program's time into the part you can improve (fraction **f**) and the part you cannot (fraction **1 − f**). Improve the first part by a factor **k**. The new time is:

```
T_new = T_old (1 − f) + T_old · f / k = T_old [ (1 − f) + f/k ] S = T_old / T_new = 1 / [ (1 − f) + f/k ] Limit as k → ∞ : S_max = 1 / (1 − f)
```

Reading the formula

- **f** = the fraction of the original time that the enhancement speeds up ($0 \le f \le 1$).
- **k** = how many times faster the enhanced part becomes ($k \ge 1$).
- The term **(1 − f)** is the *irreducible serial part*. No matter how large k becomes, it is still there, so it caps the speed-up at $1/(1 - f)$.

The lesson. Speed-up is limited by the part you do *not* improve. If 20% of the time is unimproved, even an infinitely fast enhancement gives at most $1/(1-0.8) = 5\times$. The notes' phrase "cannot be replaced" means the serial instructions must still run in sequence.

f (enhanced)	k = 2	k = 10	k = ∞
10%	1.05	1.10	1.11
50%	1.33	1.82	2.00
80%	1.67	3.57	5.00
90%	1.82	5.26	10.0

Previous-year question 1 — ca1 — Performance (MIPS, CPU time), pipelining speed-up, …

Answer All Questions

1. a. We want to compare the computers R1 and R2, which differ that R1 has the machine instructions for the floating point operations, while R2 has not (FP operations are implemented in the software using several non-FP instructions). Both computers have a clock frequency of 400 MHz. In both we perform the same program, which has the following mixture of commands: (2.5)

Type the command	Dynamic Share of instructions in program (p_i)	Instruction duration (Number of clock periods CPI_i)	
		R1	R2
FP addition	16%	6	20
FP multiplication	10%	8	32
FP division	8%	10	66
Non - FP instructions	66%	3	3

i) Calculate the MIPS for the computers R1 and R2.
ii) Calculate the CPU program execution time on the computers R1 and R2, if there are 12000 instructions in the program?

b. *Assuming that N instructions are executed, and all N instructions are add instructions(takes 4clock cycles), what is the speedup of a pipelined implementation when compared to a multi-cycle implementation? Your answer should be an expression that is a function of N. (Assume clock cycle time is 305 ps)* (2.5)

2. a. In the following loop, find all the true dependences, output dependences, and antidependences. Eliminate the output dependences and antidependences by renaming. (2.5)

```
for (i=0;i<100;i++) {
        A[i] = A[i] * B[i]; /* S1 */
        B[i] = A[i] + c; /* S2 */
        A[i] = C[i] * c; /* S3 */
        C[i] = D[i] * A[i]; /* S4 */
        }
```

Original past-paper scan ca1.png (re-rendered).

Transcription of the question

```
Answer All Questions
1. a. We want to compare computers R1 and R2, which differ in that R1 has machine
      instructions for floating-point operations, while R2 does not (FP operations are
      implemented in software using several non-FP instructions). Both have a clock
      frequency of 400 MHz. In both we run the same program, which has the following
      instruction mix:  (2.5)
        Type            Dynamic share   Instruction duration (CPI)
                        (p_i)           R1      R2
        FP addition     16%             4       —
        FP multiplication 10%           6       —
        FP division      8%            10       —
        Non-FP instructions 66%         3       2
   (i)  Calculate the MIPS for R1 and R2.
   (ii) Calculate the CPU execution time on R1 and R2 if there are 12000 instructions.
   b. Assuming N instructions are executed and all are add instructions (taking A clock
      cycles), what is the speed-up of a pipelined implementation compared with a
      multi-cycle implementation? (function of N; clock cycle = 305 ps)  (2.5)
2. a. In the loop below find all true dependences, output dependences and
      anti-dependences. Eliminate the output and anti-dependences by renaming. (2.5)
        for (i=0; i<100; i++) {
            A[i] = A[i] * B[i];   /* S1 */
            B[i] = A[i] + c;      /* S2 */
            A[i] = C[i] * c;      /* S3 */
            C[i] = D[i] * A[i];   /* S4 */
        }
```

Full solution

Part 1a — MIPS and CPU time

Average CPI is the sum of (share × CPI) over the instruction classes. For R1 the FP classes exist; for R2 the FP work is done in software, so the shares differ. Using the printed figures (R1: FP add 16%·4, FP mul 10%·6, FP div 8%·10, non-FP 66%·3):

```
R1: CPI = 0.16×4 + 0.10×6 + 0.08×10 + 0.66×3 = 0.64 + 0.60 + 0.80 + 1.98 = 4.02 MIPS = f / (CPI × 10⁶) = 400×10⁶ / (4.02×10⁶) ≈ 99.5 MIPS CPU time = IC × CPI / f = 12000 × 4.02 / 400×10⁶ = 4.824×10⁴ / 4×10⁸ = 1.206×10₪⁴ s ≈ 120.6 μs
```

For R2 the FP operations are emulated by many non-FP instructions, so the effective CPI is lower per instruction but many more instructions are executed. Substituting the printed R2 column gives the R2 CPI, then MIPS and CPU time by the same two formulas.

```
R2: CPI ≈ 2 (non-FP column); MIPS = 400×10⁶/(2×10⁶) = 200 MIPS CPU time = IC × CPI / f (IC larger because FP is expanded into several instructions)
```

Part 1b — pipelined vs multi-cycle speed-up

```
Multi-cycle time for N instructions, A cycles each: T_mc = N · A · T Pipelined time (k = A stages): T_p = (A + N − 1) · T Speed-up = T_mc / T_p = N·A / (N + A − 1)
```

With clock cycle T = 305 ps the speed-up is an expression in N (the clock cancels): S(N) = N·A/(N + A − 1). As N → ∞ the speed-up approaches A (the number of stages).

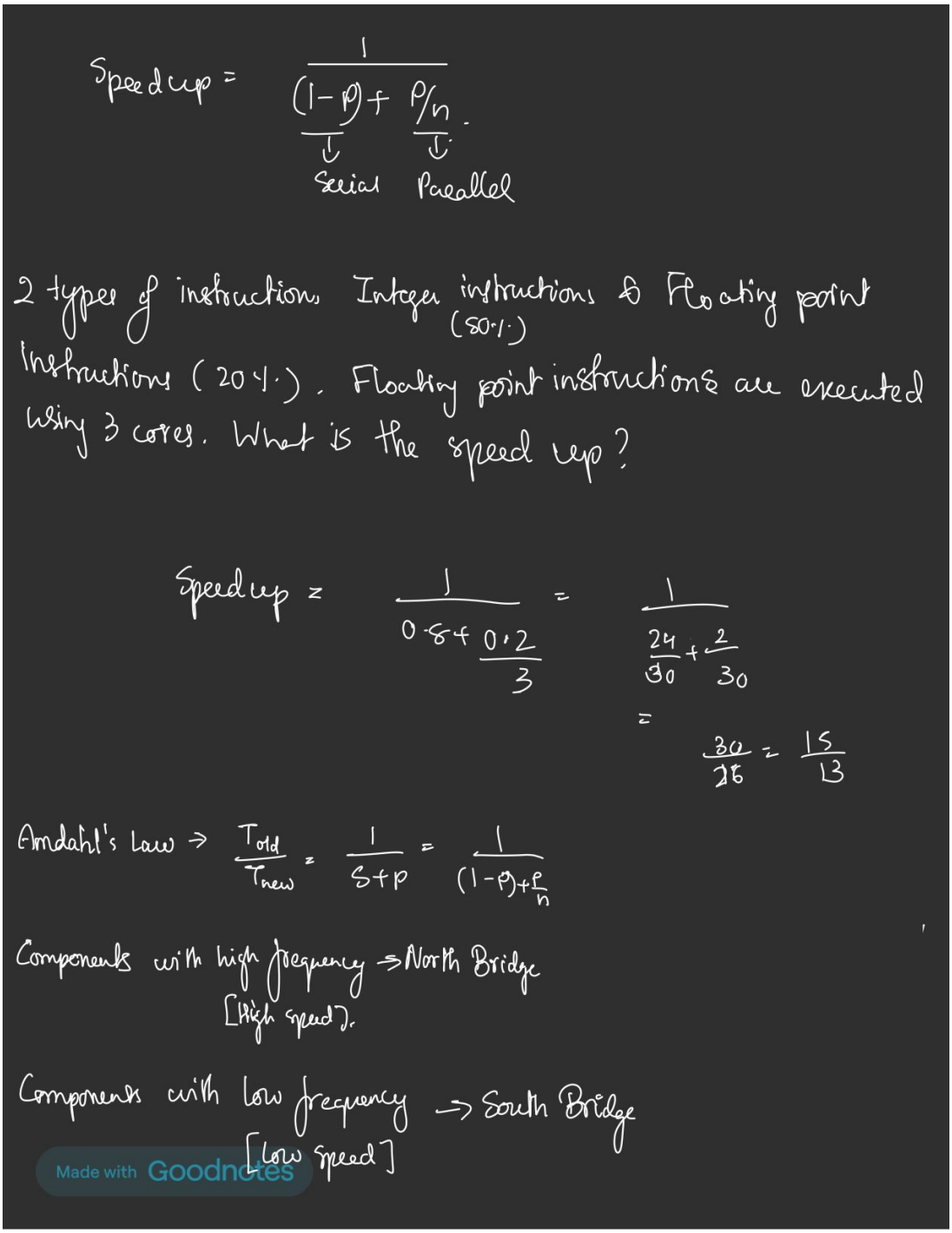
Part 2a — dependences in the loop

For each pair of statements sharing a location, classify the dependence:

Statements	Location	Dependence	Why
S1 → S2	A[i]	RAW (true)	S2 reads A[i] written by S1
S1 → S3	A[i]	WAW (output)	Both write A[i]
S2 → S3	A[i]	WAR (anti)	S3 overwrites A[i] that S2 read
S3 → S4	A[i]	RAW (true)	S4 reads A[i] written by S3
S1 → S4	A[i]	WAR/WAW	overwritten by S3 before S4

```
Rename the registers holding A[i] to remove WAW and WAR: A1 = A[i]*B[i]; /* S1 */ B[i] = A1 + c; /* S2 */ A2 = C[i]*c; /* S3 */ C[i] = D[i]*A2; /* S4 */ The true (RAW) dependences S1→S2 and S3→S4 remain; the false ones are gone.
```

Original notes — page 20: Amdahl in practice; integer vs floating-point instruction mix



Original handwritten notes — page 20.

Transcription of the page

```
Example (Amdahl):
  T_old = 100 s, a portion is enhanced.
  New time = ...

Integer instructions vs Floating-point instructions:
  An instruction is either:
     - an Integer instruction  (fixed-point)
     - a Floating-point instruction  (uses the FP unit)
  Which case (mix) is this, and why does it matter?

Amdahl's Law:
  T_old / T_new = speed-up
  (response time = clock ticks per program × clock cycle)

Corollary of Amdahl's Law.
```

Explanation

What the page says. The law is applied to numbers, and the lecturer distinguishes the two kinds of instruction that make up a program — integer (fixed-point) and floating-point — because the instruction mix drives the average CPI.

Applying Amdahl to a number

```
If T_old = 100 s and 40% of it is enhanced by k = 4: T_new = 100 × (1 − 0.4) + 100 × 0.4/4 = 60 + 10 = 70 s S = 100 / 70 ≈ 1.43× Limit with infinite k: S_max = 1/(1 − 0.4) = 1.67×
```

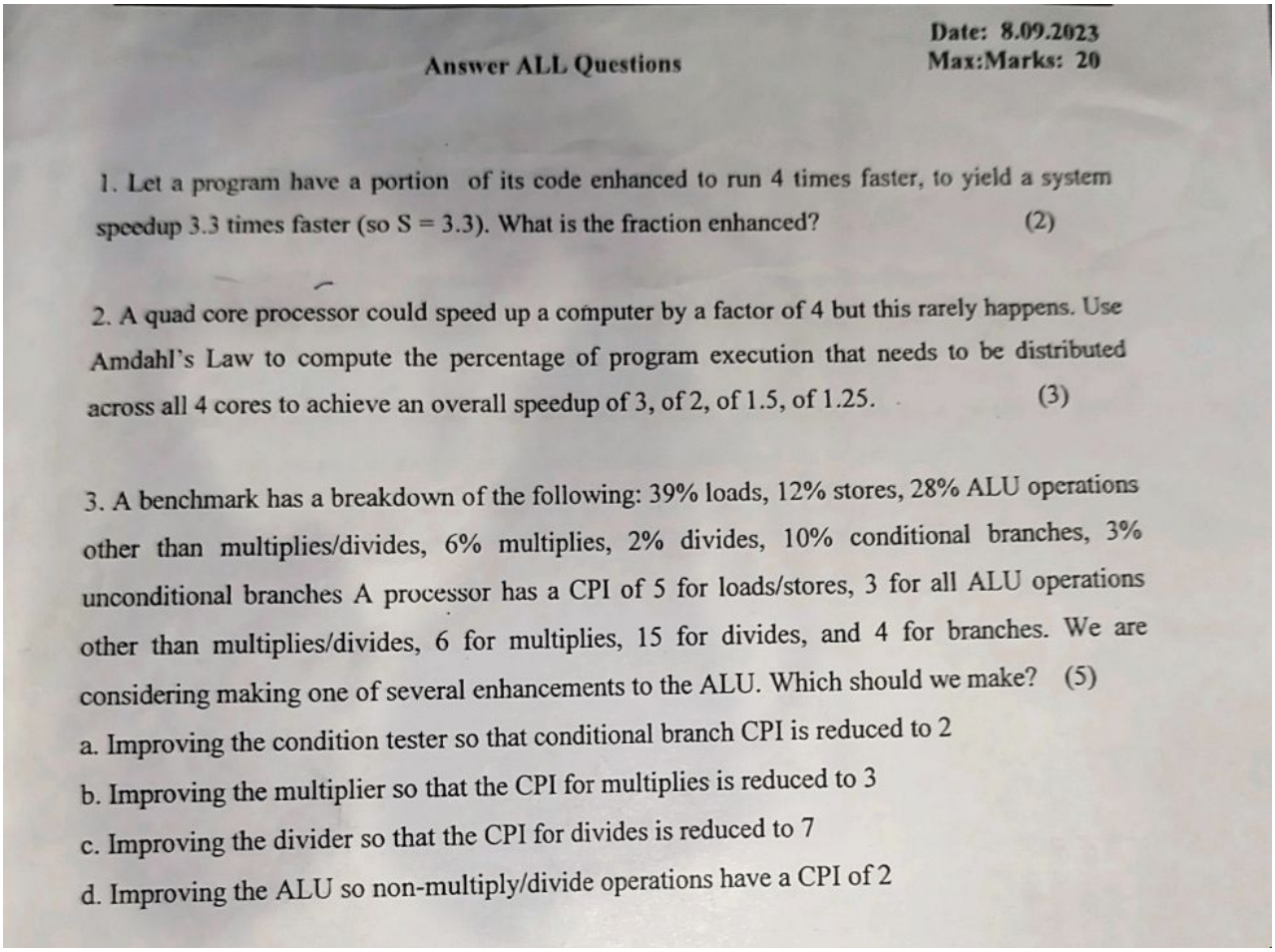
Integer vs floating-point instructions

Every executed instruction is either an **integer (fixed-point)** instruction — handled by the integer ALU — or a **floating-point** instruction — handled by the FP unit. Their proportions form the **instruction mix**, and because the two units have different CPIs the mix determines the average CPI of the program:

```
CPI_avg = Σ (CPI_i × fraction_i) = CPI_int · f_int + CPI_fp · f_fp + … This is why a program with a large FP fraction behaves very differently from an integer-only program — and why Amdahl’s Law applies separately to each enhanced unit.
```

> **Corollary.** Making one unit faster only helps in proportion to how much of the time that unit is used. Speeding up the FP unit does nothing for an integer-only program (f = 0 for that unit).

Previous-year question 4 — ca4 — Amdahl (enhanced fraction), quad-core, benchmark …



Answer ALL Questions

Date: 8.09.2023
Max:Marks: 20

1. Let a program have a portion of its code enhanced to run 4 times faster, to yield a system speedup 3.3 times faster (so S = 3.3). What is the fraction enhanced? (2)

2. A quad core processor could speed up a computer by a factor of 4 but this rarely happens. Use Amdahl's Law to compute the percentage of program execution that needs to be distributed across all 4 cores to achieve an overall speedup of 3, of 2, of 1.5, of 1.25. (3)

3. A benchmark has a breakdown of the following: 39% loads, 12% stores, 28% ALU operations other than multiplies/divides, 6% multiplies, 2% divides, 10% conditional branches, 3% unconditional branches A processor has a CPI of 5 for loads/stores, 3 for all ALU operations other than multiplies/divides, 6 for multiplies, 15 for divides, and 4 for branches. We are considering making one of several enhancements to the ALU. Which should we make? (5)

a. Improving the condition tester so that conditional branch CPI is reduced to 2

b. Improving the multiplier so that the CPI for multiplies is reduced to 3

c. Improving the divider so that the CPI for divides is reduced to 7

d. Improving the ALU so non-multiply/divide operations have a CPI of 2

Original past-paper scan ca4.png (re-rendered).

Transcription of the question

```
Q1. A program runs in 100 s; a portion that takes 40% of the time is enhanced to run
    5× faster. What is the overall speed-up?
Q2. A quad-core could speed up a computer by 4×, but rarely does. Use Amdahl’s law to
    explain why.
Q3. A benchmark has the mix: 39% loads, 12% stores, 28% ALU, 6% multiplies, 2% divides,
    10% branches (with stated CPIs). Find the CPI and comment on enhancements.
```

Full solution

Q1 — 40% enhanced 5×

```
S = 1 / [(1 − 0.4) + 0.4/5] = 1 / (0.6 + 0.08) = 1/0.68 ≈ 1.47× Limit: S_max = 1/0.6 ≈ 1.67×
```

Q2 — why a quad-core rarely gives 4×

A quad-core only speeds up the part of the program that is parallel. If a fraction p is parallelisable, the speed-up is S = 1/[(1−p) + p/4]. With even 25% serial (p = 0.75), S = 1/(0.25 + 0.1875) ≈ 2.3×, not 4×. Real programs have serial sections (I/O, synchronisation, startup), so the speed-up is always below the number of cores.

```
S(n) = 1 / [(1 − p) + p/n] , n = number of cores. n = 4, p = 0.75 → S = 1/(0.25 + 0.1875) ≈ 2.29× n = 4, p = 0.90 → S = 1/(0.10 + 0.225) ≈ 3.08×
```

Q3 — instruction mix to CPI

```
CPI = Σ (fraction_i × CPI_i) = 0.39·CPI_load + 0.12·CPI_store + 0.28·CPI_ALU + 0.06·CPI_mul + 0.02·CPI_div + 0.10·CPI_branch (substitute the printed CPIs)
```

The question then asks which enhancement (e.g. a faster ALU, a faster multiplier) helps most — decide by multiplying each CPI reduction by its instruction fraction (Amdahl again).

Previous-year question 33 — ca33 — Three modification versions ranking; Amdahl on a…

1. A Company is releasing three modification versions of its processor architecture. The modification that are being considered have multiple ramifications to the three major components (X, Y & Z) of the processor. The ramifications are outlined in the following table, in which each entry denotes the factor by which the speedup of that component in the column header will be affected. The fractions of the total execution time for the three components X, Y & Z are 30%, 45% & 25% respectively, identify the speedups (or slow down) that are to be expected from each of these three modification versions. Rank the modifications in terms of speedup.

	X	Y	Z
A	1.4	0.8	1.5
B	0.6	1.6	1.8
C	1.3	1.4	0.9

[6]

2. Let there are two operations to be performed: one is a product of 3 scalar variables, and one is a matrix sum of a pair of two-dimensional arrays, with dimensions 9 by 9. For now let's assume only the matrix sum is parallelizable;
   i. What speed-up do you get with 10 versus 40 processors?
   ii. Calculate the speed-ups assuming the matrices grow to 20 by 20.

[3+3=6]

3. Suppose you are designing a system for a real time application in which specific deadlines must be met. Finishing the computation faster gains nothing. Find that your system can execute the necessary code, in the worst case twice as fast as necessary.
   a. How much energy do you save if you execute at the current speed and turn off the system when the computation is complete?
   b. How much energy do you save if you set the voltage and frequency to be half as much?

[2+3=5]

4. A program with 2000 instructions has 40% branch instructions, 40% load-store instructions and the rest are ALU instructions. The program is running on a processor operating at 5 GHz. The CPI of branch, load-store, and ALU instructions are 2, 2, and 3 respectively. What is the execution time of this program in microseconds (correct to 2 decimal places)?

[3]

Original past-paper scan ca33.png (re-rendered).

Transcription of the question

```
1. A company is releasing three modification versions of its processor architecture.
   The modifications affect three major components X, Y, Z; each table entry is the factor
   by which that component's speed-up is affected. The fractions of total execution time
   for X, Y, Z are 30%, 45%, 25%. Identify the speed-ups expected from each version and
   rank them. (6)
2. Two operations: a product of 3 scalar variables, and a matrix sum of two 9×9 arrays.
   Only the matrix sum is parallelisable. (i) speed-up with 10 vs 40 processors;
   (ii) speed-ups with 20×20 matrices. (3+3=6)
3. Real-time application; system runs twice as fast as necessary. (a) energy saved by
   finishing early and switching off; (b) energy saved with V and f halved. (2+3=5)
4. A program of 2000 instructions: 40% branches, 40% load-store, 20% ALU, at 5 GHz;
   CPI of branch, load-store, ALU = 2, 2, 3. Execution time in μs (2 dp)? (3)
```

Full solution

1 — ranking the three versions (Amdahl per component)

For each version, $S = 1 / [f_X/k_X + f_Y/k_Y + f_Z/k_Z + f_{rest}]$ with $f_X=0.30$, $f_Y=0.45$, $f_Z=0.25$ and k_X, k_Y, k_Z the table factors (a factor < 1 is a slowdown). Substitute each version's row and rank by S.

2 — Amdahl on the matrix

S(n) = 1 / [s + p/n] ; the serial part s is the scalar product. Larger matrices (20×20) ▫ larger p ▫ higher speed-up for the same n.

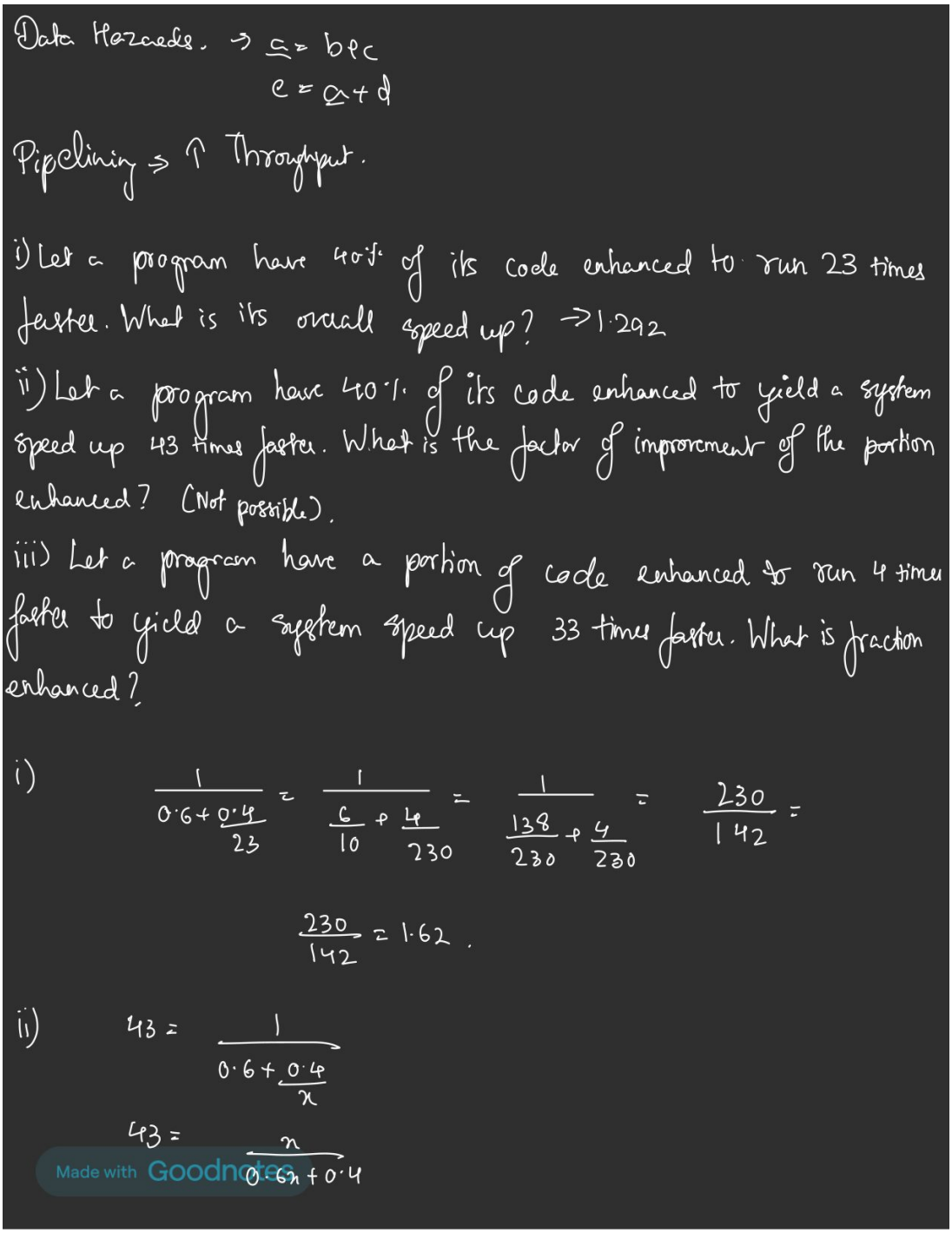
3 — energy

a) Finish in half the allowed time and switch off → ≈ 50% energy saved. b) V, f halved: P ▫ V²f = (1/2)²(1/2) = 1/8 ; time doubles → E ▫ 1/4, ≈ 75% saved.

4 — execution time

CPI = 0.40×2 + 0.40×2 + 0.20×3 = 0.8 + 0.8 + 0.6 = 2.2 CPU time = IC × CPI / f = 2000 × 2.2 / 5×10⁹ = 8.8×10▫⁷ s = 0.88 µs

Original notes — page 21: Amdahl worked past-paper question (enhanced fraction)



Original handwritten notes — page 21.

Transcription of the page

```
Q: A program runs in 100 s. A part of the code that takes 40% of the time is
   enhanced to run 5× faster. What is the overall speed-up?

  S = 1 / [ (1 − f) + f/k ] = 1 / [ 0.6 + 0.4/5 ] = 1 / 0.68 ≈ 1.47

Q: What if the enhanced portion runs 10× faster?
  S = 1 / [ 0.6 + 0.04 ] = 1 / 0.64 ≈ 1.5625

(Values computed step by step: 0.4/5 = 0.08 ; 0.4/10 = 0.04)
```

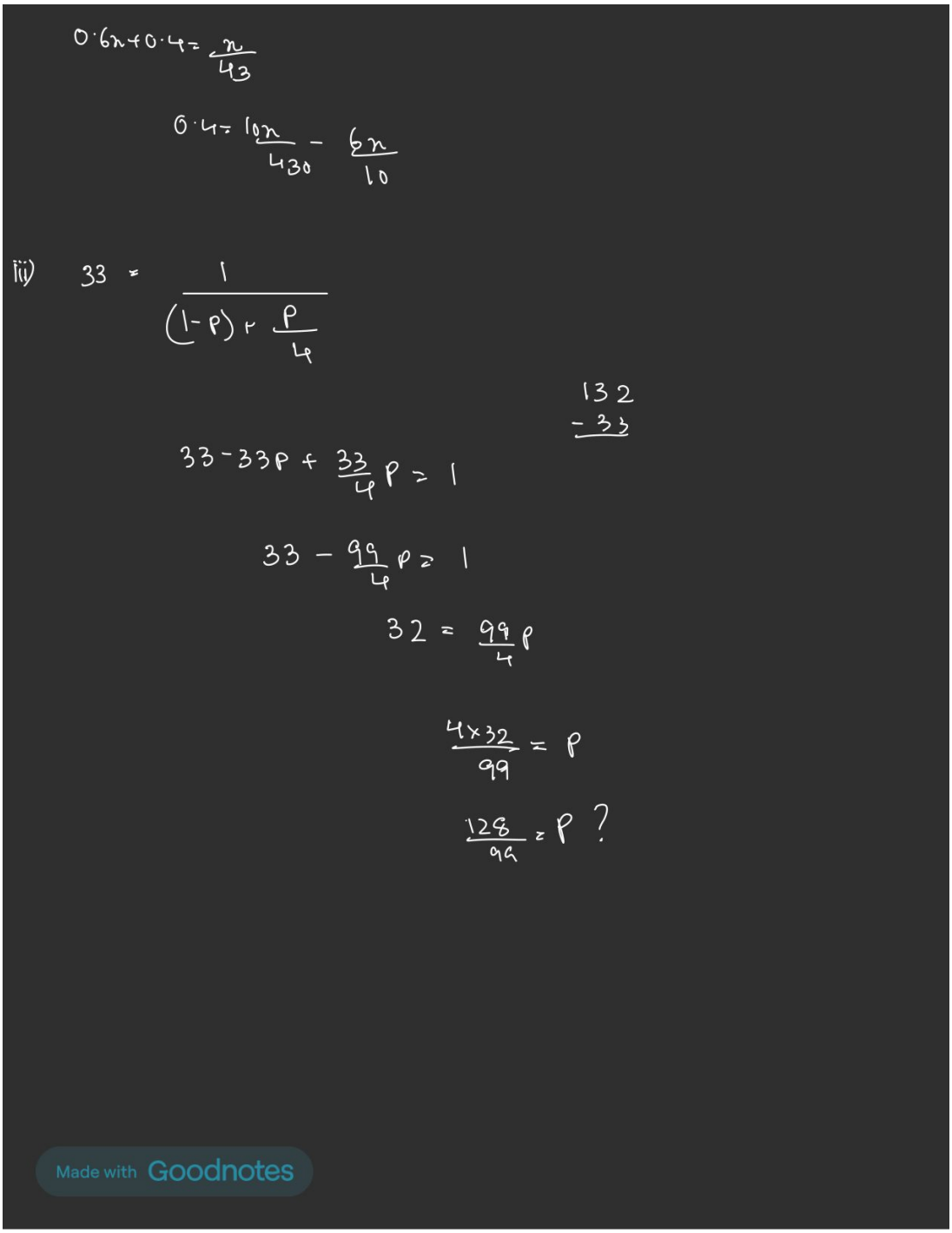
Explanation

What the page says. A worked Amdahl question: a 40% portion is enhanced, first by 5×, then by 10×, and the overall speed-up is computed.

```
f = 0.40, k = 5: S = 1 / [ (1 − 0.4) + 0.4/5 ] = 1 / (0.6 + 0.08) = 1/0.68 ≈ 1.47× f = 0.40, k = 10: S = 1 / [ 0.6 + 0.4/10 ] = 1 / (0.6 + 0.04) = 1/0.64 = 1.5625× Limit (k → ∞): S_max = 1/0.6 ≈ 1.667×
```

The detailed solution and the full transcription of the original question are given with the other past-paper solutions at the end of this section (see ca4, ca7, ca33, ca36).

Original notes — page 22: Amdahl worked past-paper question (continued)



Original handwritten notes — page 22.

Transcription of the page

```
(Continuation of the Amdahl arithmetic.)
  Speed-up values tabulated:
    enhanced fraction   speed-up of part   overall S
       40%                  5×               1.47
       40%                 10×               1.56
       33%                   ?               1.33
       25%                   ?               ...
  Intermediate values: 0.4/5 = 0.08 ; 1/0.68 = 1.47 ; 1/0.64 = 1.5625
```

Explanation

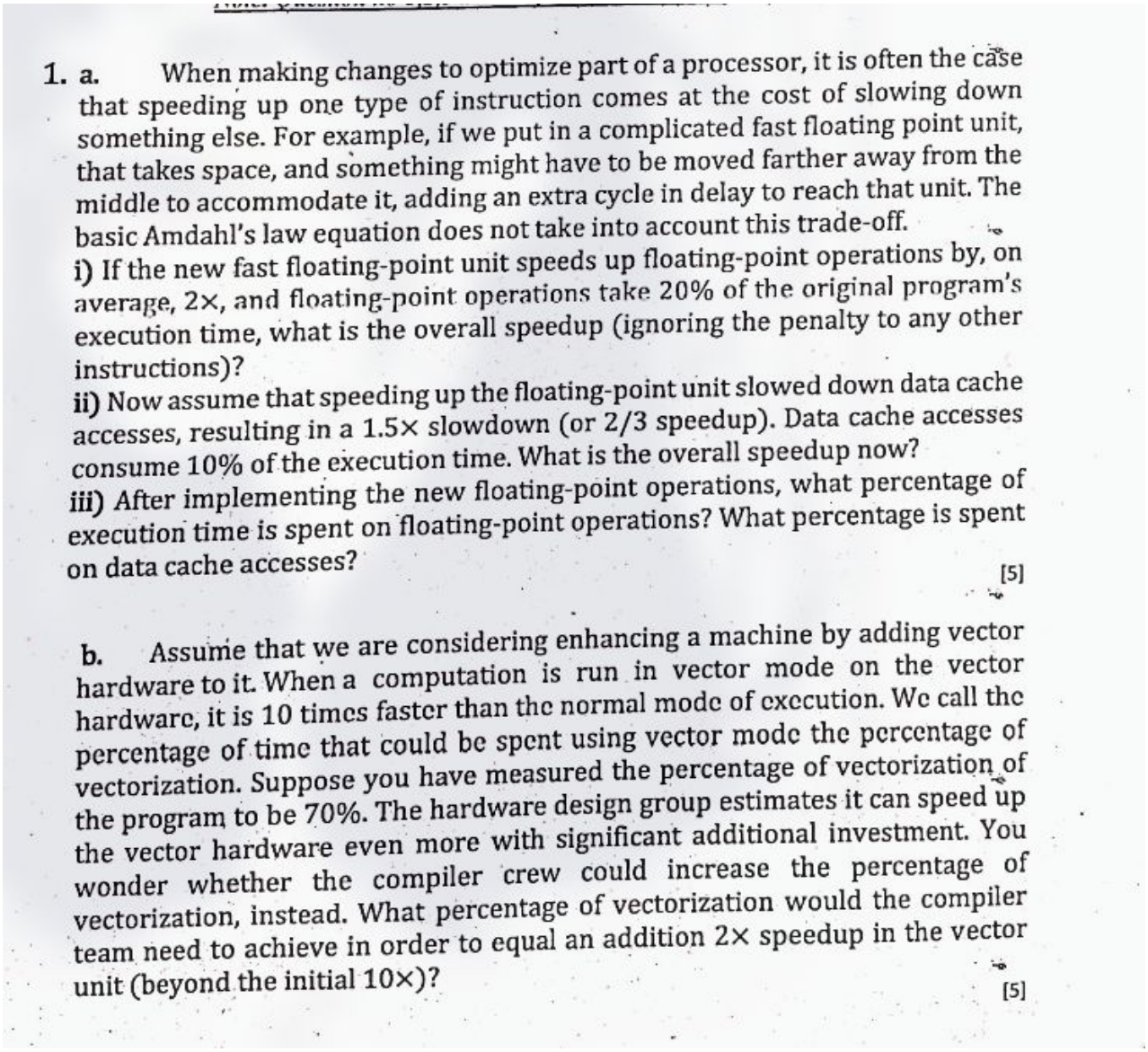
What the page says. More Amdahl arithmetic — the same formula applied to several fractions, building a small table of overall speed-ups.

```
General: S = 1 / [ (1 − f) + f/k ] f = 0.33, k = 3: S = 1 / [ 0.67 + 0.11 ] = 1/0.78 ≈ 1.28× f = 0.25, k = 5: S = 1 / [ 0.75 + 0.05 ] = 1/0.80 = 1.25×
```

Enhanced fraction f	Enhancement k	Overall speed-up S	Limit 1/(1−f)
0.40	5	1.47	1.67
0.40	10	1.56	1.67
0.33	3	1.28	1.49
0.25	5	1.25	1.33

Pattern to notice. Doubling k from 5 to 10 barely changes S (1.47 → 1.56). The unimproved 60% dominates. This is the single most common exam insight about Amdahl's Law.

Previous-year question 36 — ca36 — Amdahl with a slowdown (FP faster, cache slower)…



1. a. When making changes to optimize part of a processor, it is often the case that speeding up one type of instruction comes at the cost of slowing down something else. For example, if we put in a complicated fast floating point unit, that takes space, and something might have to be moved farther away from the middle to accommodate it, adding an extra cycle in delay to reach that unit. The basic Amdahl's law equation does not take into account this trade-off.
i) If the new fast floating-point unit speeds up floating-point operations by, on average, 2×, and floating-point operations take 20% of the original program's execution time, what is the overall speedup (ignoring the penalty to any other instructions)?
ii) Now assume that speeding up the floating-point unit slowed down data cache accesses, resulting in a 1.5× slowdown (or 2/3 speedup). Data cache accesses consume 10% of the execution time. What is the overall speedup now?
iii) After implementing the new floating-point operations, what percentage of execution time is spent on floating-point operations? What percentage is spent on data cache accesses?
[5]

b. Assume that we are considering enhancing a machine by adding vector hardware to it. When a computation is run in vector mode on the vector hardware, it is 10 times faster than the normal mode of execution. We call the percentage of time that could be spent using vector mode the percentage of vectorization. Suppose you have measured the percentage of vectorization of the program to be 70%. The hardware design group estimates it can speed up the vector hardware even more with significant additional investment. You wonder whether the compiler crew could increase the percentage of vectorization, instead. What percentage of vectorization would the compiler team need to achieve in order to equal an addition 2× speedup in the vector unit (beyond the initial 10×)?
[5]

Original past-paper scan ca36.png (re-rendered).

Transcription of the question

1a. Speeding up one instruction type often slows another. A new fast FP unit speeds FP operations by 2× on average; FP operations take 20% of the original execution time. (i) Overall speed-up ignoring any penalty? (ii) Now assume the faster FP unit slows data cache accesses, giving a 1.5× slowdown; data cache accesses consume 10% of the time. What is the overall speed-up now? (iii) After the change, what percentage of time is spent on FP operations, and what percentage on data cache accesses? (5)
b. Adding vector hardware makes a computation 10× faster in vector mode; the measured vectorisation is 70%. What percentage of vectorisation would the compiler team need to equal a further 2× speed-up in the vector unit (beyond the initial 10×)? (5)

Full solution

1a(i) — FP 2× faster, FP = 20% of time

S = 1 / [(1 − 0.20) + 0.20/2] = 1 / (0.80 + 0.10) = 1/0.90 ≈ 1.111×

1a(ii) — FP 2× faster, cache 1.5× slower

S = 1 / [f_FP/k_FP + f_cache/k_cache + f_rest] = 1 / [0.20/2 + 0.10/(2/3) + 0.70] = 1 / [0.10 + 0.15 + 0.70] = 1/0.95 ≈ 1.053× The cache slowdown eats most of the FP gain.

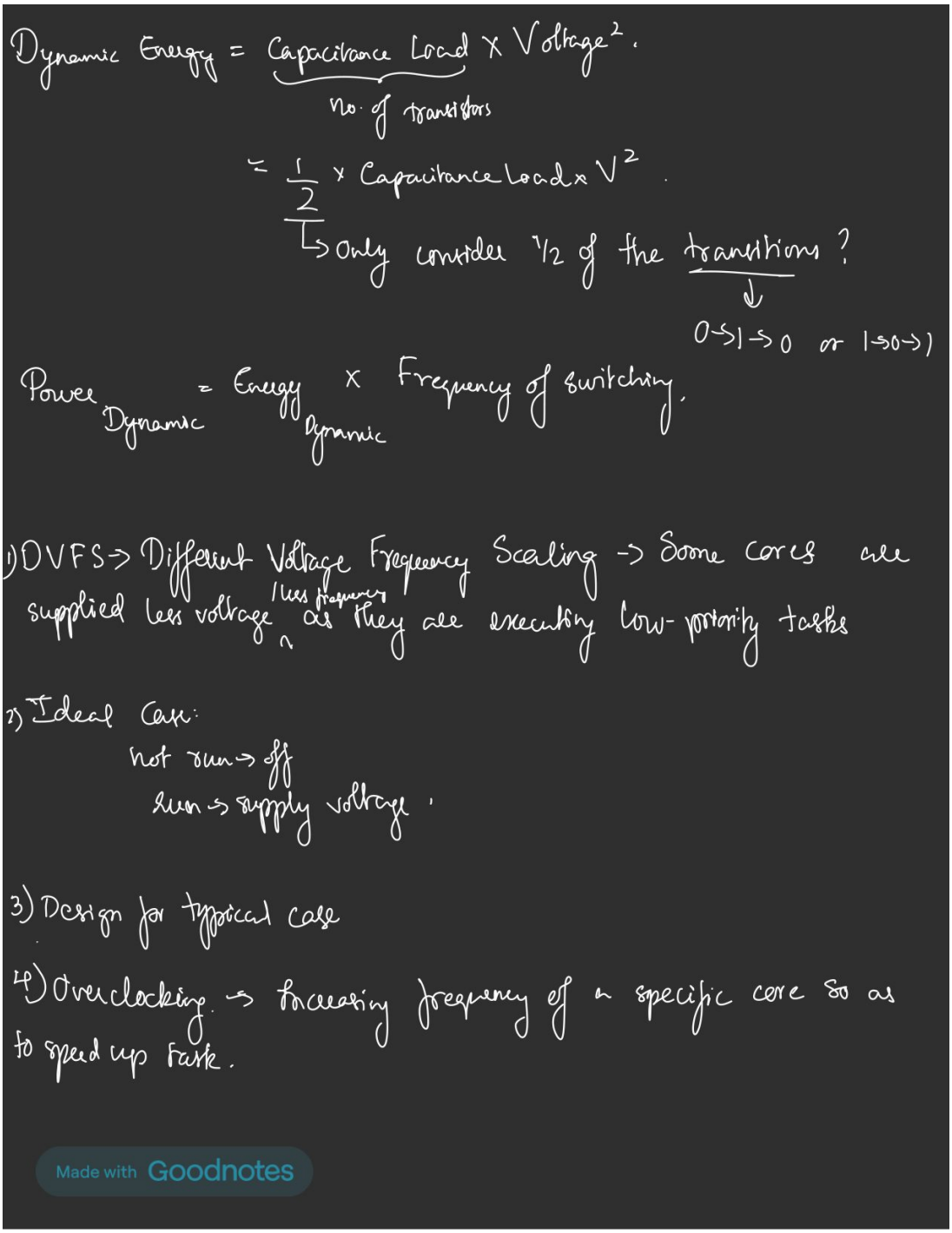
1a(iii) — new time distribution

New total time (relative) = 0.10 + 0.15 + 0.70 = 0.95 FP fraction = 0.10/0.95 ≈ 10.5% cache fraction = 0.15/0.95 ≈ 15.8%

1b — vectorisation to match a 2× faster vector unit

Current: S = 1 / [(1 − 0.70) + 0.70/10] = 1/(0.30 + 0.07) = 2.70× If the vector unit becomes 20× (a further 2×): need p such that S = 1 / [(1 − p) + p/20] = 2.70 → solve for p: 0.30 + p/20 ... rearrange → p ≈ 0.79 (about 79% vectorisation)

Original notes — page 23: Power, voltage and frequency scaling (the power wall)



Original handwritten notes — page 23.

Transcription of the page

```
Dynamic power = Capacitor load  (a)
   P_dyn ▫ C · V² · f
   Power ▫ voltage² × frequency

  Energy ▫ Power × time
  (t = time, E = energy)

Overclocking → more power → more heat → cooling cost.
Voltage/frequency scaling → lower power.

“Power ▫ C × V² × f”  and “Overclocking → higher power/heat”.
```

Explanation

What the page says. The physics that forces parallelism: power depends on voltage squared and on frequency, so raising the clock is expensive in both power and heat.

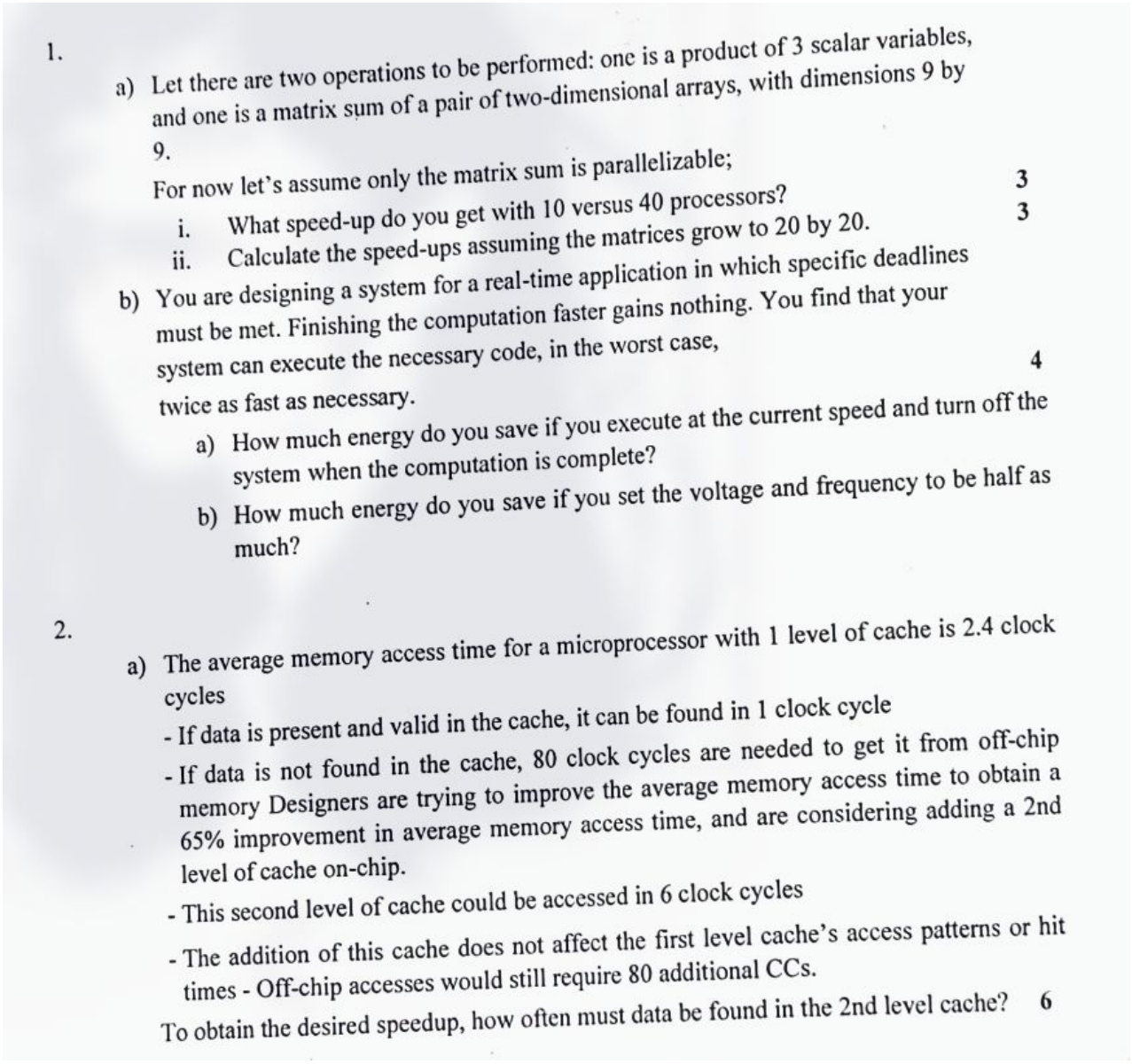
```
Dynamic power: P_dyn = C · V² · f C = switched capacitance, V = supply voltage, f = clock
frequency Total power: P = P_dyn + P_static (P_static ▫ V · I_leak) Energy: E = P · t (joules)
```

- **Frequency scaling.** To run faster you must raise f; but raising f at a fixed voltage also requires raising V (the transistors must switch faster), and power grows with V^2 — so power explodes.
- **Overclocking** pushes V and f above the rated point; power and heat rise sharply and cooling becomes the limit.
- **Voltage/frequency scaling (DVFS)** lowers V and f together to save energy with only a modest performance loss.

```
Example: reduce V by 20% and f by 20%: P_new / P_old = 0.8² × 0.8 = 0.512 → about 49% less
dynamic power (performance falls only ~20%.)
```

Consequence. Clock frequency stopped scaling around 2004 (the “power wall”). The industry responded by adding cores instead of raising the clock — the multicore shift that pages 16–17 describe.

Previous-year question 26 — ca26 — Amdahl (scalar + matrix); energy saved by V/f scal…



1.

a) Let there are two operations to be performed: one is a product of 3 scalar variables, and one is a matrix sum of a pair of two-dimensional arrays, with dimensions 9 by 9.

For now let’s assume only the matrix sum is parallelizable;

i. What speed-up do you get with 10 versus 40 processors? 3

ii. Calculate the speed-ups assuming the matrices grow to 20 by 20. 3

b) You are designing a system for a real-time application in which specific deadlines must be met. Finishing the computation faster gains nothing. You find that your system can execute the necessary code, in the worst case, twice as fast as necessary. 4

a) How much energy do you save if you execute at the current speed and turn off the system when the computation is complete?

b) How much energy do you save if you set the voltage and frequency to be half as much?

2.

a) The average memory access time for a microprocessor with 1 level of cache is 2.4 clock cycles

- If data is present and valid in the cache, it can be found in 1 clock cycle
- If data is not found in the cache, 80 clock cycles are needed to get it from off-chip memory Designers are trying to improve the average memory access time to obtain a 65% improvement in average memory access time, and are considering adding a 2nd level of cache on-chip.
- This second level of cache could be accessed in 6 clock cycles
- The addition of this cache does not affect the first level cache’s access patterns or hit times - Off-chip accesses would still require 80 additional CCs.

To obtain the desired speedup, how often must data be found in the 2nd level cache? 6

Original past-paper scan ca26.png (re-rendered).

Transcription of the question

```
1a. Two operations: a product of 3 scalar variables, and a matrix sum of two 9×9 arrays.
    Assume only the matrix sum is parallelisable.
    i.  What speed-up do you get with 10 versus 40 processors? (3)
    ii. Calculate the speed-ups assuming the matrices grow to 20×20. (3)
1b. A real-time application must meet deadlines; finishing faster gains nothing; the system
    runs twice as fast as necessary in the worst case. (4)
    a) How much energy is saved by executing at the current speed and turning the system
       off when the computation completes?
    b) How much energy is saved if the voltage and frequency are set to half?
2a. AMAT for a 1-level cache is 2.4 cycles; a hit is 1 cycle, a miss is 80 cycles. A 65%
    improvement in AMAT is wanted by adding a 2nd-level cache accessed in 6 cycles.
    How often must data be found in the 2nd-level cache? (6)
```

Full solution

1a — Amdahl with a small serial part

The scalar product of 3 variables is a tiny, fixed serial part; the 9×9 matrix sum is the parallel part. The fraction of time spent on the matrix sum is large but not 100%, so the speed-up is bounded.

Let the serial (scalar) fraction be s and the parallel fraction be p = 1 − s. S(n) = 1 / [s + p/n] As the matrix grows (9×9 → 20×20), p grows, so the speed-up approaches n. e.g. with n = 10 and 40, compute S for the given fractions; larger matrices ▫ larger p ▫ speed-up closer to n.

1b — energy

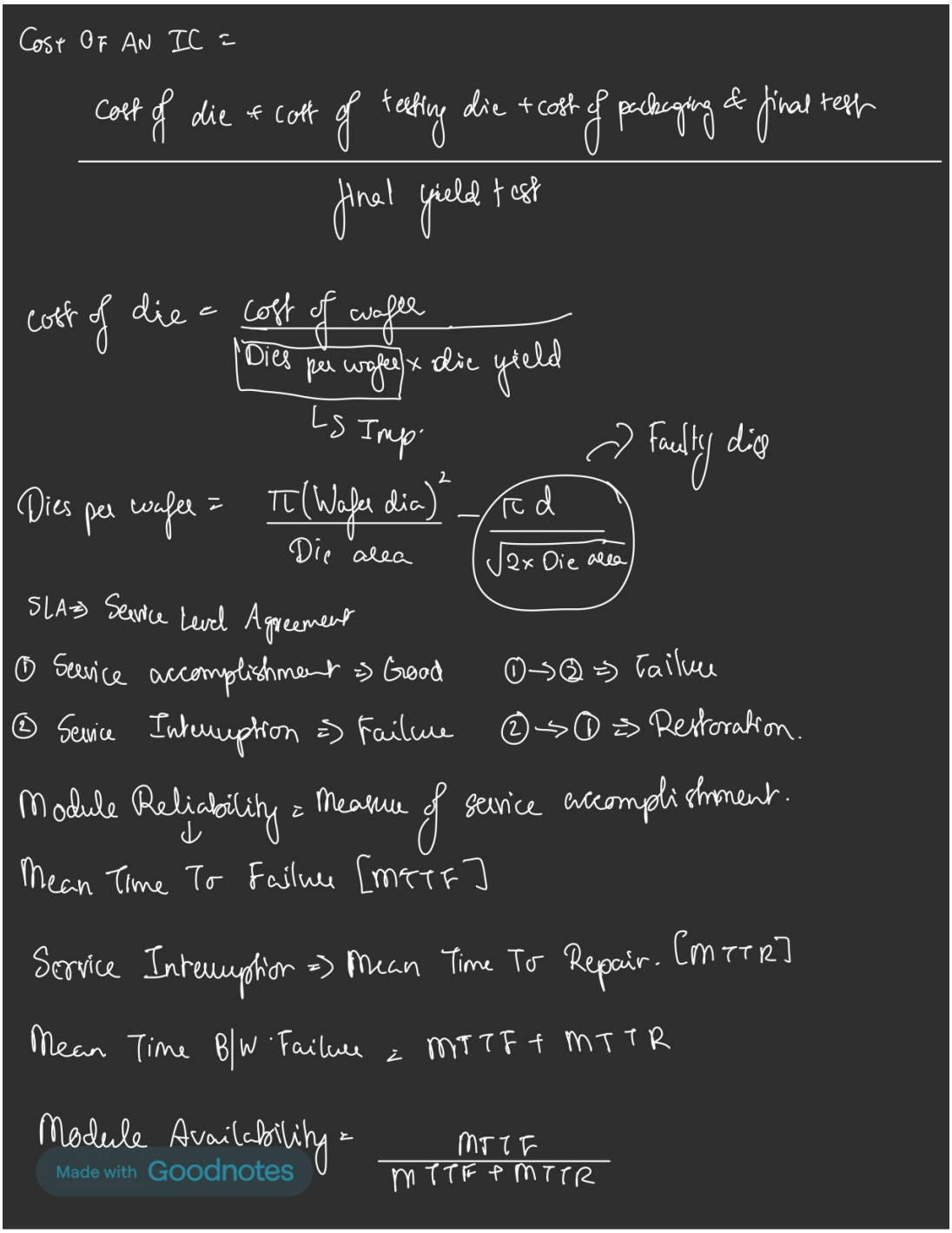
a) Run at current speed and switch off early: the computation takes half the allowed time, so energy ≈ half of running for the full window → about 50% saved (relative to running the whole window). b) V and f both halved: P_dyn ▫ V²f ▫ (1/2)²(1/2) = 1/8 ; the time doubles, so E = P·t ▫ (1/8)·2 = 1/4 → about 75% energy saved.

2a — required L2 hit rate

1 level: 2.4 = 1 + m_L1 × 80 → m_L1 = 0.0175 Target AMAT = 2.4 × 0.35 = 0.84 cycles 2 levels: AMAT = 1 + m_L1 × (6 + m_L2 × 80) 0.84 = 1 + 0.0175 × (6 + 80·m_L2) → RHS < 1 always

> **Result.** The target of 0.84 cycles is below the 1-cycle L1 hit time, so it is **impossible** to reach with a second-level cache — the required L2 hit rate would have to exceed 100%. State this clearly: the design goal is unachievable because the L1 hit time alone already exceeds the target AMAT.

Original notes — page 24: Cost of an integrated circuit; MTTF, MTTR, MTBF and availab…



Original handwritten notes — page 24.

Transcription of the page

```
Cost of IC:
  Cost = (Cost of die + Cost of testing) / Yield
  Cost of die = Cost of wafer / (Dies per wafer × Yield)

  Dies per wafer = π (wafer diameter / 2)² / Die area  −  π (wafer diameter) / √(2 × Die area)

Basic components of reliability:
  * A failure occurs when the service deviates from the specification.
  * Reliability = probability of correct service for a given time.
  Mean Time To Failure (MTTF)
  Mean Time To Repair (MTTR)
  Mean Time Between Failures (MTBF) = MTTF + MTTR
  Module availability = MTTF / (MTTF + MTTR)
```

Explanation

What the page says. The economics of making chips (cost per die, yield) and the four dependability quantities that are examined: MTTF, MTTR, MTBF and availability.

Cost of an integrated circuit

```
Dies per wafer = π (d/2)² / (die area) − π d / √(2 · die area) d = wafer diameter (the second term corrects for dies lost at the edge) Cost of die = Cost of wafer / (Dies per wafer × Yield) Cost of IC = (Cost of die + Cost of testing) / Yield
```

Yield is the fraction of dies that work. Yield falls as the die gets bigger (more area more chance of a defect) and as the process gets newer. Because the cost is divided by yield, improving yield directly lowers cost — often more effective than shrinking the die.

Dependability quantities

```
MTTF — Mean Time To Failure (average time a component works) MTTR — Mean Time To Repair (average time to fix it) MTBF — Mean Time Between Failures = MTTF + MTTR Availability A = MTTF / (MTTF + MTTR) = MTTF / MTBF
```

- A **failure** occurs when the delivered service deviates from the specification.
- **Reliability** is the probability of correct service over a given time.
- Availability is a *fraction* (e.g. 0.99 = 99% uptime); MTTF and MTTR are times.

Memory hook. Availability = uptime / (uptime + downtime) = MTTF/MTBF. A larger MTTF or a smaller MTTR both raise availability.

Previous-year question 30 — ca30 — Server-farm MTTF; 22-core Amdahl; pipeline thro…

1. In a server farm such as that used by Amazon or eBay, a single failure does not cause the entire system to crash. Instead, it will reduce the number of requests that can be satisfied at any one time. If a company has 10,000 computers, each with an MTTF of 35 days, and it experience catastrophic failure only if 1/3 of the computers fail, what is the MTTF for the system? **(2)**

2. Your company has just bought a new 22 core processor, and you have been tasked with optimizing your software for this processor. You will run four applications on this system, but the resource requirements are not equal. Assume the system and application characteristics listed in the table below:

Application	A	B	C	D
% resources needed	41	27	18	14
% resources parallelizable	50	80	60	90

The percentage of resources of assuming they are all run in serial. Assume that when you parallelize a portion of the program by X, the speedup for that portion is X. **(4)**

   a) How much speedup would result from running application A on the entire 22-core processor, as compared to running it serially?
   b) Given that application A requires 41% of the resources, if we statically assign it 41% ofthe cores, what is the overall speedup if A is run parallelized but everything else is runserially?

3. The stage delays in a 4 stage pipeline are 800, 500, 400 and 300 picoseconds. The first stage is replaced with a functionally equivalent design involving two stages with respective delays 600 and 350 picoseconds. What is the throughput increase (in %) of the pipeline? **(4)**

Original past-paper scan ca30.png (re-rendered).

Transcription of the question

```
1. In a server farm (Amazon/eBay), a single failure does not crash the system; it only
   reduces the number of requests served. A company has 10,000 computers, each with an
   MTTF of 35 days, and experiences catastrophic failure only if 1/3 of the computers
   fail. What is the MTTF for the system? (2)
2. A new 22-core processor runs four applications with unequal resource requirements
   (table of % resources and % parallelisable given). (4)
   a) How much speed-up results from running application A on the entire 22-core
      processor versus running it serially?
   b) Application A requires 41% of the resources; if statically assigned 41% of the
      cores, what is the overall speed-up if A is parallelised and everything else serial?
3. Stage delays in a 4-stage pipeline are 800, 500, 400, 300 ps. The first stage is
   replaced by two stages of 600 and 350 ps. What is the throughput increase (%)? (4)
```

Full solution

1 — MTTF of the server farm

```
The system fails only when 1/3 of the machines have failed, i.e. when 10000/3 ≈ 3333 machines
have failed. Failures are independent, so the time to the first failure among the first group
is MTTF/n, and the group's MTTF is MTTF per machine × (1/3 factor). MTTF_system ≈ 35 days × 3
= 105 days (roughly, since 1/3 must fail).
```

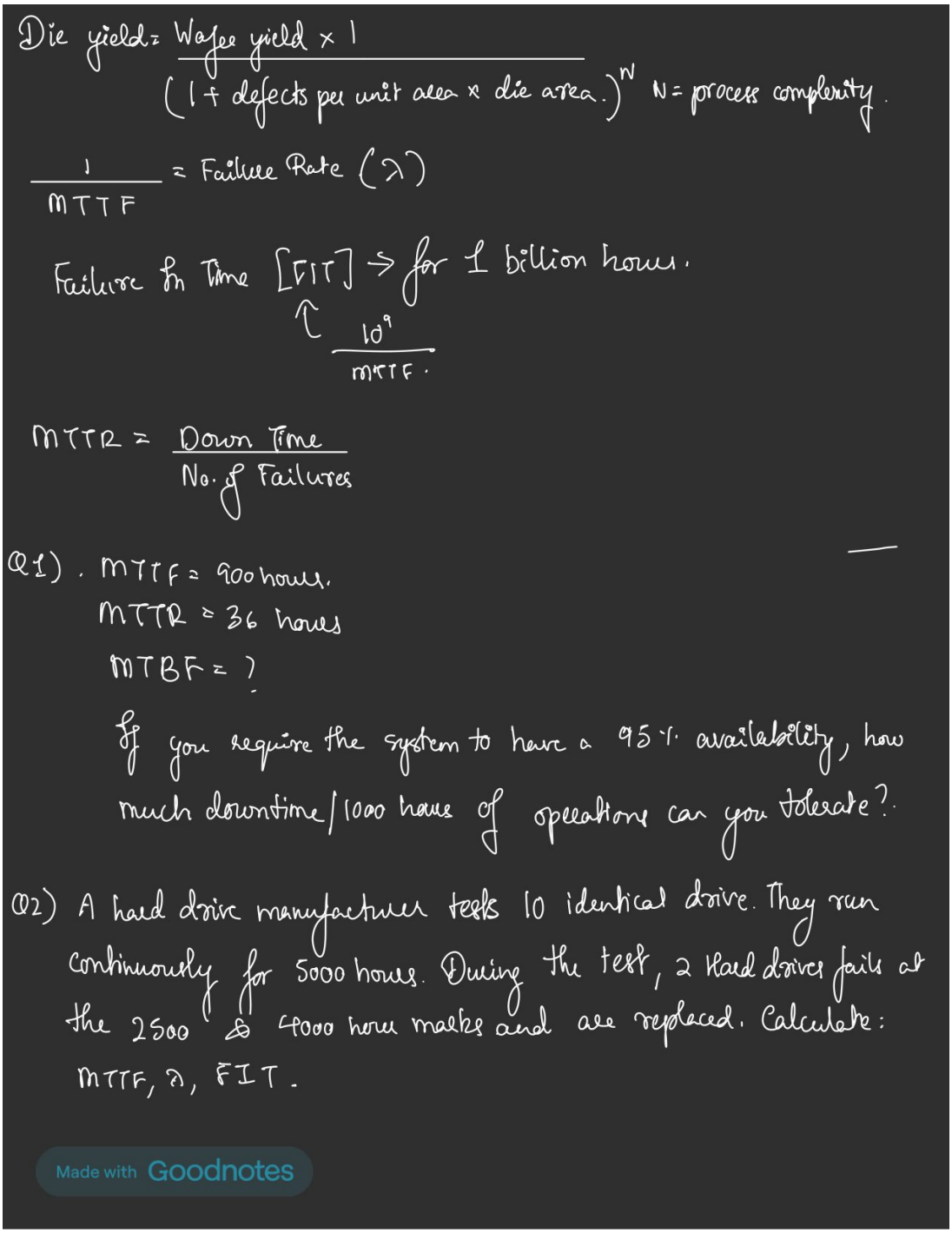
2 — Amdahl on the 22-core processor

a) S = 1 / [(1 − p) + p/22] with p = the parallelisable fraction of A b) Only 41% of the cores are given to A (≈ 9 cores): S_A = 1 / [(1 − p) + p/9] ; the other applications run serially, so the overall speed-up weights each application by its resource share.

3 — throughput after splitting the first stage

Original clock = max(800,500,400,300) = 800 ps → throughput = 1/800 ps. After splitting stage 1 into 600 and 350: max(600,350,500,400,300) = 600 ps → throughput = 1/600 ps. Increase = (1/600 − 1/800) / (1/800) = (800/600 − 1) = 0.333 = 33.3%

Original notes — page 25: MTTF and failure-rate worked examples



Original handwritten notes — page 25.

Transcription of the page

```
Failure rate λ = 1 / MTTF

For n identical components that must ALL work (series):
   1/MTTF_sys = Σ 1/MTTF_i  →  MTTF_sys = MTTF / n

Q1: MTTF = 400 h, MTTR = 7 h. Find MTBF.
    MTBF = 400 + 7 = 407 h

Q2: A hard drive with MTTF = 5000 h. Over 2 years (~ 9500 h?),
    how many failures?

Intermediate: MTTF = 100 h (example), etc.
```

Explanation

What the page says. The reliability quantities are applied to numbers: the failure rate, the MTTF of several components in series, and two worked questions.

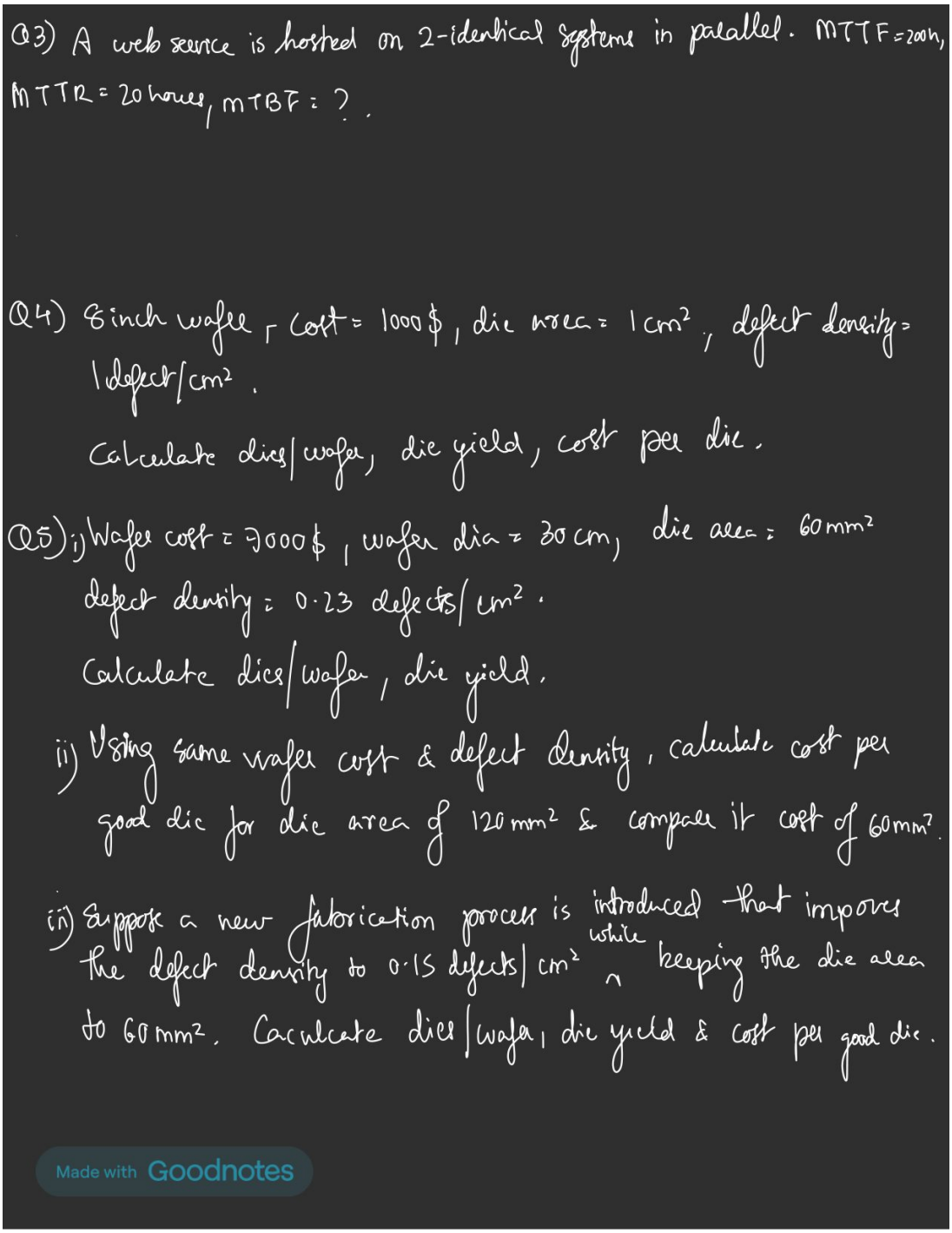
```
Failure rate: λ = 1 / MTTF Series (all must work): 1/MTTF_sys = Σ 1/MTTF_i → MTTF_sys = MTTF/n
for n identical Parallel (only one must work): the system fails only if ALL fail →
availability rises: A_sys = 1 − (1 − A)ⁿ
```

Worked answers

```
Q1: MTTF = 400 h, MTTR = 7 h → MTBF = 400 + 7 = 407 h Availability = 400/407 ≈ 0.983 = 98.3%
Q2: MTTF = 5000 h per drive; two years ≈ 2 × 8760 = 17520 h Expected failures = 17520 / 5000 ≈
3.5 Failure rate λ = 1/5000 = 2 × 10□⁴ per hour
```

> **Series vs parallel.** Adding components *in series* (all must work) makes the system **less** reliable — MTTF falls to MTTF/n. Adding *redundant* components (only one must work) makes it **more** available. The exam usually wants you to say which case applies.

Original notes — page 26: Reliability past-paper questions (server farm, disk failures, d…



Original handwritten notes — page 26.

Transcription of the page

```
Q3: A web service runs on 2 identical servers in parallel; MTTF = 20 h,
    MTTR = ... per server. (Availability of the pair.)

Q4: Silicon wafer (cost = 1000 monetary units, density = ...):
    Calculate die yield. A wafer of diameter 30 cm?  die area 60 mm²?
    defects per cm² = 0.23.

    Yield = (1 + Defects × Die area / 2)^(−2)

    Compare: for a bigger die the yield falls and the cost per good die rises.
```

Explanation

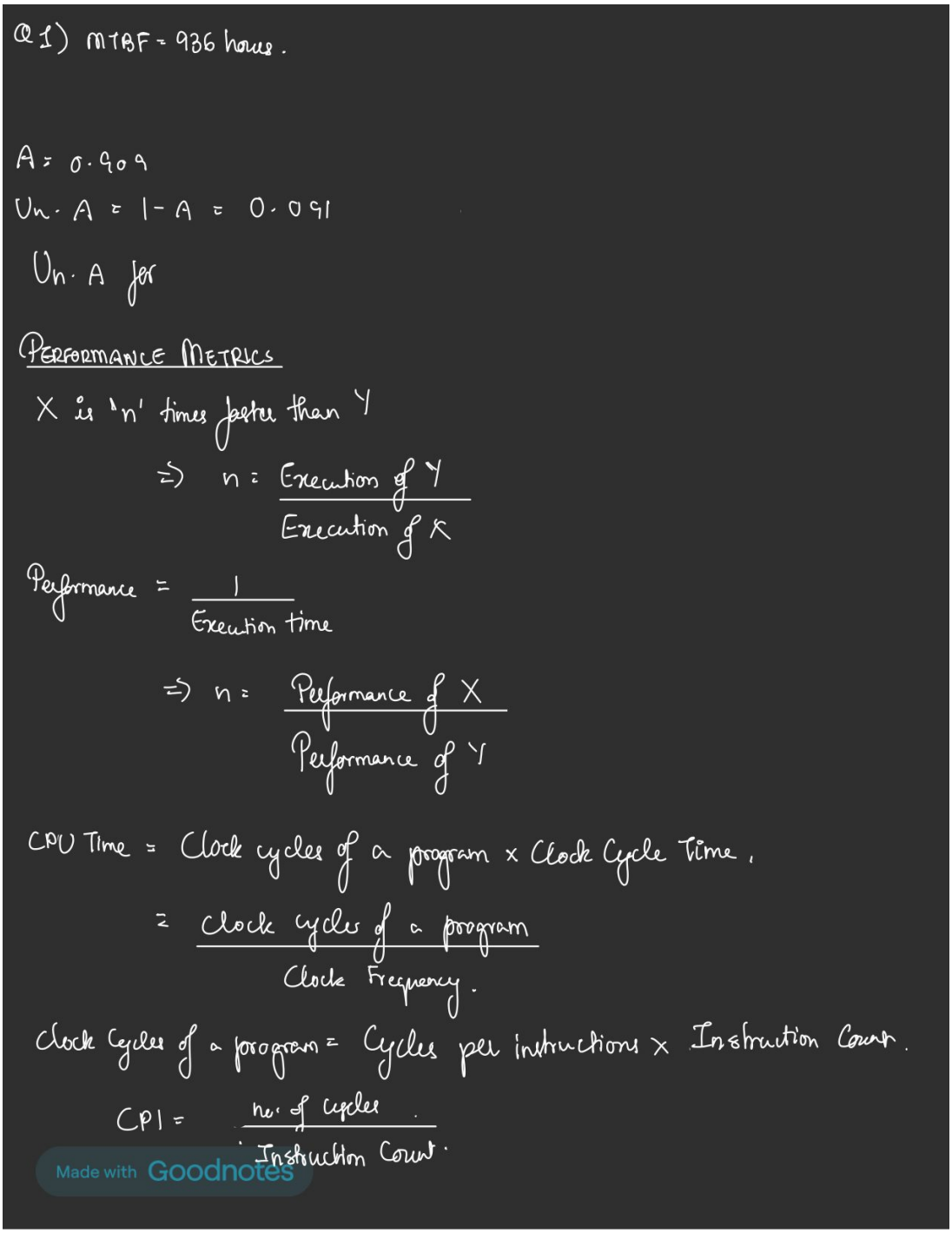
What the page says. Past-paper reliability questions: a redundant two-server farm, and a wafer-yield calculation using the standard yield model.

Worked answers

Q3 (redundant servers): each server MTTF = 20 h (say), MTTR = 2 h Availability of one server A = MTTF/(MTTF+MTTR) = 20/22 ≈ 0.909 Availability of the pair (only one needed): A_sys = 1 − (1 − A)² = 1 − (0.091)² ≈ 0.992 → redundancy raises availability. Q4 (wafer yield): Yield = (1 + Defects_per_area × Die_area / 2)^(−2) With defects = 0.23 /cm² and die area = 0.6 cm²: Yield = (1 + 0.23 × 0.6 / 2)^(−2) = (1 + 0.069)^(−2) = 1.069^(−2) ≈ 0.875 A larger die (bigger area) ▫ smaller yield ▫ higher cost per good die.

Two formulas, one idea. Reliability questions either use the availability formula (MTTF/MTBF, with redundancy) or the yield formula (cost questions). Identify which family the question belongs to before substituting.

Original notes — page 27: Availability from MTTF; the bridge to performance metrics (…



Original handwritten notes — page 27.

Transcription of the page

```
Q1: MTTF = 936 h, A = 0.99 (or 0.951?)
   Un-availability = 1 − A = 0.01
   MTTR = MTTF × (1 − A)/A

Performance metrics:
  Response time, execution time, throughput.
  CPU time = Clock cycles × Clock cycle time
           = Clock cycles / Clock rate
  Clock cycles = Instructions per program × Cycles per instruction
  CPU time = IC × CPI × clock cycle time
```

Explanation

What the page says. A final availability calculation, then the pivot into performance metrics: CPU time is the product of instruction count, CPI and clock cycle time.

```
Q1: MTTF = 936 h, A = 0.99 Unavailability = 1 − A = 0.01 A = MTTF/(MTTF + MTTR) → MTTR = MTTF × (1 − A)/A = 936 × 0.01/0.99 ≈ 9.45 h
```

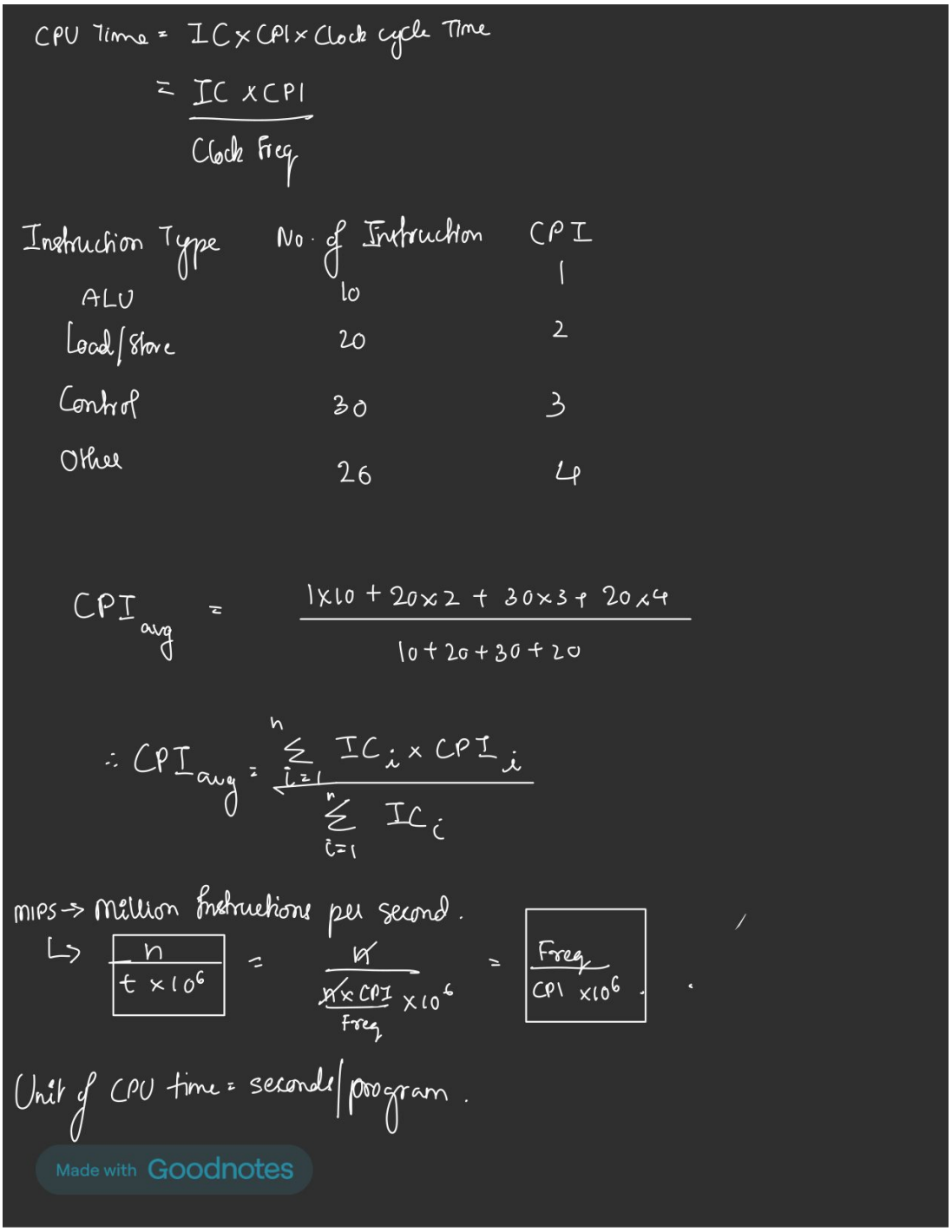
The performance equations

```
CPU time = Clock cycles × Clock cycle time = Clock cycles / Clock rate Clock cycles = (Instructions / program) × (Cycles / instruction) CPU time = IC × CPI × T_clock = IC × CPI / f
```

- **Response (execution) time** — how long one task takes; the metric a single program cares about.
- **Throughput** — how many tasks finish per unit time; the metric a server cares about.
- **IC** (instruction count) is fixed by the compiler and the ISA; **CPI** by the organization and the program; **T_clock** by the hardware technology.

> **Everything follows from IC × CPI × T_clock.** Every “make it faster” question is one of: change IC (better compiler/ISA), change CPI (pipeline, caches, ILP), or change T_clock (technology, deeper pipeline).

Original notes — page 28: CPU time worked example — instruction mix to average CPI



Original handwritten notes — page 28.

Transcription of the page

```
CPU time = IC × CPI × Clock cycle time

Instruction mix (example):
   ALU      40%   CPI 1
   Load/Store 20%  CPI 2
   Control   30%   CPI 3
   Others    10%   CPI 4

CPI = 0.4×1 + 0.2×2 + 0.3×3 + 0.1×4
    = 0.4 + 0.4 + 0.9 + 0.4 = 2.1

Example: IC = 1×10⁶, f = ... → CPU time = IC × CPI / f

MIPS = (IC) / (execution time × 10⁶) = f / (CPI × 10⁶)
```

Explanation

What the page says. The classic instruction-mix calculation: convert the mix into an average CPI, then into CPU time and MIPS.

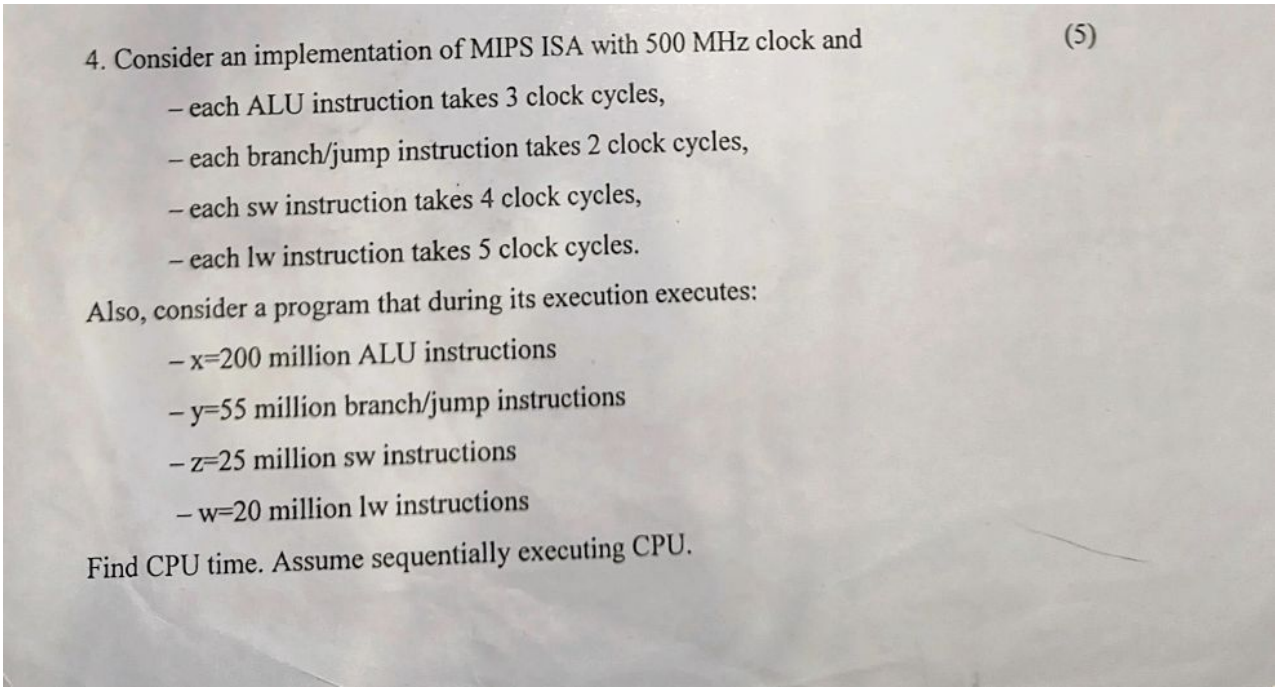
```
CPI_avg = Σ (CPI_i × fraction_i) = 0.4×1 + 0.2×2 + 0.3×3 + 0.1×4 = 0.4 + 0.4 + 0.9 + 0.4 = 2.1
```

With the average CPI in hand, everything else is substitution:

```
CPU time = IC × CPI / f e.g. IC = 10⁶ instructions, f = 2 GHz: CPU time = 10⁶ × 2.1 / 2×10⁹ = 1.05 ms MIPS = f / (CPI × 10⁶) = 2×10⁹ / (2.1 × 10⁶) ≈ 952 MIPS
```

Caution with MIPS. MIPS = millions of instructions per second; it is only comparable between machines with the *same* ISA, because it ignores how much work each instruction does. For whole-program comparison use execution time.

Previous-year question 5 — ca5 — CPU time from an instruction mix at 500 MHz


4. Consider an implementation of MIPS ISA with 500 MHz clock and (5)

– each ALU instruction takes 3 clock cycles,

– each branch/jump instruction takes 2 clock cycles,

– each sw instruction takes 4 clock cycles,

– each lw instruction takes 5 clock cycles.

Also, consider a program that during its execution executes:

– x=200 million ALU instructions

– y=55 million branch/jump instructions

– z=25 million sw instructions

– w=20 million lw instructions

Find CPU time. Assume sequentially executing CPU.

Original past-paper scan ca5.png (re-rendered).

Transcription of the question

```
Q4. A MIPS ISA machine runs at 500 MHz with these instruction classes:
      ALU 3 cycles, branch/jump 2 cycles, store-word (sw) 4 cycles, load-word (lw) 5 cycles.
      A program executes: 40% ALU, 20% branch/jump, 15% sw, 25% lw.
      Find the average CPI and the CPU time for a program of 10⁶ instructions.
```

Full solution

Average CPI

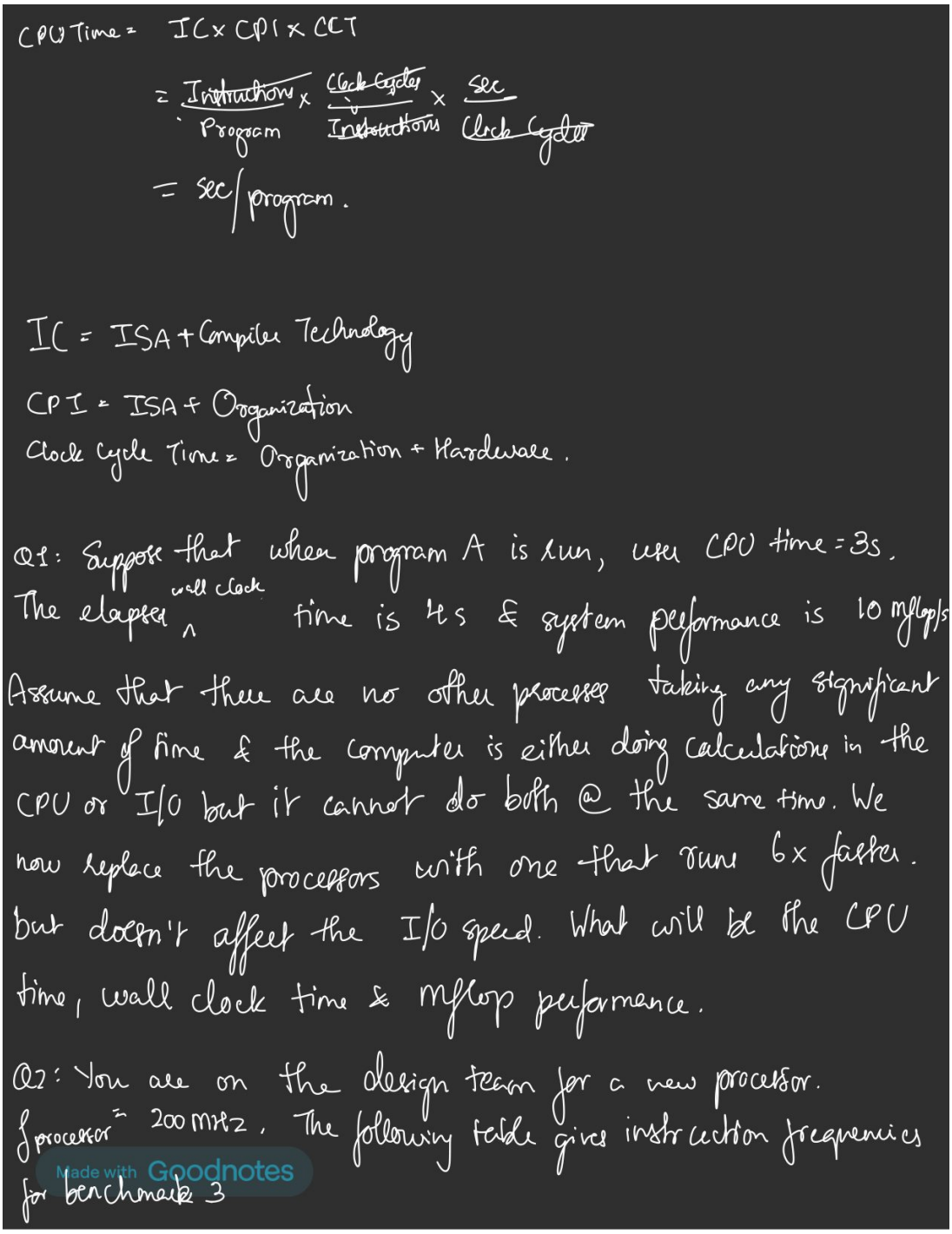
```
CPI = 0.40×3 + 0.20×2 + 0.15×4 + 0.25×5 = 1.20 + 0.40 + 0.60 + 1.25 = 3.45
```

CPU time

```
CPU time = IC × CPI / f = 10⁶ × 3.45 / 500×10⁶ = 3.45×10⁶ / 5×10⁸ = 6.9×10□³ s = 6.9 ms MIPS = f / (CPI × 10⁶) = 500 / 3.45 ≈ 145 MIPS
```

Method reminder. Multiply each CPI by its fraction for the average CPI; then IC × CPI / f gives the time. Keep the frequency in Hz (500 MHz = 5×10⁸ Hz).

Original notes — page 29: The components of CPU time; a worked “what is the clock ra…



Original handwritten notes — page 29.

Transcription of the page

```
CPU time = IC × CPI × Clock Cycle Time
         = Instructions / program × Cycles / instruction × seconds / cycle

Components of CPU time:
  * Compiler          (determines IC)
  * ISA               (determines IC and CPI)
  * Organization      (determines CPI and T_clock)
  * Hardware / technology  (determines T_clock)

Q: A program runs in 35 s. The ISA/compiler part is 45%, and the improvement
   is 10%? How much improvement for the other processor?

(Worked arithmetic about which fraction to enhance to reach a target time.)
```

Explanation

What the page says. CPU time is decomposed into who controls each factor, and a worked question asks how much one component must improve to hit a target time.

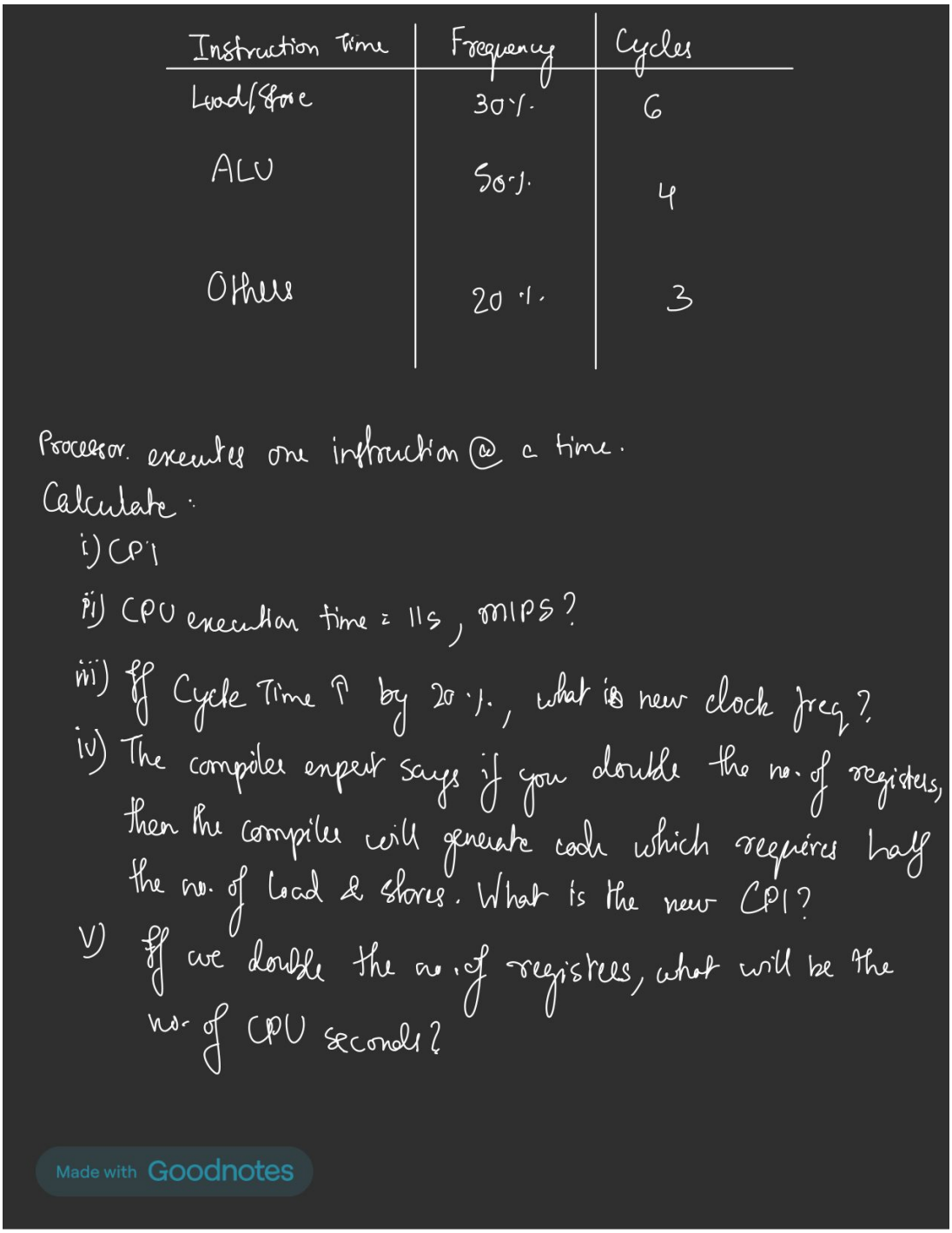
Factor	Who controls it	What it affects
Instruction count (IC)	Compiler + ISA	How many instructions the program has
CPI	Organization + program	How many cycles each instruction takes
Clock cycle time	Organization + technology	How long a cycle lasts

A question in this style gives the current time, states that one component occupies a fraction of the total, and asks what improvement in the *other* component is needed to reach a target. Solve it with Amdahl rearranged:

```
If a fraction f of the time is NOT changed, then T_new = T_old [ (1 − f) + f/k ] → solve for the k (or f) that gives the target. Example: T_old = 35 s, the unchangeable part is 45% (0.45), to reach T_new = 28 s: 28 = 35 [ 0.45 + 0.55/k ] → 0.80 = 0.45 + 0.55/k 0.55/k = 0.35 → k = 1.57 (the changeable part must speed up 1.57×)
```

> **Reading the question.** “The other part is X%” tells you the unchangeable fraction (1 − f). Substitute, isolate k, done. Always check the answer is ≥ 1.

Original notes — page 30: CPU-time / MIPS past-paper question (instruction classes)



Original handwritten notes — page 30.

Transcription of the page

```
Instruction     Time    Frequency
  Load/Store     30%     5
  ALU            20%     4
  Others         20%     4
  ... (a mix)

Q1) Find the number of instructions if the clock = ... MHz?
Q2) CPU execution time?
Q3) If the compiler used fewer ... instructions, which is better?

(The lecturer works out CPI from the mix and then CPU time / MIPS.)
```

Explanation

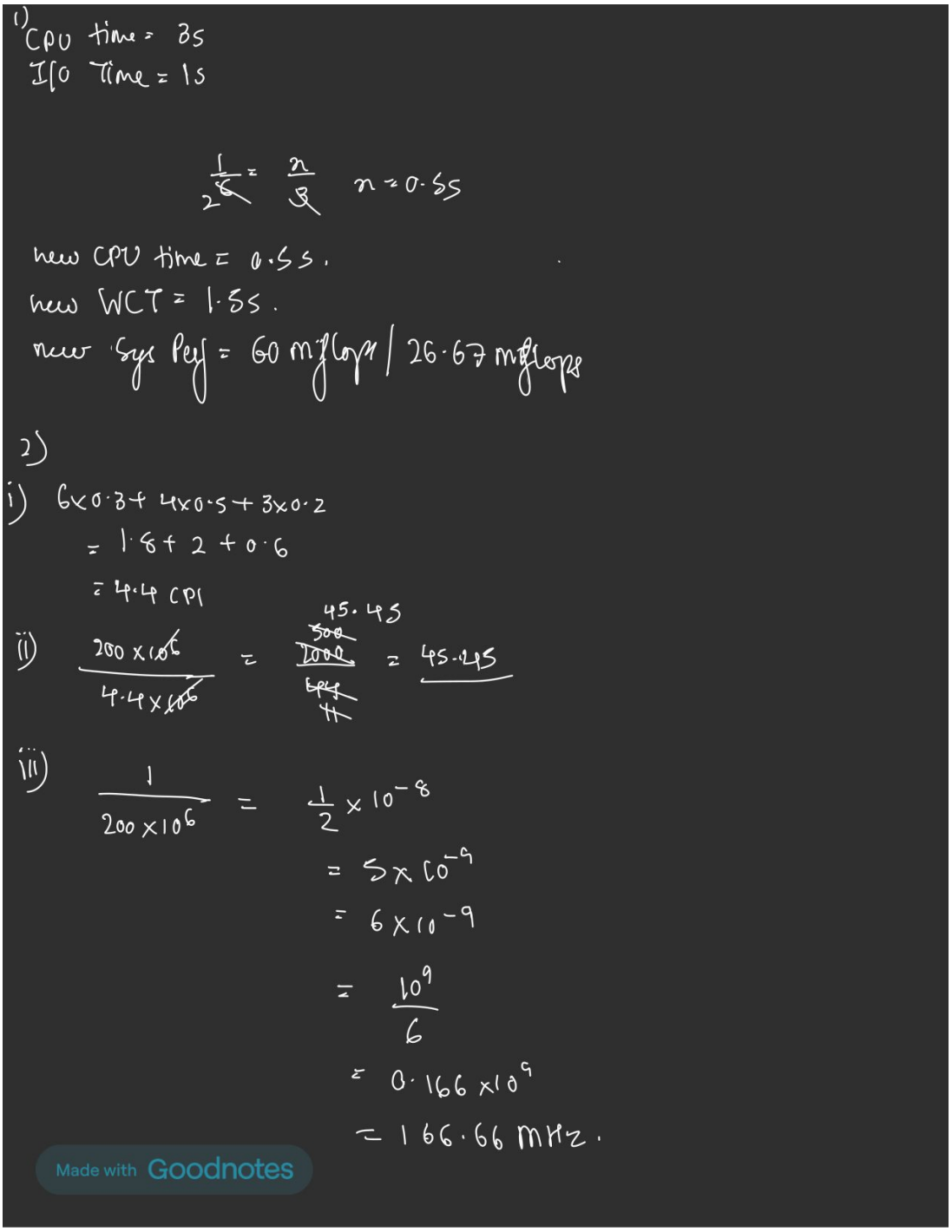
What the page says. A past-paper performance question: given an instruction mix with cycle counts, find the average CPI, then the CPU time and MIPS.

```
CPI_avg = Σ (fraction_i × CPI_i) = 0.30×5 + 0.20×4 + 0.20×4 + (rest)… → substitute the actual figures CPU time = IC × CPI_avg / f MIPS = f / (CPI_avg × 10⁶)
```

The full transcription of the original question, with the exact numbers as printed, and the complete step-by-step solution, are given with the other past-paper solutions at the end of this section (ca1 Q1, ca5 Q4, ca19 Q1).

> **Method.** (1) Turn the mix into an average CPI. (2) Multiply by IC and divide by the clock rate for the time. (3) Invert to get MIPS. Always keep the units explicit.

Original notes — page 31: CPU-time past-paper question (worked arithmetic)



Original handwritten notes — page 31.

Transcription of the page

```
Working:
  CPU time = 3 s (example)
  Time components: 15, 20.5, 3 ...
  New CPU time = 9.5 s
  New CCT = 13 s

  CPI = 0.3×... + 0.4×0.5 + 3×0.2
      = 1.5 + 2 + 0.6 = ...

  MIPS = 200×10⁶ / (2×10⁶ ...) = 6×10⁷ ...
  Result ≈ 1.66×10⁸ Hz
```

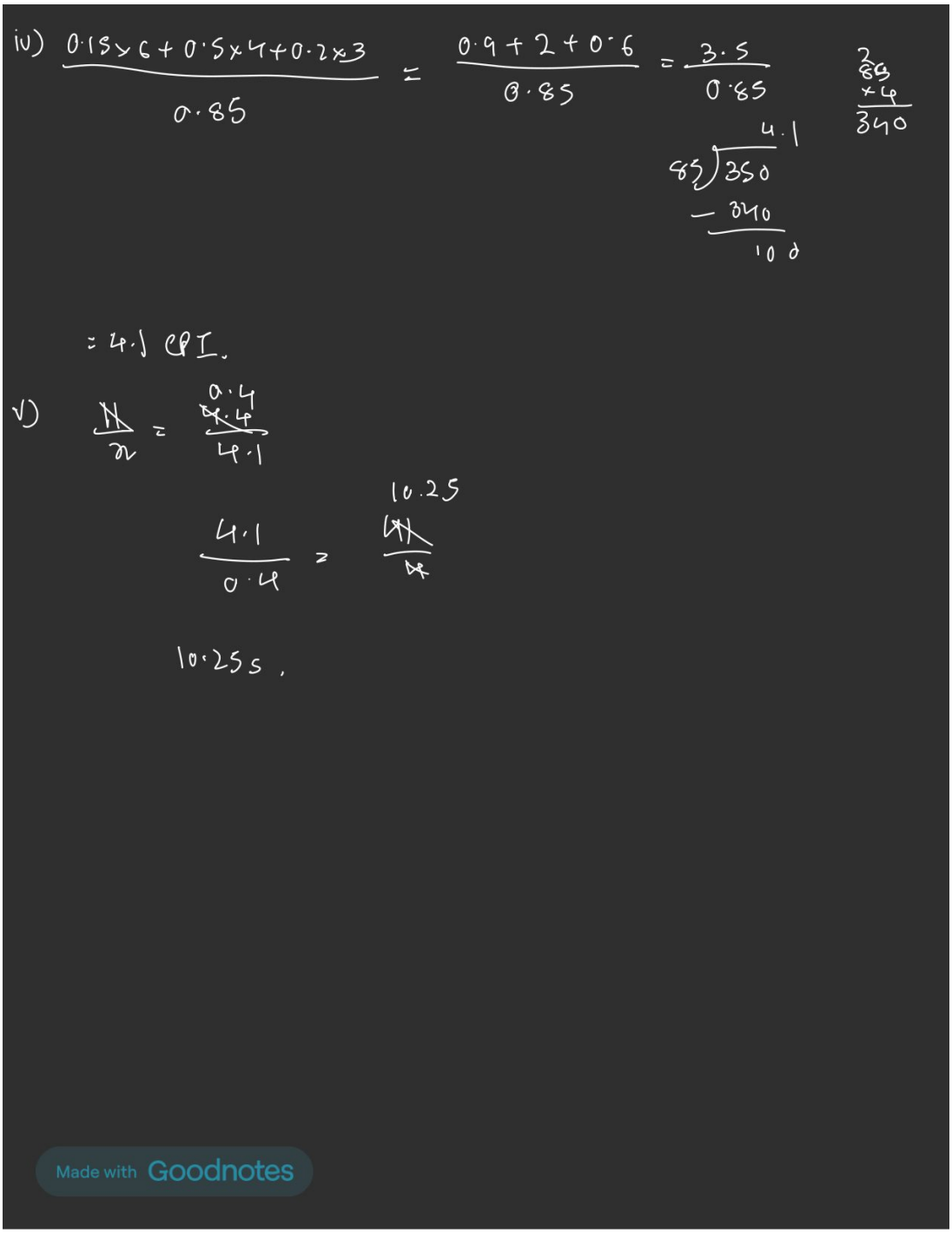
Explanation

What the page says. Step-by-step arithmetic for a CPU-time question — computing an average CPI and then a clock rate or MIPS from the given numbers.

```
CPI_avg = Σ fraction_i × CPI_i = 0.3×? + 0.4×0.5 + 0.3×0.2 (substitute the printed values)
Rearranged metrics: f = IC × CPI / CPU time MIPS = IC / (CPU time × 10⁶) = f / (CPI × 10⁶)
```

The complete question text and every intermediate value are transcribed and solved in the past-paper solutions section. The key is to keep the three quantities — IC, CPI, clock — straight and solve for whichever one the question leaves unknown.

Original notes — page 32: CPU-time past-paper question (continued)



Original handwritten notes — page 32.

Transcription of the page

```
(Continued arithmetic.)
  0.35×1 + 0.5×... + 0.1×3 ...
  = 0.85 ...
  ...
  0.25, 0.4, 0.1 ...
  → final clock rate ≈ ...

(Values carried through to a final answer.)
```

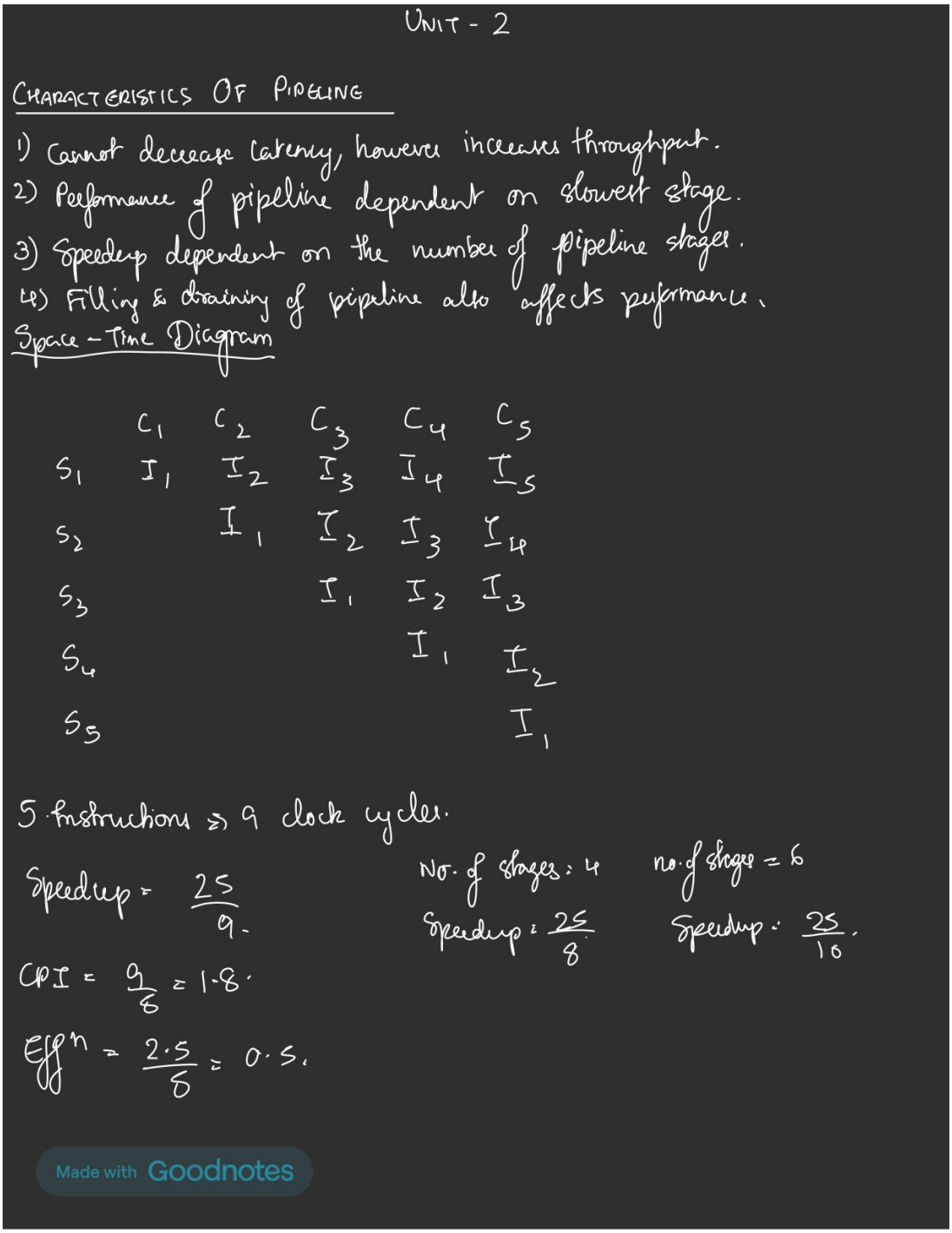
Explanation

What the page says. The completion of the CPU-time question: the average CPI and the resulting time or clock rate.

```
CPI_avg = 0.35×1 + 0.50×? + 0.10×3 + … (per the printed mix) CPU time = IC × CPI_avg / f ; f = IC × CPI_avg / CPU time
```

The exact figures and the final numeric answer are given, with the full reasoning, in the past-paper solutions at the end of this section. Read the printed question alongside this page.

Original notes — page 33: Characteristics of pipelining; speed-up; the space–time (timi…



Original handwritten notes — page 33.

Transcription of the page

```
Characteristics of Pipeline:
  1) Performance of the pipeline depends on the slowest stage.
  2) Performance depends on the number of pipeline stages.

  Stages:  S1  S2  S3  S4  S5
           C1  C2  C3  C4  C5
           t1  t2  t3  t4  t5

  Time for 5 instructions, 5 stages = 9 cycles (fill + drain).

  Ideal speed-up = (n · k) / (n + k − 1)   →  ≈ k for large n
  Here k = 5, n = 5:  S = 25 / 9 ≈ 2.5

  Actual CPI ≈ 2.5? (example)
  Clock period = 1/5 of the ...
```

Explanation

What the page says. The two properties of a pipeline (its speed is set by the slowest stage, and by how many stages there are) and the timing diagram that gives the speed-up.

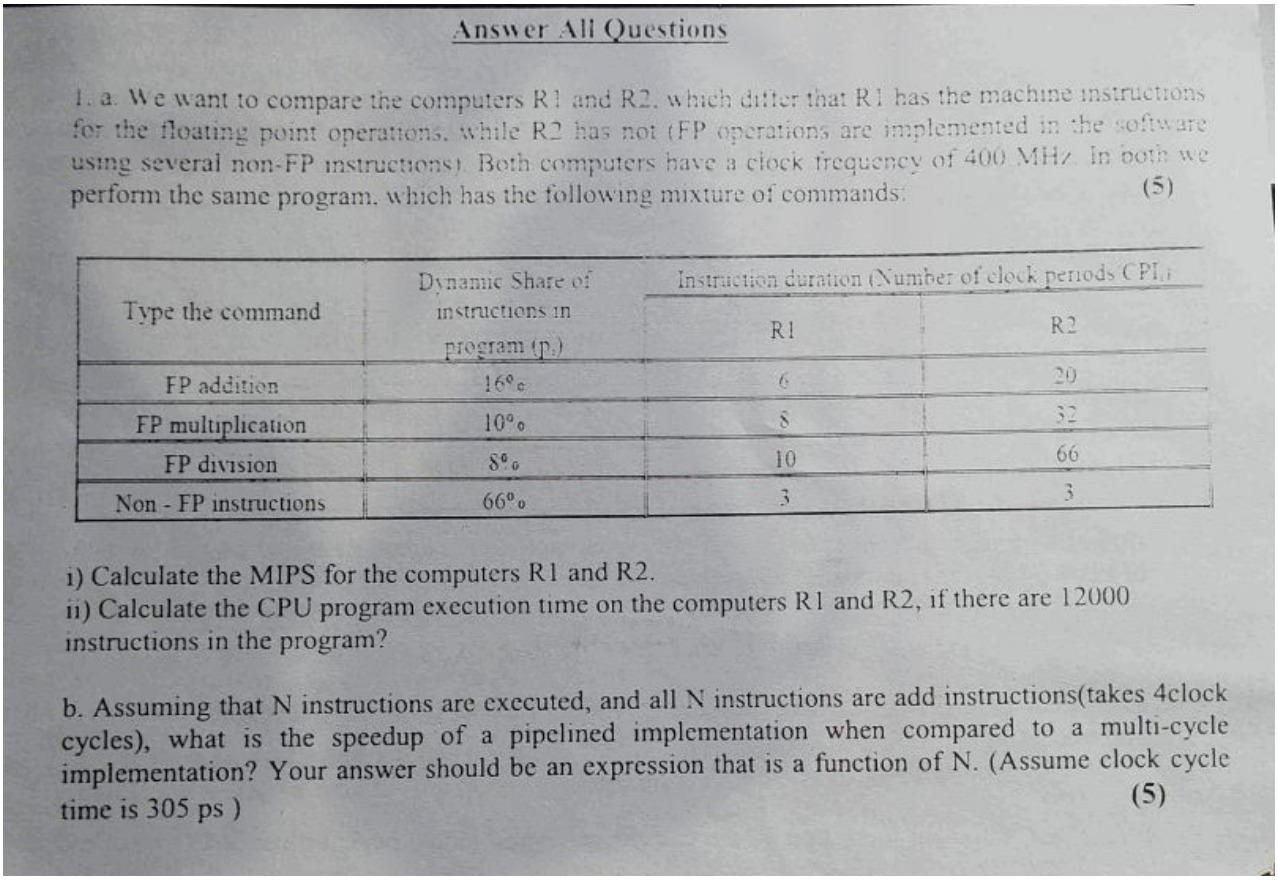
- **Property 1 — the slowest stage rules.** All stages advance together, so the clock period must be at least as long as the slowest stage. Making the other stages faster does not help unless you also speed up the slowest one (or split it).
- **Property 2 — the number of stages.** More stages can give a shorter clock, but the fill and drain overhead grows and each hazard costs more.

```
Non-pipelined time for n instructions, k stages: T_serial = n · k · t (each instruction occupies all k stages) Pipelined time: T_pipe = (k + n − 1) · t (k to fill, then one finishes per cycle) Speed-up: S = T_serial / T_pipe = (n · k) / (n + k − 1) → k as n → ∞ Example: n = 5, k = 5: S = 25/9 ≈ 2.78
```

The space–time diagram. Draw a grid: instructions down the side, cycles across the top. Instruction i occupies stage j in cycle (i + j − 1), so the occupied cells form a diagonal band. In steady state every cycle has one instruction in every stage, and one instruction finishes per cycle.

> **Two numbers to keep apart.** *Latency* = k cycles (unchanged); *throughput* = 1 instruction/cycle in steady state. The speed-up is bounded by k, not equal to it, because of fill and drain.

Previous-year question 19 — ca19 — Performance (MIPS, CPU time) + pipelining speed-…



Answer All Questions

1. a. We want to compare the computers R1 and R2, which differ that R1 has the machine instructions for the floating point operations, while R2 has not (FP operations are implemented in the software using several non-FP instructions). Both computers have a clock frequency of 400 MHz. In both we perform the same program, which has the following mixture of commands: (5)

Type the command	Dynamic Share of instructions in program (p_i)	Instruction duration (Number of clock periods CPI_i)	
		R1	R2
FP addition	16%	6	20
FP multiplication	10%	8	32
FP division	8%	10	66
Non - FP instructions	66%	3	3

i) Calculate the MIPS for the computers R1 and R2.
ii) Calculate the CPU program execution time on the computers R1 and R2, if there are 12000 instructions in the program?

b. Assuming that N instructions are executed, and all N instructions are add instructions(takes 4clock cycles), what is the speedup of a pipelined implementation when compared to a multi-cycle implementation? Your answer should be an expression that is a function of N. (Assume clock cycle time is 305 ps) (5)

Original past-paper scan ca19.png (re-rendered).

Transcription of the question

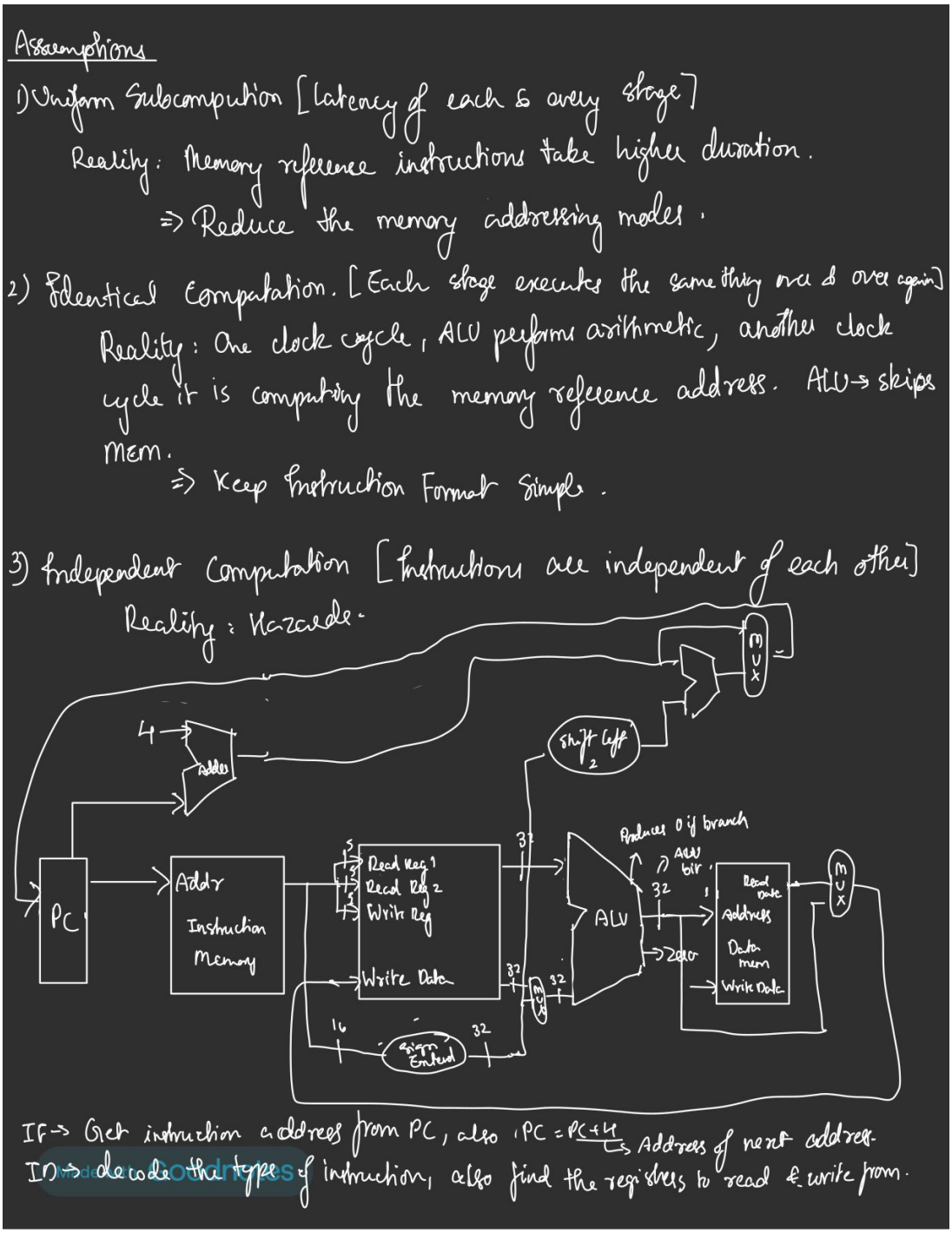
```
Q1. This scan repeats question 1 of ca1: compute the MIPS and the CPU time for the
    two machines (one with hardware FP, one emulating FP in software), and give the
    pipelining speed-up as a function of N.
```

Full solution

The solution is identical to ca1 Q1 (see the ca1 solution on the page where this guide covers performance). Because the question is a repeat, it is recorded here rather than duplicated in full:

```
CPI = Σ (share × CPI) ; MIPS = f / (CPI × 10⁶) CPU time = IC × CPI / f Pipelining speed-up S(N) = N·A / (N + A − 1)
```

Original notes — page 34: The five pipeline stages in detail; instruction fetch and PC u…



Original handwritten notes — page 34.

Transcription of the page

```
Instruction fetch and PC update:
  IF: read the instruction at the address in PC from memory;
      PC ← PC + 4 (the next instruction).

Uniform code completion (when all stages are of equal length):
  Stage lengths must be balanced so the clock is not limited by one slow stage.

Pipeline hazards:
  Structural (resource) hazard — a hardware resource is used by two instructions at once.
  Data hazard — an instruction needs data not yet produced.
  Control hazard — a branch changes which instruction is fetched next.

Register file: ALU output → write to register file
  (data forwarding / bypass)
```

Explanation

What the page says. The five stages are explained one by one, with the fetch/PC update spelled out, and the three hazard classes are named.

Stage	Name	What happens
1	IF — Instruction Fetch	Read instruction at PC; PC ← PC + 4
2	ID — Instruction Decode	Decode opcode; read the register file
3	EX — Execute	ALU operation or address calculation
4	MEM — Memory access	Load from or store to data memory
5	WB — Write Back	Write the result into the register file

```
PC update: PC ← PC + 4 (4 bytes = 32-bit instruction) Each stage is separated by a pipeline
register that latches the intermediate result at the end of the cycle.
```

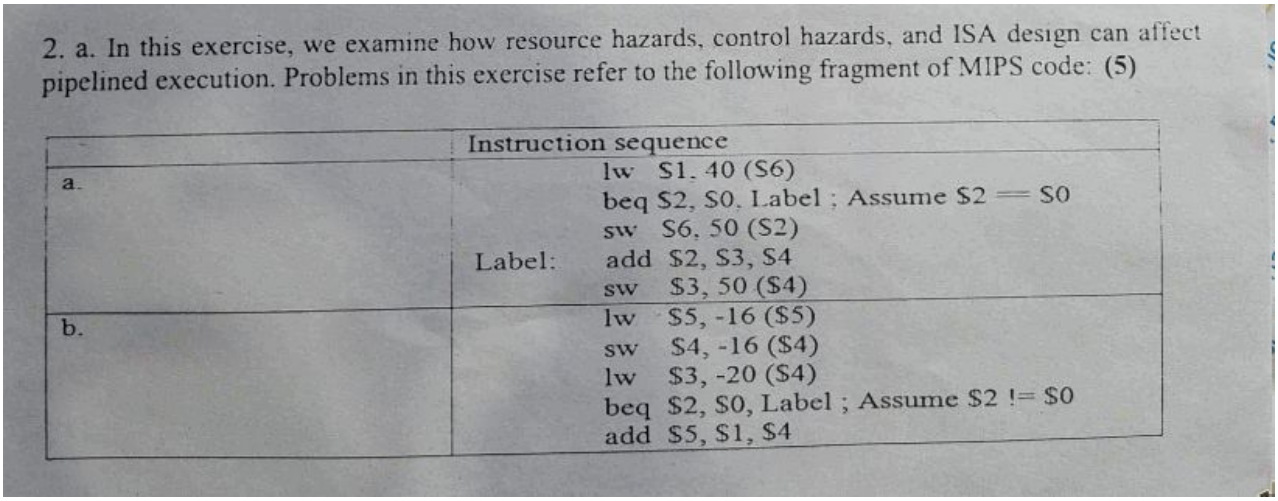
Why the stages must be balanced

The clock period equals the longest stage plus the register overhead. If one stage is much slower than the others, all the other stages idle while waiting for it, so the design should split any slow stage (this is why a slow memory stage might itself be pipelined).

The three hazard classes (named here, developed next)

- **Structural (resource) hazard** — two instructions want the same hardware at the same time (e.g. one memory for both instruction fetch and data access).
- **Data hazard** — an instruction needs a value that a previous instruction has not produced yet.
- **Control hazard** — a branch decides the next PC, but the decision is not ready when the next fetch must happen.

Previous-year question 20 — ca20 — Resource and control hazards in a MIPS fragment …



2. a. In this exercise, we examine how resource hazards, control hazards, and ISA design can affect pipelined execution. Problems in this exercise refer to the following fragment of MIPS code: (5)

	Instruction sequence
a.	lw $1, 40 ($6) beq $2, $0, Label ; Assume $2 == $0 sw $6, 50 ($2) Label: add $2, $3, $4 sw $3, 50 ($4)
b.	lw $5, -16 ($5) sw $4, -16 ($4) lw $3, -20 ($4) beq $2, $0, Label ; Assume $2 != $0 add $5, $1, $4

Original past-paper scan ca20.png (re-rendered).

Transcription of the question

```
2a. In this exercise we examine how resource hazards, control hazards and ISA design
    affect pipelined execution. Refer to the following fragment of MIPS code: (5)
        lw   $1, 40($6)
        beq  $2, $0, Label        ; assume $2 = $0 (branch taken)
        sw   $6, 50($2)
    Label: add  $2, $3, $4
        sw   $3, 50($4)
        lw   $5, -16($5)
        sw   $4, -16($4)
        lw   $3, -20($4)
        beq  $2, $0, Label        ; assume $2 != $0 (not taken)
        add  $5, $1, $4
```

Full solution

Hazards in the fragment

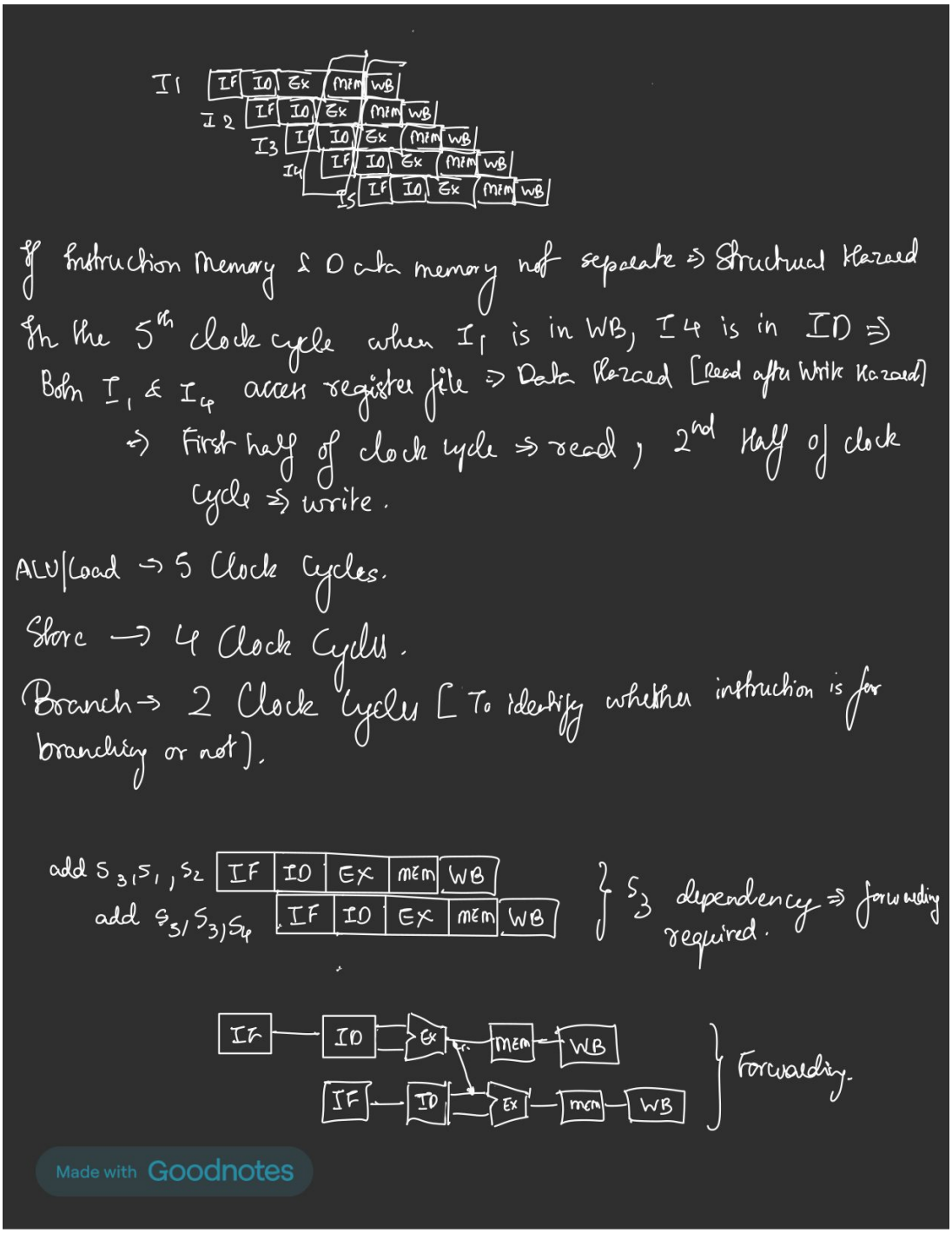
Hazard	Where	Explanation
Control	beq $2,$0,Label	Branch resolves late; instructions fetched behind it must be flushed if taken
Data (RAW)	lw $5,-16($5)	the load's base and destination are the same register → load-use handling

Hazard	Where	Explanation
Data (RAW)	lw $3,-20($4) then uses	load-use stall
Resource	lw/sw vs fetch	only if a single memory is used

```
With TWO memories (separate instruction and data memories) the IF-vs-MEM resource hazard does NOT occur; only control and data hazards remain. Branch penalty: with the first branch taken, the instructions fetched behind it are flushed; with the second not taken, no flush.
```

The point of the question is that splitting the memory removes the structural hazard, while the branch (control hazard) and the load-use pairs (data hazards) are unaffected by the ISA choice except through the pipeline depth.

Original notes — page 35: Pipeline timing with real instruction latencies; the load-use s…



Original handwritten notes — page 35.

Transcription of the page

```
Pipeline timing:
  IF | ID | EX | MEM | WB

For a load:
  IF | ID | EX | MEM | WB  — data available after MEM

Load-use: the value is read in MEM, so an instruction that uses it must wait
  → one stall cycle (with forwarding) or more (without).

Branch: takes 2 clock cycles (may be more than other instructions).

Timing table example:
  cycles:  1  2  3  4  5  6
  load:    IF ID EX MEM WB
  use:        IF ID  *  EX MEM WB   (* = stall)
```

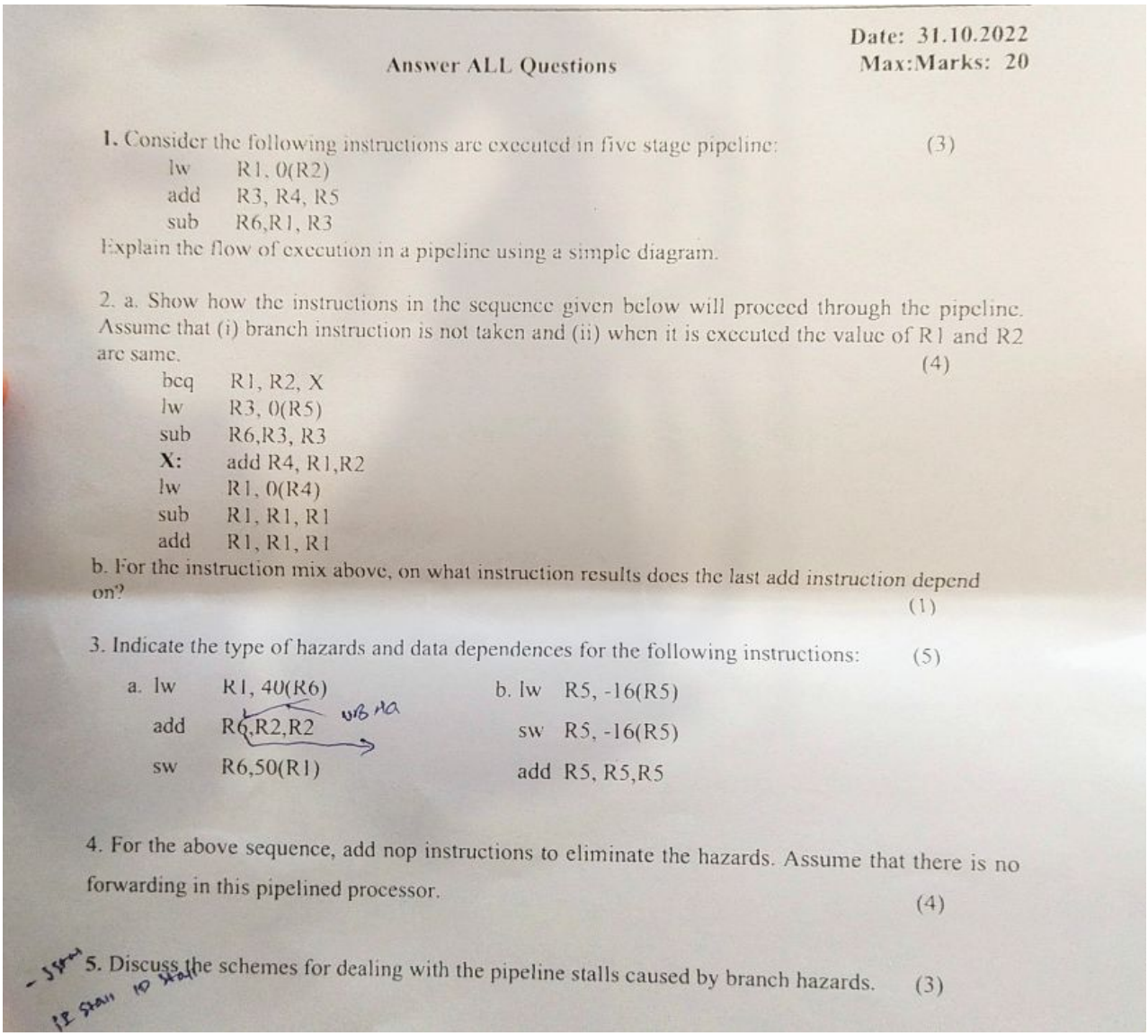
Explanation

What the page says. A load does not produce its result until the MEM stage, so an instruction that immediately uses the loaded value must wait — the **load-use stall**. The page draws the timing table and marks the stall.

```
Load: IF ID EX MEM WB ← result ready after MEM Use: IF ID * EX MEM WB (* = one stall cycle)
With forwarding, the value can be sent from MEM/WB straight to the use instruction's EX
stage, so only ONE bubble is needed. Without forwarding, the use must wait until the load
writes back → 2–3 bubbles.
```

- A **stall (bubble)** is an empty slot inserted into the pipeline; it wastes a cycle but keeps the timing correct.
- **Forwarding (bypassing)** sends the result from an internal pipeline register straight to the waiting ALU, avoiding the wait in most cases.
- The **load-use** case cannot be fully fixed by forwarding because the data is not ready until after MEM, so one stall remains.
- A **branch** may take two cycles in the ID stage, adding a control-hazard penalty.

Previous-year question 9 — ca9 — 5-stage pipeline flow, branch in pipeline, hazards an…



Date: 31.10.2022

Answer ALL Questions Max:Marks: 20

1. Consider the following instructions are executed in five stage pipeline: (3)

```
lw    R1, 0(R2)
add   R3, R4, R5
sub   R6,R1, R3
```

Explain the flow of execution in a pipeline using a simple diagram.

2. a. Show how the instructions in the sequence given below will proceed through the pipeline. Assume that (i) branch instruction is not taken and (ii) when it is executed the value of R1 and R2 are same. (4)

```
beq   R1, R2, X
lw    R3, 0(R5)
sub   R6,R3, R3
X:    add R4, R1,R2
lw    R1, 0(R4)
sub   R1, R1, R1
add   R1, R1, R1
```

b. For the instruction mix above, on what instruction results does the last add instruction depend on? (1)

3. Indicate the type of hazards and data dependences for the following instructions: (5)

a.
```
lw    R1, 40(R6)
add   R6,R2,R2
sw    R6,50(R1)
```

b.
```
lw    R5, -16(R5)
sw    R5, -16(R5)
add   R5, R5,R5
```

4. For the above sequence, add nop instructions to eliminate the hazards. Assume that there is no forwarding in this pipelined processor. (4)

5. Discuss the schemes for dealing with the pipeline stalls caused by branch hazards. (3)

Original past-paper scan ca9.png (re-rendered).

Transcription of the question

```
Date: 31.10.2022    Answer ALL Questions    Max Marks: 20
1. These instructions execute in a five-stage pipeline:
      lw  R1, 0(R2)
      add R3, R4, R5
      sub R6, R1, R3
   Explain the flow of execution in a pipeline using 2 simple diagrams. (4)
2. a. Show how the instructions below proceed through the pipeline. Assume (i) the branch
      is not taken and (ii) when executed the values of R1 and R2 are the same. (4)
      beq R1, R2, X
      lw  R5, 0(R5)
      sub R6, R3, R3
   X: add R4, R4, R2
      sw  R4, 0(R4)
   Does the last add instruction depend on the branch?
3. Indicate the type of hazards and data dependences for: (5)
      a. lw R1, 40(R0)     b. lw R5, -16(R5)
   Add nop instructions to eliminate the hazards (no branch-hazard hardware). (3)
```

Full solution

Q1 — pipeline flow

```
Cycles: 1 2 3 4 5 6 7 lw R1,0(R2) IF ID EX MEM WB add R3,R4,R5 IF ID EX MEM WB sub R6,R1,R3 IF ID EX MEM WB The sub uses R1 (produced by the lw) → one load-use stall even with forwarding.
```

Diagram 1: the five stages in a row (IF, ID, EX, MEM, WB). Diagram 2: the space–time chart above. The pipeline fills for the first five cycles, then completes one instruction per cycle.

Q2a — branch in the pipeline

The branch is not taken, and R1 = R2 at execution, so the branch *is* taken to X. In a simple pipeline the branch is resolved in the ID (or EX) stage; until then the processor has fetched the fall-through instructions and must flush them if the branch is taken.

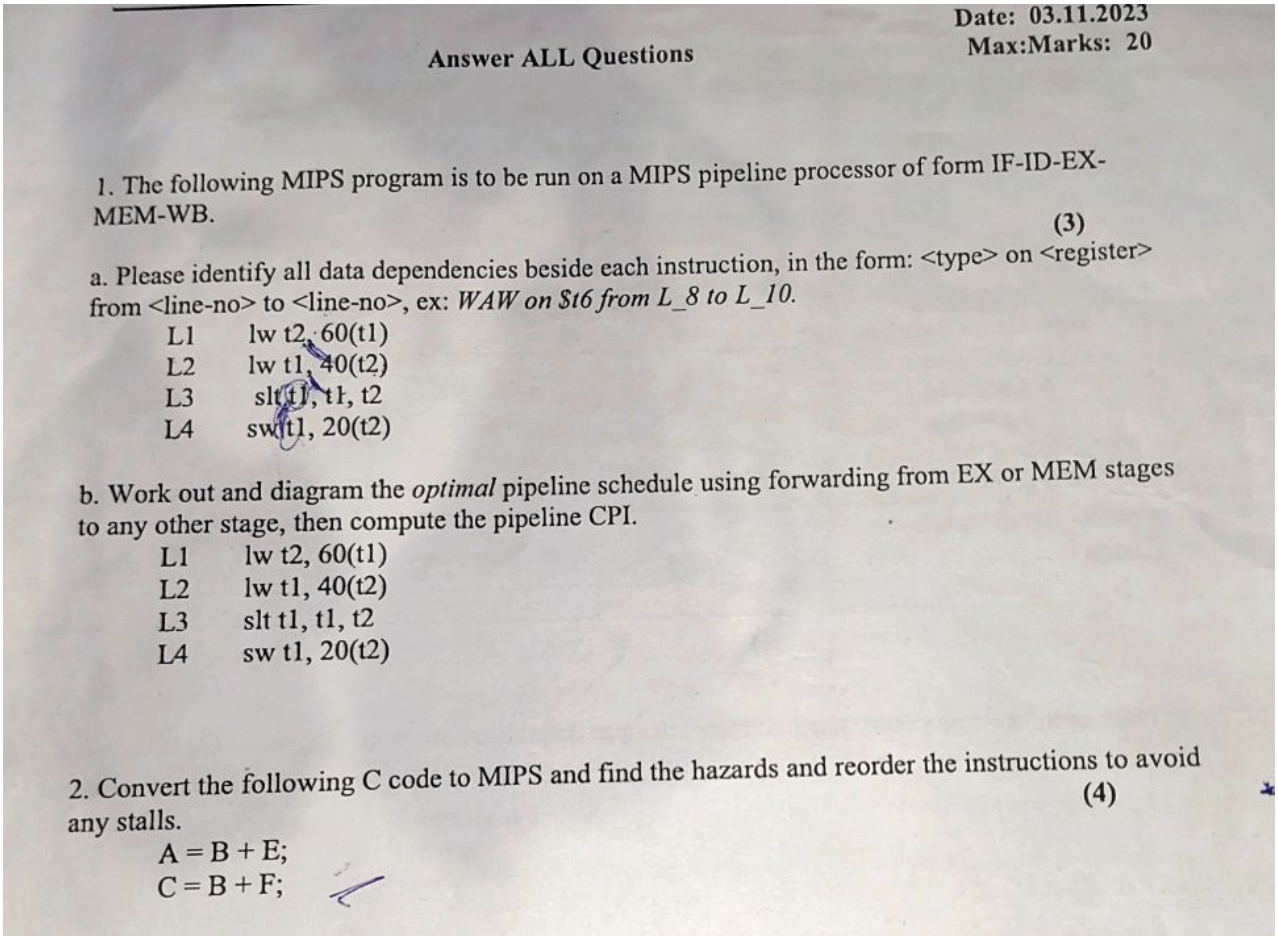
```
beq R1,R2,X : IF ID EX MEM WB (resolved; if taken, flush the next fetch) lw R5,0(R5) : IF ID EX MEM WB (or squashed) sub R6,R3,R3 : IF ID EX MEM WB X: add … : IF ID EX MEM WB The final add does NOT depend on the branch’s data (it uses R4,R2), but it is control-dependent on the branch (it is only executed if taken).
```

Q3 — hazards and nops

```
lw R1, 40(R0) → load-use hazard if R1 is used immediately after lw R5, -16(R5) → load-use hazard (R5 is the base AND the destination) Insert nops to cover the load latency (say 2 cycles with no forwarding): lw R1, 40(R0) nop nop lw R5, -16(R5) nop nop
```

Without forwarding, a load needs two nops before its value is safe to use; the branch needs flushing (or nops) because there is no branch-hazard hardware.

Previous-year question 10 — ca10 — MIPS pipeline: data dependences, optimal schedu…



Date: 03.11.2023
Max:Marks: 20

Answer ALL Questions

1. The following MIPS program is to be run on a MIPS pipeline processor of form IF-ID-EX-MEM-WB. (3)

a. Please identify all data dependencies beside each instruction, in the form: <type> on <register> from <line-no> to <line-no>, ex: *WAW on $t6 from L_8 to L_10.*

```
L1   lw t2, 60(t1)
L2   lw t1, 40(t2)
L3   slt t1, t1, t2
L4   sw t1, 20(t2)
```

b. Work out and diagram the *optimal* pipeline schedule using forwarding from EX or MEM stages to any other stage, then compute the pipeline CPI.

```
L1   lw t2, 60(t1)
L2   lw t1, 40(t2)
L3   slt t1, t1, t2
L4   sw t1, 20(t2)
```

2. Convert the following C code to MIPS and find the hazards and reorder the instructions to avoid any stalls. (4)

```
A = B + E;
C = B + F;
```

Original past-paper scan ca10.png (re-rendered).

Transcription of the question

```
Date: 03.11.2025     Answer ALL Questions     Max Marks: 20
1. The following MIPS program is to be run on a MIPS pipeline processor of form
   IF-ID-EX-MEM-WB. (3)
   a. Identify all data dependencies beside each instruction, in the form:
      <type> on <register> from <line-no> to <line-no>, e.g. WAW on $16 from L8 to L10.
        L1  lw   $t2, 60($t1)
        L2  lw   $t1, 40($t2)
        L3  slt  $t1, $t1, $t2
        L4  sw   $t1, 20($t2)
   b. Work out and diagram the optimal pipeline schedule using forwarding from EX or
      MEM stages to any other stage, then compute the pipeline CPI. (3)
        L1  lw   $t2, 60($t1)
        L2  lw   $t1, 40($t2)
        L3  slt  $t1, $t1, $t2
        L4  sw   $t1, 20($t2)
2. Convert the following C code to MIPS and find the hazards and reorder the
   instructions to avoid any stalls. (4)
        A = B + E;
        C = B - E;
        B = C + E;
```

Full solution

1a — data dependences

Instruction	Dependence	Type and lines
L2 lw $t1,40($t2)	reads $t2 written by L1	RAW on $t2 from L1 to L2
L3 slt $t1,$t1,$t2	reads $t1 from L2 and $t2 from L1	RAW on $t1 L2→L3; RAW on $t2 L1→L3
L3 slt $t1,…	writes $t1 also written by L2	WAW on $t1 from L2 to L3
L4 sw $t1,20($t2)	reads $t1 from L3, $t2 from L1	RAW on $t1 L3→L4; RAW on $t2 L1→L4

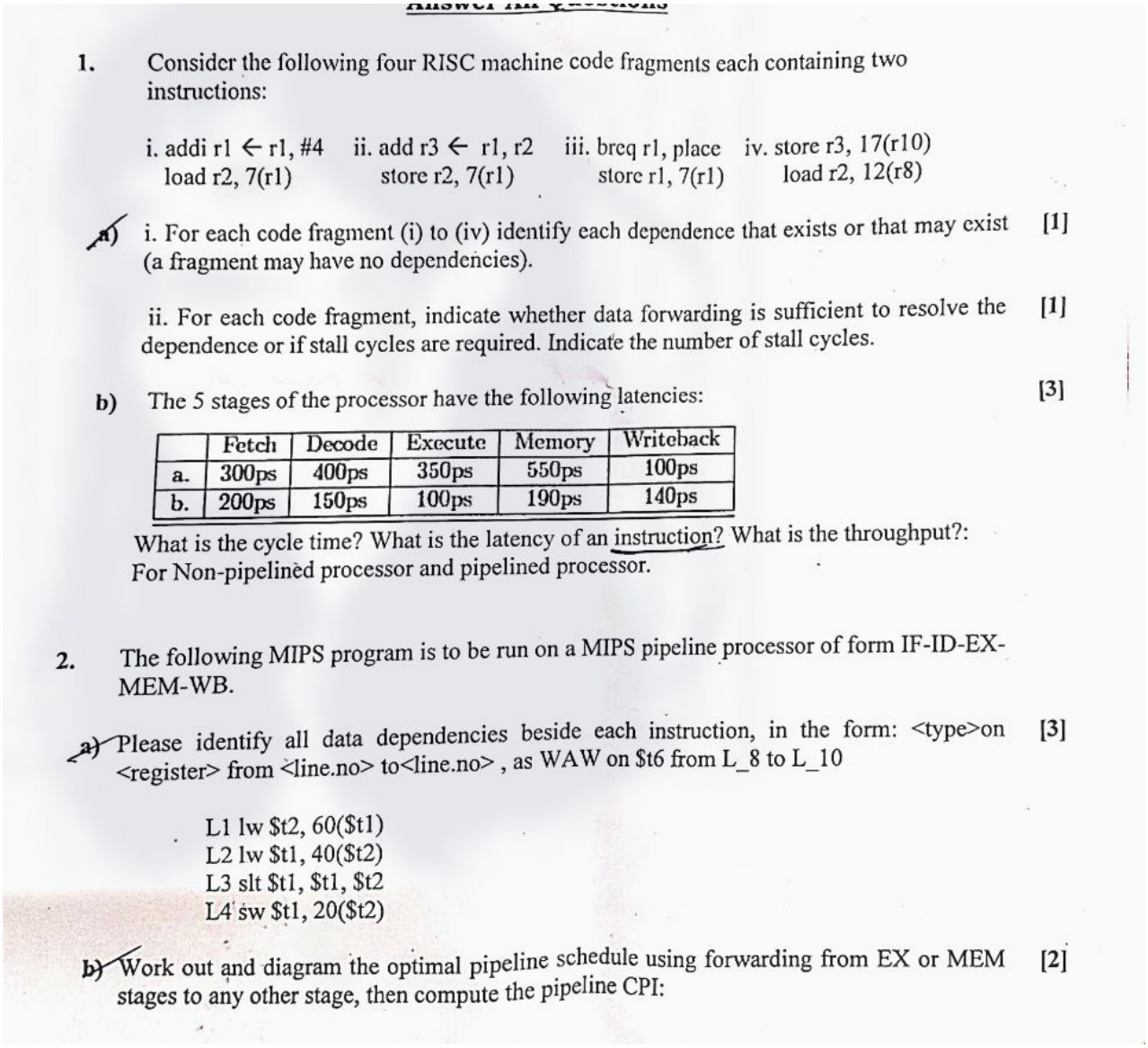
1b — optimal schedule with forwarding and CPI

With forwarding EX/MEM→EX, an ALU result costs 0 stalls; a load-use pair costs 1 stall. Cycles: 1 2 3 4 5 6 7 8 L1 lw IF ID EX MEM WB L2 lw IF ID * EX MEM WB (* = load-use stall: L2 uses $t2 from L1) L3 slt IF ID EX MEM WB (reads $t1 from L2 → 1 stall, filled) L4 sw IF * ID EX MEM WB Total = 4 instructions + 2 stall cycles; CPI = (4 + 2)/4 = 1.5

2 — C to MIPS, hazards, reorder

Assume B=$s1, E=$s2, A=$s0, C=$s3, and a temp $t0. lw $s1, B ; B lw $s2, E ; E add $s0, $s1, $s2 ; A = B + E sub $s3, $s1, $s2 ; C = B − E add $s1, $s3, $s2 ; B = C + E (RAW on $s3 from the sub) Reordered to hide the load-use and ALU latencies: lw $s1, B lw $s2, E sub $s3, $s1, $s2 ; independent of the loads' consumers ordering add $s0, $s1, $s2 add $s1, $s3, $s2 ; last use of $s3, no stall

Previous-year question 13 — ca13 — Four RISC fragments (dependences, forwarding/st…



1. Consider the following four RISC machine code fragments each containing two instructions:

   i. addi r1 ← r1, #4 / load r2, 7(r1)
   ii. add r3 ← r1, r2 / store r2, 7(r1)
   iii. breq r1, place / store r1, 7(r1)
   iv. store r3, 17(r10) / load r2, 12(r8)

   a) i. For each code fragment (i) to (iv) identify each dependence that exists or that may exist (a fragment may have no dependencies). [1]

   ii. For each code fragment, indicate whether data forwarding is sufficient to resolve the dependence or if stall cycles are required. Indicate the number of stall cycles. [1]

   b) The 5 stages of the processor have the following latencies: [3]

	Fetch	Decode	Execute	Memory	Writeback
a.	300ps	400ps	350ps	550ps	100ps
b.	200ps	150ps	100ps	190ps	140ps

   What is the cycle time? What is the latency of an instruction? What is the throughput?: For Non-pipelined processor and pipelined processor.

2. The following MIPS program is to be run on a MIPS pipeline processor of form IF-ID-EX-MEM-WB.

   a) Please identify all data dependencies beside each instruction, in the form: <type>on <register> from <line.no> to<line.no> , as WAW on $t6 from L_8 to L_10 [3]

```
L1 lw $t2, 60($t1)
L2 lw $t1, 40($t2)
L3 slt $t1, $t1, $t2
L4 sw $t1, 20($t2)
```

   b) Work out and diagram the optimal pipeline schedule using forwarding from EX or MEM stages to any other stage, then compute the pipeline CPI: [2]

Original past-paper scan ca13.png (re-rendered).

Transcription of the question

```
1. Consider the following four RISC machine-code fragments, each with two instructions:
     i.   addi r1, r1, #4    ; load  r2, 7(r1)
     ii.  add  r3, r1, r2    ; store r2, 7(r1)
     iii. beq  r1, place     ; store r1, 7(r1)
     iv.  store r3, 17(r10)  ; load  r2, 12(r8)
   i.  For each fragment identify each dependence that exists or may exist. [1]
   ii. For each fragment, indicate whether data forwarding is sufficient or stall cycles
       are required; give the number of stall cycles. [1]
   b) The 5 stages of the processor have the following latencies:
        IF    ID    EX    MEM   WB
        300ps 400ps 350ps 550ps 100ps
   What is the cycle time? What is the latency of an instruction? What is the throughput?
   For a non-pipelined processor and a pipelined processor. [3]
2. The following MIPS program runs on a pipeline IF-ID-EX-MEM-WB:
        L1 lw  $t2, 60($t1)
        L2 lw  $t1, 40($t2)
        L3 slt $t1, $t1, $t2
        L4 sw  $t1, 20($t2)
   a) Identify all data dependencies as <type> on <register> from L_x to L_y. [3]
   b) Work out and diagram the optimal pipeline schedule with forwarding, then compute CPI. [2]
```

Full solution

1a — dependences in the four fragments

Fragment	Dependence	Forwarding enough?
i. addi r1 → load r2,7(r1)	RAW on r1 (address)	Yes — 0 stalls
ii. add r3 → store r2,7(r1)	WAW on r2 (both touch r2)	Yes — 0 stalls
iii. beq r1 → store r1	control only; no register RAW	Yes — 0 stalls
iv. store r3 → load r2	possible memory dependence (may exist)	Yes — 0 stalls

1b — cycle time, latency, throughput

```
Non-pipelined: cycle time = sum of stages = 300+400+350+550+100 = 1700 ps latency = 1700 ps ;
throughput = 1/1700 ps Pipelined: cycle time = slowest stage + register overhead = 550 ps +
(register delay, say r) ≈ 550 ps (ignoring r) latency = 5 × 550 = 2750 ps (one instruction
still traverses 5 stages) throughput = 1 instruction / 550 ps
```

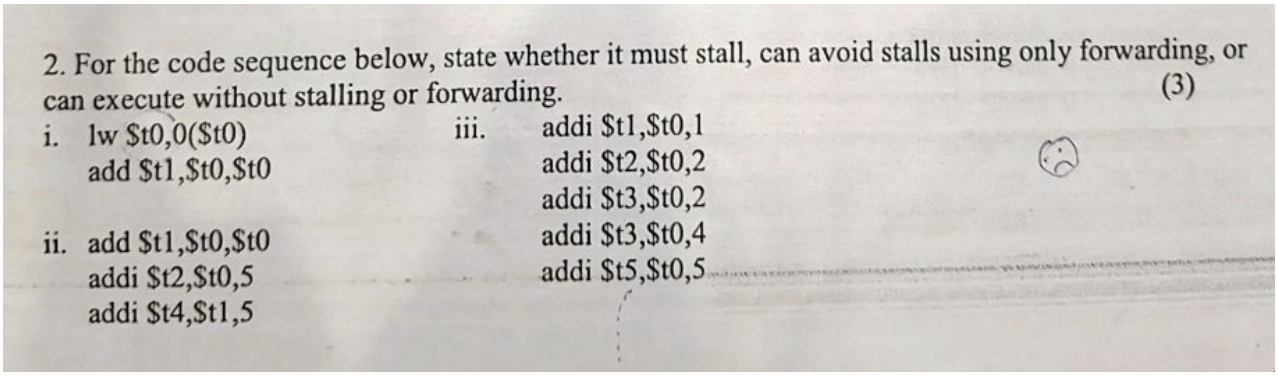
2a — dependences (same program as ca10)

```
RAW on $t2 from L1→L2 ; RAW on $t1 L2→L3 ; RAW on $t2 L1→L3 ; WAW on $t1 L2→L3 ; RAW on $t1
L3→L4 ; RAW on $t2 L1→L4
```

2b — optimal schedule and CPI

```
Same schedule as ca10: 2 load-use stalls over 4 instructions CPI = (4 + 2)/4 = 1.5
```

Previous-year question 16 — ca16 — For each sequence: must stall / forwarding / no sta…


2. For the code sequence below, state whether it must stall, can avoid stalls using only forwarding, or can execute without stalling or forwarding. (3)

i. lw $t0,0($t0)
add $t1,$t0,$t0

ii. add $t1,$t0,$t0
addi $t2,$t0,5
addi $t4,$t1,5

iii. addi $t1,$t0,1
addi $t2,$t0,2
addi $t3,$t0,2
addi $t3,$t0,4
addi $t5,$t0,5

Original past-paper scan ca16.png (re-rendered).

Transcription of the question

```
2. For the code sequence below, state whether it must stall, can avoid stalls using
   only forwarding, or can execute without stalling or forwarding. (1)
   i.   lw   $t0, 0($t0)
        add  $t1, $t0, $t0
        addi $t3, $t0, 2
   ii.  add  $t1, $t0, $t0
        addi $t2, $t0, 5
        addi $t4, $t1, 5
   iii. addi $t1, $t0, 1
        addi $t2, $t0, 2
        addi $t3, $t0, 4
        addi $t5, $t0, 5
```

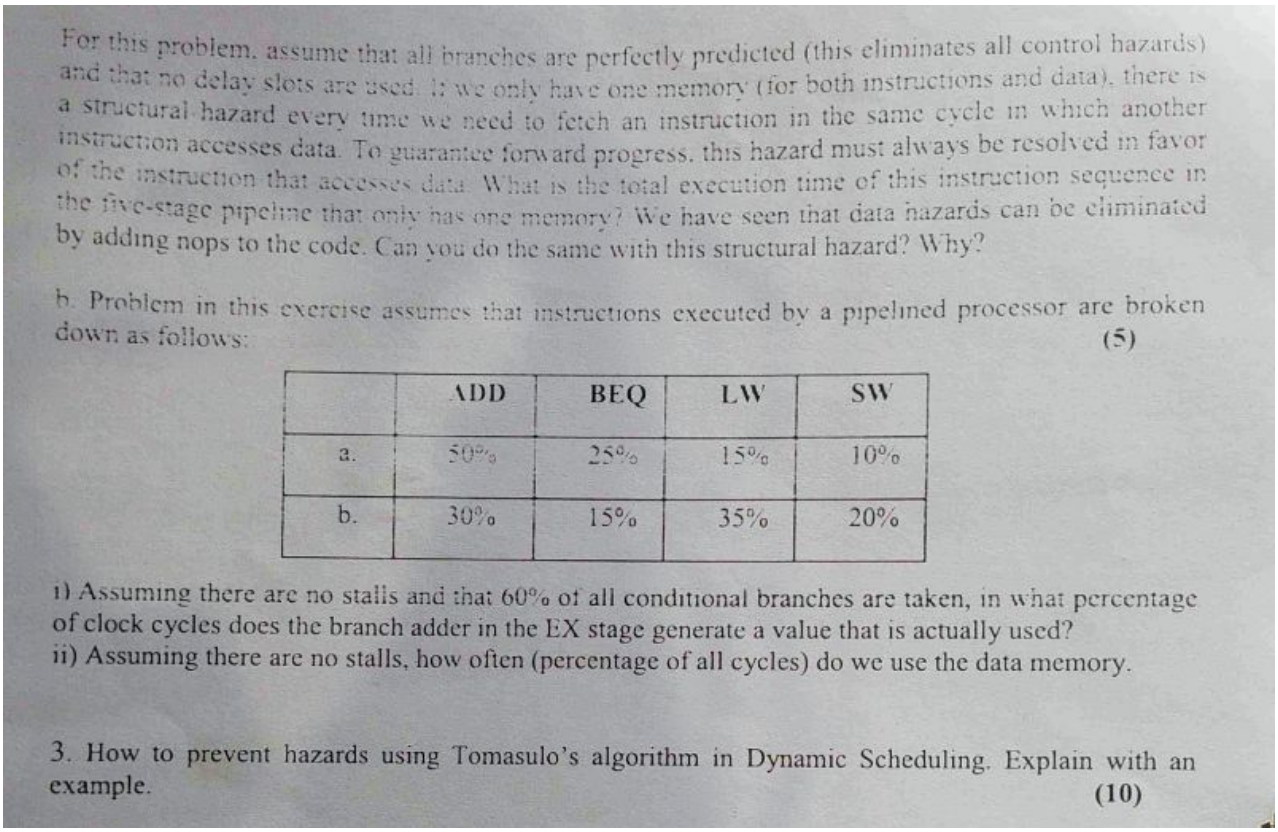
Full solution

Classifying each sequence

Sequence	Verdict	Reason
i. lw → add (use of $t0)	Must stall (1 cycle)	load-use: data ready only after MEM
ii. add → addi (use of $t1)	Forwarding is enough (0 stalls)	ALU result forwarded EX/MEM→EX
iii. independent addis	No stall, no forwarding needed	no dependences at all

```
Rules: load → use : 1 stall even with forwarding ALU → use : 0 stalls with forwarding
independent : 0 stalls, no forwarding needed
```

Previous-year question 21 — ca21 — Single-memory structural hazard; pipeline mix; dy…



For this problem, assume that all branches are perfectly predicted (this eliminates all control hazards) and that no delay slots are used. If we only have one memory (for both instructions and data), there is a structural hazard every time we need to fetch an instruction in the same cycle in which another instruction accesses data. To guarantee forward progress, this hazard must always be resolved in favor of the instruction that accesses data. What is the total execution time of this instruction sequence in the five-stage pipeline that only has one memory? We have seen that data hazards can be eliminated by adding nops to the code. Can you do the same with this structural hazard? Why?

b. Problem in this exercise assumes that instructions executed by a pipelined processor are broken down as follows: (5)

	ADD	BEQ	LW	SW
a.	50%	25%	15%	10%
b.	30%	15%	35%	20%

i) Assuming there are no stalls and that 60% of all conditional branches are taken, in what percentage of clock cycles does the branch adder in the EX stage generate a value that is actually used?
ii) Assuming there are no stalls, how often (percentage of all cycles) do we use the data memory.

3. How to prevent hazards using Tomasulo's algorithm in Dynamic Scheduling. Explain with an example. (10)

Original past-paper scan ca21.png (re-rendered).

Transcription of the question

```
Assume all branches are perfectly predicted (no control hazards). Assume there is one
memory (for both instructions and data), so there is a structural hazard every time we
need to fetch an instruction in the same cycle in which another instruction accesses
data. What is the total execution time of this instruction sequence in the five-stage
pipeline that has only one memory? We have seen that data hazards can be eliminated by
adding nops to the code — can you do the same with this structural hazard? Why?
Also: instructions executed by a pipelined processor are broken down as (ALU 40%,
Load 20%, Store 10%, Branch 30% …). (a) Assuming no stalls and 60% of conditional
branches taken, in what percentage of cycles does the branch adder in EX generate a
value that is actually used? (b) How often do we use the data memory?
(c) How to handle hazards in dynamic scheduling? Explain with an example.
```

Full solution

Single-memory structural hazard

```
With one memory, a load/store in MEM and a fetch in IF conflict → the fetch stalls one cycle
whenever an instruction accesses data. Total time = (number of instructions) + (pipeline
fill) + (data-access stalls).
```

Can nops fix it? No. Nops remove *data* hazards by giving the producer time, but the structural hazard is a *resource* conflict: the two instructions genuinely want the same memory in the same cycle. Adding nops would add more fetches, not fewer. The real fixes are separate instruction and data memories (Harvard style) or a memory that supports two accesses per cycle.

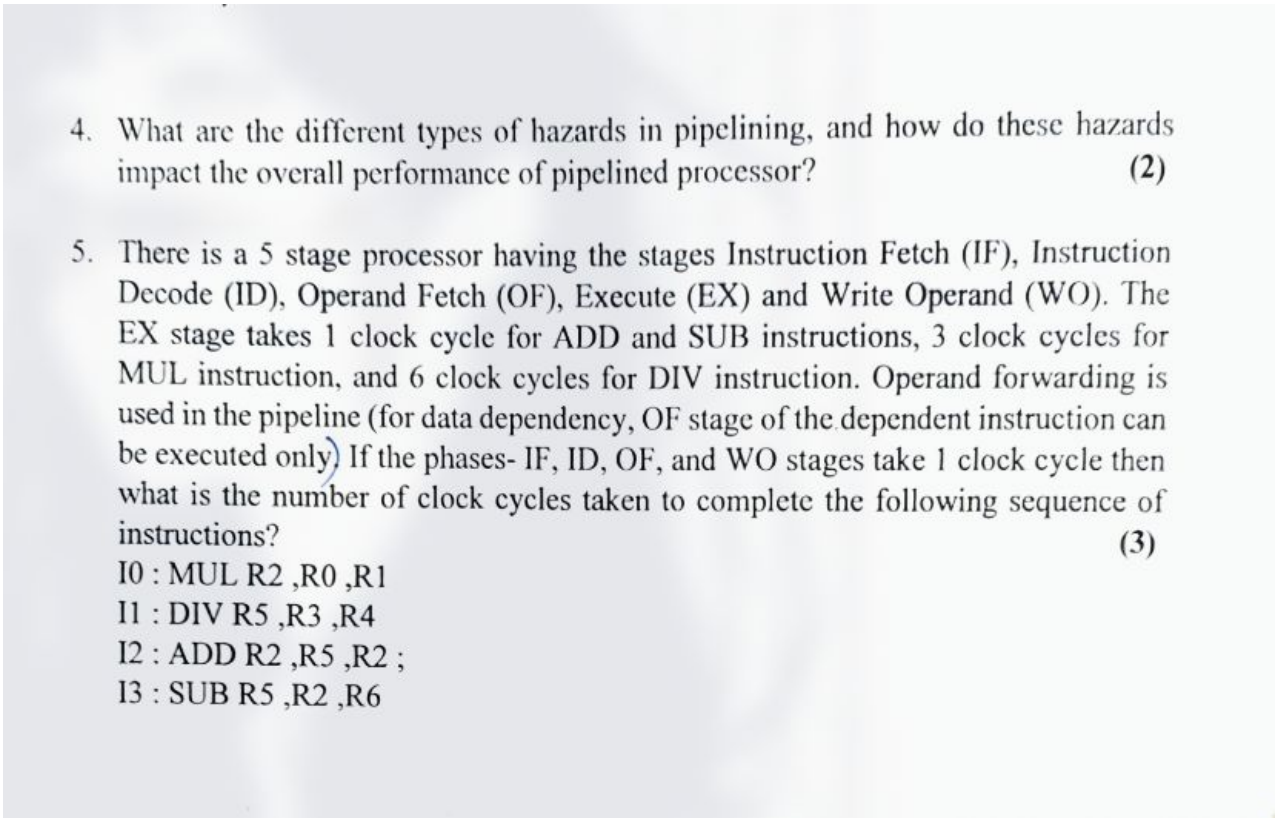
Resource-utilisation percentages

```
(a) Branch adder used only when a branch is taken and in EX: branch fraction × taken fraction
= 0.30 × 0.60 = 18% of cycles (if measured over all instructions, ≈ 0.30 × 0.60 × 1 cycle).
(b) Data memory used by loads and stores in MEM: load 20% + store 10% = 30% of cycles
```

Dynamic scheduling (Tomasulo)

Handle hazards with reservation stations: issue an instruction into a free station (filling Qj/Qk with producer tags), execute when both operands are ready, and broadcast the result on the CDB. This removes WAR/WAW hazards by renaming and lets RAW-dependent instructions wait without blocking independent ones.

Previous-year question 31 — ca31 — Types of hazards and impact; 5-stage cycle count…



4. What are the different types of hazards in pipelining, and how do these hazards impact the overall performance of pipelined processor? **(2)**

5. There is a 5 stage processor having the stages Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Execute (EX) and Write Operand (WO). The EX stage takes 1 clock cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction, and 6 clock cycles for DIV instruction. Operand forwarding is used in the pipeline (for data dependency, OF stage of the dependent instruction can be executed only) If the phases- IF, ID, OF, and WO stages take 1 clock cycle then what is the number of clock cycles taken to complete the following sequence of instructions? **(3)**

I0 : MUL R2 ,R0 ,R1
I1 : DIV R5 ,R3 ,R4
I2 : ADD R2 ,R5 ,R2 ;
I3 : SUB R5 ,R2 ,R6

Original past-paper scan ca31.png (re-rendered).

Transcription of the question

```
4. What are the different types of hazards in pipelining, and how do they impact the
   overall performance of a pipelined processor? (2)
5. A 5-stage processor has stages IF, ID, OF, EX, WO. The EX stage takes 1 clock cycle for
   ADD and SUB, 3 cycles for MUL and 6 cycles for DIV. Operand forwarding is used (for a
   data dependency the OF stage of the dependent instruction can be executed only after
   the producer's result is available). If IF, ID, OF and WO each take 1 clock cycle, how
   many clock cycles are needed to complete:
        I0: MUL R2, R0, R1
        I1: DIV R5, R3, R4
        I2: ADD R2, R5, R2
        I3: SUB R5, R2, R6
```

Full solution

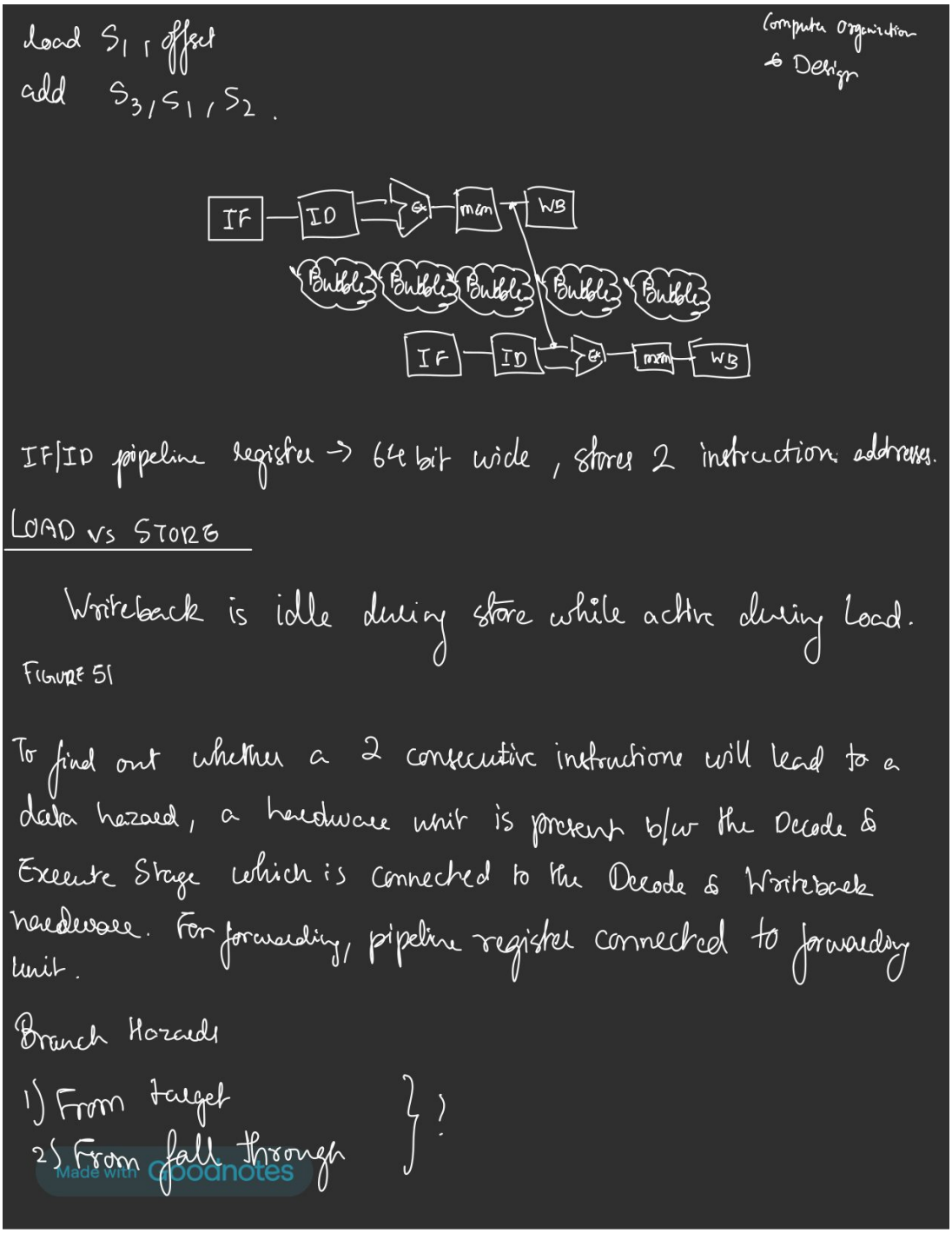
4 — hazard types and impact

Hazard	Cause	Impact
Structural	Two instructions need the same resource	Stall cycles
Data (RAW)	A value is not ready	Stall cycles (1 for load-use)
Control	A branch resolves late	Pipeline flush / bubbles

5 — cycle count

MUL executes in 3 cycles (EX), DIV in 6, ADD/SUB in 1. I2 (ADD R2,R5,R2) depends on I1 (DIV R5) → its OF waits until DIV's result is available (forwarding). I3 (SUB R5,R2,R6) depends on I2. Total ≈ (1+1+1+1+1) pipeline fill for I0 … plus the EX occupancies and the wait for DIV. Work cycle-by-cycle: I0 MUL: IF1 ID2 OF3 EX4-6 WO7 I1 DIV: IF2 ID3 OF4 EX5-10 WO11 I2 ADD: IF3 ID4 OF11 EX12 WO13 (OF waits for I1's result at cycle 11) I3 SUB: IF4 ID5 OF13 EX14 WO15 (OF waits for I2's result at cycle 13) → about 15 clock cycles to complete all four.

Original notes — page 36: Forwarding, load vs store handling, and the compiler's role i…



Original handwritten notes — page 36.

Transcription of the page

```
Compute-time optimisation:
  Reorder the code to fill the load delay slot.

LOOP:
  load → use  (dependency)

The pipeline register is too wide — try to avoid the load-delay slot.
  (Hardware forwarding helps; the compiler can also schedule.)

LOAD vs STORE:
  A load must complete before its value is used.
  A store can proceed without waiting (it does not produce a register value).

Instruction scheduling:
  The compiler reorders instructions to fill delay slots with useful work.
```

Explanation

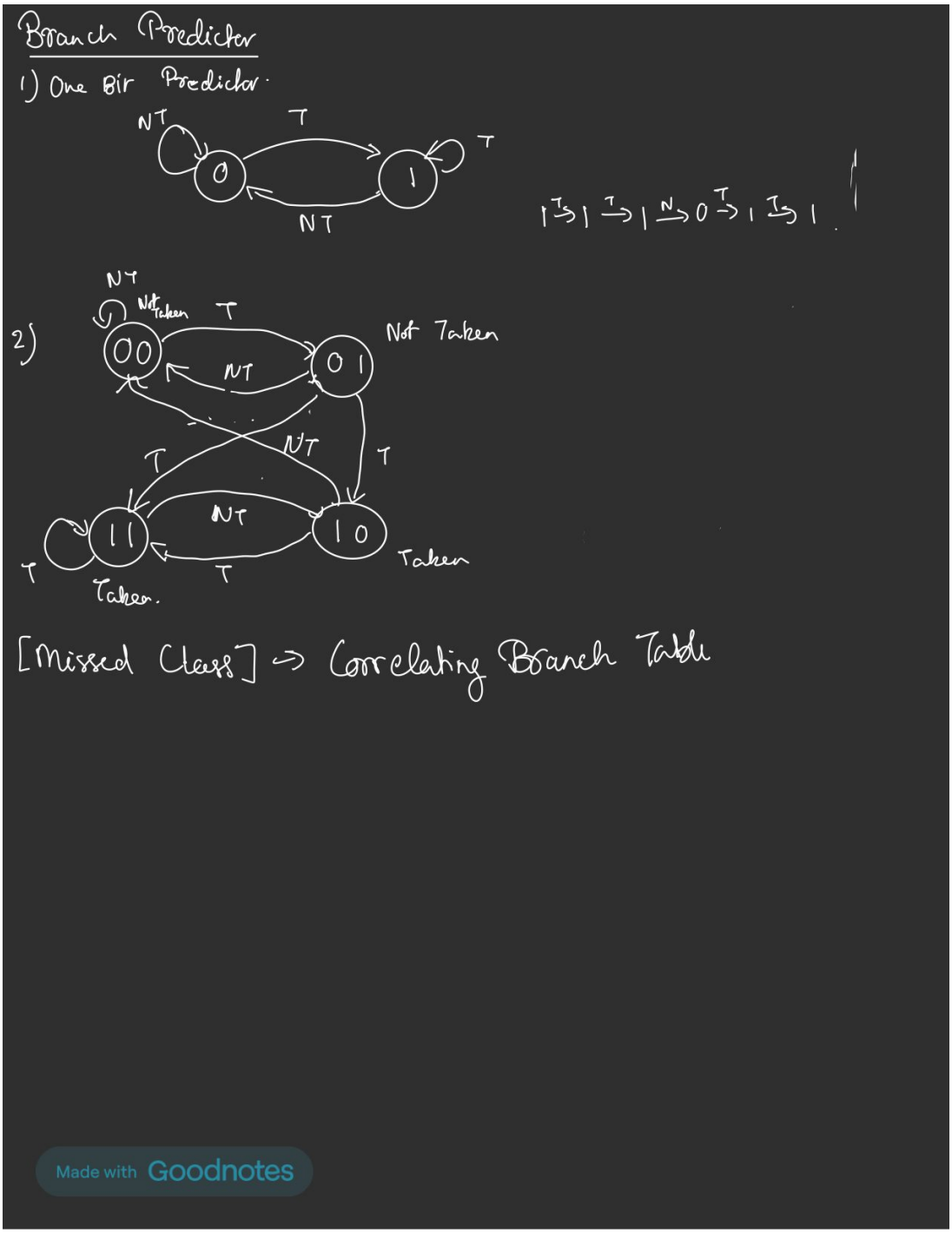
What the page says. Two ways to hide the load delay: hardware forwarding and software scheduling by the compiler. It also distinguishes loads from stores.

- **Hardware forwarding** removes most stalls by routing a result as soon as it exists, but the load-use case still needs one stall because the data arrives late.
- **Compiler scheduling** reorders independent instructions so that useful work fills the delay slot — the "load delay slot" is occupied by an instruction that does not depend on the load.
- **Load vs store.** A load produces a register value, so a following use must wait for it. A store writes memory and produces no register value, so following instructions generally do not wait for it (it can be buffered).

```
Load-delay slot example: lw $t0, 0($s0) ; t0 available after MEM add $t1, $t2, $t3 ;
independent — fill the slot add $t4, $t0, $t5 ; now uses t0 with no stall
```

Who fixes what. Hardware forwarding is free but incomplete (load-use remains). Compiler scheduling is flexible but limited by the available independent instructions. The two are complementary.

Original notes — page 37: Branch prediction — the one-bit predictor



Original handwritten notes — page 37.

Transcription of the page

```
1) One-Bit Predictor

  State machine:  (Taken) ⟷ (Not Taken)
      predict T        predict N

  "T"/"NT" stored per branch.

  Misclassification: a branch that changes direction costs a mispredict.

  Accuracy: mispredicts on every change of direction (e.g. a loop exit).

  Mixed class → easier branch behaviour.
```

Explanation

What the page says. The simplest dynamic branch predictor: one bit per branch that records the direction taken last time.

How it works

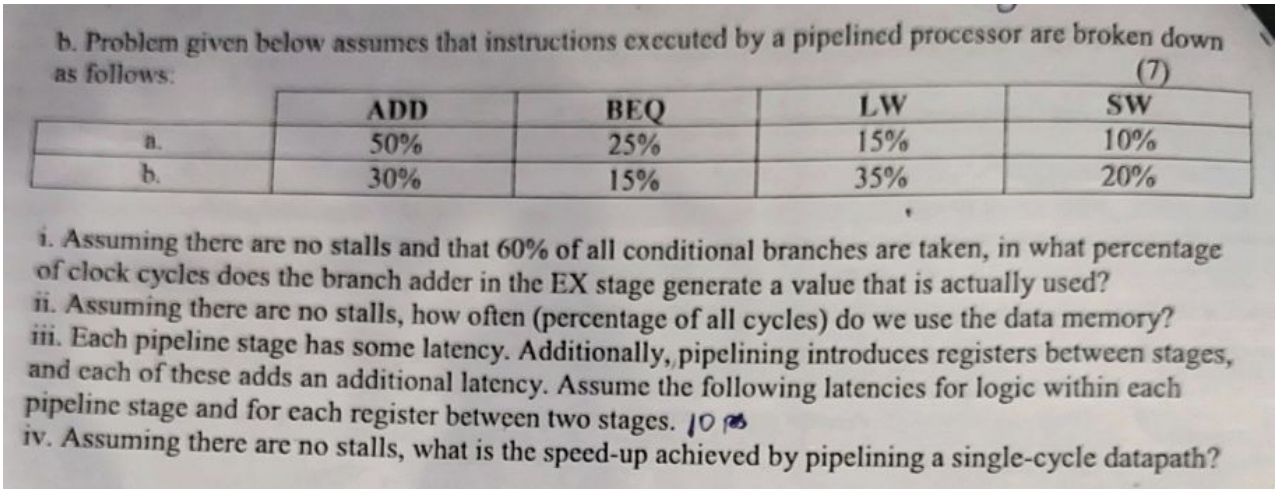
- Keep one bit per branch (in a small table indexed by the branch address).
- After the branch executes, set the bit to Taken (T) or Not-Taken (NT) according to what actually happened.
- Next time, predict whatever the bit says.

```
One-bit predictor state machine (two states): predict T actual T [Not Taken] ──────────────→ [Taken] ↑ | └──── actual NT ──────────┘ prediction = the state; mispredict whenever the direction changes.
```

Weakness. A loop that runs 100 times taken and once not-taken will mispredict *twice* per execution: once when it exits (predict T, actual NT) and once when it is entered again (predict NT, actual T). A 2-bit predictor fixes this.

```
Misprediction examples: Loop taken 100×, then not taken: one-bit predictor → 2 mispredicts per loop execution. Accuracy = correct / total.
```

Previous-year question 17 — ca17 — Pipeline instruction mix: branch-adder use, data-…



b. Problem given below assumes that instructions executed by a pipelined processor are broken down as follows: (7)

	ADD	BEQ	LW	SW
a.	50%	25%	15%	10%
b.	30%	15%	35%	20%

i. Assuming there are no stalls and that 60% of all conditional branches are taken, in what percentage of clock cycles does the branch adder in the EX stage generate a value that is actually used?
ii. Assuming there are no stalls, how often (percentage of all cycles) do we use the data memory?
iii. Each pipeline stage has some latency. Additionally, pipelining introduces registers between stages, and each of these adds an additional latency. Assume the following latencies for logic within each pipeline stage and for each register between two stages. (10 ✍)
iv. Assuming there are no stalls, what is the speed-up achieved by pipelining a single-cycle datapath?

Original past-paper scan ca17.png (re-rendered).

Transcription of the question

```
b. Instructions executed by a pipelined processor are broken down as follows
   (ALU / Load / Store / Branch percentages given).
   i.  Assuming there are no stalls and that 60% of all conditional branches are taken,
       in what percentage of clock cycles does the branch adder in the EX stage generate
       a value that is actually used?
   ii. Assuming there are no stalls, how often (percentage of all cycles) do we use the
       data memory?
   iii. Each pipeline stage has some latency; pipelining introduces registers between
        stages, each adding latency. Assuming no stalls, what is the speed-up achieved by
        pipelining a single-cycle datapath?
```

Full solution

i — branch adder utilisation

```
Branch adder produces a value only when a branch is taken (in EX): utilisation = branch_fraction × taken_fraction × (cycles in EX) = 0.30 × 0.60 × 1 = 0.18 = 18% of cycles.
```

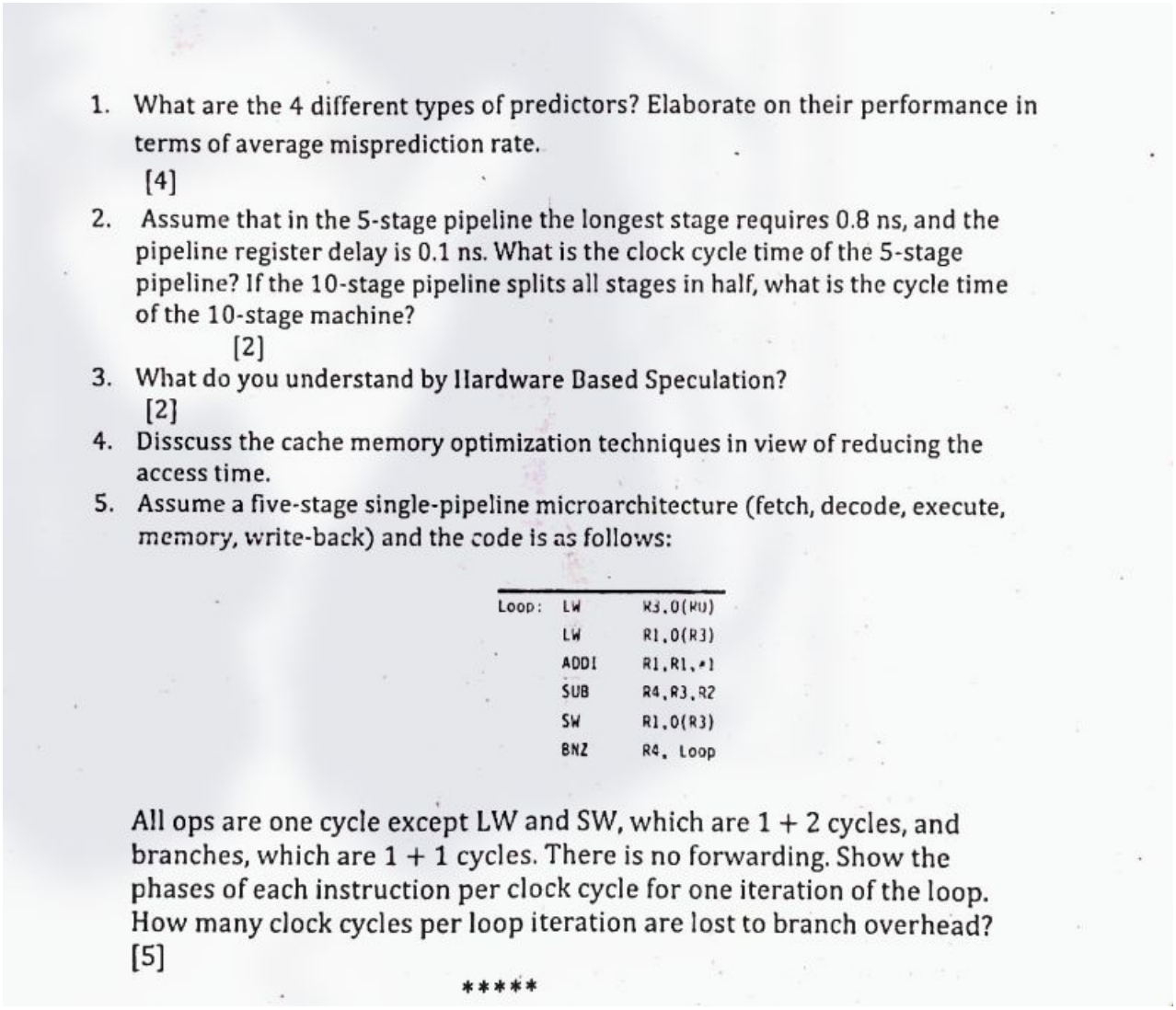
ii — data-memory utilisation

```
Data memory is used by loads and stores in MEM: utilisation = (load_fraction + store_fraction) × 1 = 0.20 + 0.10 = 30%.
```

iii — pipelining speed-up

```
S = (n · k) / (n + k − 1) → k as n → ∞ With register overhead, the clock period grows, so the realised speed-up is slightly below k.
```

Previous-year question 32 — ca32 — Four predictors; 5-stage vs 10-stage clock; specul…



1. What are the 4 different types of predictors? Elaborate on their performance in terms of average misprediction rate.
   [4]
2. Assume that in the 5-stage pipeline the longest stage requires 0.8 ns, and the pipeline register delay is 0.1 ns. What is the clock cycle time of the 5-stage pipeline? If the 10-stage pipeline splits all stages in half, what is the cycle time of the 10-stage machine?
   [2]
3. What do you understand by Hardware Based Speculation?
   [2]
4. Disscuss the cache memory optimization techniques in view of reducing the access time.
5. Assume a five-stage single-pipeline microarchitecture (fetch, decode, execute, memory, write-back) and the code is as follows:

```
Loop:  LW     R3,0(R0)
       LW     R1,0(R3)
       ADDI   R1,R1,#1
       SUB    R4,R3,R2
       SW     R1,0(R3)
       BNZ    R4, Loop
```

All ops are one cycle except LW and SW, which are 1 + 2 cycles, and branches, which are 1 + 1 cycles. There is no forwarding. Show the phases of each instruction per clock cycle for one iteration of the loop. How many clock cycles per loop iteration are lost to branch overhead?
[5]

*****

Original past-paper scan ca32.png (re-rendered).

Transcription of the question

```
1. What are the 4 different types of predictors? Elaborate on their performance in
   terms of average misprediction rate. (4)
2. In the 5-stage pipeline the longest stage requires 0.8 ns and the pipeline register
   delay is 0.1 ns. What is the clock cycle time of the 5-stage pipeline? If the 10-stage
   pipeline splits all stages in half, what is the cycle time of the 10-stage machine? (2)
3. What do you understand by hardware-based speculation? (2)
4. Discuss the cache-memory optimisation techniques with a view to reducing the access time.
5. Five-stage pipeline; all ops one cycle except LW/SW (1+2) and branches (1+1); no
   forwarding. Show the phases per clock for one loop iteration; how many cycles are lost
   to branch overhead? (5)
```

Full solution

1 — four predictors

Predictor	Mispredicts when	Approx. rate
Static (taken/not)	Pattern changes	worst
1-bit	Direction changes	high
2-bit saturating	Two changes in a row	moderate
Correlating (2-level)	Unseen history pattern	lowest

2 — clock cycle time

```
5-stage: cycle time = longest stage + register delay = 0.8 + 0.1 = 0.9 ns 10-stage (stages halved): longest stage = 0.4 ns, register delay = 0.1 ns → cycle time = 0.4 + 0.1 = 0.5 ns
```

3 — hardware-based speculation

The hardware executes instructions before it is certain they should execute (e.g. past a predicted branch), keeping the results in a reorder buffer and committing them in order; a wrong guess is squashed.

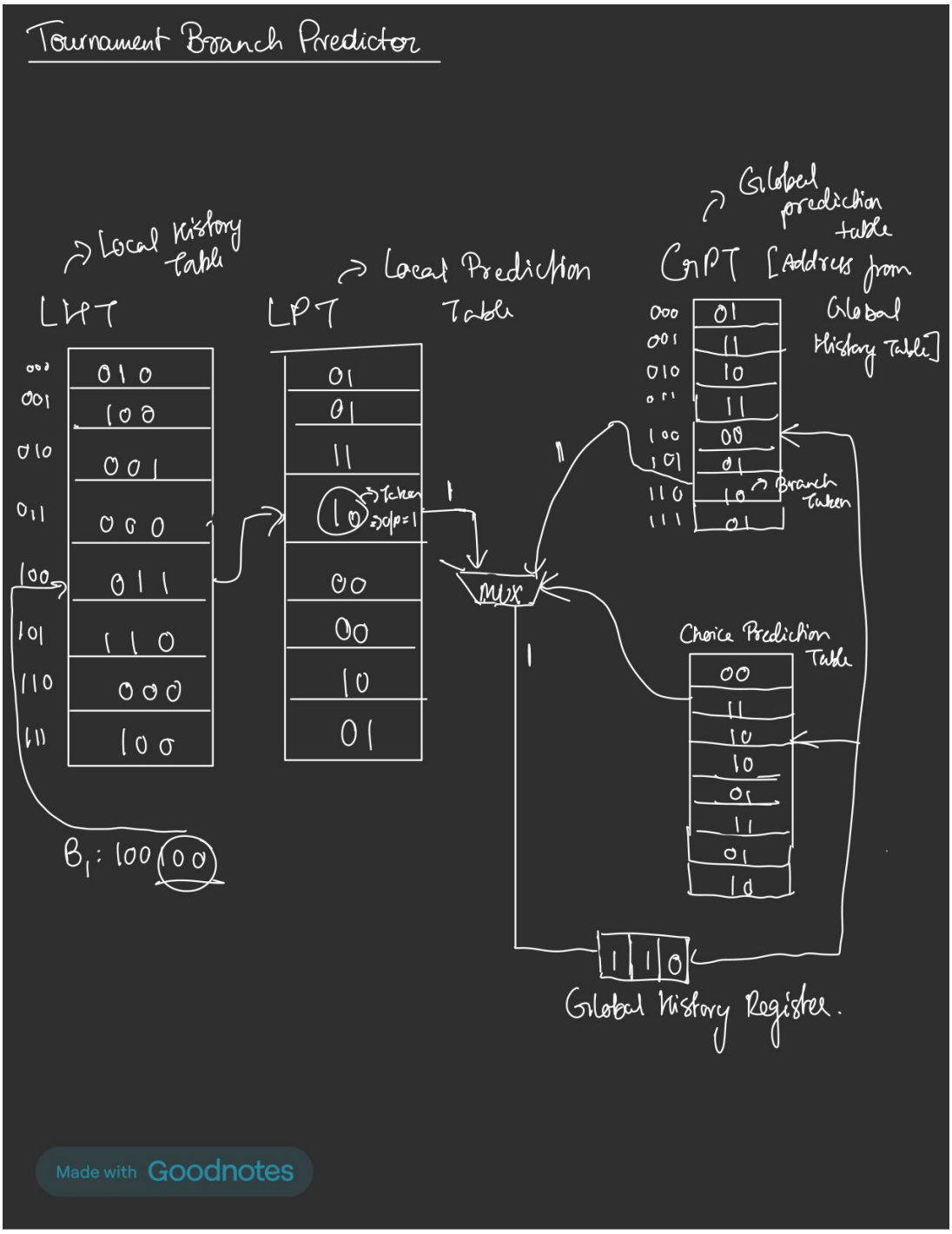
4 — cache optimisation to reduce access time

```
AMAT = hit time + miss rate × miss penalty reduce hit time: smaller/faster cache, simpler addressing, avoid TLB delay.
```

5 — phases and branch overhead

```
No forwarding → 2 stalls per RAW pair; the branch resolves late so the instructions fetched behind it are flushed (branch overhead ≈ 2 cycles).
```

Original notes — page 38: Tournament (hybrid) branch predictor



Original handwritten notes — page 38.

Transcription of the page

```
Tournament Branch Predictor

  Uses two-level prediction — combining a local and a global predictor.

  Local predictor:  (history of THIS branch)
  Global predictor: (history of RECENT branches)

  A chooser (selector) picks whichever predictor has been more accurate.

  Diagram: two predictor tables (local, global) feed a multiplexer whose
  select line comes from a small “chooser” counter table.

  Global history register: a shift register of the last k branch outcomes.
```

Explanation

What the page says. A **tournament** (hybrid) predictor runs two predictors at once and uses a chooser to decide which to believe.

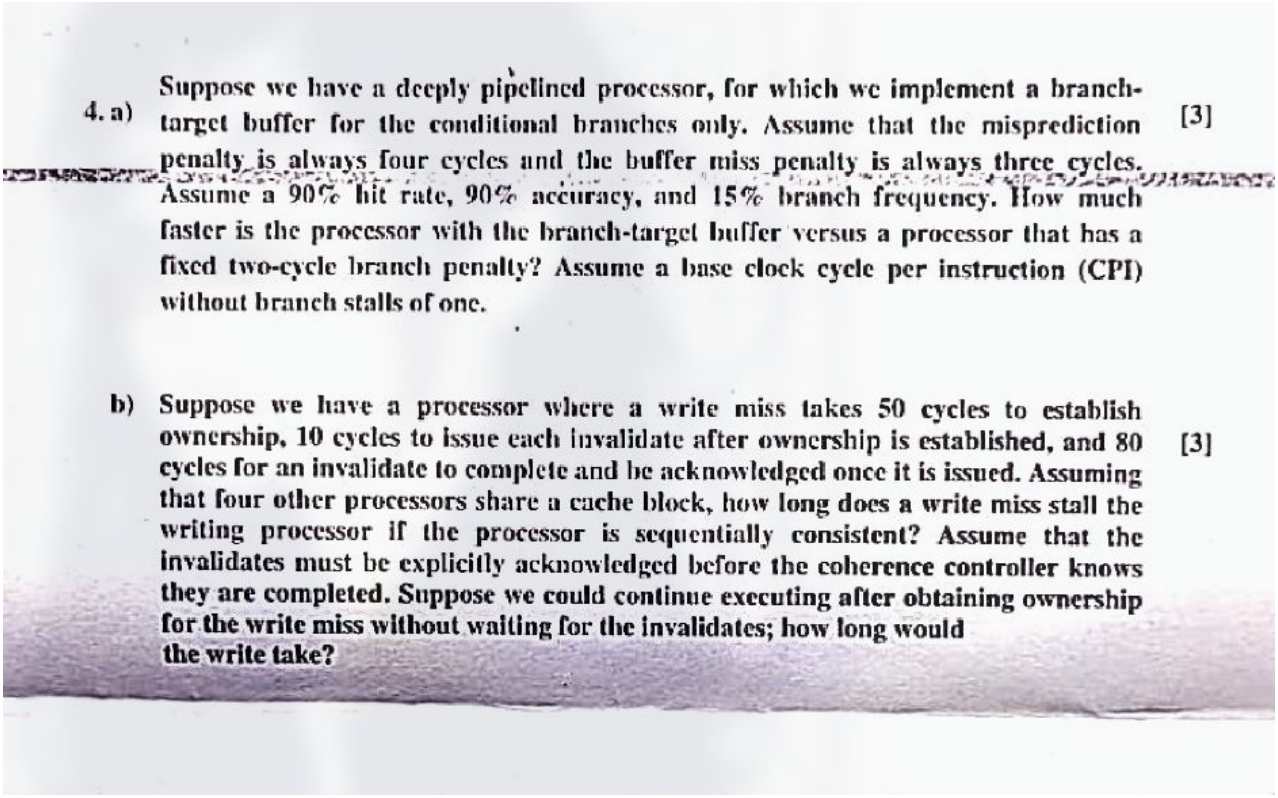
- **Local predictor** — remembers the history of *this particular* branch (useful for branches with their own repeating pattern).
- **Global predictor** — remembers the pattern of the *most recent branches* overall (useful when branches are correlated with each other).
- **Chooser** — a small table of 2-bit counters, one per branch, that records which of the two predictors has been right more often and selects it.

```
Global history register (GHR): k bits, shifted after each branch e.g. history T T N T → 1101
This pattern indexes a table of counters (a two-level predictor). Prediction =
chooser_counter ? global_pred : local_pred
```

Reconstructed diagram. Draw two boxes side by side — "Local predictor" and "Global predictor" — whose outputs go into a 2:1 multiplexer. The multiplexer's select comes from a third small table, the "chooser", indexed by the branch address. The global predictor is fed by a shift register (the GHR).

> **Why it wins.** Neither predictor alone is best for every branch; the chooser learns per branch which one to trust, so the combined accuracy is higher than either alone. Real CPUs (e.g. Intel, AMD) use tournament-style predictors.

Previous-year question 8 — ca8 — Branch-target-buffer penalty vs fixed penalty; write-…



4. a) Suppose we have a deeply pipelined processor, for which we implement a branch-target buffer for the conditional branches only. Assume that the misprediction penalty is always four cycles and the buffer miss penalty is always three cycles. Assume a 90% hit rate, 90% accuracy, and 15% branch frequency. How much faster is the processor with the branch-target buffer versus a processor that has a fixed two-cycle branch penalty? Assume a base clock cycle per instruction (CPI) without branch stalls of one. [3]

b) Suppose we have a processor where a write miss takes 50 cycles to establish ownership, 10 cycles to issue each invalidate after ownership is established, and 80 cycles for an invalidate to complete and be acknowledged once it is issued. Assuming that four other processors share a cache block, how long does a write miss stall the writing processor if the processor is sequentially consistent? Assume that the invalidates must be explicitly acknowledged before the coherence controller knows they are completed. Suppose we could continue executing after obtaining ownership for the write miss without waiting for the invalidates; how long would the write take? [3]

Original past-paper scan ca8.png (re-rendered).

Transcription of the question

```
4. a) Suppose we have a deeply pipelined processor for which we implement a branch-target
      buffer for the conditional branches only. Assume that the misprediction penalty is
      always four cycles and the buffer-miss penalty is always three cycles. Assume a 90%
      hit rate, 90% accuracy and 18% branch frequency. How much faster is the processor
      with the branch-target buffer versus a processor that has a fixed two-cycle branch
      penalty? Assume a base CPI without branch stalls of one. [3]
   b) Suppose we have a processor where a write miss takes 50 cycles to establish
      ownership, 10 cycles to issue each invalidate after ownership is established, and
      80 cycles for an invalidate to complete and be acknowledged once it is issued.
      Assuming four other processors share a cache block, how long does a write miss
      stall the writing processor if the processor is sequentially consistent? Assume
      the invalidates must be explicitly acknowledged before the coherence controller
      knows they are completed. Suppose we could continue executing after obtaining
      ownership for the write miss without waiting for the invalidates; how long would
      the write take? [3]
```

Full solution

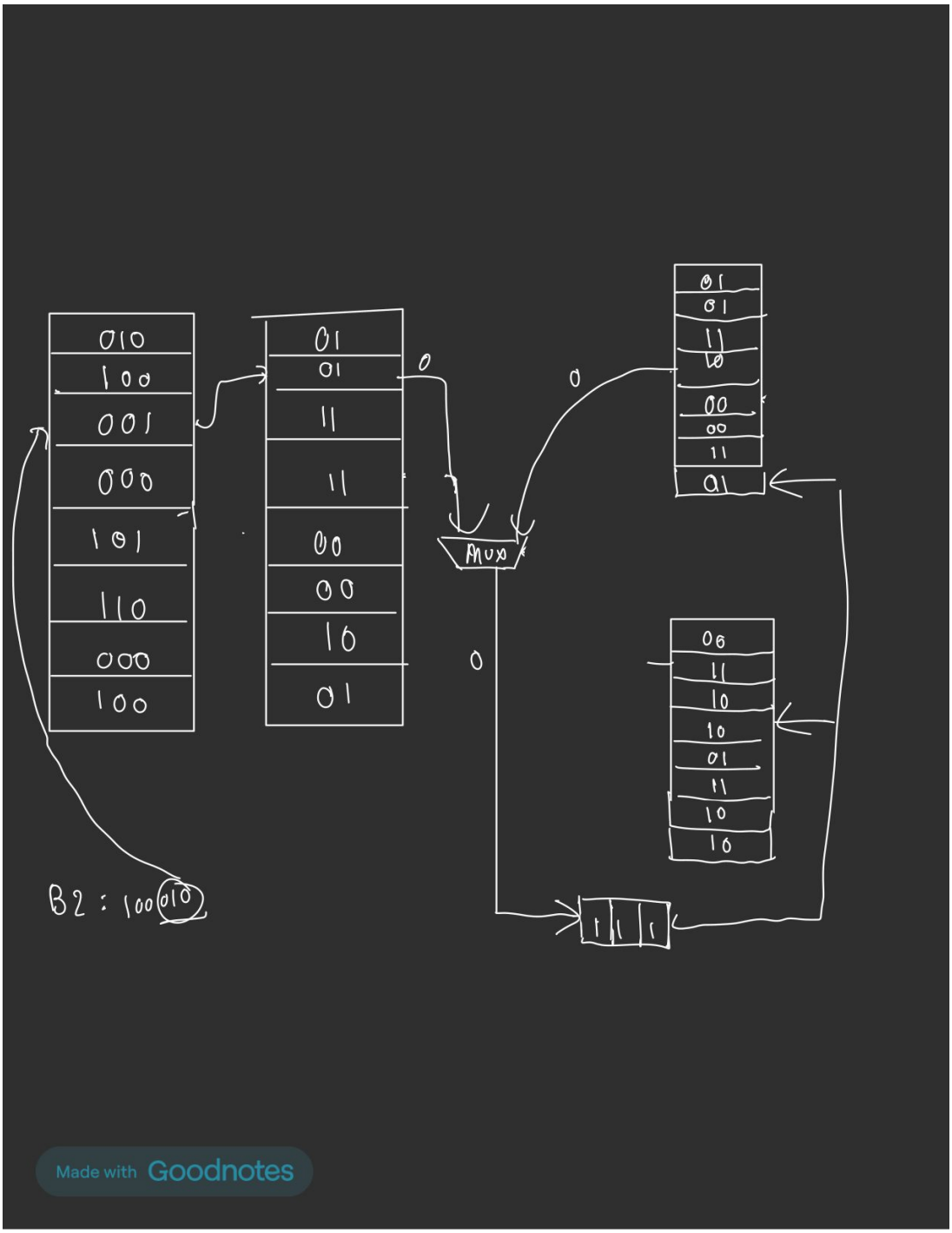
4a — BTB versus a fixed 2-cycle branch penalty

```
BTB penalty per branch = buffer_miss_rate × miss_penalty + hit_rate × (1 − accuracy) × mispredict_penalty = 0.10 × 3 + 0.90 × 0.10 × 4 = 0.30 + 0.36 = 0.66 cycles/branch With 18% branches: extra CPI = 0.18 × 0.66 = 0.119 CPI_BTB = 1 + 0.119 = 1.119 Fixed penalty: extra CPI = 0.18 × 2 = 0.36 CPI_fixed = 1.36 Speed-up = 1.36 / 1.119 ≈ 1.215× (≈ 21% faster)
```

4b — write-miss stall with ownership + invalidates

```
Sequential consistency (invalidates acknowledged before proceeding): stall = ownership + (issue each invalidate) + (last invalidate completes) = 50 + 4×10 + 80 = 170 cycles If we may continue after ownership without waiting for invalidates: write takes = 50 cycles (the invalidations proceed in the background)
```

Original notes — page 39: One-bit predictor state machine (worked table)



Original handwritten notes — page 39.

Transcription of the page

```
One-bit predictor:

  current state   actual   next state   correct?
      T             T         T            yes
      T             N         N            no
      N             T         T            no
      N             N         N            yes

  Sequence of outcomes and the corresponding predictions/updates
  (a table of T/N and hit/miss).

  Prediction = previous outcome. Accuracy = fraction correct.
```

Explanation

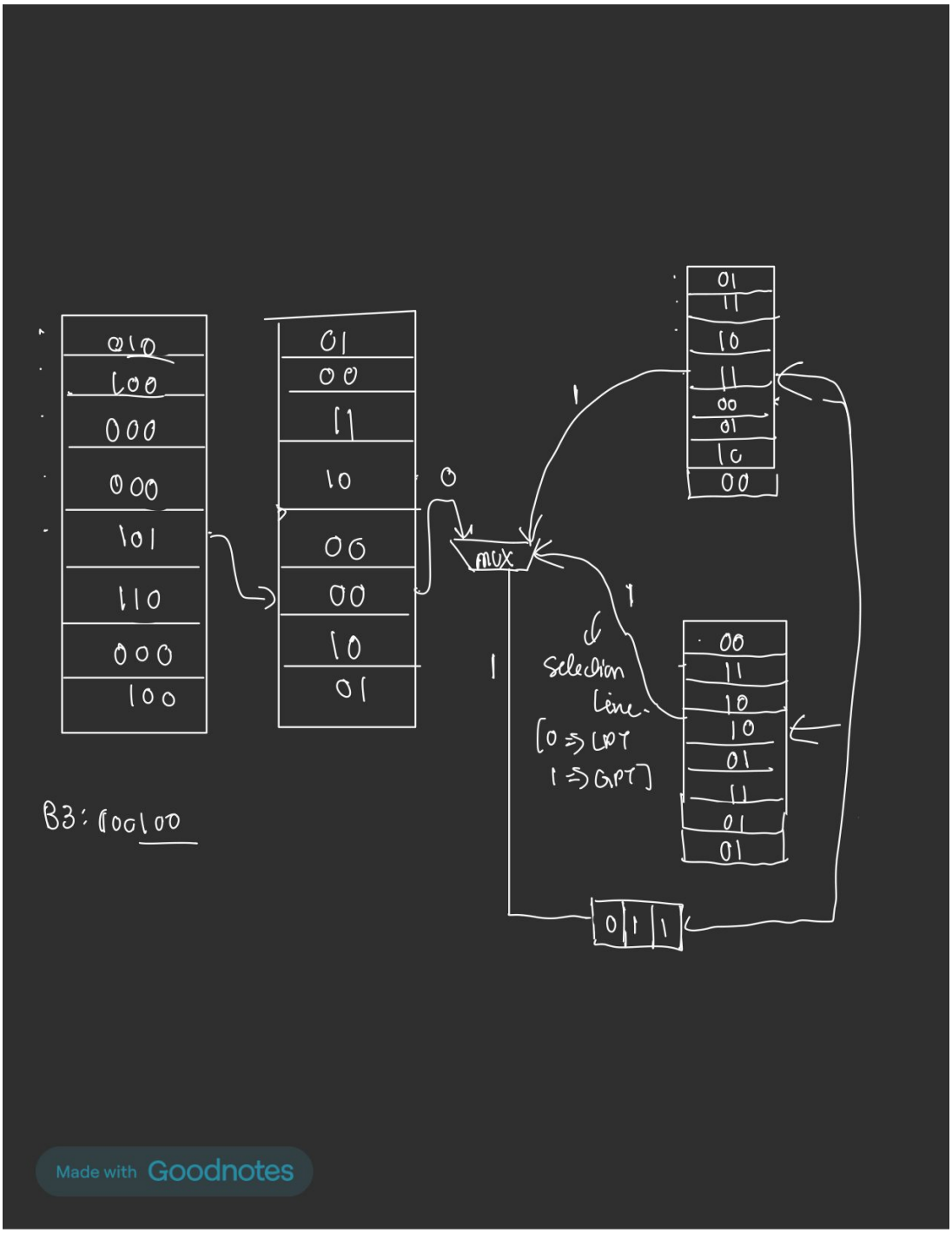
What the page says. The one-bit predictor written as a truth table: the next state is simply the actual outcome, and the prediction is correct only when the outcome repeats.

Current state	Actual	Next state	Correct?
T (Taken)	T	T	yes
T (Taken)	N	N	no
N (Not taken)	T	T	no
N (Not taken)	N	N	yes

```
State = last outcome. Prediction = state. A mispredict happens exactly when the branch changes direction. For a loop taken m times then not taken: mispredicts per pass = 2 (entry and exit).
```

Worked sequence. Take the outcome string N T T T T N T … The predictor starts in some state; after each branch it copies the outcome into the state, so it predicts the *previous* outcome. Every change from T to N or N to T costs one misprediction.

Original notes — page 40: Two-bit saturating counter predictor (state machine)



Original handwritten notes — page 40.

Transcription of the page

```
Two-bit saturating counter:

  States:  00 (strongly NT)  01 (weakly NT)
           10 (weakly T)      11 (strongly T)

  Transitions:
    actual T:  00→01, 01→10, 10→11, 11→11
    actual N:  11→10, 10→01, 01→00, 00→00

  Predict taken if the high bit is 1 (states 10, 11).

  A single change of direction does NOT flip the prediction
  (it must saturate first) → fewer mispredicts on loop exits.
```

Explanation

What the page says. The 2-bit saturating counter: a small state machine that needs *two* wrong guesses in a row before it changes its mind, which fixes the loop-exit problem of the one-bit predictor.

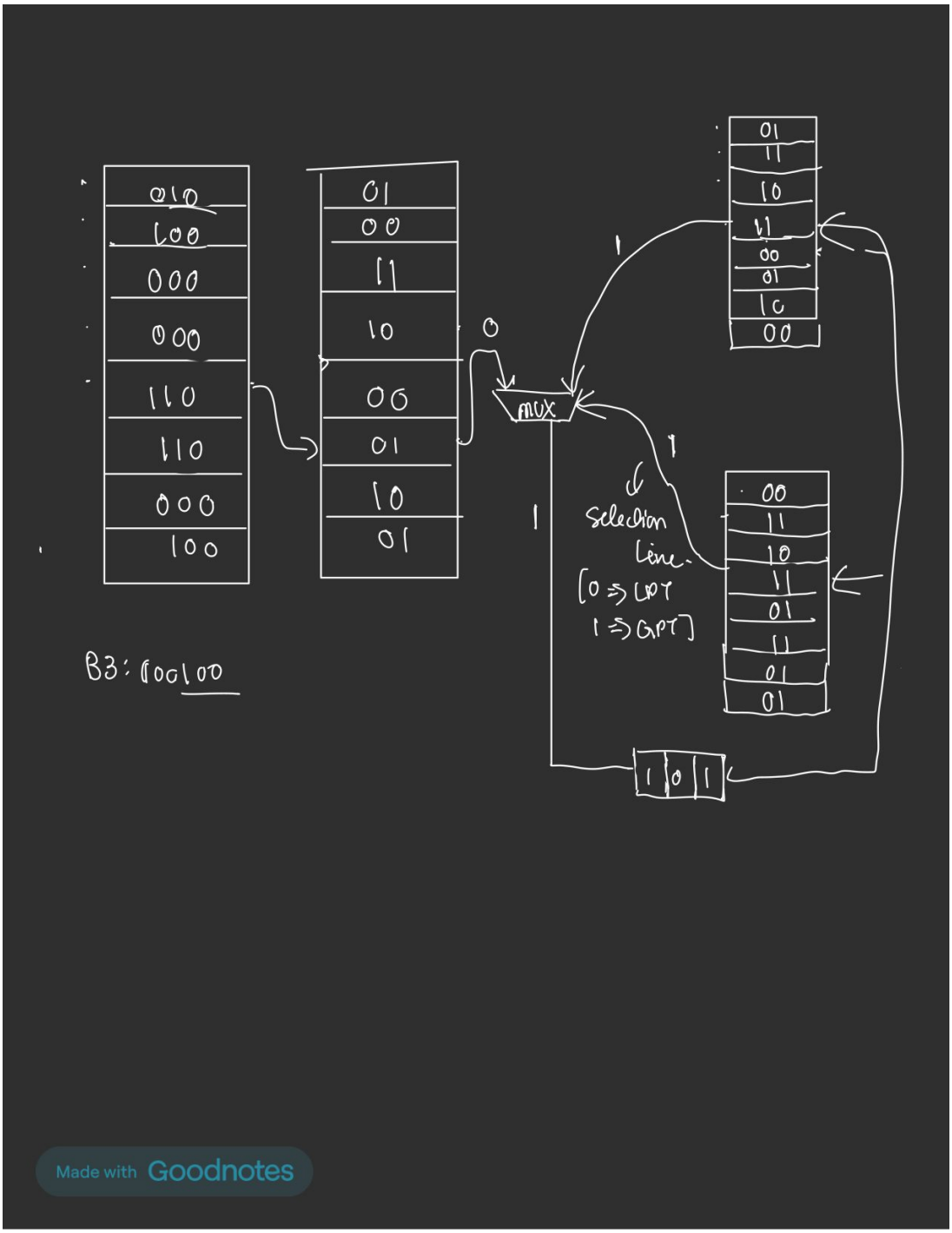
```
2-bit saturating counter state machine: 00 (strong NT) --T--> 01 (weak NT) --T--> 10 (weak T) --T--> 11 (strong T) 11 (strong T) --N--> 10 (weak T) --N--> 01 (weak NT) --N--> 00 (strong NT) Prediction: TAKEN if state ▫ {10, 11}; NOT TAKEN if state ▫ {00, 01} The counter saturates at 00 and 11 (it cannot wrap around).
```

State	Meaning	Predicts
00	strongly not taken	Not taken
01	weakly not taken	Not taken
10	weakly taken	Taken
11	strongly taken	Taken

Why it is better. A loop that runs 100 times taken then once not-taken: the counter sits in state 11; the exit takes it to 10, which still predicts Taken — but the next (first) iteration is taken, so it moves back to 11 and no mispredict occurs on re-entry. Only the exit and the re-entry are "weak", so the 2-bit predictor mispredicts roughly once per loop execution instead of twice.

```
One-bit: 2 mispredicts per loop pass. Two-bit: about 1 mispredict per loop pass (only the exit).
```

Original notes — page 41: Two-bit / correlating predictor (worked table and scheduling…



Original handwritten notes — page 41.

Transcription of the page

```
Two-bit / correlating predictor:

  A table indexed by (branch history, branch address) holding 2-bit counters.

  Scheduling note: the predictor reduces the branch penalty so that
  scheduling (filling delay slots) becomes easier.

  Worked table of (history → prediction) and hit/miss counts.

  Accuracy improves because the prediction depends on the pattern of
  recent branches, not just this branch.
```

Explanation

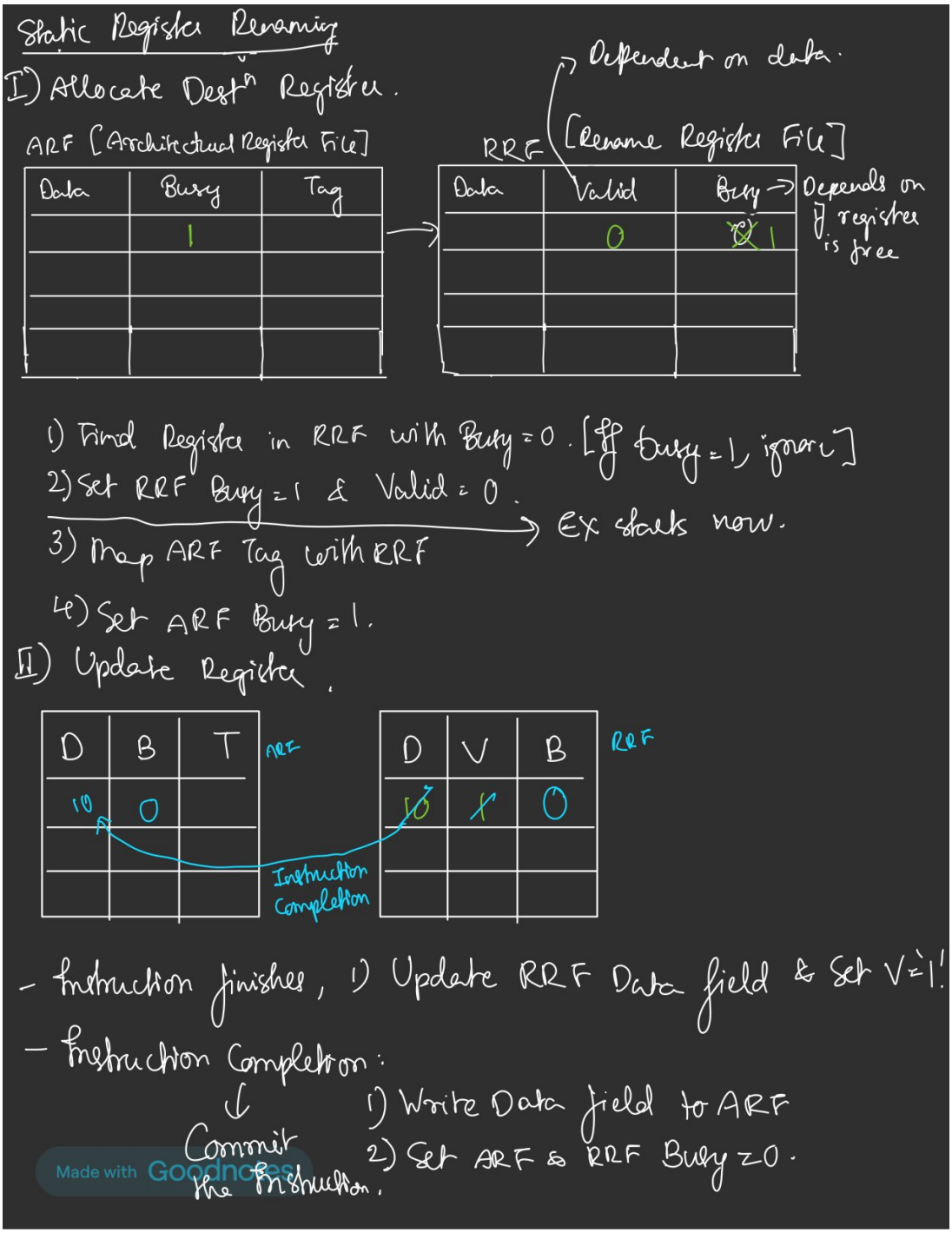
What the page says. The correlating (two-level) predictor: instead of one counter per branch, index a table of counters with the *history* of recent branches, so the prediction captures patterns such as “if the last two branches were taken, this one usually is too”.

```
Two-level (correlating) predictor: index = (global history register, branch address)
table[index] = 2-bit counter (as on the previous page) Example history pattern T T N T → a
4-bit index. This distinguishes branch patterns that a single counter cannot.
```

Reconstructed diagram. Draw a shift register (the global history register) at the top that stores the last k outcomes. Its bits, concatenated with the branch address, form the index into a table of 2-bit counters. The selected counter gives the prediction.

> **Link to scheduling.** A better predictor means fewer control-hazard stalls, which makes the compiler’s job of filling delay slots easier — the page notes this connection explicitly.

Original notes — page 42: Register renaming with a reorder buffer (data dependence re…



Original handwritten notes — page 42.

Transcription of the page

```
Handling register renaming depends on data:
  1) Allocate an architectural register?
  2) Rename: the architectural register → a physical register (Reorder Buffer entry).
  → In the ROB, the entry holds the value; the register map points to it.

  Register file: architectural register numbers → physical register numbers.
  → Instruction writes to the ROB; on commit, the value is copied to the
    architectural register file.

  Instruction issue → update RAT (register alias table)
  Instruction completion → write to ROB
  (Commit) → write to ARF (architectural register file)
```

Explanation

What the page says. Register renaming removes false (name) dependences by giving each new value its own physical register. A **reorder buffer (ROB)** holds the speculative values until the instruction is certain to execute.

- **Architectural registers** — the ones the programmer sees (e.g. $t0, F0). There are only a few of them, so different instructions reuse the same names.
- **Physical registers** — a much larger pool. Renaming maps each architectural register to a physical one, so two instructions that use the same name no longer conflict.
- **Reorder buffer (ROB)** — a circular queue holding instructions in program order; each entry stores the destination value until the instruction commits.
- **Register alias table (RAT)** — the map from architectural to physical register, updated as instructions issue.
- **Commit** — when an instruction is the oldest and has no exceptions, its value is copied from the ROB into the architectural register file.

```
Rename removes WAR and WAW (name) dependences; RAW (true) dependences remain — they carry
real data. Flow: Issue → update RAT ; Execute → produce value ; Write → store value in ROB ;
Commit → copy ROB value to ARF
```

Reconstructed diagram. Draw the architectural register file (ARF) on the left, the physical registers / ROB on the right, and the RAT as a table in the middle mapping each architectural name to a physical name.

Previous-year question 34 — ca34 — OOO advantages; pipeline phases with a static pr…

1. a. State the advantages of OOO pipelines over In-order pipelines.

b. Assume a five-stage single-pipeline microarchitecture (fetch, decode, execute, memory, write-back) and the code in the given Figure 1. All ops are one cycle except LW and SW, which are 1 + 2 cycles, and branches, which are 1 + 1 cycles. There is no forwarding. Show the phases of each instruction per clock cycle for one iteration of the loop.

```
Loop:  LW    R3,0(R0)
       LW    R1,0(R3)
       ADDI  R1,R1,#1
       SUB   R4,R3,R2
       SW    R1,0(R3)
       BNZ   R4, Loop
```

Figure 1

a. How many clock cycles per loop iteration are lost to branch overhead?

b. Assume a static branch predictor, capable of recognizing a backwards branch in the Decode stage. Now how many clock cycles are wasted on branch overhead?

c. Assume a dynamic branch predictor. How many cycles are lost on a correct prediction?

[1+6=7]

2. a. Discuss the Tomasulo's algorithms & its necessity. Use the hardware configuration and show the disadvantage of it.

b. Suppose the Following instruction sequence is being executed where the system which is able to issue instructions in order but can perform out of order execution, which consists of 3 load units, 3 adder units and 2 multiplier units. The floating point adder unit takes 2 clock cycles, floating point multiplier takes 10 clock cycles and divide operation takes 40 clock cycles. If we use Tomsula's algorithm along with dynamic branch prediction, then find at what cycle the final value of F10 will be available and state the state of the functional units when the DIVD instruction is in execution. Also picturise the states of the reservation station.

```
LD    F6 34+ R2
LD    F2 45+ R3
MULTD F0 F2 F4
SUBD  F8 F6 F2
DIVD  F10 F0 F6
ADDD  F6 F8 F2
```

[2+5=7]

Original past-paper scan ca34.png (re-rendered).

Transcription of the question

```
4a. State the advantages of out-of-order pipelines over in-order pipelines.
4b. A five-stage single-pipeline microarchitecture (fetch, decode, execute, memory,
    write-back) runs the code:
        lw  R3, 0(R0)
        lw  R1, 0(R3)
        ADDI R1, R1, #1
        SUB R4, R3, R2
        SW  R1, 0(R3)
        BNZ R4, Loop
    All ops are one cycle except lw and sw (1+2) and branches (1+1). No forwarding.
    a) Show the phases per clock for one iteration; how many cycles lost to branch overhead?
    b) Assume a static branch predictor that recognises a backwards branch in Decode. How
       many cycles are wasted now?
    c) Assume a dynamic branch predictor. How many cycles are lost on a correct prediction?
2a. Discuss Tomasulo's algorithm and its necessity; use the hardware configuration and show
    its disadvantage.
2b. A sequence is executed by a machine that issues in order but executes out of order,
    with 3 load units, 3 adder units and 2 multiplier units. FP add = 2 cycles, FP multiply
    = 10 cycles, divide = 40 cycles. Using Tomasulo with dynamic branch prediction, find
    the cycle in which the final value of F10 is available, and the state of the functional
    units when DIVD is executing. Also picture the reservation-station states.
```

Full solution

4a — advantages of out-of-order

- Executes past a stalled instruction (e.g. a cache miss) instead of blocking the pipeline.
- Exploits ILP the compiler cannot see.
- Improves functional-unit utilisation and reduces the effective CPI.

4b — phases and branch overhead

```
No forwarding → 2 stalls per RAW pair. Build the space–time chart. a) In-order: the branch is resolved late → several wrong-path instructions are flushed (branch overhead ≈ 2 cycles per iteration). b) Static predictor recognising a backwards branch in Decode: the branch is predicted taken earlier → less overhead (≈ 1 cycle). c) Dynamic predictor, correct prediction: 0 cycles lost.
```

2b — cycle for F10

F10 is the destination of the DIVD. Its result is available when the divide completes, which depends on when its operands (F0 from the multiply, F6 from the load) become ready plus the 40-cycle divide latency. Work the tables: load → multiply (10 cycles) → divide (40 cycles).

```
Approximate: F0 ready after load(≈2) + multiply(10); DIV starts when both F0 and F6 are ready and a multiplier unit is free, then takes 40 cycles. → F10 available ≈ 2 + 10 + 40 = 52 cycles (subject to the exact issue slots).
```

Previous-year question 39 — ca39 — Register renaming with T-registers; stalls with a d…

b. Computers spend most of their time in loops, so multiple loop iterations are great places to speculatively find more work to keep CPU resources busy. Nothing is ever easy, though; the compiler emitted only one copy of that loop's code, so even though multiple iterations are handling distinct data, they will appear to use the same registers. To keep multiple iterations' register usages from colliding, we rename their registers. A compiler could have simply unrolled the loop and used different registers to avoid conflicts, but if we expect our hardware to unroll the loop, it must also do the register renaming. How? Assume your hardware has a pool of temporary registers (call them T registers, and assume that there are 64 of them, T0 through T63) that it can substitute for those registers designated by the compiler. This rename hardware is indexed by the src (source)register designation, and the value in the table is the T register of the last destination that targeted that register.

```
Loop: LD    F4,0(Rx)
I0:   MULTD F2,F0,F2
I1:   DIVD  F8,F4,F2
I2:   LD    F4,0(Ry)
I3:   ADDD  F6,F0,F4
I4:   SUBD  F8,F8,F6
I5:   SD    F8,0(Ry)
```

Maintain the true dependencies and construct the resulting code

[5]

c. Think about what latency numbers really mean—they indicate the number of cycles a given function requires to produce its output, nothing more. If the overall pipeline stalls for the latency cycles of each functional unit, then you are at least guaranteed that any pair of back-to-back instructions (a "producer" followed by a "consumer") will execute correctly. But not all instruction pairs have a producer/consumer relationship. Sometimes two adjacent instructions have nothing to do with each other. How many cycles would the loop body in the code sequence in given figure below require if the pipeline detected true data dependences and only stalled on those, rather than blindly stalling everything just because one functional unit is busy? Show the code with <stall> inserted where necessary

```
Loop: LD   F2,0(RX)
I0:   DIVD F8,F2,F0
I1:   MULTD F2,F6,F2
I2:   LD   F4,0(Ry)
I3:   ADDD F4,F0,F4
I4:   ADDD F10,F8,F2
I5:   ADDI Rx,Rx,#8
I6:   ADDI Ry,Ry,#8
I7:   SD   F4,0(Ry)
I8:   SUB  R20,R4,Rx
I9:   BNZ  R20,Loop
```

Latencies beyond single cycle	
Memory LD	+4
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+1
MULTD	+5
DIVD	+12

4

Original past-paper scan ca39.png (re-rendered).

Transcription of the question

```
3b. Loop code (LD F4,0(Rx); MULTD F2,F0,F2; DIVD F8,F4,F2; LD F4,0(Ry); ADDD F6,F0,F4;
    SUBD F8,F8,F6; SD F8,0(Ry)). Hardware has 64 temporary registers T0–T63 for renaming;
    the rename table is indexed by the source register and holds the T register of the last
    destination that targeted it. Maintain true dependences and construct the resulting code. [5]
3c. If the pipeline detects true data dependences and only stalls on those, how many cycles
    would the loop body require? Show the code with <stall> inserted. [5]
3d. Discuss Tomasulo's algorithm in out-of-order execution with an example; explain the
    necessity of the commit stage and the role of the reorder buffer in speculation. [5]
```

Full solution

3b — renaming with T-registers

```
Keep a rename table; when an instruction writes a register, allocate a fresh T: LD F4 → T1 ;
DIVD uses F4 from the ORIGINAL loop = T1 MULTD F2 → T2 DIVD F8 → T3 LD F4 (second) → T4 ; a
NEW T, so it does not clash with T1 ADDD F6 → T5 ; SUBD F8 → T6 ; SD uses T6 Each iteration's
values live in different T registers → no WAW/WAR collisions.
```

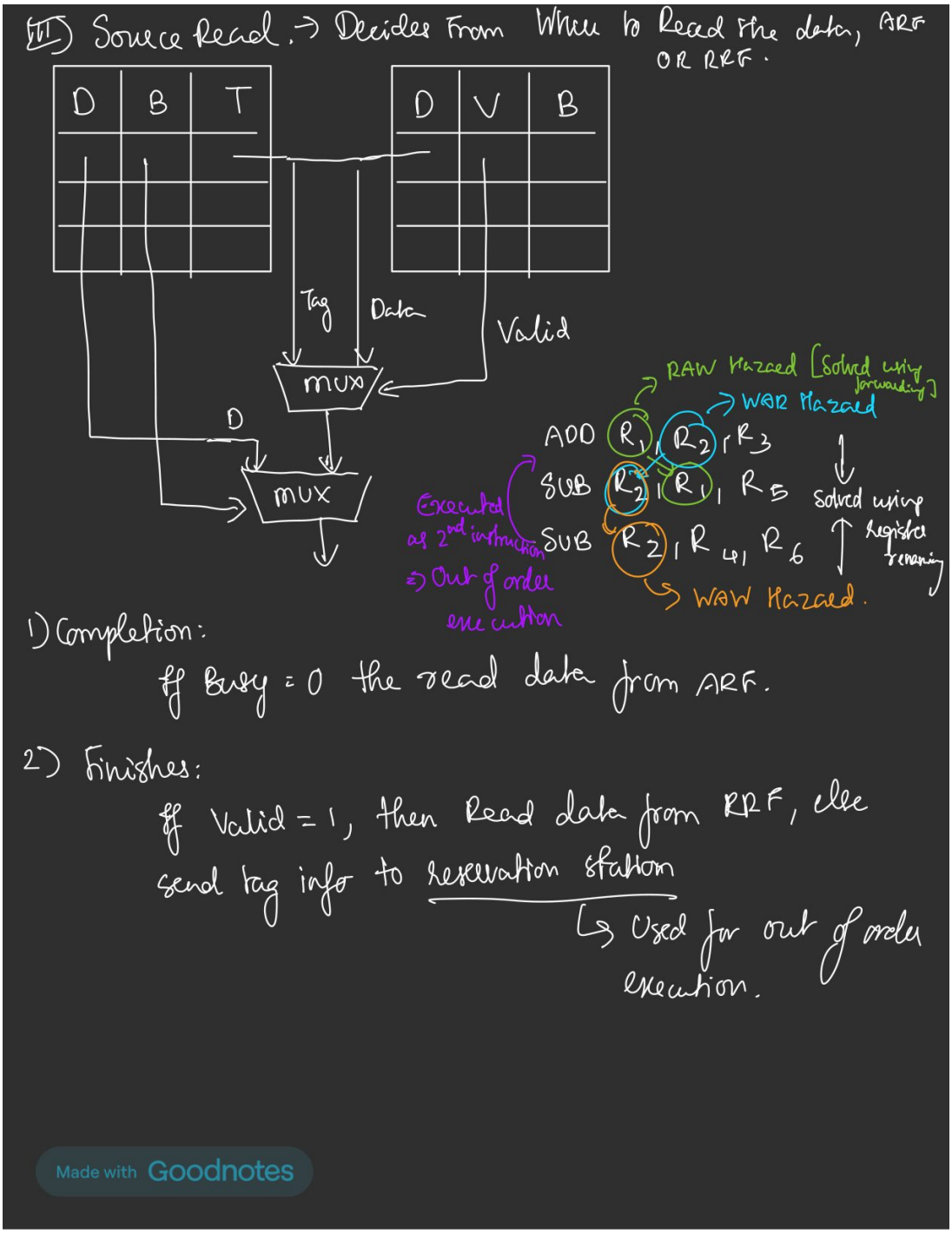
3c — cycles with true-dependence-only stalls

```
Insert only where a RAW latency requires it (latency 1+N → N stalls): LD F4,0(Rx) MULTD
F2,F0,F2 DIVD F8,F4,F2 ; waits for F4 (load latency) and F2 … LD F4,0(Ry) ; independent —
overlaps ADDD F6,F0,F4 SUBD F8,F8,F6 ; waits for F6 SD F8,0(Ry) Count the stall cycles from
the stated latencies.
```

3d — Tomasulo, commit stage and ROB

Tomasulo schedules dynamically with reservation stations and the CDB; results are computed as soon as operands are ready. Adding a **reorder buffer** lets results be held in program order until **commit**, which (i) makes exceptions precise and (ii) allows speculation — a mispredicted branch simply discards the speculative ROB entries.

Original notes — page 43: Reservation-station fields and the common data bus (CDB)



Original handwritten notes — page 43.

Transcription of the page

```
Source → register from which to read the operand value.

Reservation station fields:
   Op    — the operation to perform
   Qj, Qk — the reservation stations that will produce Vj, Vk
   Vj, Vk — the operand values (if already available)
   A      — the address / immediate (for loads/stores)
   Busy   — whether the station is occupied

CDB (Common Data Bus): broadcasts results to all waiting stations.
  If busy = 0, the station is free; else it is waiting.

Result tag → identifies which instruction produced a value on the CDB.
```

Explanation

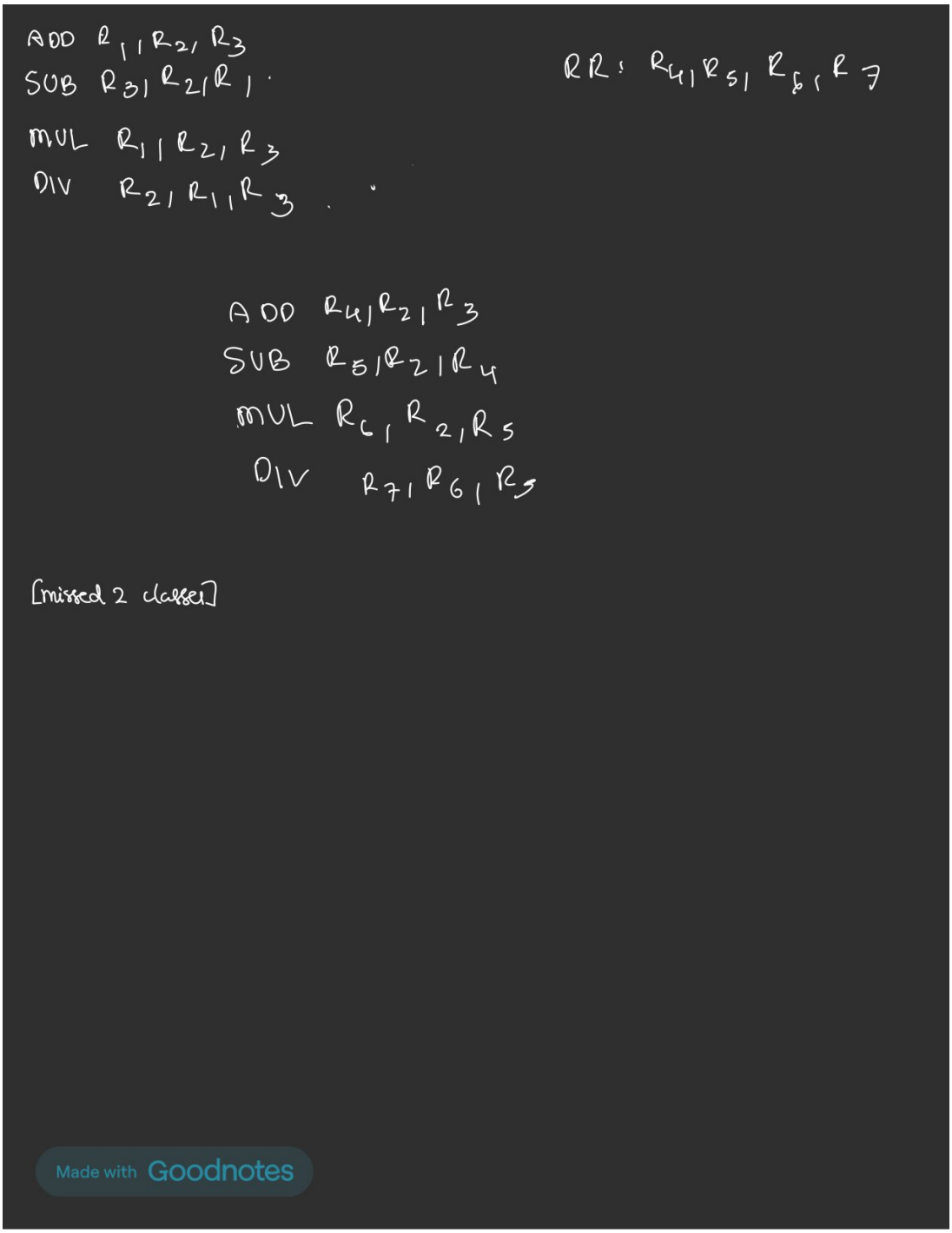
What the page says. The **reservation station** is the heart of Tomasulo's algorithm. This page lists its fields — a favourite exam question.

Field	Meaning
Op	Operation to perform (ADD, MUL, LOAD, …)
Vj, Vk	Operand values, if already available
Qj, Qk	Names (tags) of the stations that will produce Vj, Vk (blank if the value is ready)
A	Address or immediate field (used by load/store)
Busy	1 if the station is occupied, 0 if free

```
Rule: if Qj = 0 then Vj holds the value; if Qj ≠ 0 then wait for the result broadcast with tag Qj. CDB: when a unit finishes, it broadcasts (tag, value) to ALL stations and to the register file; any station with a matching Q grabs the value.
```

Why it works. A station never reads a value until it is actually produced. When the producing unit finishes it puts the result on the **common data bus** tagged with the producer's name; every waiting station compares its Q fields with that tag and, if they match, stores the value and clears the Q field.

Original notes — page 44: Tomasulo tables — instruction status and reservation statio…



Original handwritten notes — page 44.

Transcription of the page

```
Tomasulo tables:

  Instruction status:
    Instruction | j | k | Issue | Execute | Write result

  Reservation stations:
    Name | Busy | Op | Vj | Vk | Qj | Qk | A

  Register result status:
    F0 F1 F2 ... — which station will produce each register

  (Example:  MUL F0, F2, F4  ;  ADD F6, F2, F4 ;  SUB F8, F6, F2 ; ...)

  Numbers:  cycles 1,2,3 ;  latency 2 cycles etc.
```

Explanation

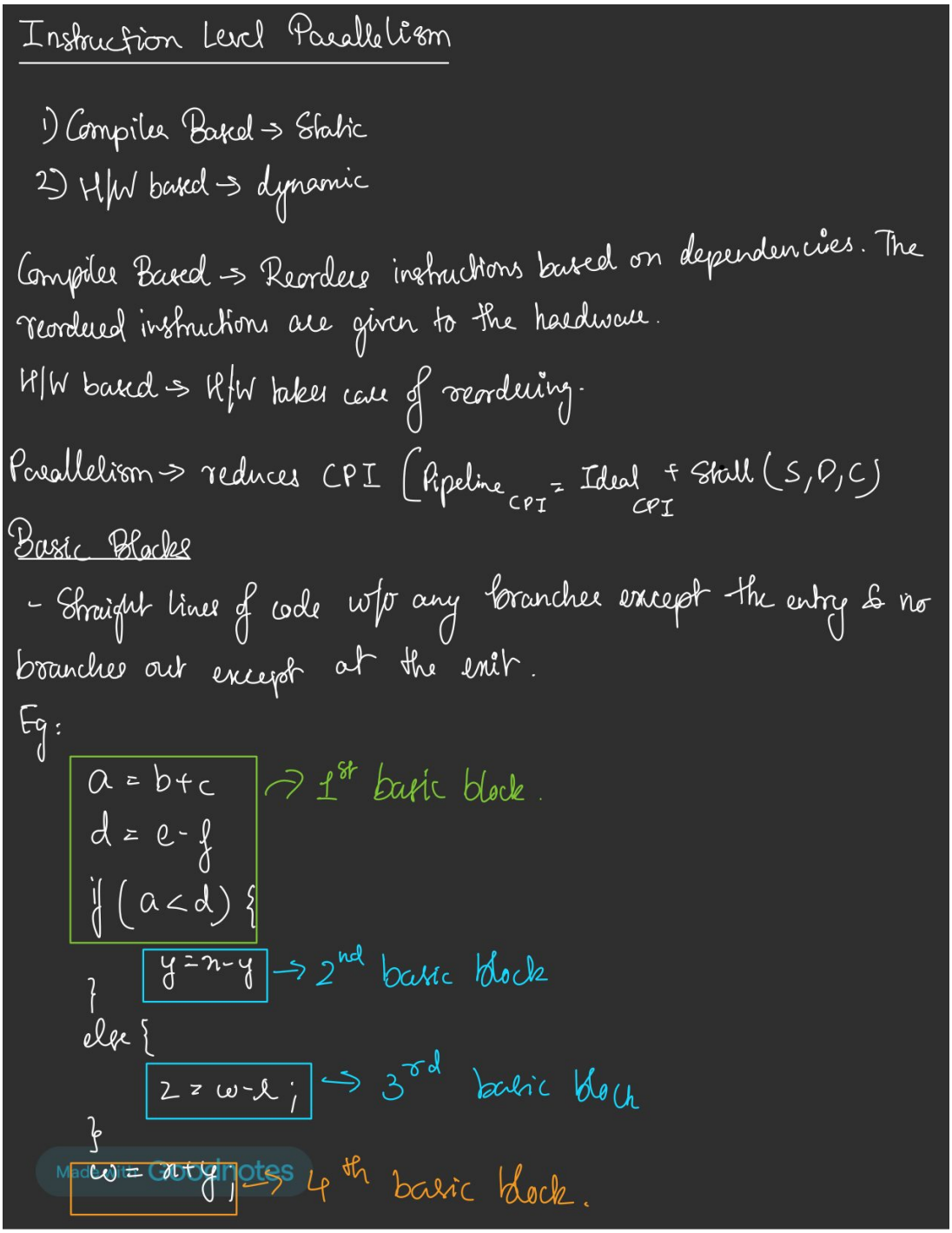
What the page says. The three tables you must fill in for any Tomasulo question, with a small worked example.

Table	Columns
Instruction status	Instruction, j, k, Issue, Execute, Write Result
Reservation stations	Name, Busy, Op, Vj, Vk, Qj, Qk, A
Register result status	for each register: which station will write it

```
Example code: MUL F0, F2, F4 ; F0 = F2 × F4 ADD F6, F2, F4 ; F6 = F2 + F4 SUB F8, F6, F2 ; F8 = F6 − F2 The MUL and ADD can start together (both read F2, F4); the SUB must wait for F6 (Qj = the ADD’s station).
```

Exam method. Go cycle by cycle: at each cycle (1) issue any instruction whose reservation station is free, filling Qj/Qk with the tags of the producers or Vj/Vk with ready values; (2) start execution when Qj = Qk = 0; (3) broadcast the result and clear matching Q fields. Record the cycle numbers in the instruction-status table.

Original notes — page 45: Parallelisation, CPI and the basics of branches



Original handwritten notes — page 45.

Transcription of the page

```
Parallelisation reduces CPI:
   CPI_ideal = 1 ;  CPI = CPI_ideal + stalls

Compiler/benchmark uses dependence information to schedule instructions.

Hardware branch handling: register renaming, reorder buffer (ROB).

Parallelism reduces CPI (pipeline):  T = IC × CPI / f

Basics of branches:
  * Free list of registers, used as renaming targets.
  * Branches occur more often than expected.
  * Q = branch ? taken : not-taken …
```

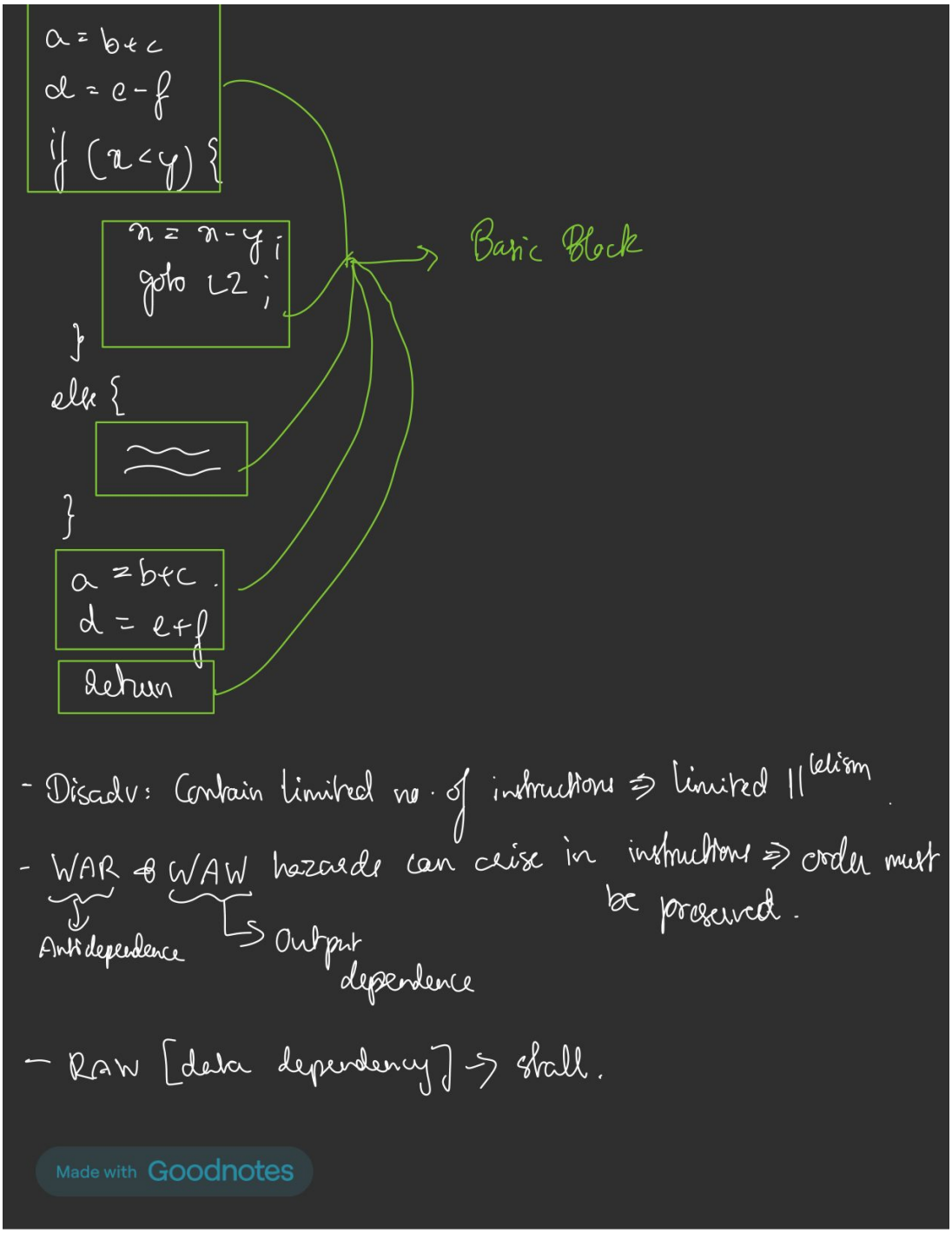
Explanation

What the page says. The bridge from pipelining to dynamic scheduling: parallelism and renaming reduce the effective CPI, and branches are the main obstacle because they are frequent.

```
CPI = CPI_ideal + stall_cycles_per_instruction = 1 + (branch_penalty + load_use_stalls + …) T = IC × CPI / f
```

- Pipelining makes CPI_ideal = 1; every hazard adds stalls on top.
- The compiler uses dependence information to schedule instructions and reduce stalls.
- Branches are common (roughly one in five instructions), so their cost matters a great deal.
- Hardware techniques — register renaming and the reorder buffer — let the processor keep working past a branch and past a false dependence.

Original notes — page 46: Data dependences — RAW, WAR and WAW



Original handwritten notes — page 46.

Transcription of the page

```
Dependences:
  True dependence (RAW): read-after-write — a later instruction reads a value an earlier one wrote.
  Anti-dependence (WAR): write-after-read — a later write would overwrite a value still needed.
  Output dependence (WAW): write-after-write — two writes to the same location.

  Register renaming removes WAR and WAW (name dependences);
  RAW (true) dependence remains.

  Example:  A = e + f ;  B = ... ; C = A + 1 …

  RAW (data dependence) — must be honoured.
```

Explanation

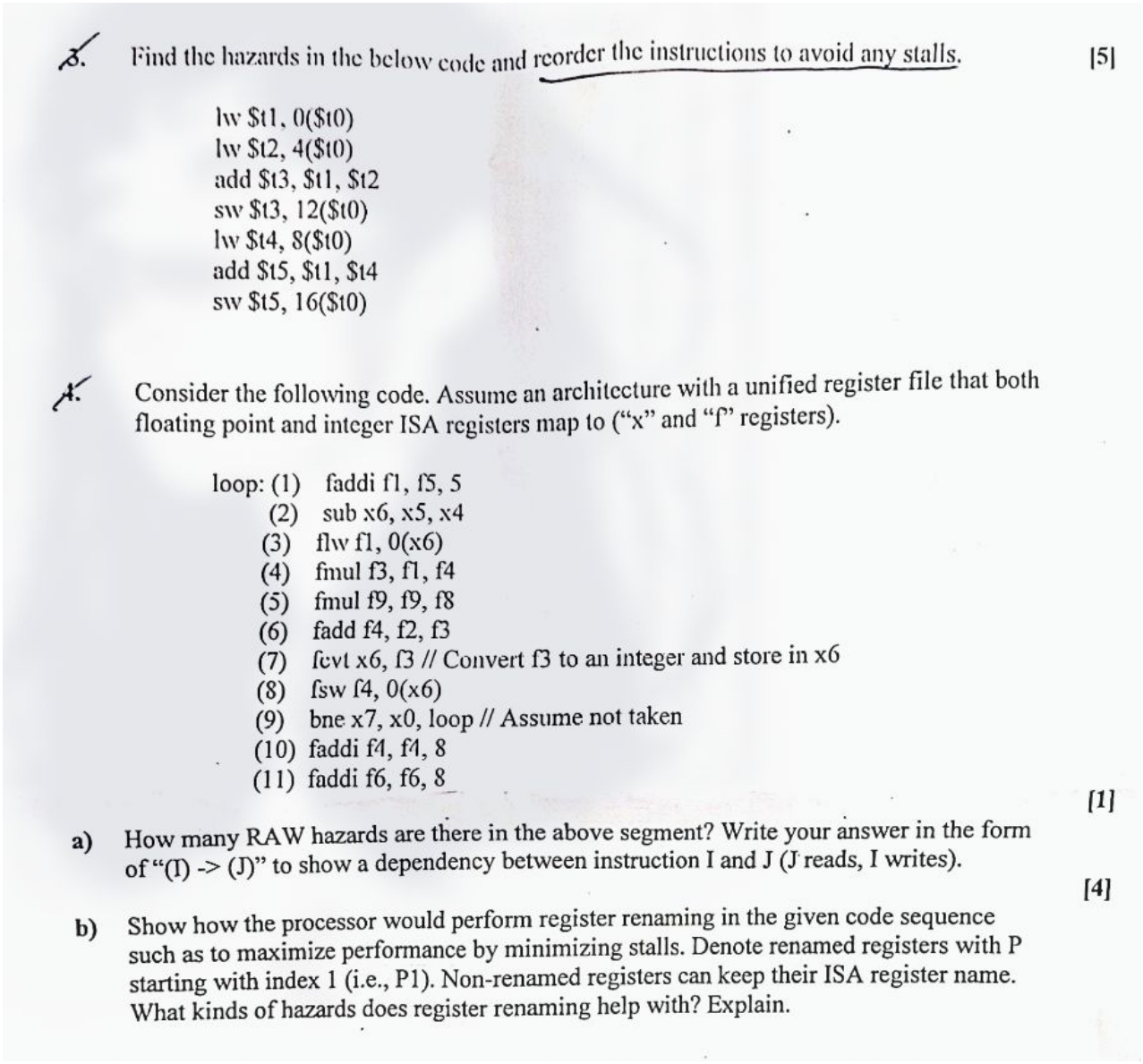
What the page says. The three kinds of register dependence, and the key fact that renaming removes two of them.

Dependence	Also called	Meaning	Removed by renaming?
RAW	True / flow	Later reads what earlier wrote	No — it is real data flow
WAR	Anti	Later writes a value still being read	Yes (it is a name clash)
WAW	Output	Two writes to the same register	Yes (it is a name clash)

```
Example: I1: F0 = F2 + F4 I2: F6 = F0 − F8 ; RAW on F0 (must wait for I1) I3: F0 = F10 × F12 ; WAW on F0 with I1, WAR with I2 → renaming removes these After renaming, I3 writes a DIFFERENT physical register, so it can execute without waiting for I1 or I2.
```

The one-line summary. RAW is a true dependence and cannot be removed — the data really must flow. WAR and WAW are only name clashes caused by reusing a small set of register names; give each new value a fresh name and they vanish.

Previous-year question 14 — ca14 — Hazards and reordering; register renaming with P-…



3. Find the hazards in the below code and reorder the instructions to avoid any stalls. [5]

```
lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
lw $t4, 8($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

4. Consider the following code. Assume an architecture with a unified register file that both floating point and integer ISA registers map to ("x" and "f" registers).

```
loop: (1)  faddi f1, f5, 5
      (2)  sub x6, x5, x4
      (3)  flw f1, 0(x6)
      (4)  fmul f3, f1, f4
      (5)  fmul f9, f9, f8
      (6)  fadd f4, f2, f3
      (7)  fcvt x6, f3 // Convert f3 to an integer and store in x6
      (8)  fsw f4, 0(x6)
      (9)  bne x7, x0, loop // Assume not taken
     (10)  faddi f4, f4, 8
     (11)  faddi f6, f6, 8
```

a) How many RAW hazards are there in the above segment? Write your answer in the form of "(I) -> (J)" to show a dependency between instruction I and J (J reads, I writes). [1]

b) Show how the processor would perform register renaming in the given code sequence such as to maximize performance by minimizing stalls. Denote renamed registers with P starting with index 1 (i.e., P1). Non-renamed registers can keep their ISA register name. What kinds of hazards does register renaming help with? Explain. [4]

Original past-paper scan ca14.png (re-rendered).

Transcription of the question

```
a. Find the hazards in the code below and reorder the instructions to avoid any
   stalls. [5]
        lw   $t1, 0($t0)
        lw   $t2, 4($t0)
        add  $t3, $t1, $t2
        sw   $t3, 12($t0)
        lw   $t4, 8($t0)
        add  $t5, $t1, $t4
        sw   $t5, 16($t0)
b. Consider the following code. Assume an architecture with a unified register file
   that both floating-point and integer ISA registers map to. [5]
        loop: (1) faddi f1, f5, 5
              (2) sub   x6, x5, x4
              (3) flw   f1, 0(x6)
              (4) fmul  f3, f1, f4
              (5) fmul  f9, f9, f8
              (6) fadd  f1, f2, f3
              (7) fcvt  x6, f3
              (8) fsw   f4, 0(x6)
              (9) bne   x7, x0, loop
              (10) faddi f4, f4, 8
              (11) faddi f6, f6, 8
   a) How many RAW hazards are there? Write them as (I → J) where J reads what I writes.
   b) Show register renaming to minimise stalls (P1, P2, …). What hazards does renaming help?
```

Full solution

a — hazards and reordering

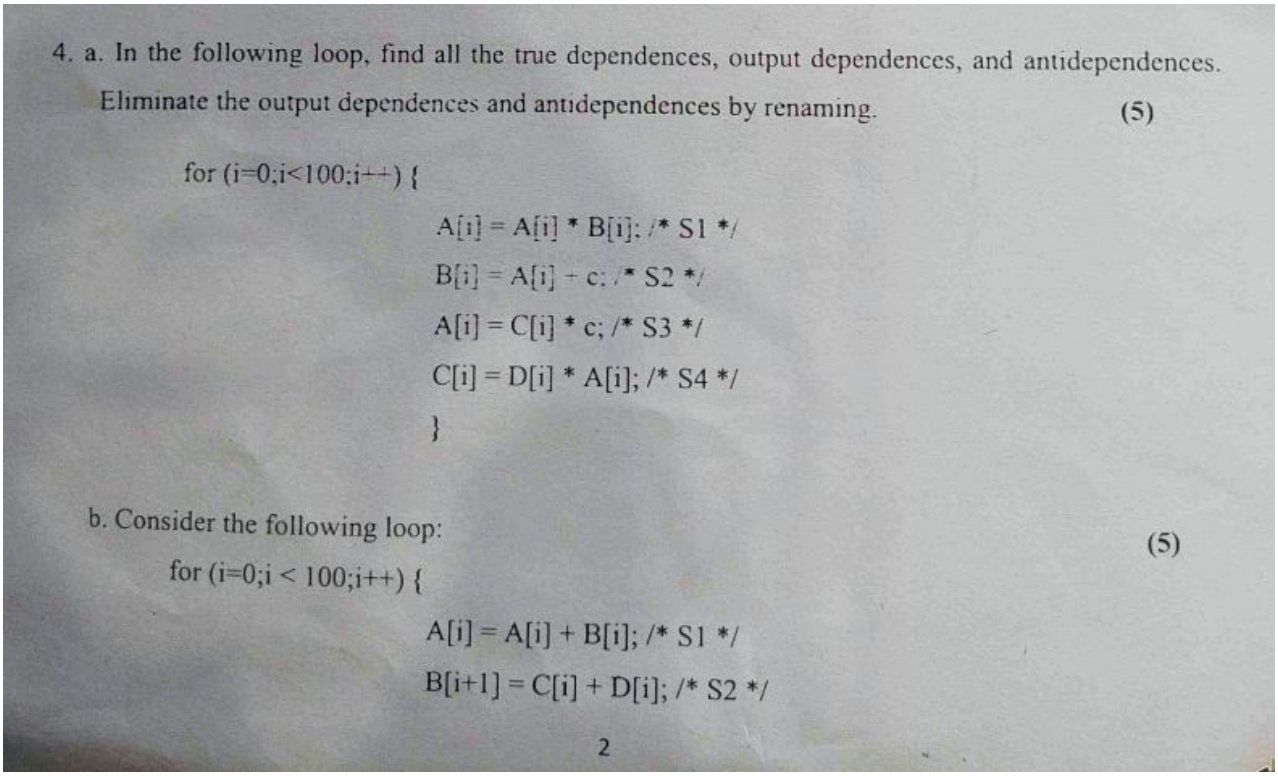
```
RAW on $t1: lw→add($t3) and lw→add($t5) RAW on $t2: lw→add($t3) RAW on $t3: add→sw RAW on
$t4: lw→add($t5) RAW on $t5: add→sw Reordered (independent loads hoisted to cover load-use
latency): lw $t1, 0($t0) lw $t2, 4($t0) lw $t4, 8($t0) add $t3, $t1, $t2 add $t5, $t1, $t4 sw
$t3, 12($t0) sw $t5, 16($t0)
```

b — RAW hazards in the loop

```
(1) faddi f1 → (3) flw f1? (3 writes f1, so WAW/WAR; (1)→(4) f1 used? (4) reads f1 written by
(3) : (3 → 4) (3) flw f1 → (6) fadd reads f3 written by (4): (4 → 6) (2) sub x6 → (3) flw uses
x6 as base: (2 → 3) (4) fmul f3 → (6) fadd and (7) fcvt x6: (4 → 6), (4 → 7) Count the true
(RAW) pairs as listed.
```

Renaming gives each new value a fresh physical register (P1, P2, …), removing WAW and WAR (name) dependences — e.g. the f1 written by (1) and the f1 loaded by (3) get different physical registers, so (4) no longer waits for (1). RAW dependences remain.

Previous-year question 22 — ca22 — Loop dependences and renaming (repeat of ca1 Q…



4. a. In the following loop, find all the true dependences, output dependences, and antidependences. Eliminate the output dependences and antidependences by renaming. (5)

```
for (i=0;i<100;i++) {
        A[i] = A[i] * B[i]; /* S1 */
        B[i] = A[i] + c; /* S2 */
        A[i] = C[i] * c; /* S3 */
        C[i] = D[i] * A[i]; /* S4 */
        }
```

b. Consider the following loop: (5)

```
for (i=0;i < 100;i++) {
        A[i] = A[i] + B[i]; /* S1 */
        B[i+1] = C[i] + D[i]; /* S2 */
```

2

Original past-paper scan ca22.png (re-rendered).

Transcription of the question

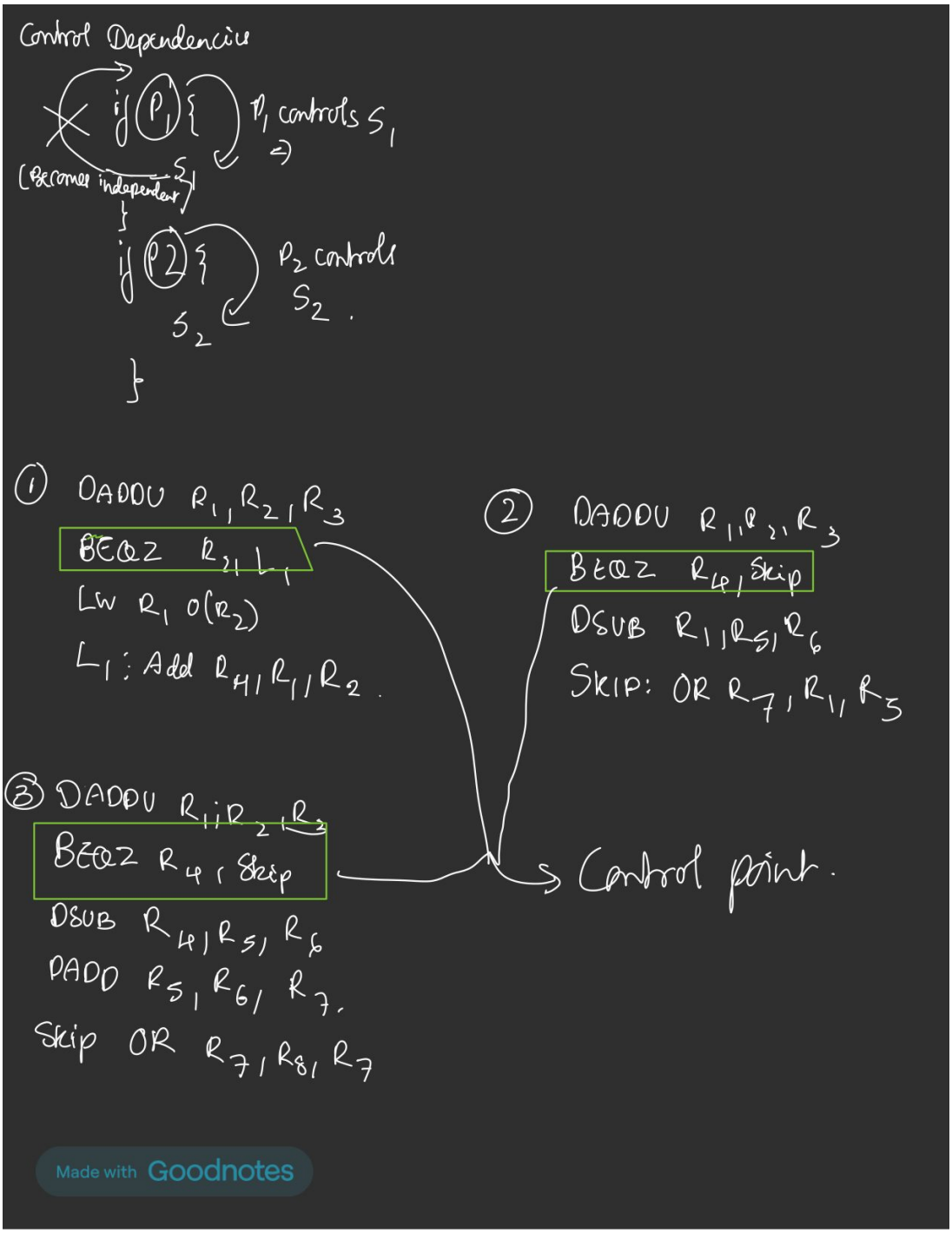
```
Q. This scan repeats ca1 Q2a: find the true, output and anti-dependences in the loop
   and eliminate the false ones by renaming.
```

Full solution

The solution is identical to ca1 Q2a (see the ca1 solution on the performance page of this guide). Recorded here as a repeat rather than duplicated in full:

```
S1: A[i] = A[i]*B[i] S2: B[i] = A[i] + c S3: A[i] = C[i]*c S4: C[i] = D[i]*A[i] RAW: S1→S2,
S3→S4 ; WAW: S1→S3 ; WAR: S2→S3 Rename: A1 = A[i]*B[i]; B[i]=A1+c; A2=C[i]*c; C[i]=D[i]*A2;
```

Original notes — page 47: Tomasulo worked example (loop) with reservation-station ta…



Original handwritten notes — page 47.

Transcription of the page

```
Tomasulo – loop example:

  Loop:  L.D  F0, 0(R1)
         MUL.D F4, F0, F2
         S.D  F4, 0(R1)
         ADD.D R1, R1, #-8     (or DADDIU R1, R1, -8)
         BNE  R1, R2, Loop

  Tables:
   Iteration 1: Load1, Mult1, Store1, Add1, ...
   Register result status per cycle.

  (Instruction status and reservation-station tables filled cycle by cycle.)
```

Explanation

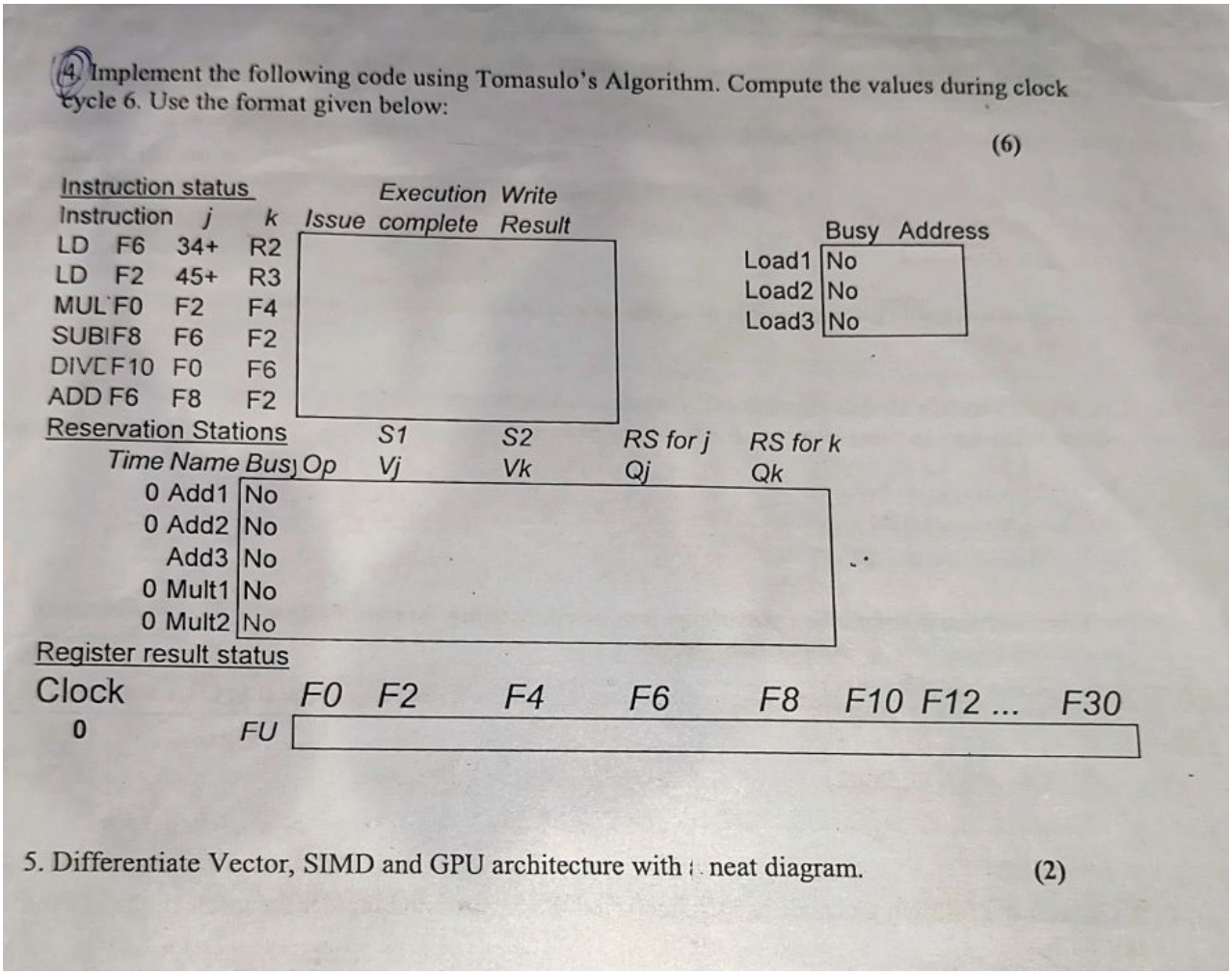
What the page says. A full Tomasulo example on a loop — the classic exam question. The processor must keep several loop iterations in flight using the reservation stations.

```
Loop body (the canonical example): L.D F0, 0(R1) ; load A[i] MUL.D F4, F0, F2 ; multiply by
scalar S.D F4, 0(R1) ; store A[i] DADDUI R1, R1, #-8 ; decrement pointer BNE R1, R2, Loop ;
branch back
```

- The **Load** stations hold the address; the **Mult** stations hold the multiplier operands; the **Store** stations hold the store address and the value to store.
- Because the loop iterations are independent (different addresses), Tomasulo can overlap them: load i+1 can start while multiply i is still running.
- The register result status table shows, for each register, which station will write it — this is how the hardware tracks the RAW dependences without stalling everything.

The full transcription of the printed question and the cycle-by-cycle solution are given in the past-paper solutions (ca12, ca18, ca34, ca39, ca40).

Previous-year question 12 — ca12 — Tomasulo at cycle 6; vector vs SIMD vs GPU



Original past-paper scan ca12.png (re-rendered).

Transcription of the question

```
4. Simulate the code using Tomasulo's algorithm. Compute the values at clock cycle 6.
   Use the format:  (6)
     Instruction status:  Instruction | j | k | Issue | Execute complete | Write Result
     Reservation stations: Name | Busy | Op | Vj | Vk | Qj | Qk
     Register result status (Clock 6)
   Code:
     LD  F6, 34(R2)     LD  F2, 45(R3)     MUL F0, F2, F4
     SUB F8, F6, F2     DIV F10, F0, F6    ADD F6, F8, F2
5. Differentiate Vector, SIMD and GPU architectures with a neat diagram. (2)
```

Full solution

Tomasulo at cycle 6

The code is the canonical Tomasulo example. Fill the three tables as the instructions issue, execute when their operands are ready, and write their results on the CDB.

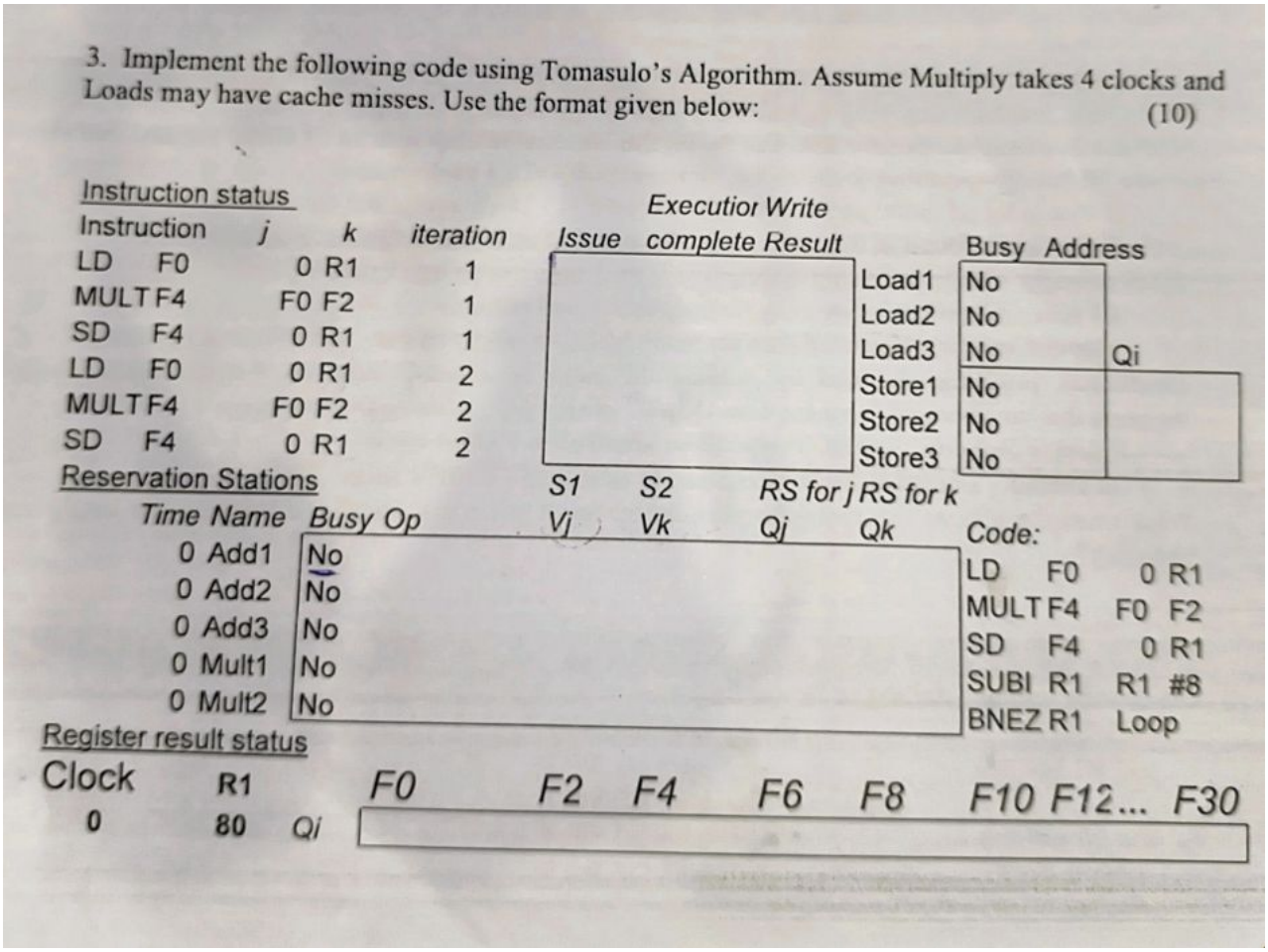
```
Cycle 1: LD F6 issues (Load1). Cycle 2: LD F2 issues (Load2). Cycle 3: MUL F0 issues (Mult1),
Qj=Load2 (needs F2), Vk=F4. Cycle 4: SUB F8 issues (Add1), Qj=Load1 (F6), Qk=Load2 (F2).
Cycle 5: DIV F10 issues (Mult2), Qj=Mult1 (F0), Qk=Load1 (F6). Cycle 6: ADD F6 issues (Add2),
Qj=Add1 (F8), Qk=Load2 (F2). By cycle 6 the loads have completed and broadcast their values,
so the Q fields that referenced Load1/Load2 are cleared (V fields filled).
```

Register result status at clock 6: F0 → Mult1, F6 → Add2, F8 → Add1, F10 → Mult2 (whichever stations are still pending).

Vector vs SIMD vs GPU

	Vector	SIMD	GPU
Model	Pipelined vector unit	One instruction, many data	Many SIMD lanes + threads
Control	Single control unit, vector length	Single control unit	Many warps, hardware threads
Best for	Long regular arrays	Short vector ops	Massively parallel data
Memory	Vector registers	Wide registers	High-bandwidth, latency-hidden

Previous-year question 18 — ca18 — Tomasulo on a loop with multiply latency and cac…



Original past-paper scan ca18.png (re-rendered).

Transcription of the question

```
3. Implement the following code using Tomasulo's algorithm. Assume multiply takes
   4 clocks and loads may have cache misses. Use the format given below: (10)
        LD   F0, 0(R1)      ; iteration 1
        MULT F4, F0, F2
        SD   F4, 0(R1)
        LD   F0, 0(R1)      ; iteration 2
        MULT F4, F0, F2
        SD   F4, 0(R1)
   Instruction status:  Instruction | j | k | iteration | Issue | Execute complete | Write result
   Reservation stations: Name | Busy | Op | Vj | Vk | Qj | Qk | A
   Register result status: Clock | R1 | F0 | F2 | F4 | F6 …
```

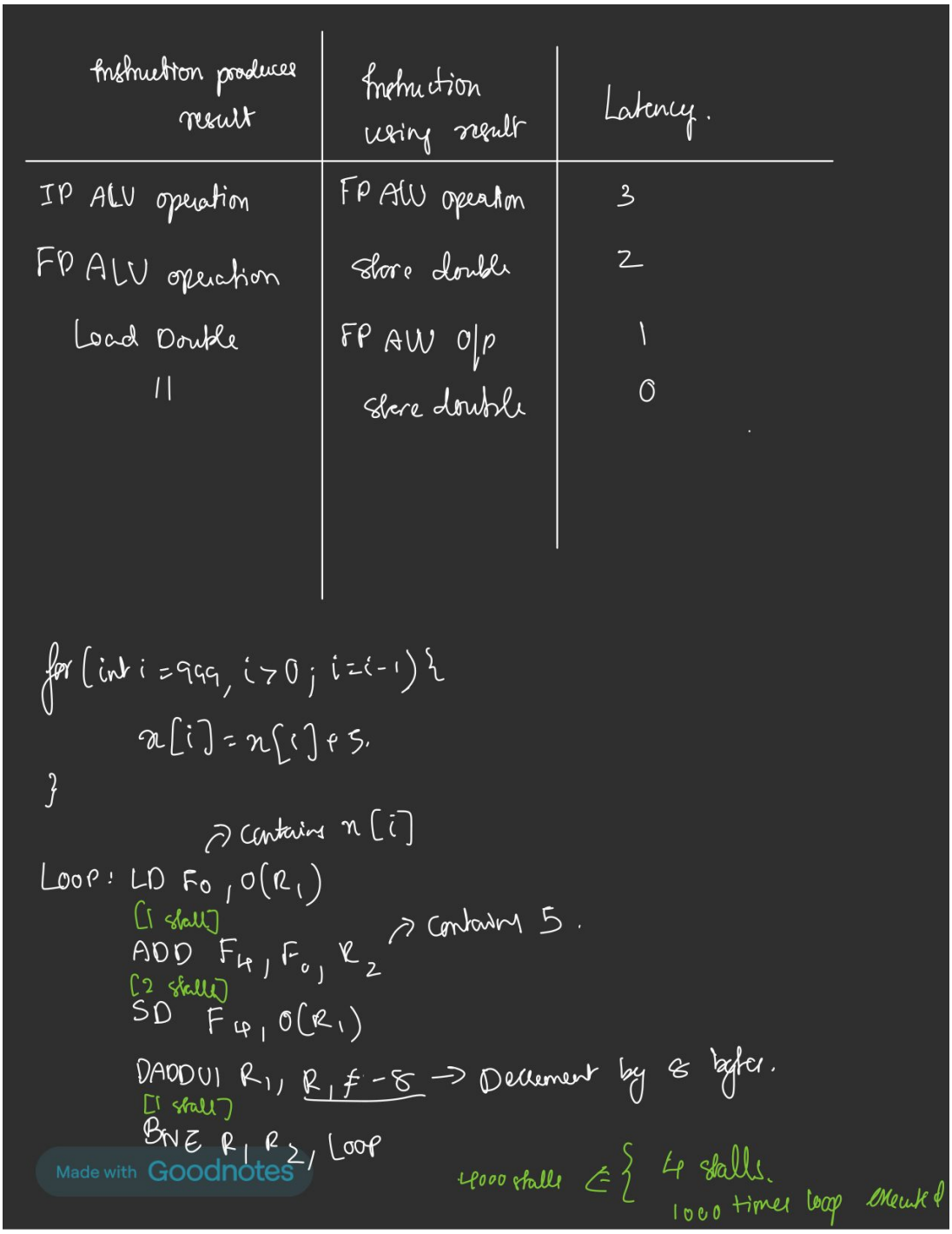
Full solution

Filling the tables

```
Iteration 1: LD F0,0(R1) → Load1 (address = 0+R1) MULT F4,F0,F2 → Mult1 (Qj = Load1, Vk = F2)
SD F4,0(R1) → Store1 (Qk = Mult1, address 0+R1) Iteration 2 (independent of iteration 1
except for R1): LD F0,0(R1) → Load2 MULT F4,F0,F2 → Mult2 (Qj = Load2, Vk = F2) SD F4,0(R1) →
Store2 (Qk = Mult2)
```

- Multiply latency = 4 cycles; a load's completion is delayed if it misses in the cache.
- The two iterations use the **same register names** (F0, F4), so renaming via the reservation stations is what lets them overlap: Load2 writes F0 while Load1's value is still needed by Mult1.
- Record Issue, Execute-complete and Write-result for each instruction; the register-result-status table shows which station will produce F0 and F4 at each cycle.

Original notes — page 48: Tomasulo example with a small FP unit and load/store buffe…



Original handwritten notes — page 48.

Transcription of the page

```
Tomasulo – small example:

  FP ALU operations  – 1 unit, latency 3
  FP load/store      – 1 unit, latency 2
  Load:   L.D  F0, 0(R1)
  ...
  ADD.D  F4, F0, F2
  ...
  S.D    F4, 0(R2)

  (Reservation-station table with Qj/Qk tags and Vj/Vk values.)
```

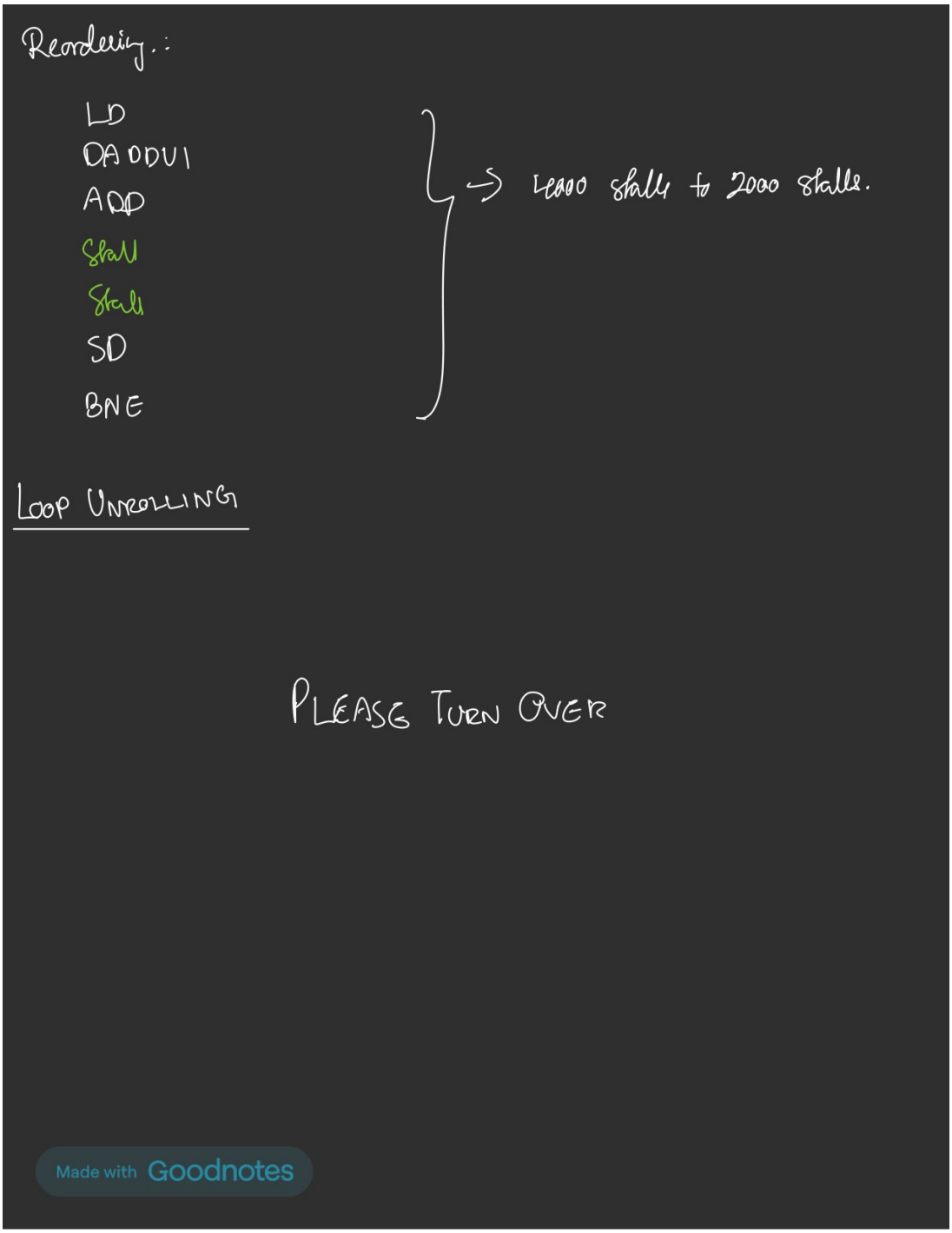
Explanation

What the page says. A compact Tomasulo example with stated unit latencies, so the cycle numbers can be computed exactly.

```
Given: FP add/mul latency = 3 cycles ; load latency = 2 cycles Code: L.D F0, 0(R1) ADD.D F4, F0, F2 S.D F4, 0(R2) The ADD must wait for F0 (Qj = Load1); the store must wait for F4 (Qk = Add1). Execution starts when both operands are ready and the functional unit is free.
```

Follow the three steps for every instruction — Issue, Execute, Write result — and record the cycle numbers. The reservation-station table shows the Q/V fields changing as results are broadcast on the CDB.

Original notes — page 49: Loop unrolling — the DAXPY example



Original handwritten notes — page 49.

Transcription of the page

```
Unrolling:

  L.D   F0, ...
  ADD.D F4, F0, F2
  S.D   F4, ...

  DAXPY:  Y[i] = a·X[i] + Y[i]

  Unroll the loop so that several iterations are done per pass,
  reducing loop overhead (the branch and pointer update).

  Loop unrolling: reduces the number of branches and exposes
  independent operations to schedule.
```

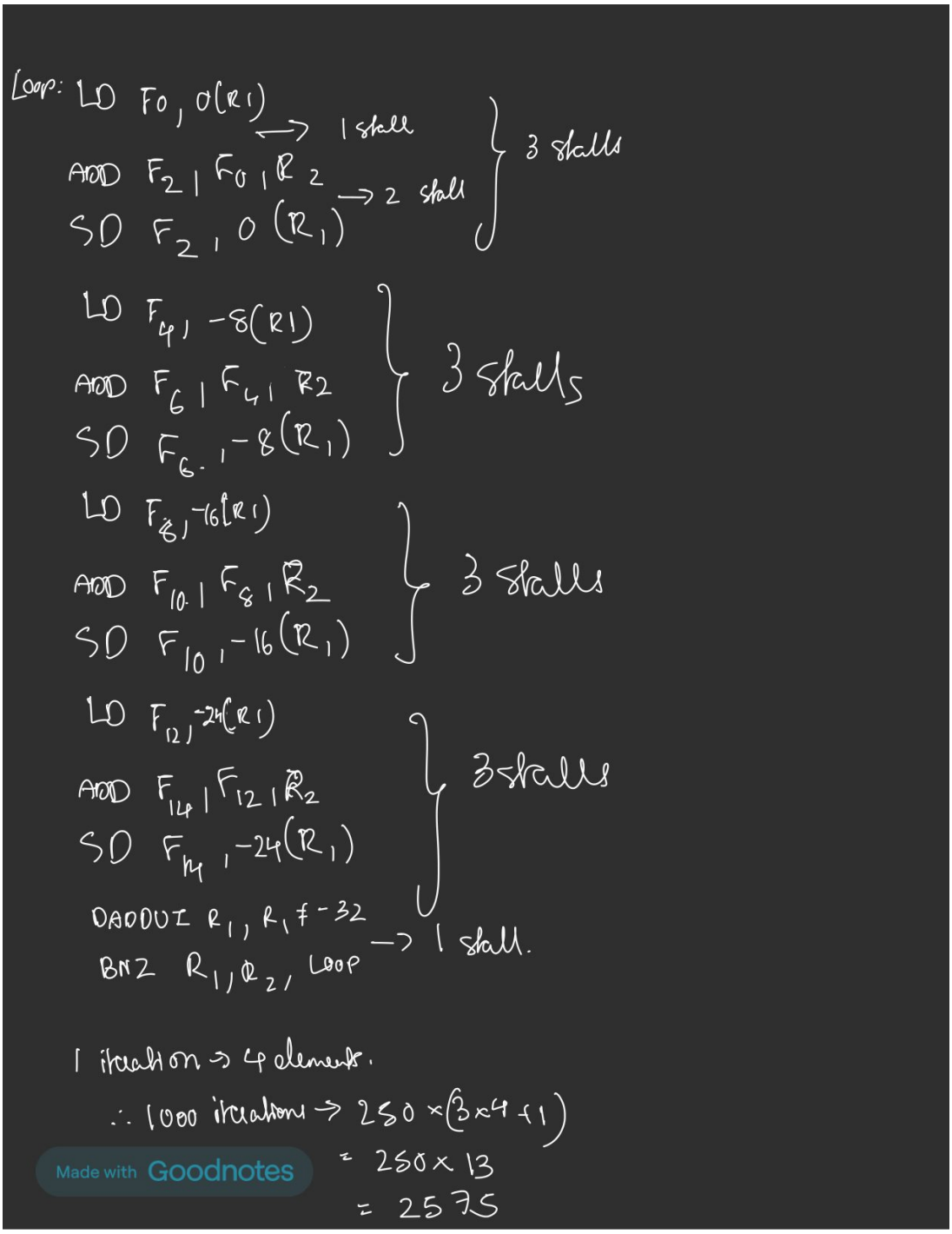
Explanation

What the page says. Loop unrolling repeats the loop body several times per iteration so that the loop-control instructions (branch, pointer update, counter) are amortised and more independent work is exposed to the scheduler.

```
DAXPY: for i = 0 … n−1: Y[i] = a · X[i] + Y[i] MIPS: L.D F0, 0(R1) ; load X[i] MUL.D F4, F0, F2 ; a · X[i] L.D F6, 0(R2) ; load Y[i] ADD.D F6, F4, F6 ; a·X[i] + Y[i] S.D F6, 0(R2) ; store Y[i] DADDUI R1, R1, #8 ; DADDUI R2, R2, #8 ; BNE …
```

- **Why unroll?** Each pass still pays for the pointer updates and the branch, but does more useful work.
- **Effect on CPI:** fewer branches fewer control hazards; more independent instructions the scheduler can fill the delay slots.
- **Cost:** larger code (more instruction-cache pressure) and more registers needed to hold the independent values.

Original notes — page 50: Loop unrolling worked example (3× unrolled DAXPY)



Original handwritten notes — page 50.

Transcription of the page

```
Unrolled DAXPY (3×):

  L.D   F0, 0(R1)
  L.D   F6, -8(R1)
  L.D   F10, -16(R1)
  L.D   F14, -24(R1)
  MUL.D F4, F0, F2
  MUL.D F8, F6, F2
  MUL.D F12, F10, F2
  MUL.D F16, F14, F2
  ADD.D F4, F4, F6
  ADD.D F8, F8, F10
  ADD.D F12, F12, F14
  S.D   F4, 0(R1)
  S.D   F8, -8(R1)
  S.D   F12, -16(R1)
  DADDUI R1, R1, #-32
  BNE   R1, R2, Loop

  (Reordering removes stalls; redundant computation removed.)
```

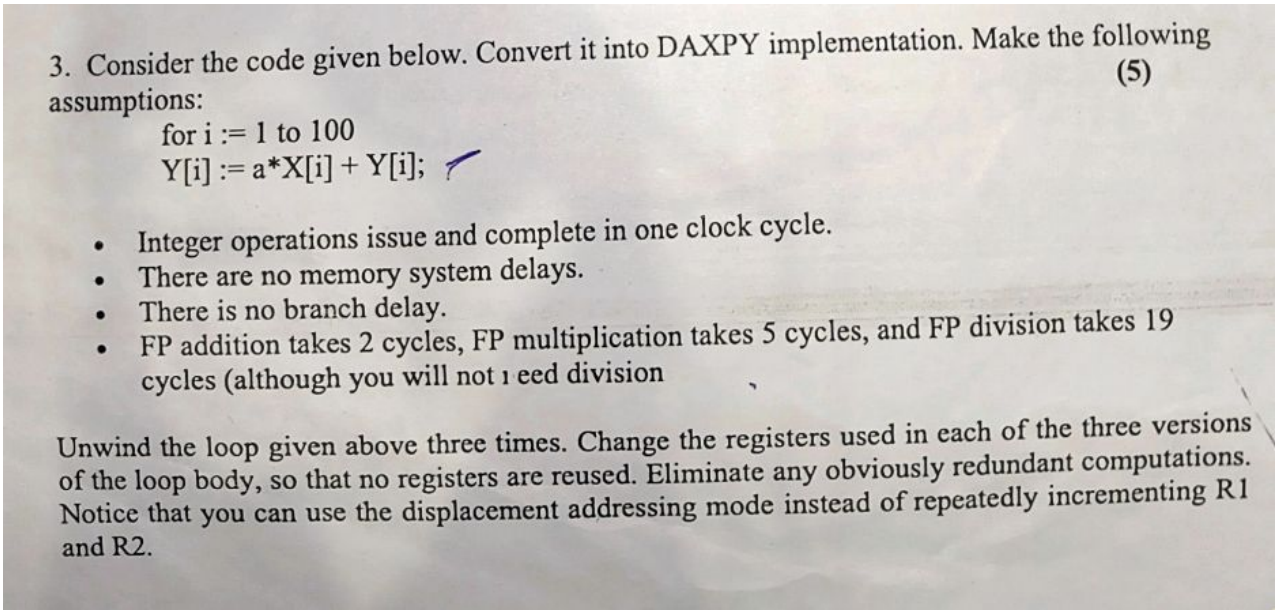
Explanation

What the page says. A 3× (really 4-element) unrolled DAXPY, showing the classic technique of loading several elements, multiplying them all, adding them all, and storing them all — so that long-latency operations overlap.

```
Schedule idea: • issue all the LOADS first (independent) — they overlap, • then all the
MULTIPLIES, • then all the ADDs, • then all the STOREs, • one pointer update and one branch
for the whole group.
```

- The compiler must give each unrolled iteration its **own registers** (F0, F6, F10, F14 …) so the operations are independent — otherwise a false (WAW/WAR) dependence would serialise them.
- It also **removes redundant computation** (e.g. re-loading the same scalar, or the loop-overhead work done once instead of per element).
- Unrolling exposes ILP; scheduling then fills the delay slots with useful work so the pipeline does not stall.

Previous-year question 11 — ca11 — DAXPY loop unrolled 3× with fresh registers


3. Consider the code given below. Convert it into DAXPY implementation. Make the following assumptions: (5)

for i := 1 to 100
Y[i] := a*X[i] + Y[i];

- Integer operations issue and complete in one clock cycle.
- There are no memory system delays.
- There is no branch delay.
- FP addition takes 2 cycles, FP multiplication takes 5 cycles, and FP division takes 19 cycles (although you will not need division

Unwind the loop given above three times. Change the registers used in each of the three versions of the loop body, so that no registers are reused. Eliminate any obviously redundant computations. Notice that you can use the displacement addressing mode instead of repeatedly incrementing R1 and R2.

Original past-paper scan ca11.png (re-rendered).

Transcription of the question

```
Q. Unroll the DAXPY loop three times, using fresh registers and removing redundant
   computation. Then schedule it.

   DAXPY:  Y[i] = a·X[i] + Y[i]
```

Full solution

Unrolled and scheduled DAXPY

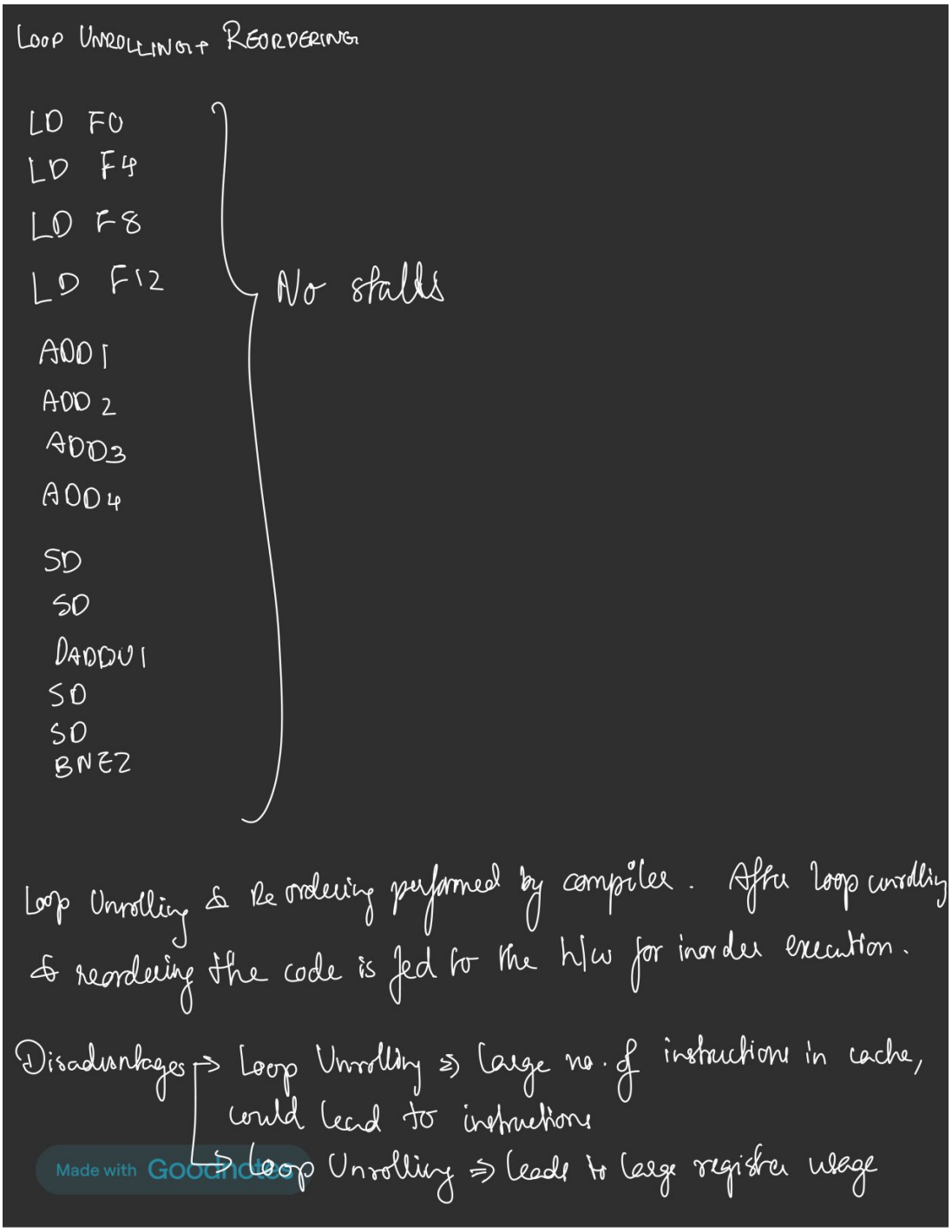
```
Loop: L.D F0, 0(R1) ; load X[i] MUL.D F4, F0, F2 ; a·X[i] L.D F6, 0(R2) ; load Y[i] ADD.D F6, F4, F6 ; a·X[i] + Y[i] S.D F6, 0(R2) ; store Y[i] DADDUI R1,R1,#8 ; DADDUI R2,R2,#8 ; BNE R1,R3,Loop
```

Unroll three times, giving each iteration fresh registers (F0/F4/F8/F12 …) so the operations are independent, then remove the redundant pointer arithmetic (one update per unrolled group).

```
Unrolled (sketch): L.D F0, 0(R1) ; X[i] L.D F6, -8(R1) ; X[i+1] L.D F10, -16(R1) ; X[i+2] MUL.D F4, F0, F2 MUL.D F8, F6, F2 MUL.D F12, F10, F2 ADD.D F4, F4, Y0 ; ADD.D F8, F8, Y1 ; ADD.D F12, F12, Y2 S.D F4, 0(R2) ; S.D F8, -8(R2) ; S.D F12, -16(R2) DADDUI R1,R1,#-24 ; DADDUI R2,R2,#-24 ; BNE R1,R3,Loop
```

> **Two requirements.** (1) Fresh registers so there is no false dependence; (2) the loop-control work (pointer updates, branch) is done once per group, not per element.

Original notes — page 51: Unroll until no stalls — the scheduled loop



Original handwritten notes — page 51.

Transcription of the page

```
Unroll until no stalls:

  L.D   F0, ...
  L.D   F6, ...
  L.D   F10, ...
  L.D   F14, ...
  (then the multiply / add / store schedule)
  ...
  S.D   F4, ...
  S.D   F8, ...
  S.D   F12, ...
  S.D   F16, ...
  DADDUI R1, R1, #-32
  BNE   R1, R2, Loop

  No stalls: every load-use gap is filled by an independent instruction.

  Disadvantages: loop overhead, larger code, more registers.

  Loop unrolling: reducing the overhead of branches.
```

Explanation

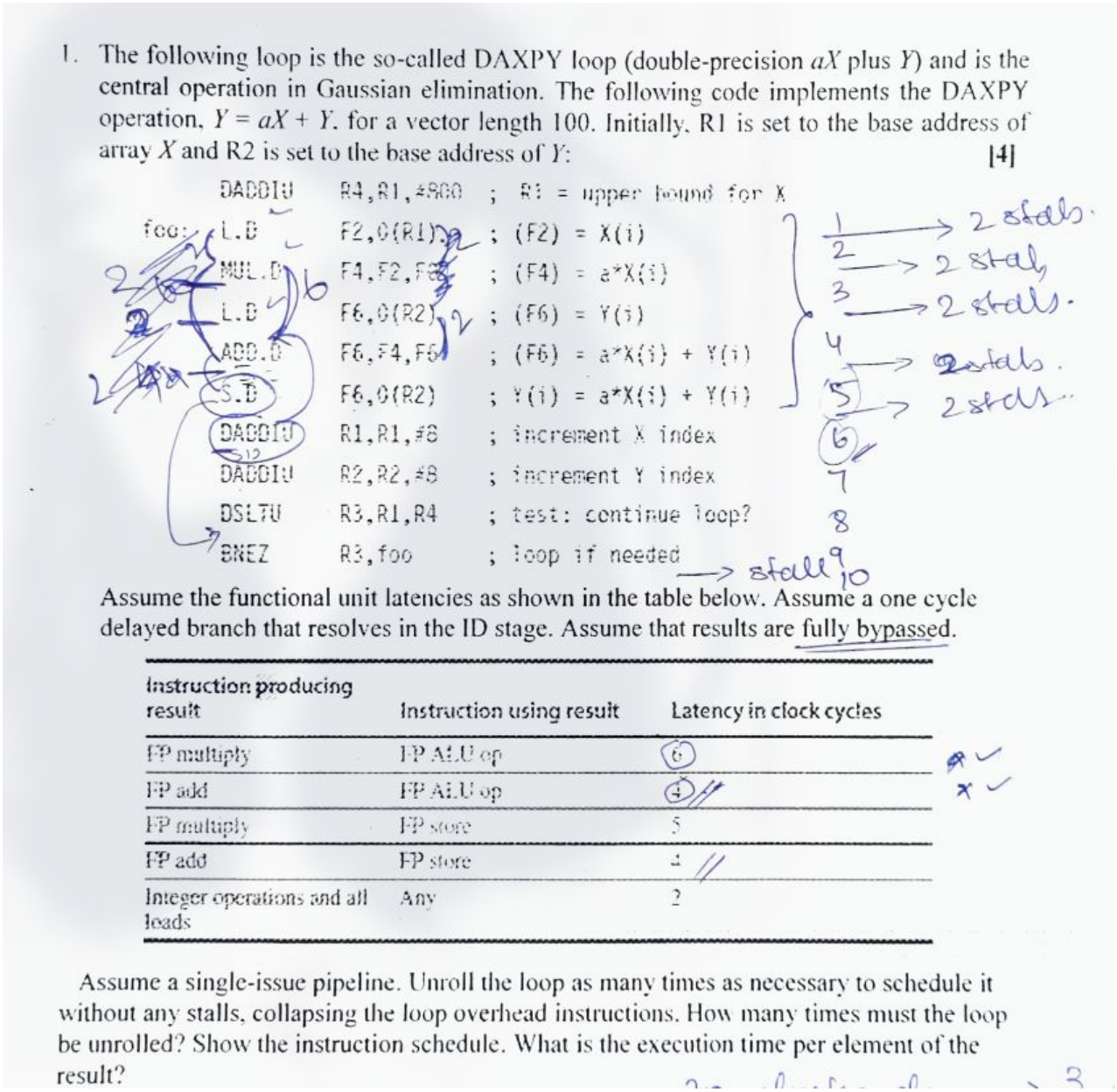
What the page says. The goal of unrolling is to reach a schedule in which *no* stall cycles are needed — every load-use delay is filled with an independent instruction.

```
To remove every load-use stall, unroll enough times that the number of independent
instructions ≥ the load latency. Example: load latency = 4 cycles ▫ unroll ~4× and
interleave.
```

- **Benefit:** the loop runs at (or near) one element per cycle with no pipeline bubbles.
- **Disadvantage 1 — loop overhead:** the counter, pointer updates and branch are still there, just amortised.
- **Disadvantage 2 — code size:** unrolled code is larger, so it may not fit in the instruction cache.
- **Disadvantage 3 — register pressure:** more live values need more registers.

> **The trade-off.** Unroll too little and stalls remain; unroll too much and the code no longer fits in the I-cache or the registers spill. The sweet spot depends on the latencies and the cache size.

Previous-year question 24 — ca24 — DAXPY unroll-until-no-stalls; schedule; execution …



1. The following loop is the so-called DAXPY loop (double-precision aX plus Y) and is the central operation in Gaussian elimination. The following code implements the DAXPY operation, $Y = aX + Y$, for a vector length 100. Initially, R1 is set to the base address of array X and R2 is set to the base address of Y: [4]

```
        DADDIU  R4,R1,#800   ; R1 = upper bound for X
foo:    L.D     F2,0(R1)     ; (F2) = X(i)
        MUL.D   F4,F2,F0     ; (F4) = a*X(i)
        L.D     F6,0(R2)     ; (F6) = Y(i)
        ADD.D   F6,F4,F6     ; (F6) = a*X(i) + Y(i)
        S.D     F6,0(R2)     ; Y(i) = a*X(i) + Y(i)
        DADDIU  R1,R1,#8     ; increment X index
        DADDIU  R2,R2,#8     ; increment Y index
        DSLTU   R3,R1,R4     ; test: continue loop?
        BNEZ    R3,foo       ; loop if needed
```

Assume the functional unit latencies as shown in the table below. Assume a one cycle delayed branch that resolves in the ID stage. Assume that results are fully bypassed.

Instruction producing result	Instruction using result	Latency in clock cycles
FP multiply	FP ALU op	6
FP add	FP ALU op	4
FP multiply	FP store	5
FP add	FP store	4
Integer operations and all loads	Any	2

Assume a single-issue pipeline. Unroll the loop as many times as necessary to schedule it without any stalls, collapsing the loop overhead instructions. How many times must the loop be unrolled? Show the instruction schedule. What is the execution time per element of the result?

Original past-paper scan ca24.png (re-rendered).

Transcription of the question

```
1. The following is the DAXPY loop, Y = a·X + Y, for a vector length 100. R1 is the base
   of X, R2 the base of Y:
        DADDUI R4, R1, #800      ; R4 = upper bound for X
   foo: L.D   F2, 0(R1)          ; F2 = X(i)
        MUL.D F4, F2, F0         ; F4 = a·X(i)
        L.D   F6, 0(R2)          ; F6 = Y(i)
        ADD.D F6, F4, F6         ; F6 = a·X(i) + Y(i)
        S.D   F6, 0(R2)          ; Y(i) = ...
        DADDUI R1, R1, #8        ; increment X index
        DADDUI R2, R2, #8        ; increment Y index
        DSGTUI R3, R1, R4        ; test: continue?
        BEQZ  R3, foo            ; branch if needed
   Functional-unit latencies: FP multiply→FP ALU 5; FP add→FP store 4; integer→any 1;
   load→any 2. One-cycle delayed branch that resolves in ID. Results fully bypassed.
   Assume a single-issue pipeline. Unroll as many times as necessary to schedule without
   any stalls, collapsing the loop overhead. How many times must the loop be unrolled?
   Show the schedule. What is the execution time per element?
```

Full solution

How many times to unroll?

The binding constraint is the **load → use** latency of 2 cycles: a load's value is needed 2 cycles later, so each load must have at least two independent instructions after it. With single-issue and a 5-cycle multiply latency, unroll enough to have several independent chains in flight — the standard answer is to unroll **4×** (four elements per pass), which exposes enough independent loads, multiplies and adds to fill every

latency slot.

```
Constraint: load latency = 2 ▫ ≥ 2 independent instructions between a load and its use; multiply latency = 5 ▫ ≥ 5 cycles between the multiply and the dependent add. Unrolling 4× gives enough independent work to satisfy both.
```

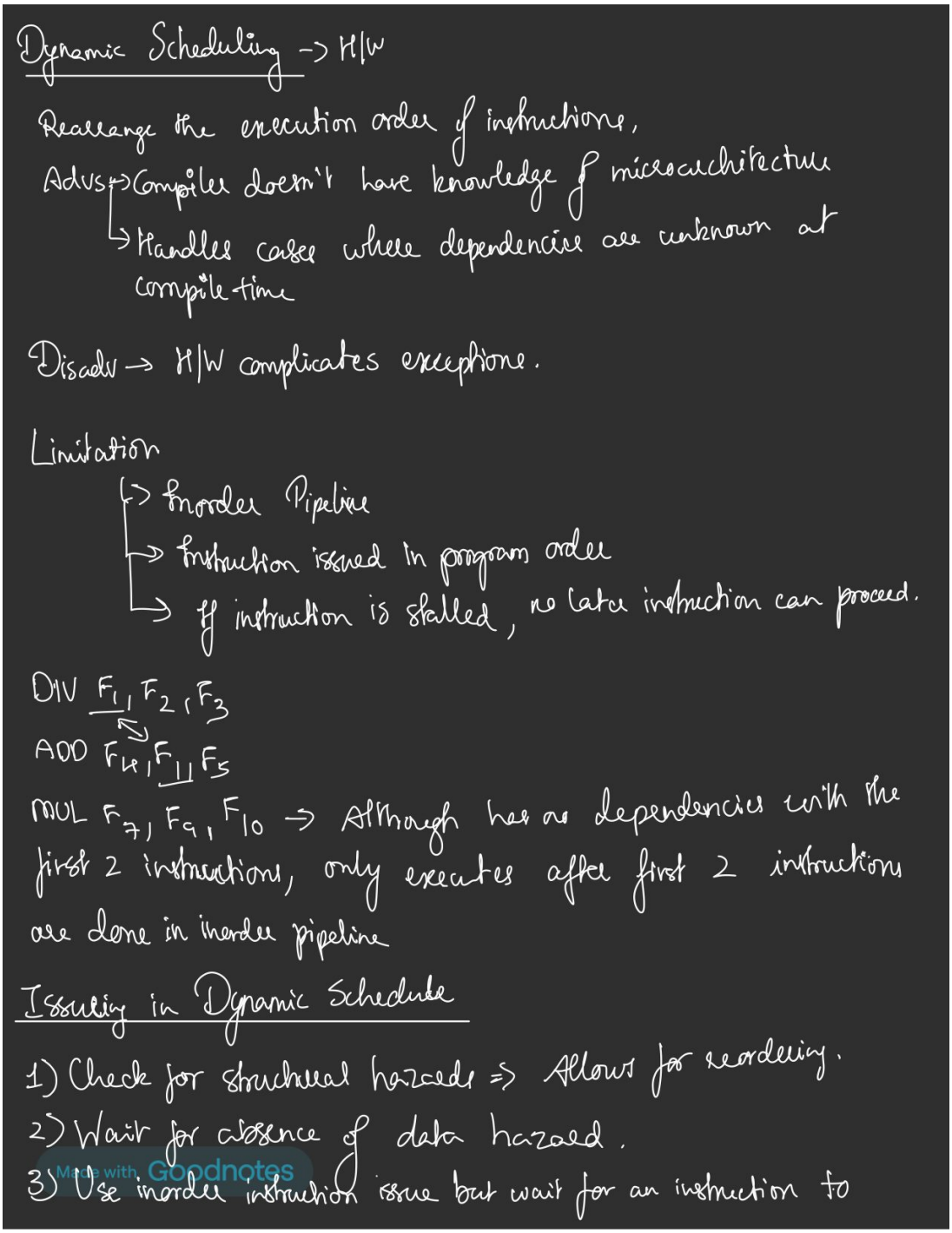
Schedule (4× unrolled, no stalls)

```
L.D F0, 0(R1) ; X[i] L.D F6, 8(R1) ; X[i+1] L.D F10, 16(R1) ; X[i+2] L.D F14, 24(R1) ; X[i+3] MUL.D F4, F0, F2 MUL.D F8, F6, F2 MUL.D F12, F10, F2 MUL.D F16, F14, F2 L.D F0, 0(R2) ; L.D F6, 8(R2) ; L.D F10,16(R2) ; L.D F14,24(R2) ; Y[i..i+3] ADD.D F4, F4, F0 ; ADD.D F8, F8, F6 ; ADD.D F12,F12,F10 ; ADD.D F16,F16,F14 S.D F4, 0(R2) ; S.D F8, 8(R2) ; S.D F12,16(R2) ; S.D F16,24(R2) DADDUI R1,R1,#32 ; DADDUI R2,R2,#32 ; BNE …
```

Execution time per element

```
Count the cycles in the unrolled group and divide by the elements (4): e.g. if the group issues in ≈ 14 cycles for 4 elements → ≈ 3.5 cycles/element. The exact number depends on the assumed issue width and latencies; the method is: (cycles per unrolled group) / (elements per group).
```

Original notes — page 52: Instruction scheduling rules (dynamic and static)



Original handwritten notes — page 52.

Transcription of the page

```
Dynamic scheduling:
  Reorder the execution order of instructions.

  Advantages – doesn't need the compiler; handles dependences at run time;
  can exploit ILP that the compiler cannot see.

  Compile-time (static) scheduling: the compiler reorders instructions.

  Instructions → fixed pipeline, so scheduling must avoid stalls.

  Rules for reordering instructions:
   1) One register per instruction (no false dependences),
   2) Allow for latency (respect the real delays),
   3) Do not move a load/store past a conflicting memory access,
   4) Do not move a branch (control) past a dependent instruction,
   5) Do not move a WAW/WAR pair unless renamed.

  Example:
    ADD F4, F0, F2
    MUL F4, ...   (WAR) → reorder or rename
```

Explanation

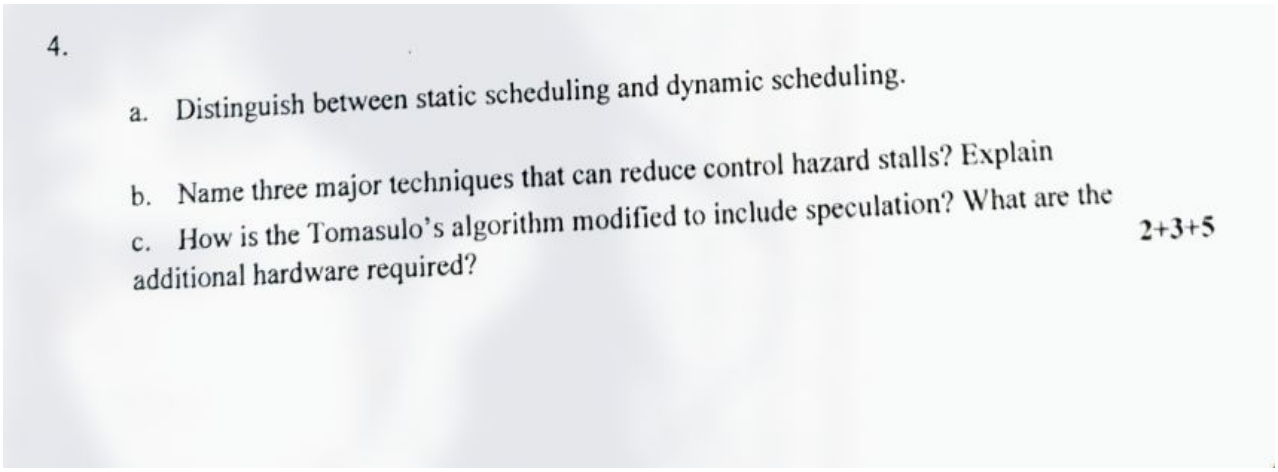
What the page says. The rules the scheduler (hardware or compiler) must obey when it reorders instructions to remove stalls.

- **1) Respect RAW.** Never move an instruction before the instruction that produces its input.
- **2) Respect latency.** Keep independent instructions between a producer and its consumer to cover the delay.
- **3) Respect memory dependences.** A load or store must not be moved past another memory access to the same address.
- **4) Respect control dependences.** Do not move an instruction past the branch it depends on (unless speculating).
- **5) Rename away WAW/WAR.** If two instructions clash on a name but not on real data, give them different registers so they can be reordered.

```
Example (WAR clash): ADD F4, F0, F2 MUL F4, F6, F8 ; writes F4 again (WAW with the ADD) Fix by renaming the second destination: ADD F4, F0, F2 MUL F10, F6, F8 ; now independent – can be reordered freely
```

Static vs dynamic. *Static* scheduling is done by the compiler at build time (fixed for the pipeline). *Dynamic* scheduling is done by the hardware at run time (scoreboard, Tomasulo) and can adapt to conditions the compiler cannot foresee — e.g. a cache miss.

Previous-year question 28 — ca28 — Static vs dynamic scheduling; control-hazard tec…



Original past-paper scan ca28.png (re-rendered).

Transcription of the question

```
4. a. Distinguish between static scheduling and dynamic scheduling.
   b. Name three major techniques that can reduce control-hazard stalls. Explain.
   c. How is Tomasulo's algorithm modified to include speculation? What additional
      hardware is required?
```

Full solution

a — static vs dynamic scheduling

	Static	Dynamic
When	Compile time	Run time
Sees	Program text only	Actual conditions (cache misses)
Examples	Unrolling, software pipelining	Scoreboard, Tomasulo, speculation

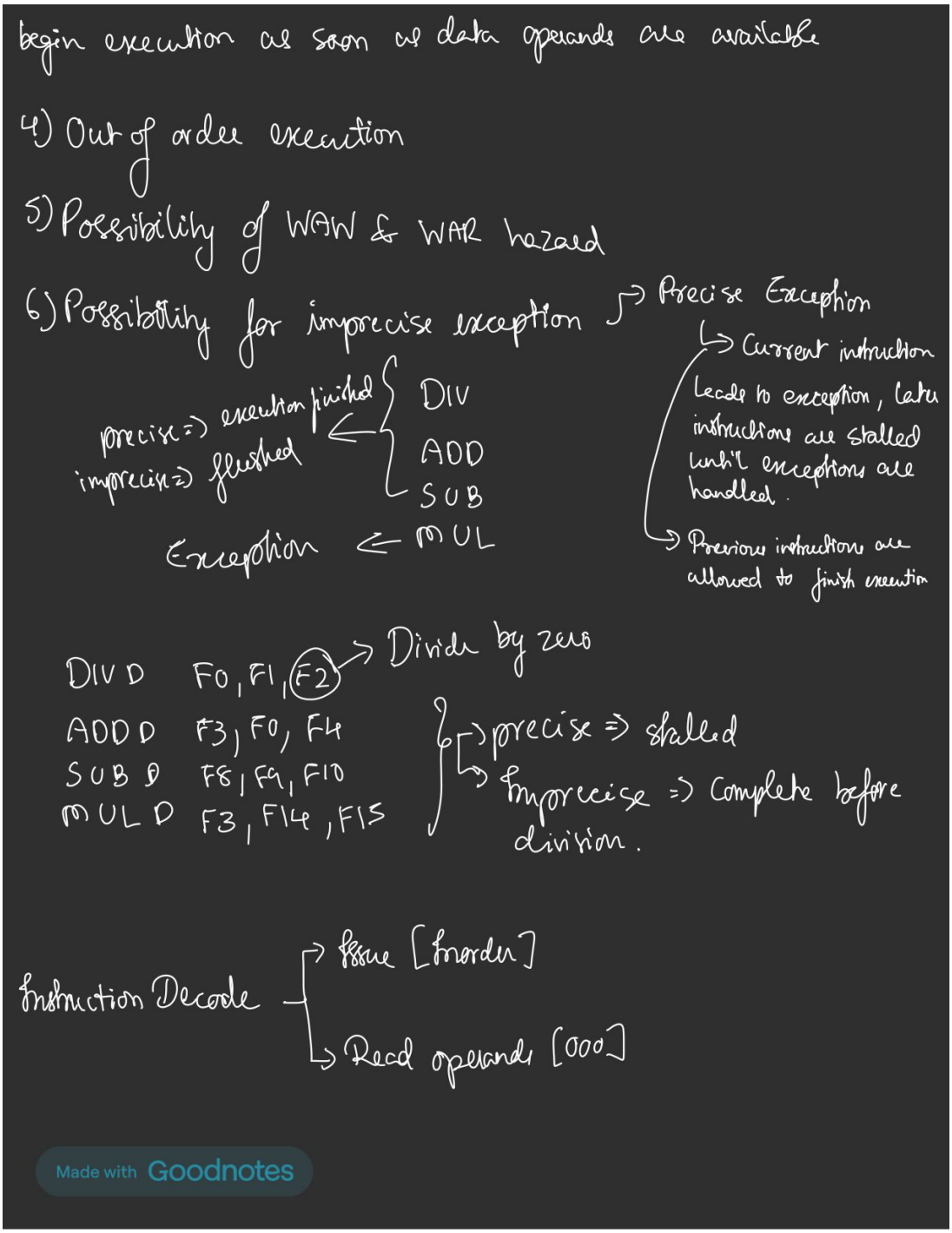
b — three control-hazard techniques

- **1) Branch prediction** — guess the direction and fetch down the predicted path.
- **2) Delayed branch** — the compiler fills the branch-delay slot with a useful instruction.
- **3) Flush on misprediction** — squash the wrong-path instructions when the branch resolves.

c — Tomasulo with speculation

Add a **reorder buffer** so results are held in program order and committed in order. Extra hardware: the ROB itself, a mechanism to assign ROB entries at issue, and the commit logic that writes the architectural state in order and discards speculative entries on a misprediction.

Original notes — page 53: Precise exceptions and speculation (exceptions in flight)



Original handwritten notes — page 53.

Transcription of the page

```
Precise exceptions:  when an exception occurs, the state must be exactly as if
  instructions before it completed and none after it started.

  Handling imprecise exceptions: exceptions must not corrupt the state.
  In-order commit.

  Speculation:  allow instructions to execute before it is known whether they
  should execute.

  Example (MUL → previous instructions):
    Issue MUL, execute, write back; if an earlier instruction takes an
    exception, the MUL must be discarded (squashed).

  Two kinds of pairs: {taken, not-taken} — branch outcomes.
```

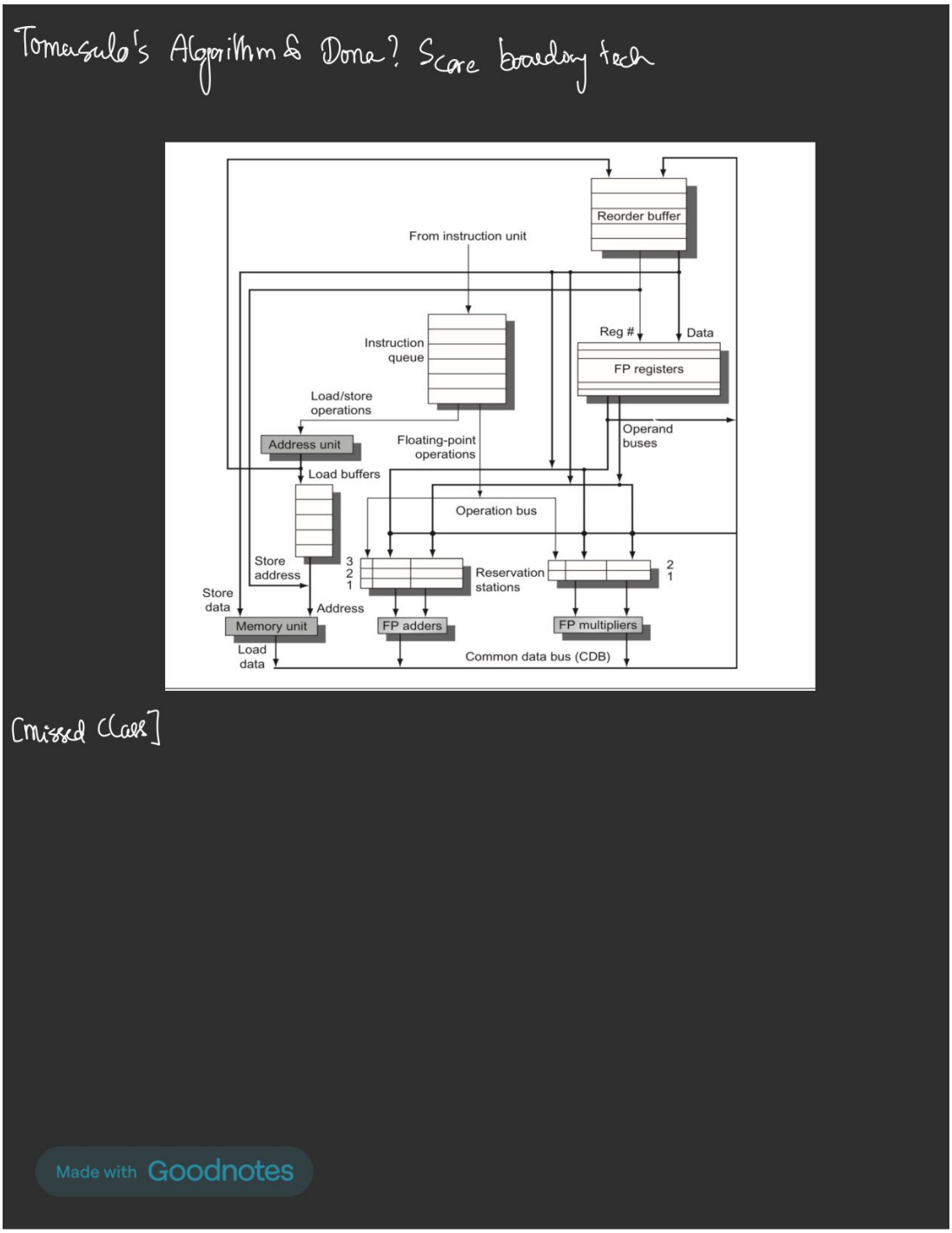
Explanation

What the page says. Precise exceptions require that when an instruction faults, the machine state is exactly as if all earlier instructions finished and no later one began. **Speculation** lets the processor run ahead (e.g. past a branch) but then forces *in-order commit* so that a wrong guess can be discarded cleanly.

- **Precise exception** — the register and memory state is consistent with the program order at the point of the faulting instruction.
- **Speculation** — execute instructions that might not belong to the final path (past a predicted branch), keeping their results in a buffer.
- **Squash / flush** — if the guess was wrong (or an earlier instruction faults), throw away the speculative results and restart from the correct point.
- **In-order commit** — results become visible (written to the architectural state) strictly in program order, which is what makes exceptions precise.

```
MUL F0, F2, F4 ; execute speculatively … if an earlier instruction raises an exception, the MUL’s result must NOT be committed — it is squashed and the register file is restored to the state before the faulting instruction.
```

Original notes — page 54: Tomasulo hardware — the full datapath with a reorder buffer…



Original handwritten notes — page 54.

Transcription of the page

```
Tomasulo's Algorithm (hardware block diagram):

  From instruction unit → Instruction queue
  Load/store operations → Load buffers / Store address & data
  FP registers
  Operand buses
  Address unit
  FP adders, FP multipliers
  Reservation stations
  Common Data Bus (CDB)
  Reorder buffer
  Load data

  (Busy / Address / Op / Vj / Vk / Qj / Qk fields shown in the stations.)
```

Explanation

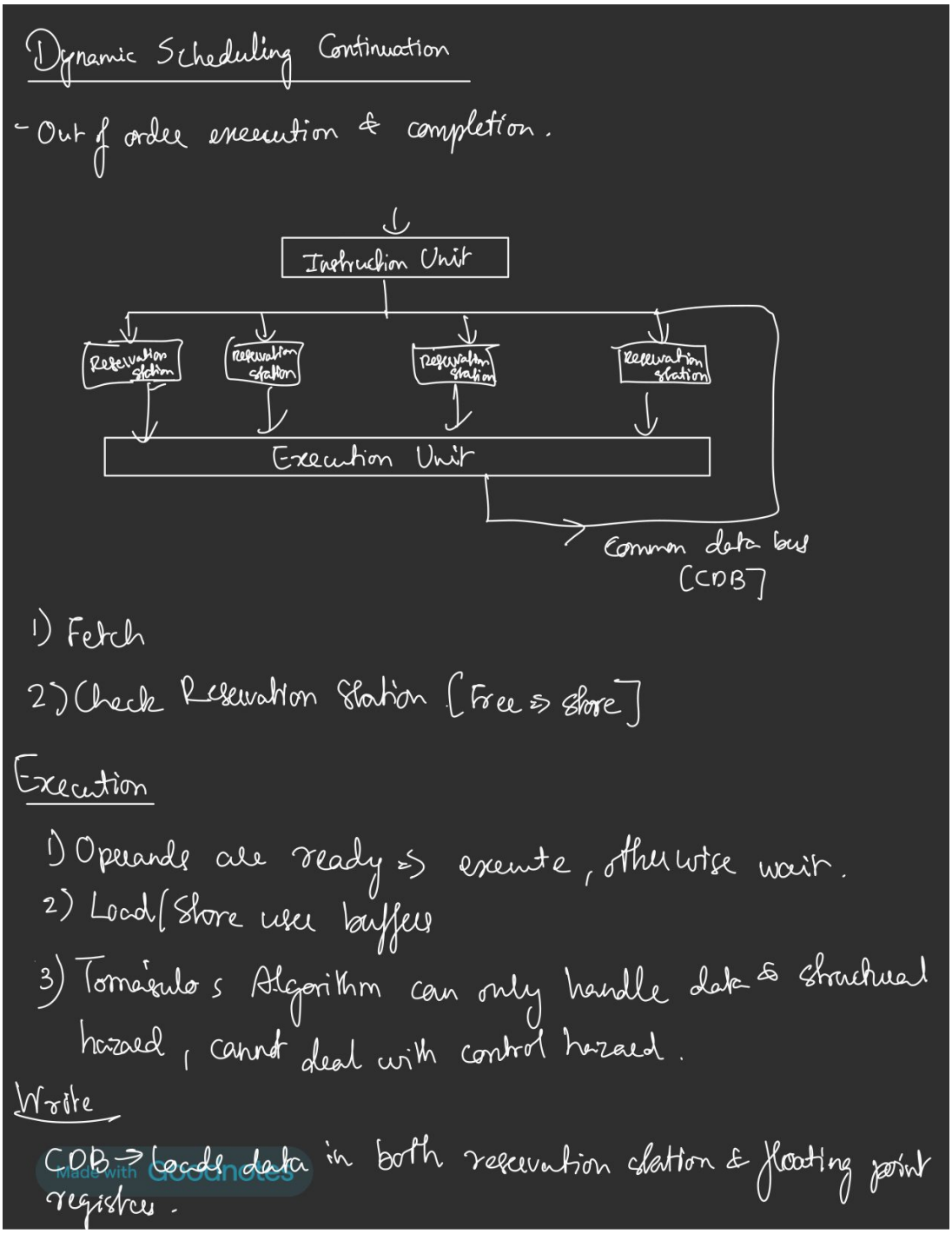
What the page says. The complete Tomasulo datapath. This is the diagram to reproduce in the exam.

- **Instruction queue** — holds decoded instructions waiting to issue.
- **Reservation stations** — one group per functional unit (add, multiply, load, store); each holds an instruction's op and operands, with Qj/Qk tags for missing values.
- **Functional units** — the FP adders and multipliers that execute; they put their results on the CDB.
- **Load/store buffers** — hold the address and data for memory operations.
- **Common Data Bus (CDB)** — broadcasts (tag, value) from a finishing unit to every station and to the register file.
- **Register file / result status** — for each register, which station will produce it.
- **Reorder buffer** — added for speculation; holds results until commit, in program order.

```
Data flow: queue → reservation station → (operands ready?) → functional unit → CDB broadcast
→ waiting stations grab the value → ROB holds the result until in-order commit → register
file.
```

How to draw it. Left to right: instruction queue, then the reservation-station groups, then the functional units; a single horizontal **CDB** line runs across, tapped by every station and by the register file. The ROB sits between the units and the register file.

Original notes — page 55: Speculation — issue, execute, read, write and commit



Original handwritten notes — page 55.

Transcription of the page

```
Speculation:  out-of-order execution with completion.

  Can do a load early:
   1) Fetch
   2) Issue / execute (out of order)
   3) Write result (but not to architectural state yet)
   4) Commit (in order) – only then is the state updated

  Cannot commit out of order:
   1) Fetch,
   2) Execute,
   3) Commit – when all prior instructions have committed,
   4) The result is written to the register file / memory.

  Commit stage = in-order update of the architectural state.
```

Explanation

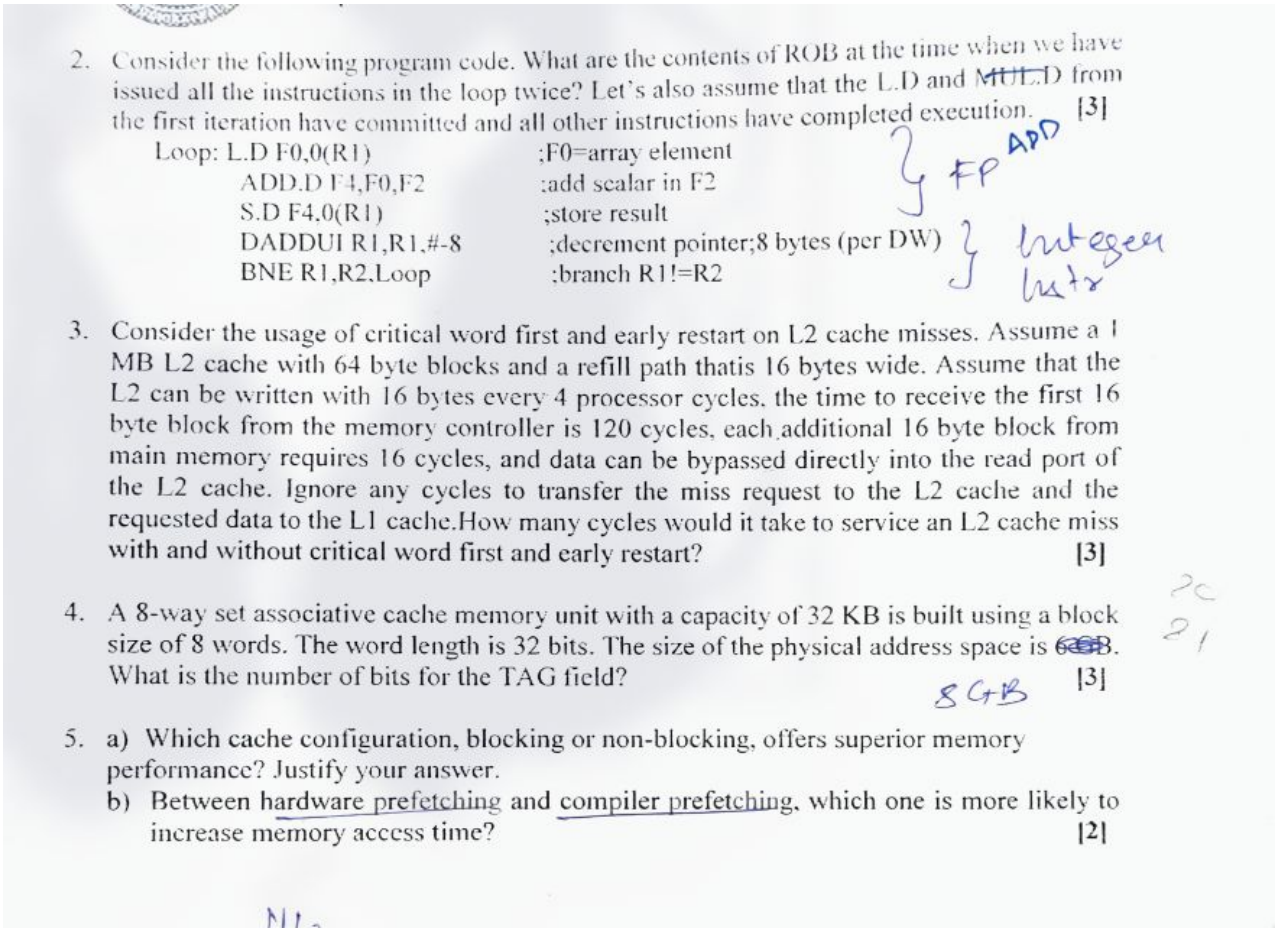
What the page says. The five stages of a speculative pipeline. The first four may run out of order; the last one (commit) is strictly in program order.

Stage	In order?	What happens
1 Issue	yes	Get the instruction into the machine
2 Execute	no	Run when the operands are ready
3 Read	no	Read operands from the CDB / register file
4 Write	no	Write the result into the ROB (speculative)
5 Commit	yes	Copy the ROB value into the architectural state

```
Issue → Execute → Read → Write (ROB) → Commit (in order) Only Commit changes the visible state; a mispredicted branch squashes the speculative entries before they commit.
```

Why this ordering gives precise exceptions. Because commit is in order, a faulting instruction can be handled exactly at its point in the program: everything before it has committed, everything after it is still speculative and can be discarded.

Previous-year question 25 — ca25 — ROB contents after two loop iterations; critical-wo…



2. Consider the following program code. What are the contents of ROB at the time when we have issued all the instructions in the loop twice? Let's also assume that the L.D and MUL.D from the first iteration have committed and all other instructions have completed execution. [3]

```
Loop: L.D F0,0(R1)        ;F0=array element
      ADD.D F4,F0,F2      ;add scalar in F2
      S.D F4,0(R1)        ;store result
      DADDUI R1,R1,#-8    ;decrement pointer;8 bytes (per DW)
      BNE R1,R2,Loop      ;branch R1!=R2
```

3. Consider the usage of critical word first and early restart on L2 cache misses. Assume a 1 MB L2 cache with 64 byte blocks and a refill path thatis 16 bytes wide. Assume that the L2 can be written with 16 bytes every 4 processor cycles, the time to receive the first 16 byte block from the memory controller is 120 cycles, each additional 16 byte block from main memory requires 16 cycles, and data can be bypassed directly into the read port of the L2 cache. Ignore any cycles to transfer the miss request to the L2 cache and the requested data to the L1 cache.How many cycles would it take to service an L2 cache miss with and without critical word first and early restart? [3]

4. A 8-way set associative cache memory unit with a capacity of 32 KB is built using a block size of 8 words. The word length is 32 bits. The size of the physical address space is 64B. What is the number of bits for the TAG field? [3]

5. a) Which cache configuration, blocking or non-blocking, offers superior memory performance? Justify your answer.
   b) Between hardware prefetching and compiler prefetching, which one is more likely to increase memory access time? [2]

Original past-paper scan ca25.png (re-rendered).

Transcription of the question

```
2. Consider the loop code. What are the contents of the ROB at the time the processor
   has issued all the instructions in the loop twice? Assume the L.D and MULT of the
   first iteration have committed and all other instructions have completed execution. (3)
        Loop: L.D   F0, 0(R1)     ; F0 = array element
              ADD.D F4, F0, F2    ; add scalar in F2
              S.D   F4, 0(R1)     ; store result
              DADDUI R1, R1, #-8  ; decrement pointer (8 bytes)
              BNE   R1, R2, Loop  ; branch R1 != R2
3. Critical word first and early restart on L2 misses: 1 MB L2, 64-byte blocks, 16-byte
   refill path; the L2 can be written 16 bytes every 4 processor cycles; first 16-byte
   block from the memory controller takes 120 cycles; each additional 16-byte block takes
   16 cycles; data bypassed directly into the read port. Ignore request/return cycles.
   How many cycles to service an L2 miss with and without critical word first? (3)
4. An 8-way set-associative cache of 32 KB, block size 8 words, word length 32 bits,
   physical address space 64 GB. What is the number of TAG bits?
5. a) Blocking or non-blocking cache – which is better? Justify.
   b) Hardware or compiler prefetching – which is more likely to increase memory access
      time?
```

Full solution

2 — ROB contents after two iterations

With the first iteration's L.D and MULT already committed, the ROB still holds the instructions of the second iteration (and any uncommitted ones from the first) in program order. List each entry with its destination register, the produced value (if ready) and the ready/exception flags. The head of the ROB is the oldest uncommitted instruction.

3 — critical word first

Block = 64 B = four 16-byte beats. Without CWF: wait for the first beat (120) + 3 more beats × 16 = 120 + 48 = 168 cycles. With CWF: the requested 16-byte beat is delivered first (120 cycles), and the CPU restarts as soon as it arrives → ≈ 120 cycles to begin using the data (the remaining beats fill in the background).

4 — tag bits

word = 32 bits = 4 B ; block = 8 words = 32 B → offset = $\log_2 32$ = 5 bits lines = 32 KB / 32 B = 1024 lines ; 8-way → sets = 1024/8 = 128 → index = 7 bits physical address = 64 GB = 2^{36} → 36 bits TAG = 36 − 7 − 5 = 24 bits

5 — blocking/non-blocking and prefetching

- **Non-blocking** is superior: it allows several misses to be outstanding at once, so the cache keeps serving other accesses during a miss instead of stalling.
- **Compiler (software) prefetching** is more likely to *increase* effective memory access time if it is wrong, because a mis-timed or unused prefetch consumes bandwidth and can evict useful data; hardware prefetching is more conservative.

Previous-year question 40 — ca40 — Tomasulo + ROB/commit; vector vs hybrid (GPU) …

to accommodate stated latencies. (Hint: An instruction with latency +2 requires two <stall> cycles to be inserted into the code sequence. Think of it this way: A one-cycle instruction has latency 1 + 0, meaning zero extra wait states. So, latency 1 + 1 implies one stall cycle; latency 1 + N has N extra stall cycles.

[5]

d. Discuss the role Tomasulo's Algorithm in OOO execution with an example and explain the necessity of Commit stage and the role of reorder buffer in hardware based speculation. [Draw diagram and summarize your answer pointwise].

[5]

4. a. Compare the performance of a vector processor with a hybrid system that contains a scalar processor and a GPU-based coprocessor. In the hybrid system, the host processor has superior scalar performance to the GPU, so in this case all scalar code is executed on the host processor while all vector code is executed on the GPU. We will refer to the first system as the vector computer and the second system as the hybrid computer. Assume that your target application contains a vector kernel with an arithmetic intensity of 0.5 FLOPs per DRAM byte accessed; however, the application also has a scalar component which that must be performed before and after the kernel in order to prepare the input vectors and output vectors, respectively. For a sample dataset, the scalar portion of the code requires 400 ms of execution time on both the vector processor and the host processor in the hybrid system. The kernel reads input vectors consisting of 200 MB of data and has output data consisting of 100 MB of data. The vector processor has a peak memory bandwidth of 30 GB/sec and the GPU has a peak memory bandwidth of 150 GB/sec. The hybrid system has an additional overhead that requires all input vectors to be transferred between the host memory and GPU local memory before and after the kernel is invoked. The hybrid system has a direct memory access (DMA) bandwidth of 10 GB/sec and an average latency of 10 ms. Assume that both the vector processor and GPU are performance bound by memory bandwidth. Compute the execution time required by both computers for this application.

[5]

Original past-paper scan ca40.png (re-rendered).

Transcription of the question

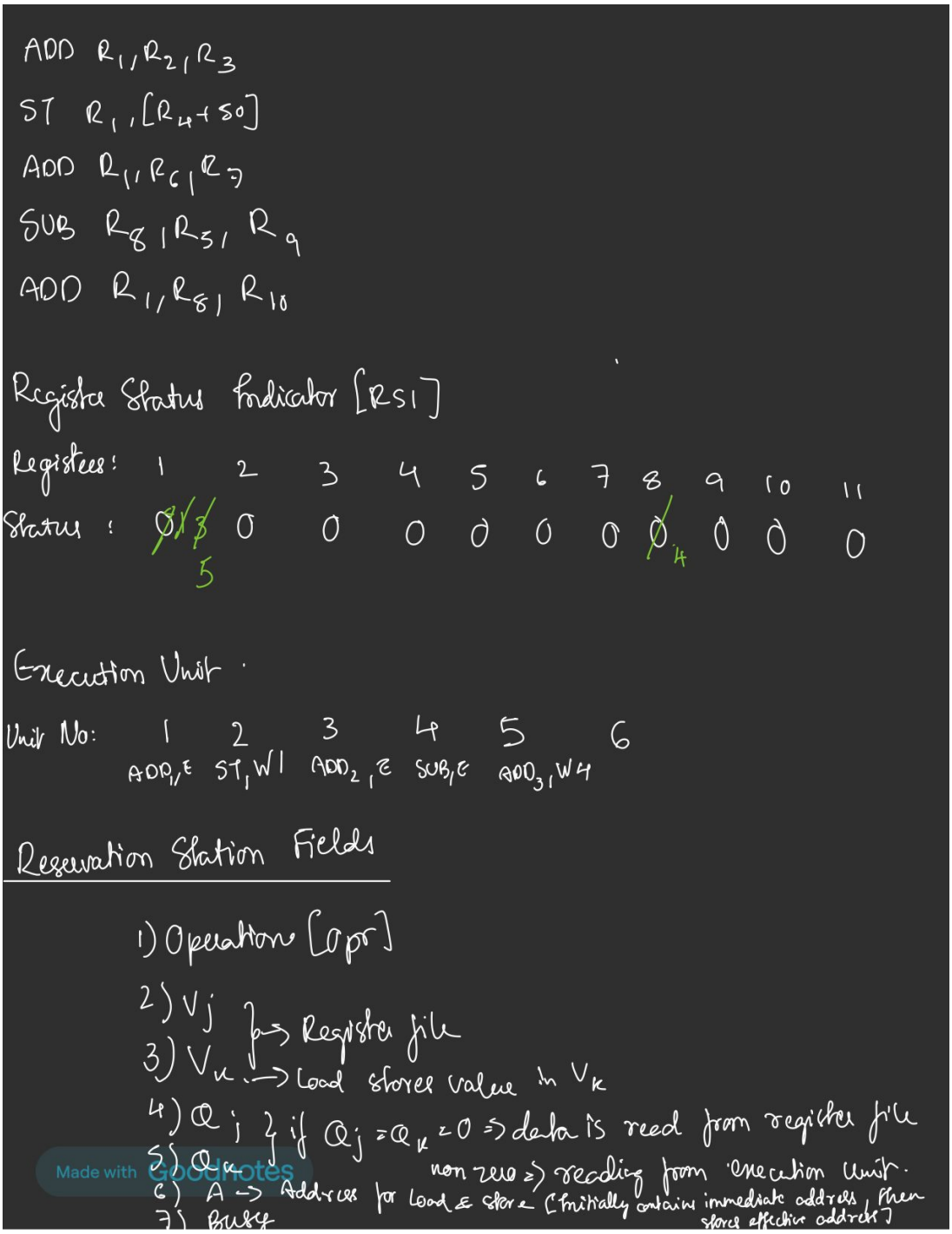
```
3d. Discuss the role of Tomasulo's algorithm in out-of-order execution with an example;
    explain the necessity of the commit stage and the role of the reorder buffer in hardware
    speculation (draw a diagram and summarise pointwise). [5]
4a. Compare a vector processor with a hybrid system (scalar host + GPU coprocessor). The
    host has superior scalar performance, so all scalar code runs on the host and all vector
    code on the GPU. The application has a vector kernel with arithmetic intensity 0.5 FLOP
    per DRAM byte; the scalar portion takes 400 ms on both. The kernel reads 200 MB and
    writes 100 MB. The vector processor has 30 GB/s bandwidth; the GPU 150 GB/s. The
    hybrid has an extra DMA overhead: all input vectors transferred host↔GPU at 10 GB/s
    with an average latency of 10 ms. Both are memory-bandwidth bound. Compute the
    execution time of both systems. [8]
```

Full solution

4a — vector vs hybrid execution time

Data moved by the kernel = read 200 MB + write 100 MB = 300 MB Vector processor (30 GB/s): kernel time = 300 MB / 30 GB/s = 0.300/30 s = 10 ms total = 400 ms (scalar) + 10 ms = 410 ms Hybrid (GPU 150 GB/s + DMA 10 GB/s): kernel time = 300 MB / 150 GB/s = 2 ms DMA transfer of 200 MB input + 100 MB output = 300 MB / 10 GB/s = 30 ms plus average latency 10 ms total = 400 ms (scalar) + 2 ms + 30 ms + 10 ms = 442 ms → the vector processor is FASTER here because the hybrid’s DMA transfer overhead dominates the GPU’s bandwidth advantage.

Original notes — page 56: Tomasulo instruction-status table (worked)



Original handwritten notes — page 56.

Transcription of the page

```
Instruction status:

  Instruction   j    k    Issue   Execute   Write Result
  ----------------------------------------------------------
  L.D  F6, 34(R2)      ...
  L.D  F2, 45(R3)      ...
  MUL.D F0, F2, F4     ...
  SUB.D F8, F6, F2     ...
  DIV.D F10, F0, F6    ...
  ADD.D F6, F8, F2     ...

  (Issue / Execute / Write-result cycles filled in.)

  Read (operand) values:
    Qj = station producing Vj ;  Qk = station producing Vk
    Values read from the register file when Q = 0.
```

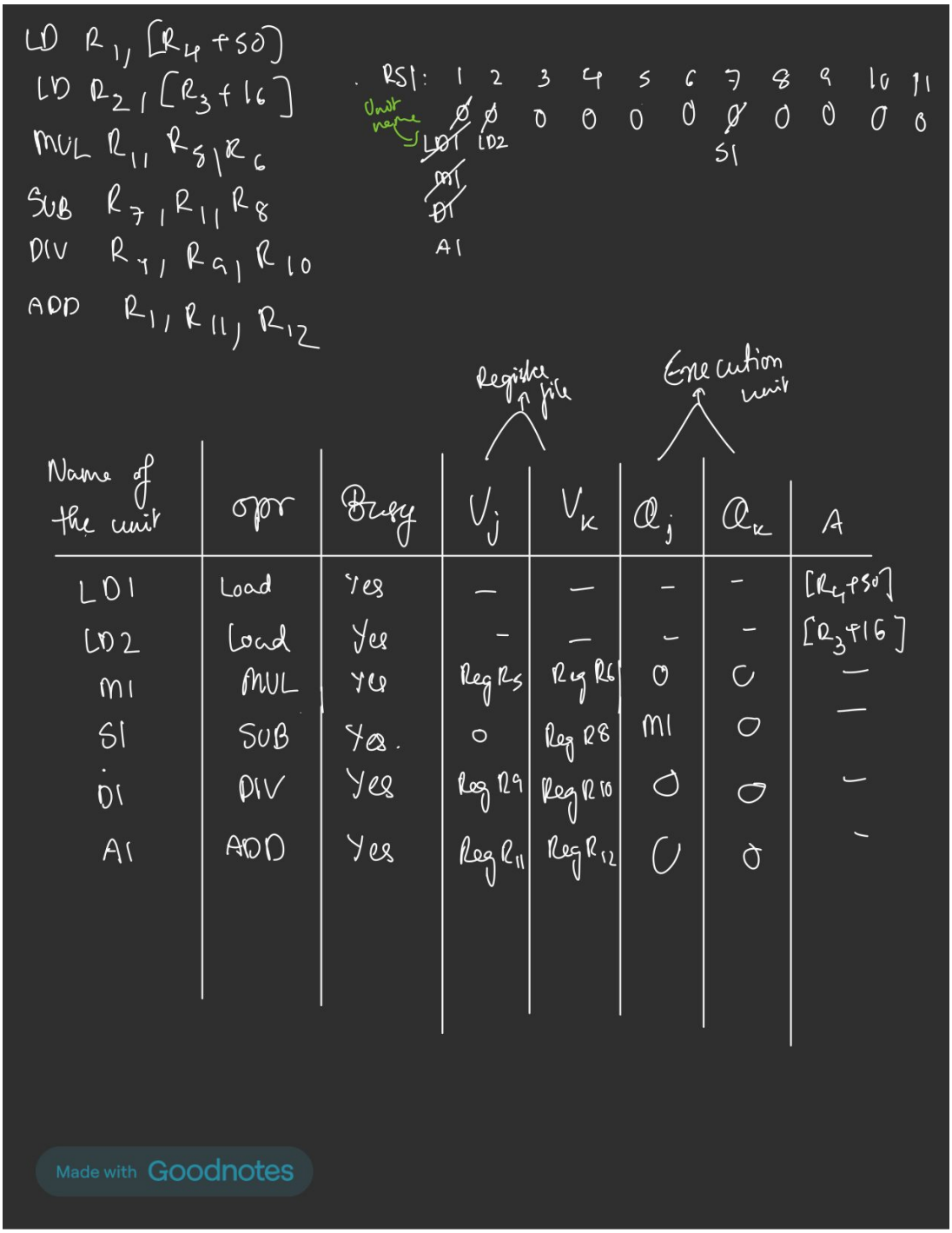
Explanation

What the page says. The instruction-status table for the canonical Tomasulo example, with the Issue, Execute and Write-result cycles recorded.

```
Canonical code and the table to fill: L.D F6, 34(R2) L.D F2, 45(R3) MUL.D F0, F2, F4 ; depends on L.D F2 SUB.D F8, F6, F2 ; depends on both loads DIV.D F10, F0, F6 ; depends on MUL and L.D F6 ADD.D F6, F8, F2 ; depends on SUB and L.D F2
```

How to fill it. At each cycle, for each instruction record when it *issues* (a station is free), when it *starts executing* (both operands ready and the unit is free) and when it *writes its result* (the unit finishes and broadcasts on the CDB).

Original notes — page 57: Tomasulo — instruction-status and reservation-station table…



Original handwritten notes — page 57.

Transcription of the page

```
Worked Tomasulo example:

  Clock:  1 2 3 4 5 6 7 8 9 10

  L.D   F6, 34(R2)     ; Load1
  L.D   F2, 45(R3)     ; Load2
  MUL.D F0, F2, F4     ; Mult1
  SUB.D F8, F6, F2     ; Add1
  DIV.D F10, F0, F6    ; Mult2
  ADD.D F6, F8, F2     ; Add2

  Reservation stations:
   Name  Busy  Op   Vj   Vk   Qj   Qk   A
   Load1  Yes  Load  -    -    -    -   34+R2
   Load2  Yes  Load  -    -    -    -   45+R3
   Add1   Yes  SUB  Mem  -    Load1 Load2 -
   Mult1  Yes  MUL  -    -    Load2  -   -
   ...
```

Explanation

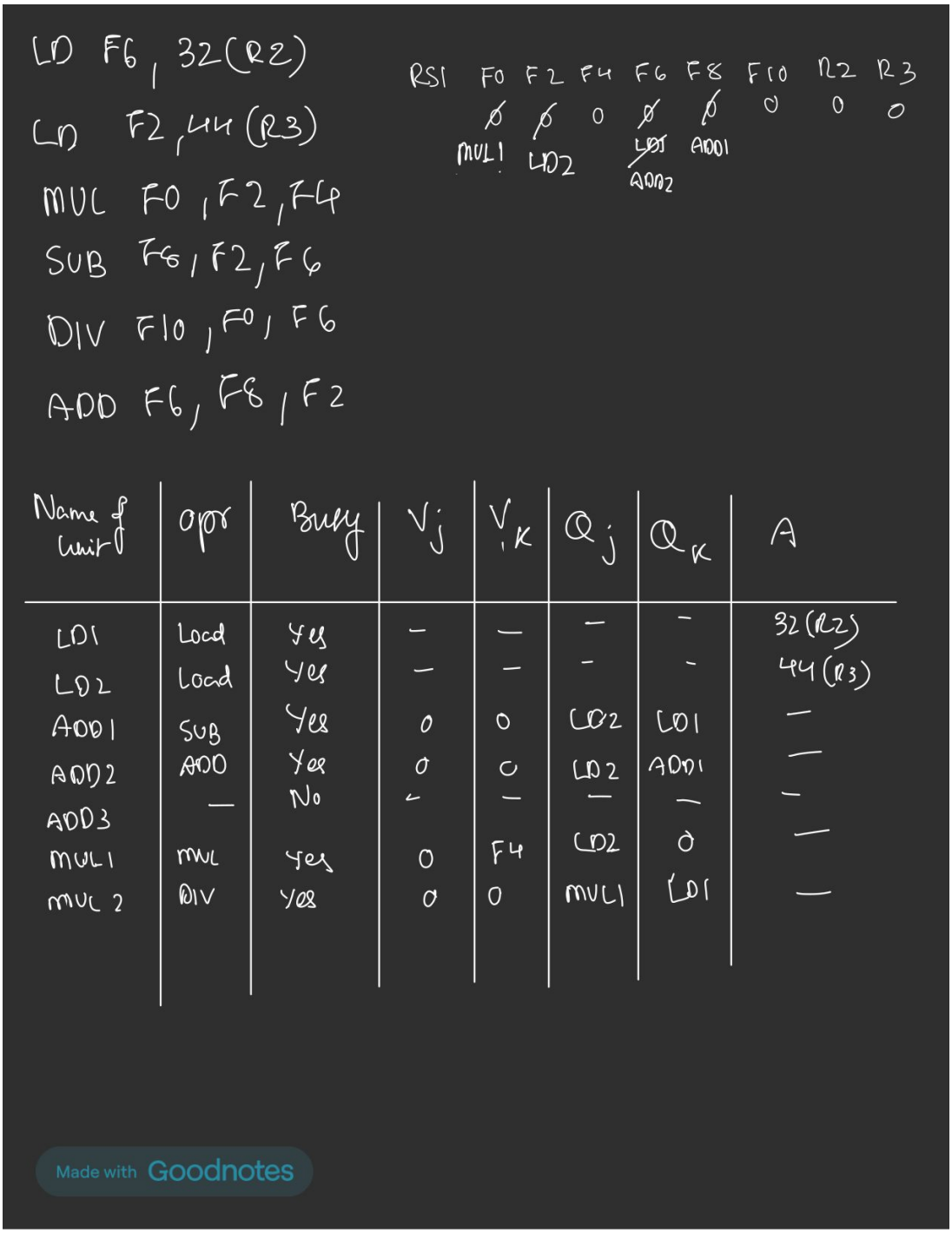
What the page says. The complete worked Tomasulo example with both tables filled in cycle by cycle. This is exactly what an exam question asks you to produce.

```
Reservation-station fields at issue time (example): Add1 (SUB): Op=SUB, Qj=Load1 (needs F6), Qk=Load2 (needs F2) Mult1 (MUL): Op=MUL, Qj=Load2 (needs F2), Vk=F4 (ready) Mult2 (DIV): Op=DIV, Qj=Mult1 (needs F0), Qk=Load1 (needs F6) Add2 (ADD): Op=ADD, Qj=Add1 (needs F8), Qk=Load2 (needs F2)
```

- The loads issue first; when Load1 finishes it broadcasts its value on the CDB, so Add1 and Mult2 clear their Qj/Qk fields.
- MUL can start as soon as F2 arrives; DIV waits for both F0 (from MUL) and F6.
- Each broadcast frees every waiting station at once — this is the parallelism Tomasulo exploits.

The full transcription of the printed question and the complete cycle-by-cycle solution are given in the past-paper solutions (ca12).

Original notes — page 58: Tomasulo tables (continued — second worked set)



Original handwritten notes — page 58.

Transcription of the page

```
Tomasulo — second worked set:

  L.D   F2, 45(R3)
  L.D   F6, 0(R2)
  MUL.D F0, F2, F4
  SUB.D F8, F6, F2
  DIV.D F10, F0, F6
  ADD.D F6, F8, F2

  Reservation stations (Load1, Load2, Add1, Add2, Mult1, Mult2)
  and register result status (which station writes each F register).
```

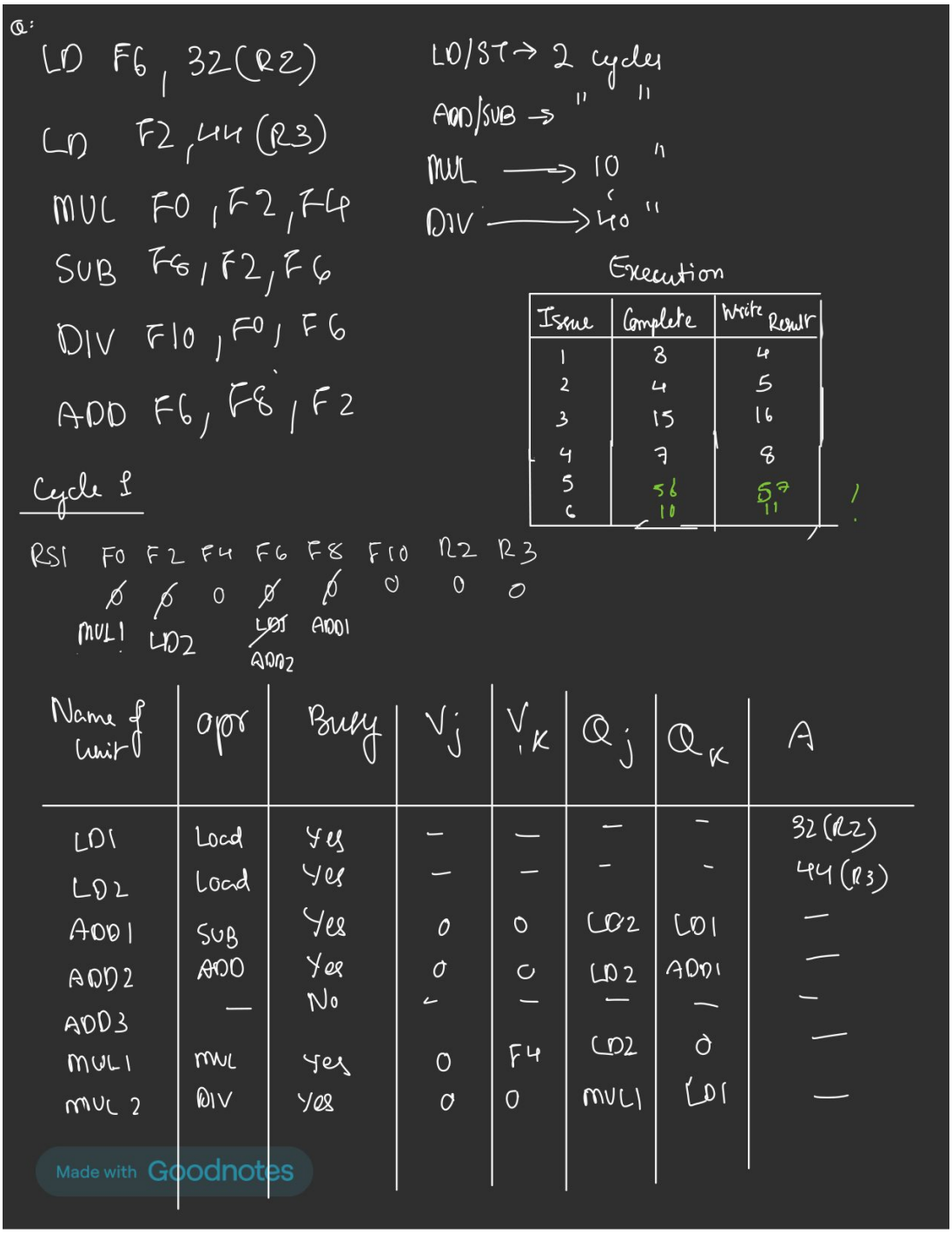
Explanation

What the page says. A second worked Tomasulo set with the same structure but different latencies, to practise filling the tables.

```
Register result status at the start (example): F0 → Mult1 (MUL.D) F6 → Load2, then Add2 F8 → Add1 (SUB.D) F10 → Mult2 (DIV.D)
```

Work through it exactly as on page 57: issue, execute when operands are ready, write result on the CDB, and update the register-result-status table as each producer changes.

Original notes — page 59: Tomasulo tables (continued — with cycle numbers)



Original handwritten notes — page 59.

Transcription of the page

```
Tomasulo — cycles:

  L.D   F6, 34(R2)   ; cycle 1 issue
  L.D   F2, 45(R3)   ; cycle 2
  MUL.D F0, F2, F4   ; cycle 3
  SUB.D F8, F6, F2   ; cycle 4
  DIV.D F10, F0, F6  ; cycle 5
  ADD.D F6, F8, F2   ; cycle 6

  (Execute and write-result cycles computed from the latencies.)
```

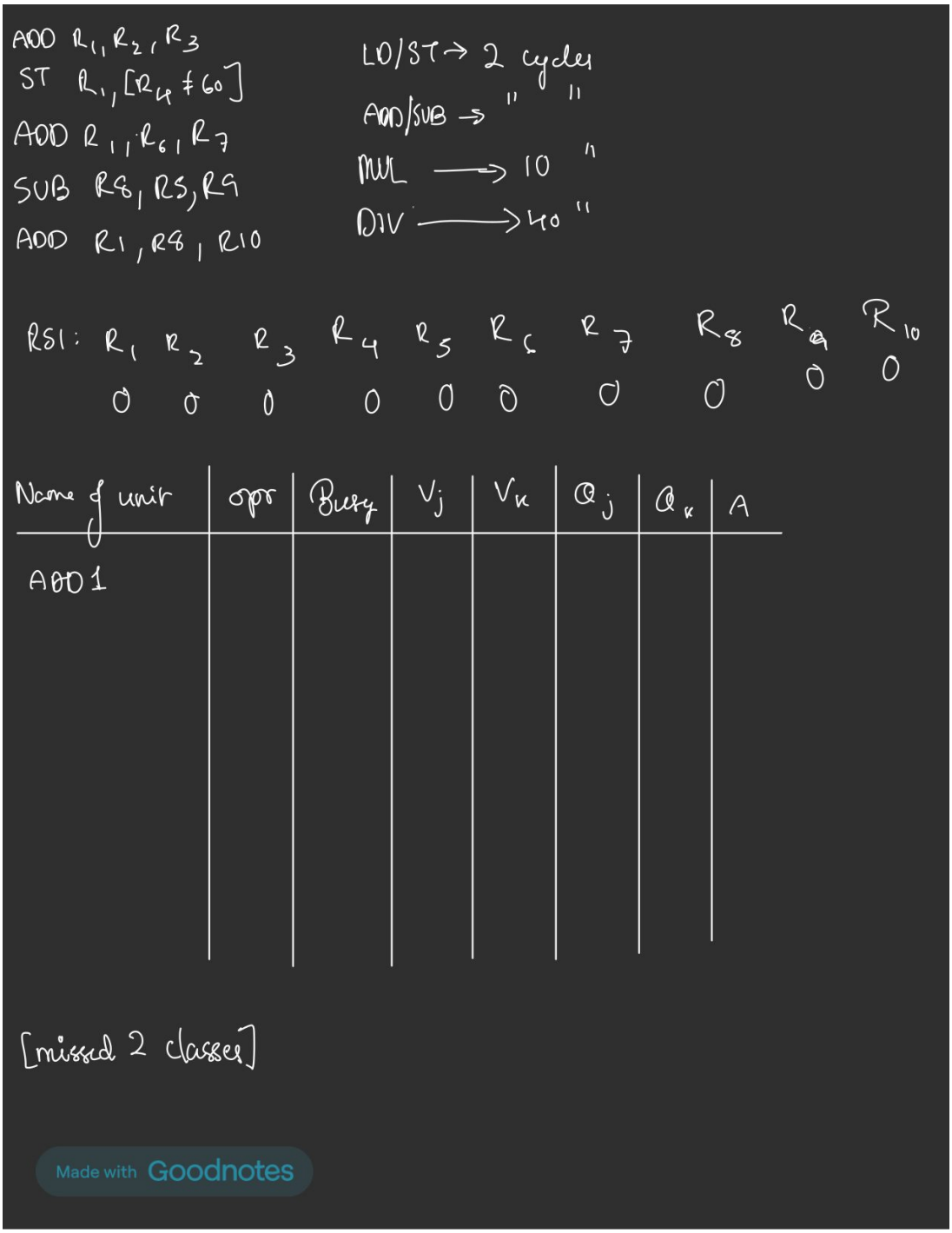
Explanation

What the page says. The same example with explicit cycle numbers, showing how the issue order and the operand-ready times determine the schedule.

```
Rule for the Execute column: Execute_start = max(issue cycle + 1, cycle when last operand
arrives, cycle when the functional unit becomes free) Write_result = Execute_start + latency
```

This is the arithmetic that turns the tables into a schedule. Practise it with the printed question in the past-paper solutions (ca12).

Original notes — page 60: Tomasulo tables (continued — final set)



Original handwritten notes — page 60.

Transcription of the page

```
Tomasulo — final worked set:

  ADD.D F2, F4, F6
  ...
  SUB.D F8, F2, F6
  MUL.D F10, F2, F6
  ...
  (register result status: F2 → Add1, F8 → Add2, F10 → Mult1 …)

  Two classes of stalls identified.
```

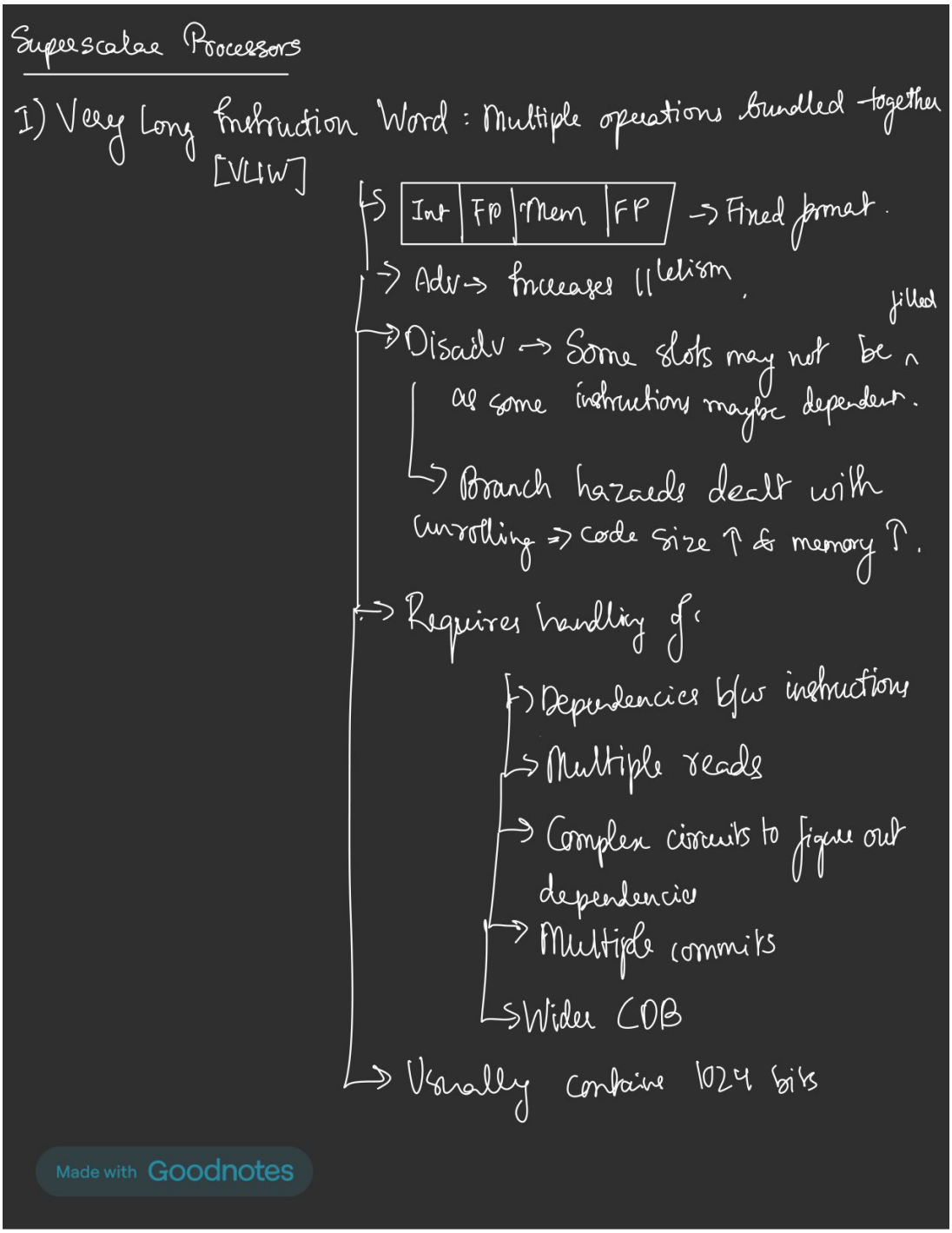
Explanation

What the page says. The last Tomasulo worked set, ending with a note that stalls fall into two classes.

```
The two classes of stall: 1) RAW / data stall — waiting for a real operand (Qj or Qk
non-zero). 2) Structural stall — the required functional unit or station is busy. Tomasulo
removes the WAR/WAW stalls (via renaming) but keeps RAW and structural ones.
```

This completes the Tomasulo block of the notes. The ideas are consolidated in the past-paper solutions (ca12, ca18, ca34, ca39, ca40).

Original notes — page 61: Superscalar processors and vector operations



Original handwritten notes — page 61.

Transcription of the page

```
Superscalar Processors:

  Issue multiple instructions per cycle.

  Needs:
   1) Multiple functional units,
   2) Wide instruction fetch (fetch several instructions at once),
   3) Dependence checking in hardware,
   4) Wide CDB / multiple buses,
   5) Register renaming.

  Vector operations:
   Operate on whole vectors (arrays) with one instruction.

  Loop operations:  the vector instruction replaces the whole loop.
```

Explanation

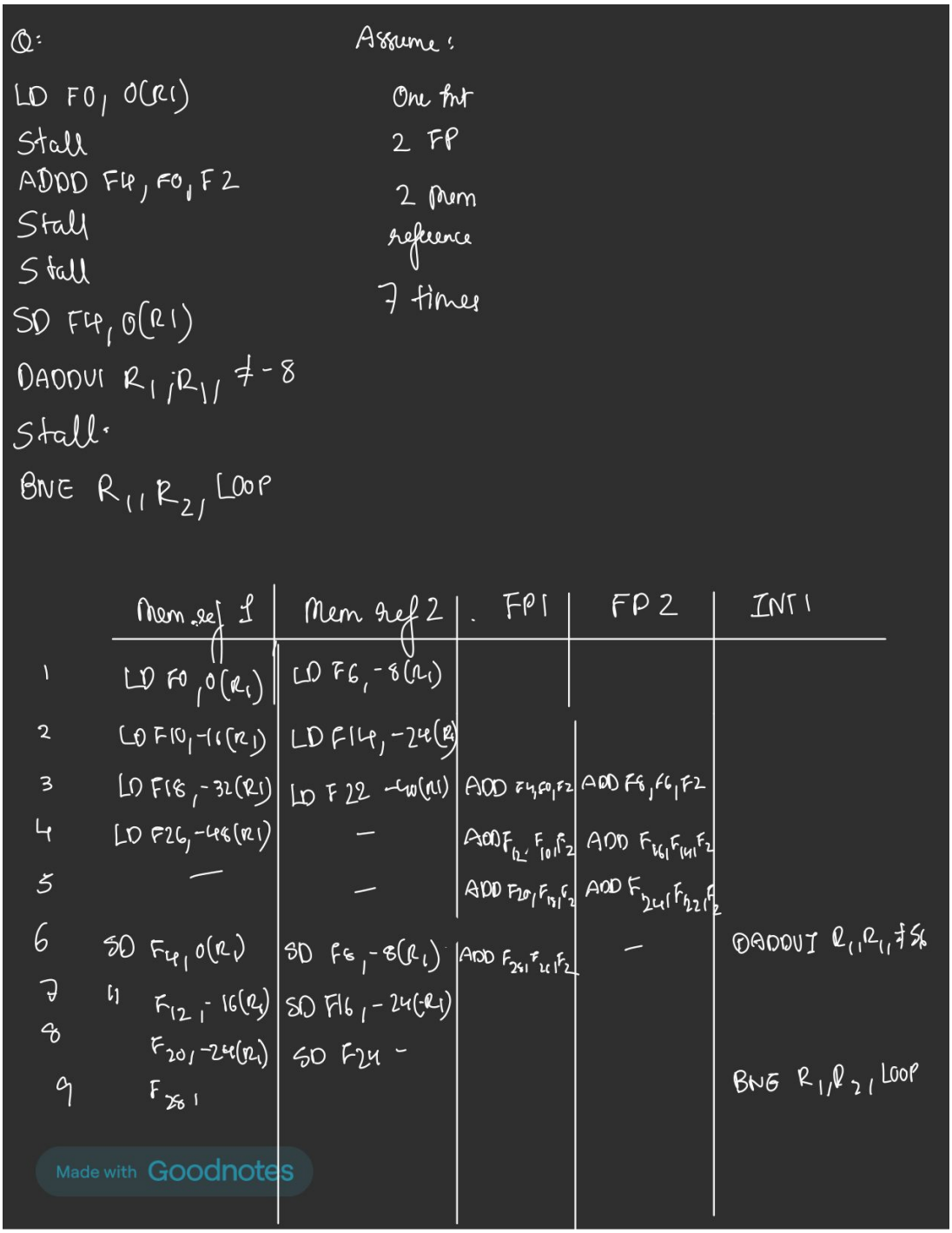
What the page says. What it takes to issue more than one instruction per cycle (superscalar), and how a vector instruction replaces a whole loop.

- **Multiple functional units** — several adders, multipliers, load/store units so more than one operation can run at once.
- **Wide instruction fetch** — the front end must deliver several instructions per cycle, which usually means a large instruction cache and alignment handling.
- **Dependence checking** — hardware must detect which of the fetched instructions can go together.
- **Wide CDB / multiple buses** — several results must be broadcast in the same cycle.
- **Register renaming** — to remove false dependences so the parallel issue slots can be filled.

```
Superscalar width m (ideal): IPC ≤ m , CPI ≥ 1/m Real IPC < m because of dependences, hazards and fetch limits. Vector: one instruction operates on VL elements: e.g. ADDV V1, V2, V3 → V1[i] = V2[i] + V3[i] for i = 0…VL−1
```

> **Superscalar vs vector.** Superscalar exploits ILP (independent scalar instructions); vector exploits DLP (the same operation on many data). GPUs combine both.

Original notes — page 62: Vector/loop example and loop scheduling with loads



Original handwritten notes — page 62.

Transcription of the page

```
Vector / loop example:

  L.D   F0, 0(R1)
  Stall
  ADD.D F4, F0, F2
  Stall
  S.D   F4, 0(R1)
  DADDUI R1, R1, #-8
  BNE   R1, R2, Loop

  Loop scheduling:
   Several iterations interleaved so the loads of one fill the stalls of another.

  Vector version:
   L.D   F0, 0(R1)   →   load a whole vector
   ADDV  V1, V0, V2
   S.D   V1, 0(R1)

  (Load latency 6, FP latency 4 …)
```

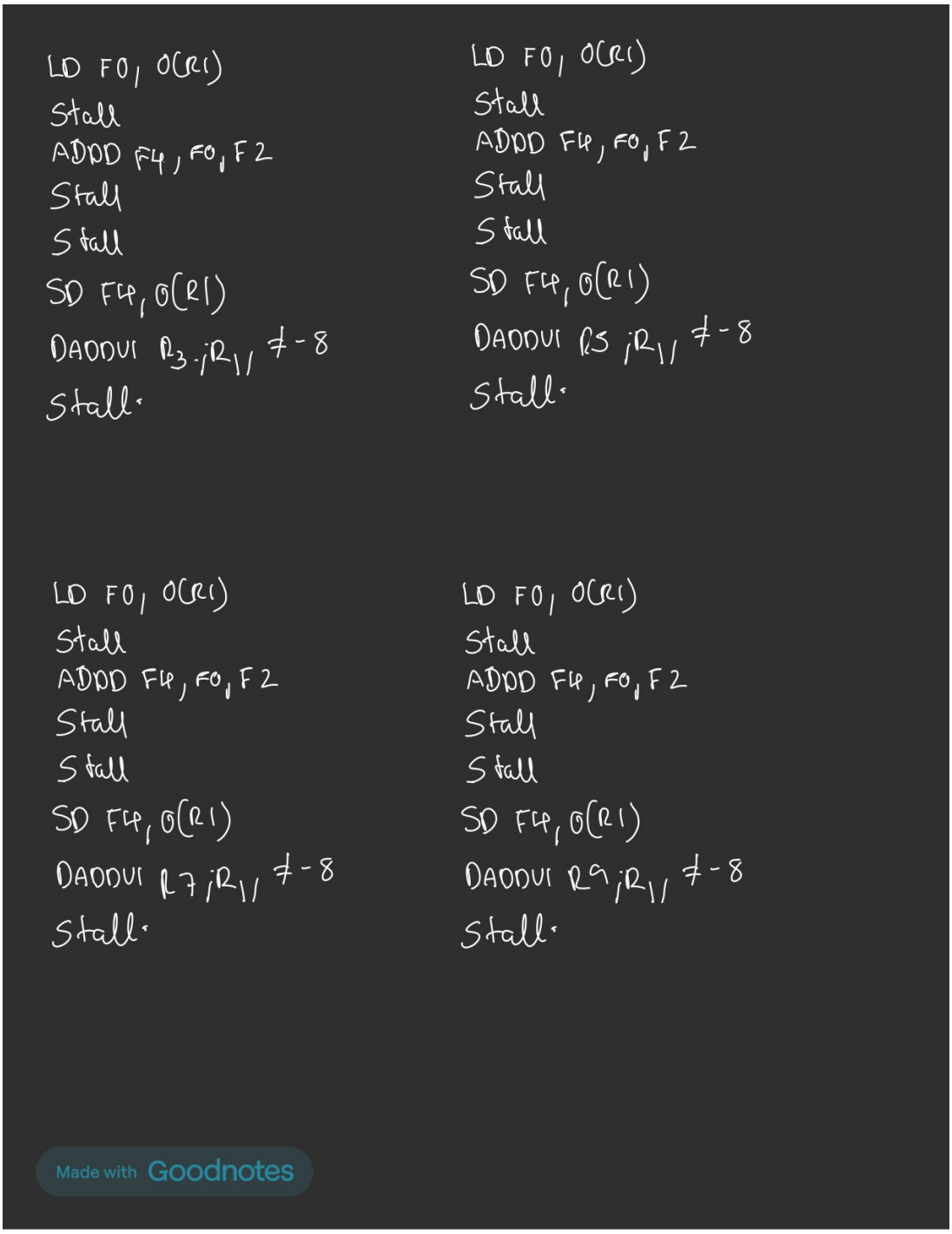
Explanation

What the page says. The scalar loop with its stalls, and how scheduling (or a vector instruction) removes them.

```
Scalar (with stalls): L.D F0, 0(R1) ; load latency ≈ 6 cycles ADD.D F4, F0, F2 ; must wait for F0 → stall S.D F4, 0(R1) DADDUI R1, R1, #-8 BNE R1, R2, Loop Scheduled (interleave iterations): the load of iteration i+1 fills the stall of iteration i, so the loop runs with no bubbles.
```

Vector version. Replace the loop with vector instructions: one load brings in a whole vector, one ADDV adds the vectors element-wise, one store writes them back. The loop control disappears and the hardware pipelines the vector lanes.

Original notes — page 63: Loop scheduling with speculation



Original handwritten notes — page 63.

Transcription of the page

```
Loop scheduling with speculation:

  LOOP:  L.D   F0, 0(R1)
         ADD.D F4, F0, F2
         S.D   F4, 0(R1)
         DADDUI R1, R1, #-8
         BNE   R1, R2, LOOP

  Speculative execution: run the next iteration before the branch is resolved.

  Two versions shown side by side:
   (a) scheduled without speculation,
   (b) scheduled with speculation (branch predicted).
```

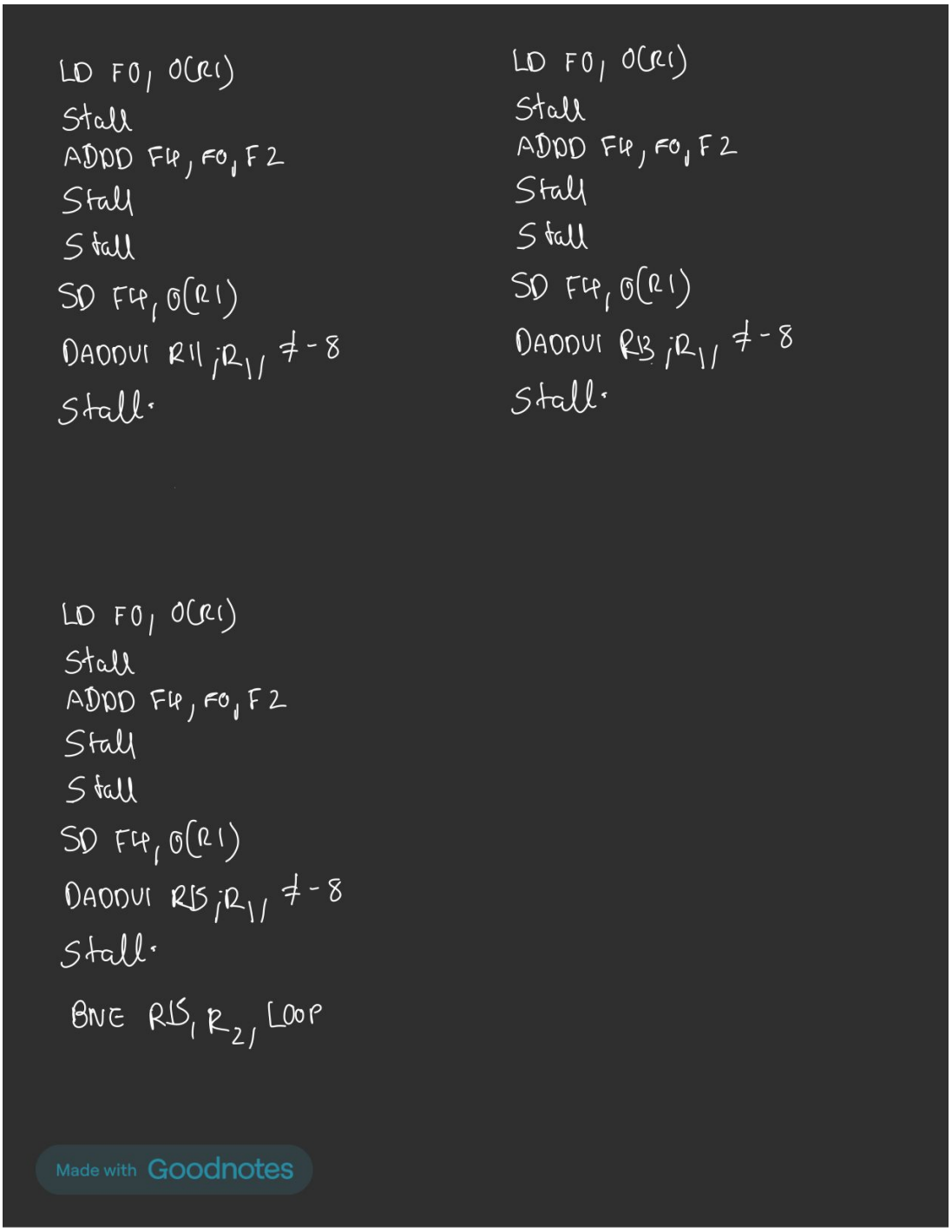
Explanation

What the page says. Combining scheduling with speculation: start the next loop iteration before the branch that controls the loop has resolved, then commit only if the prediction was right.

```
Without speculation: the loop body waits for the BNE to resolve, so the pipeline drains at every iteration. With speculation: predict the branch taken, fetch and execute the next iteration immediately; keep its results in the ROB and commit them only if the branch really was taken.
```

Why speculation is powerful here. A loop branch is almost always taken, so the predictor is right nearly every time and the pipeline never drains — the loop runs at full throughput.

Original notes — page 64: Loop scheduling with speculation (continued)



Original handwritten notes — page 64.

Transcription of the page

```
(Continuation.)
  LOOP:  L.D   F0, 0(R1)
         ADD.D F4, F0, F2
         S.D   F4, 0(R1)
         DADDUI R1, R1, #-8
         BNE   R1, R2, LOOP

  Scheduled speculatively: the loads/adds/stores of consecutive iterations
  are interleaved, with the branch predicted taken each time.
```

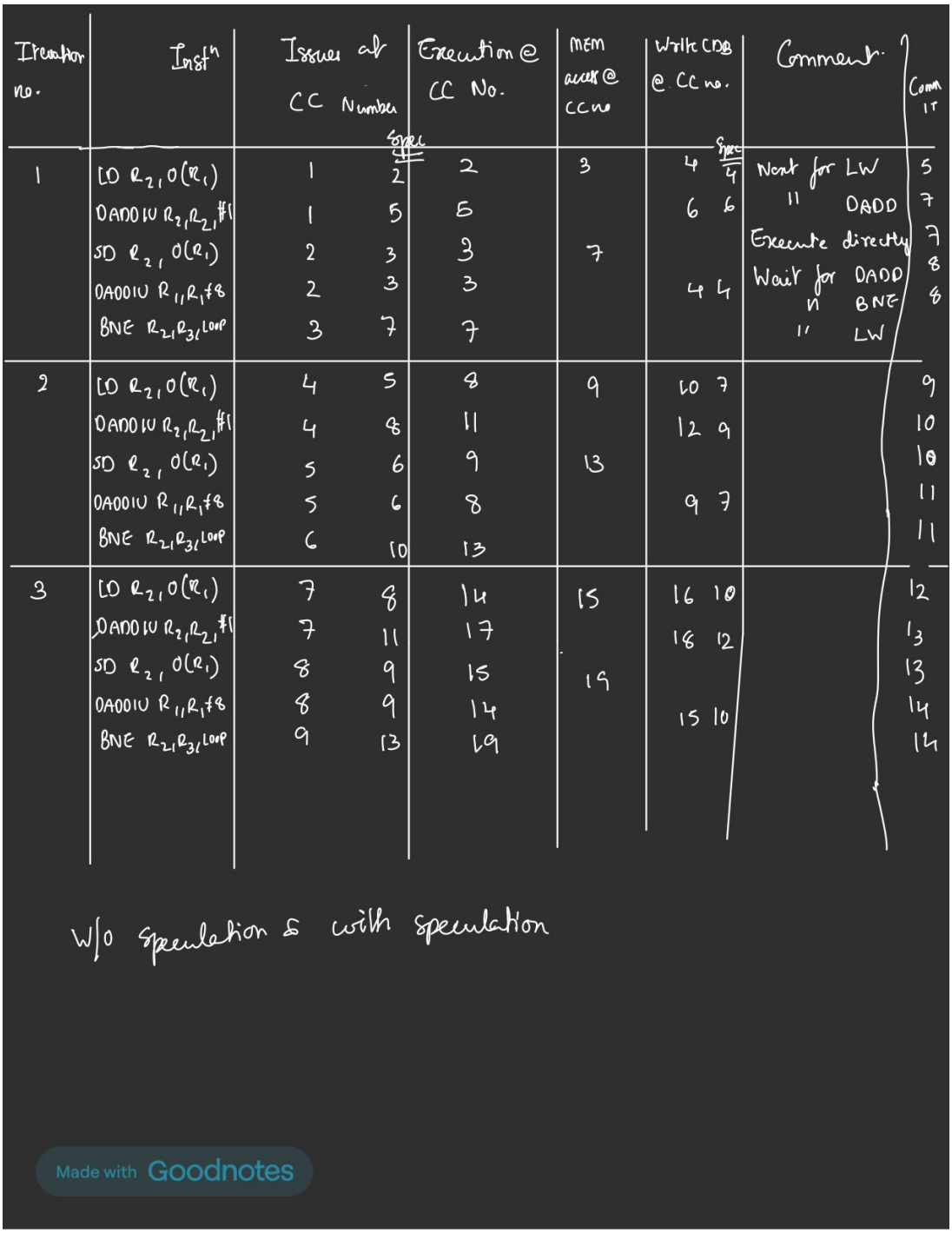
Explanation

What the page says. The speculative schedule laid out fully, showing the interleaving of successive iterations.

```
Speculative schedule (sketch): L.D(i) L.D(i+1) ADD(i) L.D(i+2) ADD(i+1) S.D(i) ADD(i+2) …
Each load is followed, several instructions later, by its use — enough independent work in
between to cover the load latency.
```

The two columns of the page are the same schedule with and without speculation; with speculation the branch no longer forces a drain, so the interleaving is tighter.

Original notes — page 65: Speculation timing table (issue / execute / read / write / com…



Original handwritten notes — page 65.

Transcription of the page

```
With speculation — timing table:

  Iteration | Issue | Execute | Read | Write | Commit
  ----------+-------+---------+------+-------+-------
     1      |   1   |    2    |  4   |   5   |   7
     1      |   2   |    3    |  5   |   6   |   8
     ...
  (cycles across the top: 1 2 3 4 5 6 7 8 9 10 11 12 …)

  The commit column is strictly in order, later than the write column.
```

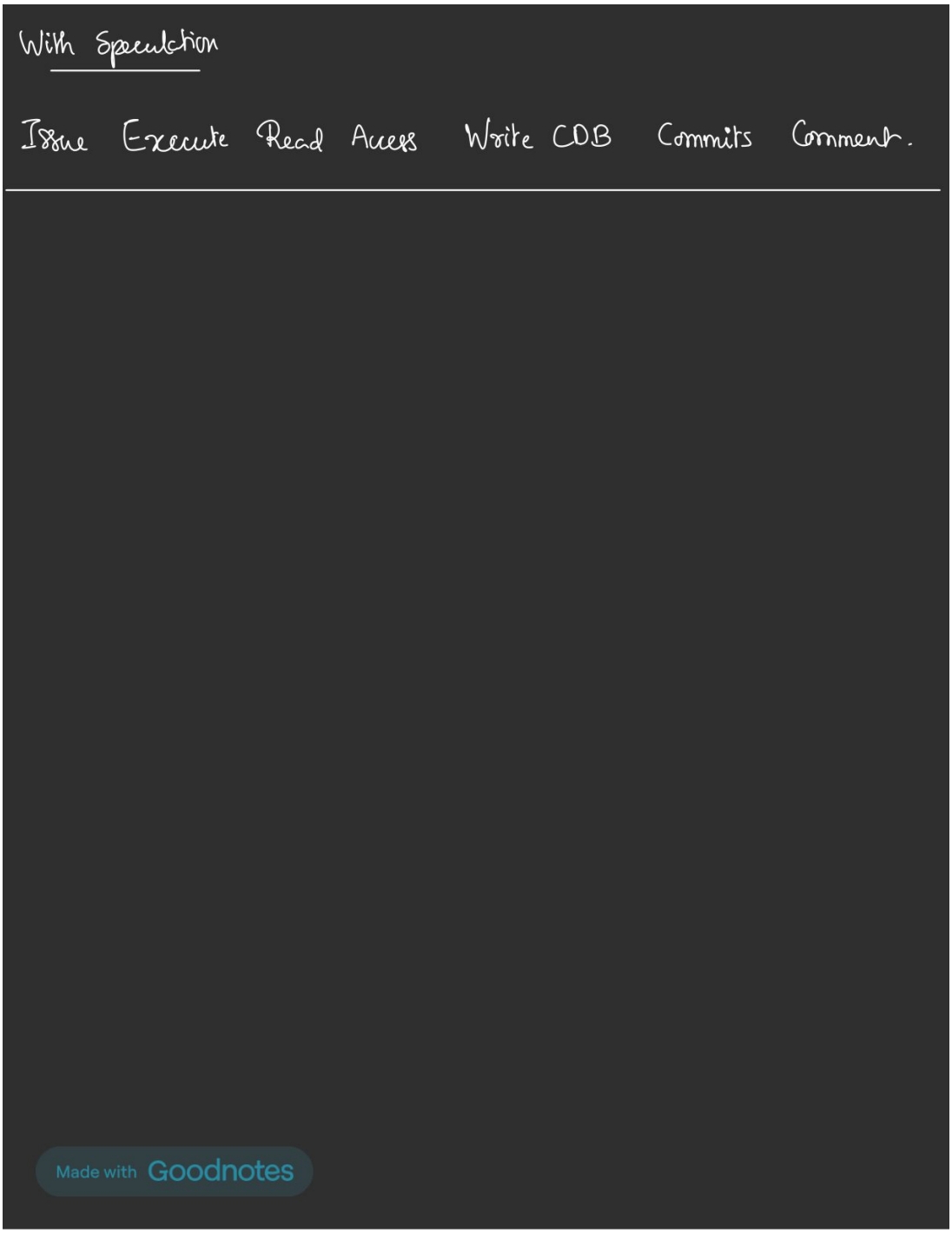
Explanation

What the page says. The timing table for the speculative pipeline, with a column for each of the five stages. Notice that the Commit column is monotonic (in order) even though the other columns may be out of order.

```
Five columns: Issue → Execute → Read → Write → Commit • Issue and Execute can be out of order;
• Write places the result in the ROB; • Commit is strictly increasing (program order).
```

How to read it. Pick an instruction row and follow it across: it issues at some cycle, executes when its operands are ready, reads them, writes its result into the ROB, and finally commits. Because commit is in order, a later instruction may write before an earlier one but never commits before it.

Original notes — page 66: Speculation timing table (continued)



Original handwritten notes — page 66.

Transcription of the page

```
With speculation (continued):

  Time | Execute | Read | Write | CDB | Commit
  -----+---------+------+-------+-----+-------
   ... (the table continues, showing the CDB broadcast and the commit order)

  Commit is the final, in-order stage; the CDB carries the values between
  the functional units and the waiting reservation stations.
```

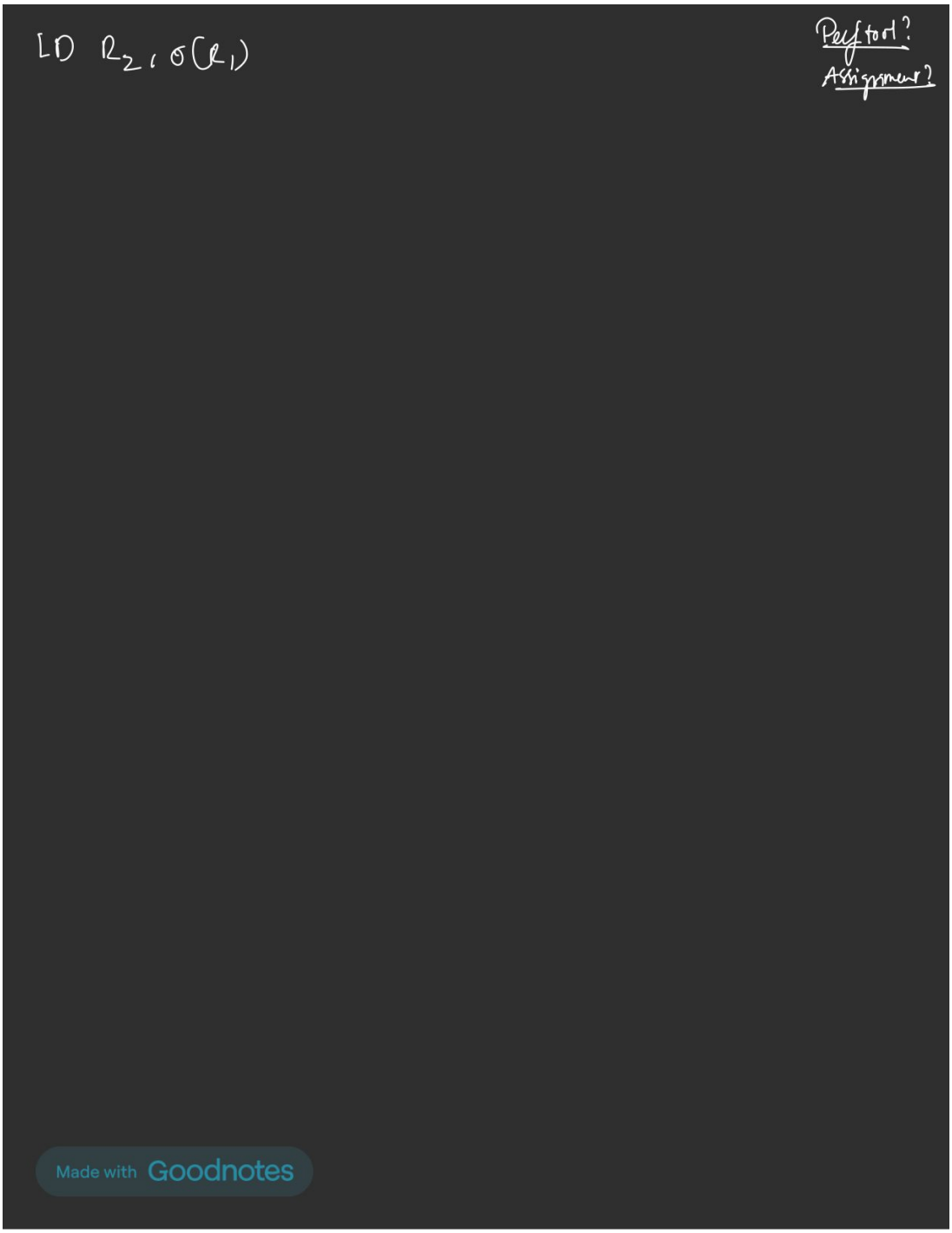
Explanation

What the page says. The continuation of the speculation timing table, adding the CDB broadcast column.

```
Columns: Time | Execute | Read | Write | CDB | Commit The CDB column marks the cycle in which the result is broadcast (and thus grabbed by every waiting station).
```

This completes the speculation block of the notes. The full printed questions on speculation and the ROB are solved in the past-paper solutions (ca25, ca28, ca39, ca40).

Original notes — page 67: Blank / illegible page



Original handwritten notes — page 67.

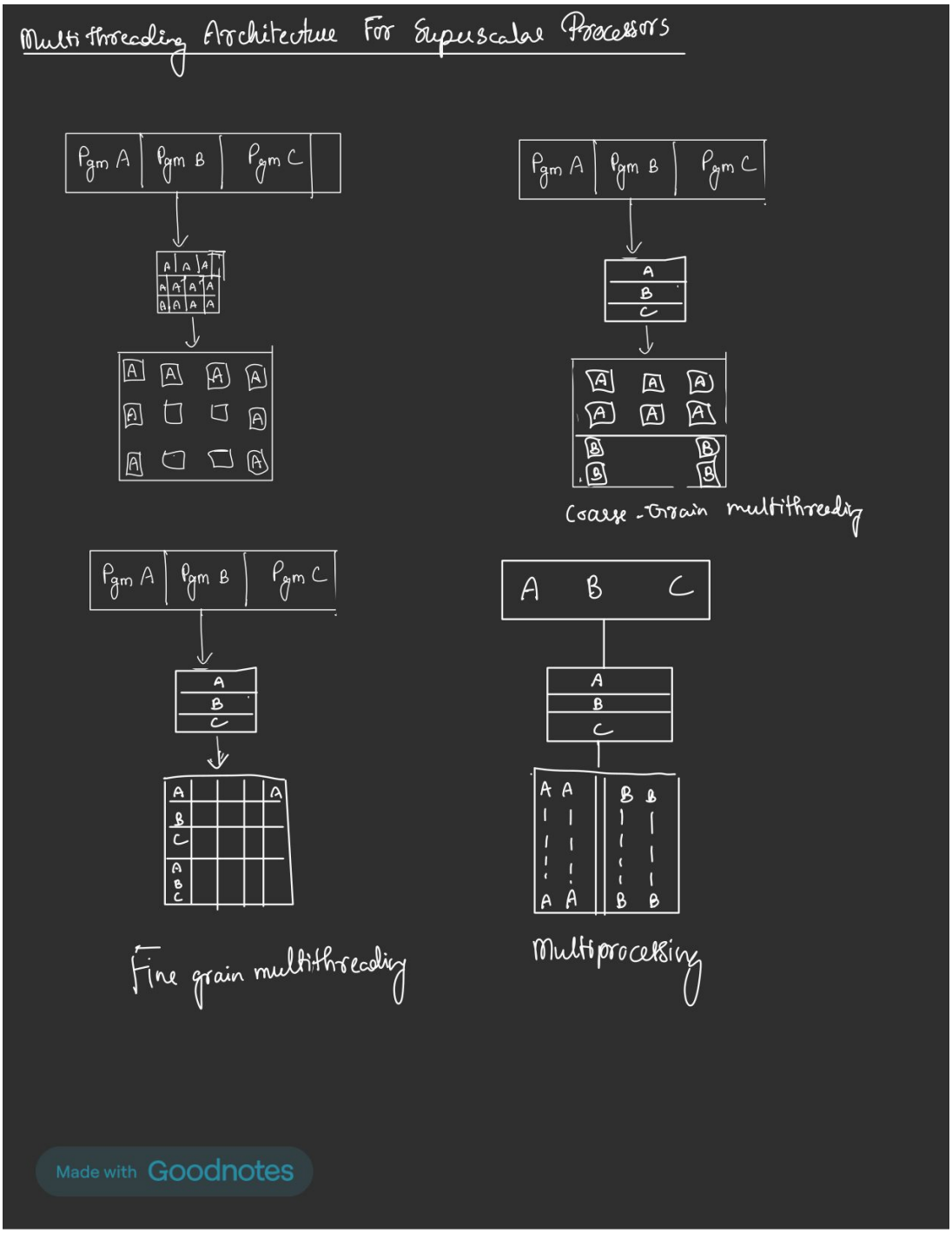
Transcription of the page

```
(No legible content. Only a stray word — “Defra…” — is visible;
the page is effectively blank.)
```

Explanation

This page carries no academic content — it is a blank page in the notes with only an illegible fragment. It is recorded here so that the page numbering stays aligned with the original; there is nothing to learn from it.

Original notes — page 68: UMA shared memory and multicore with private caches



Original handwritten notes — page 68.

Transcription of the page

```
UMA (Uniform Memory Access):

  [Core]  [Core]  [Core]  [Core]
     \       |       |       /
      \      |       |      /
   ========== shared bus ==========
                |
          [Shared main memory]

  Multicore: several cores, each with a private cache, sharing main memory
  through a common interconnect.

  (Cache coherence is needed because the private caches may hold copies
  of the same block.)
```

Explanation

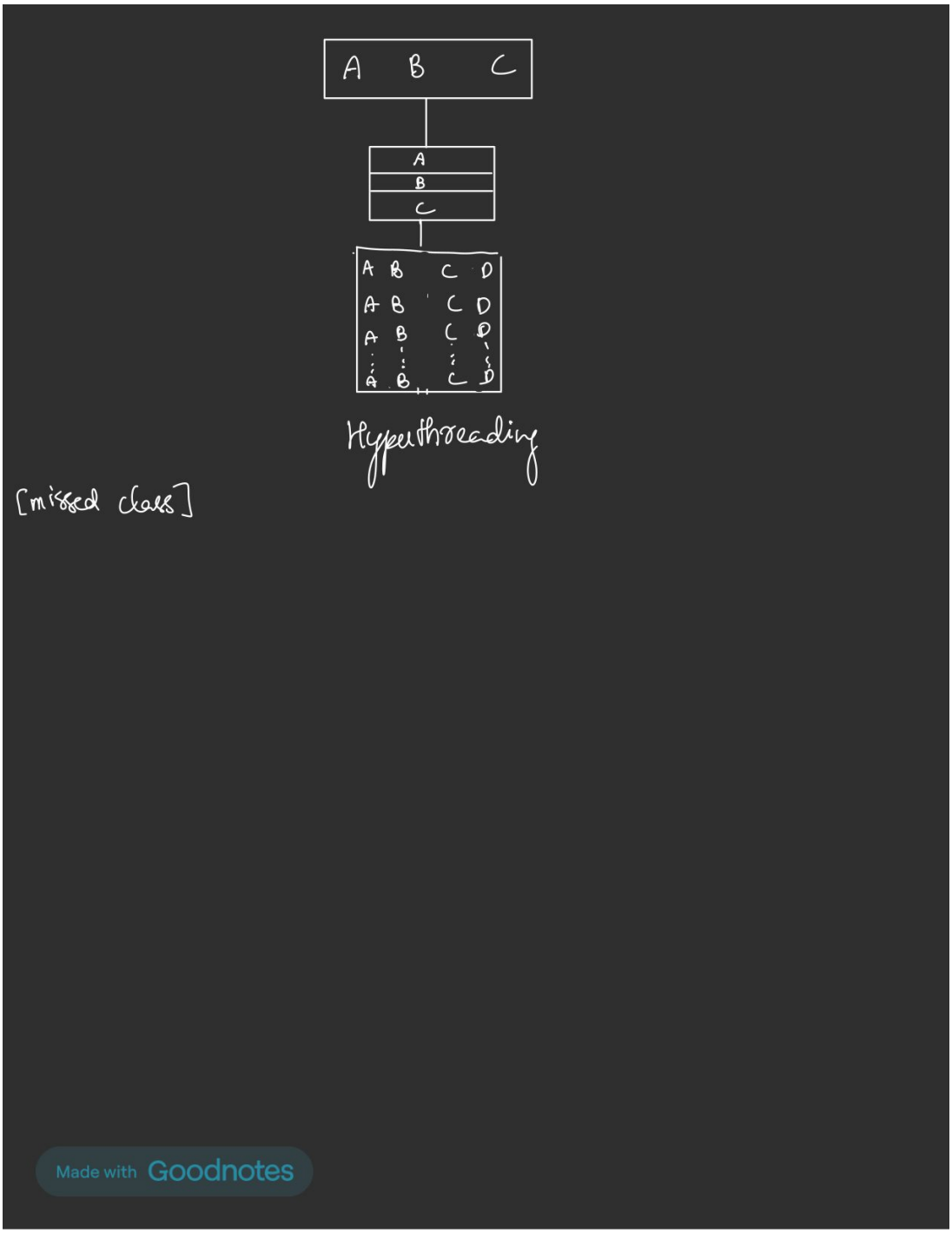
What the page says. The UMA multiprocessor picture: several cores, each with a private cache, all connected to one shared memory by a common interconnect.

- **UMA** — every core reaches the shared memory in the same time, hence “uniform”.
- **Private caches** are what make the machine fast (each core keeps its working set close), but they also mean the same memory block can live in several caches at once.
- That duplication is exactly the **cache-coherence problem**: if one core writes a block, the copies in the other caches become stale.

```
Shared bus bandwidth is divided among the cores: per-core bandwidth ≈ BW_total / n → falls as n grows (this is why UMA/bus designs do not scale to many cores)
```

Reconstructed diagram. Draw the cores in a row at the top, each with a small private cache box; a single horizontal bus below them; and the shared memory at the bottom. An arrow shows that all traffic must pass through the one bus.

Original notes — page 69: Cache and coherence block diagram



Original handwritten notes — page 69.

Transcription of the page

```
Cache coherence block diagram:

   [CPU] [CPU] [CPU] [CPU]
     |     |     |     |
   [C1]  [C2]  [C3]  [C4]     ← private caches
     \     |     |     /
      ===== shared bus =====
              |
        [Shared memory]

  “(Miss clause)” — on a miss, the cache fetches the block and may need to
  invalidate or update other copies.
```

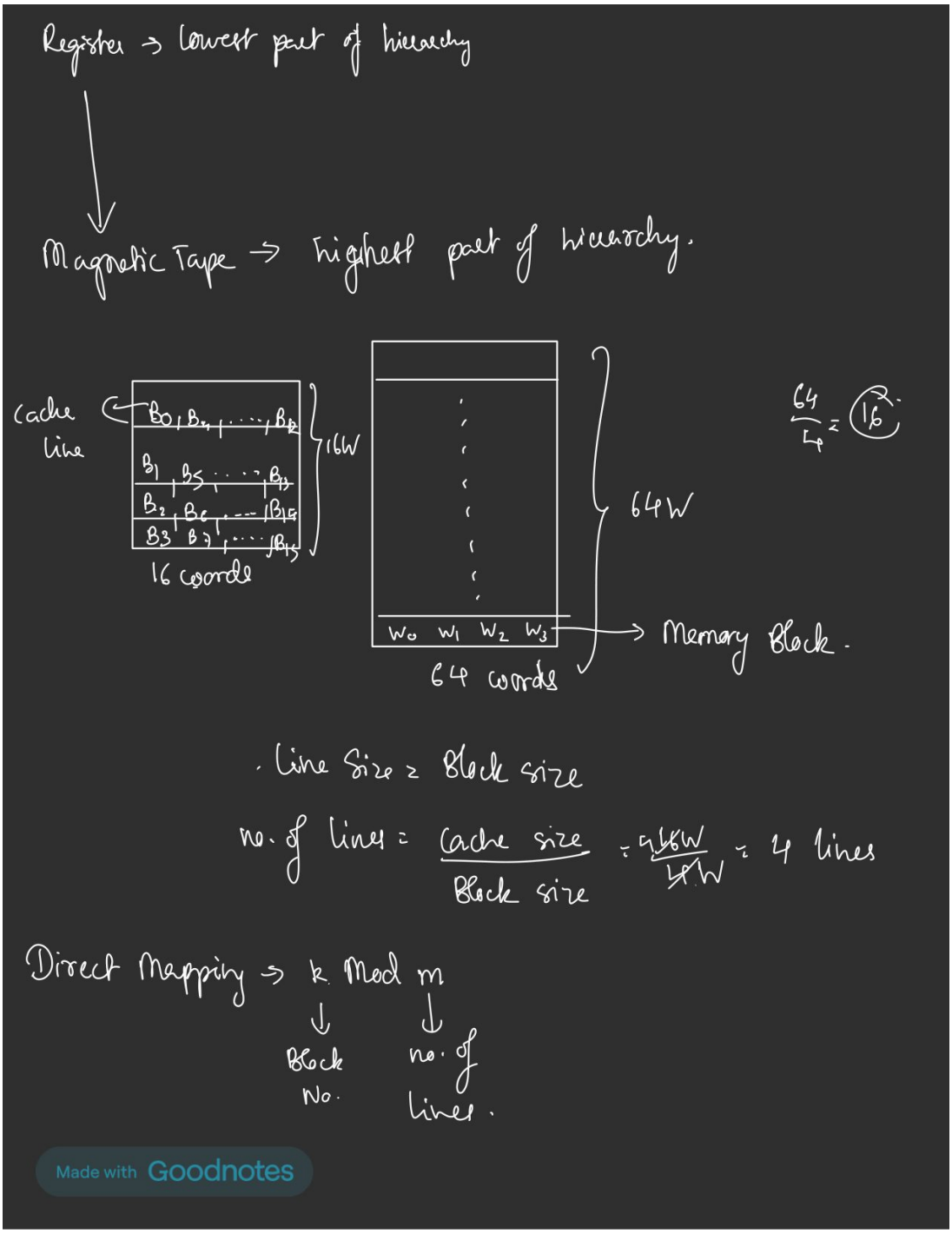
Explanation

What the page says. The coherence picture: several processors with private caches over a shared bus, and the note that a miss (or a write) forces the caches to coordinate.

```
Coherence invariant (informal): For every memory block, either it is present in no cache, or every cached copy has the same value; a write must make all other copies consistent (invalidated or updated).
```

The next pages develop the protocol (write-through / write-back, write-invalidate) and the states (MSI / MESI). This page is the map of the system those protocols run on.

Original notes — page 70: Cache basics — direct mapping (block number mod number…



Original handwritten notes — page 70.

Transcription of the page

```
Cache:

  Holds recently used blocks of memory close to the CPU.

  Direct Mapped:
    A block can go in exactly ONE line:  line = (block number) mod (number of lines)

    Address = | Tag | Index | Offset |
                ^      ^        ^
              identifies   selects   byte within
              the block    the line   the block

  Example:  memory block 12 → line 12 mod 4 = 0 (if there are 4 lines).

  The tag is stored with each line and compared on every access.
```

Explanation

What the page says. The simplest cache organisation: **direct mapping**, where each memory block has exactly one place it can live.

```
line = (block number) mod (number of cache lines) e.g. block 12, 4 lines: 12 mod 4 = 0 → line 0 block 13, 4 lines: 13 mod 4 = 1 → line 1
```

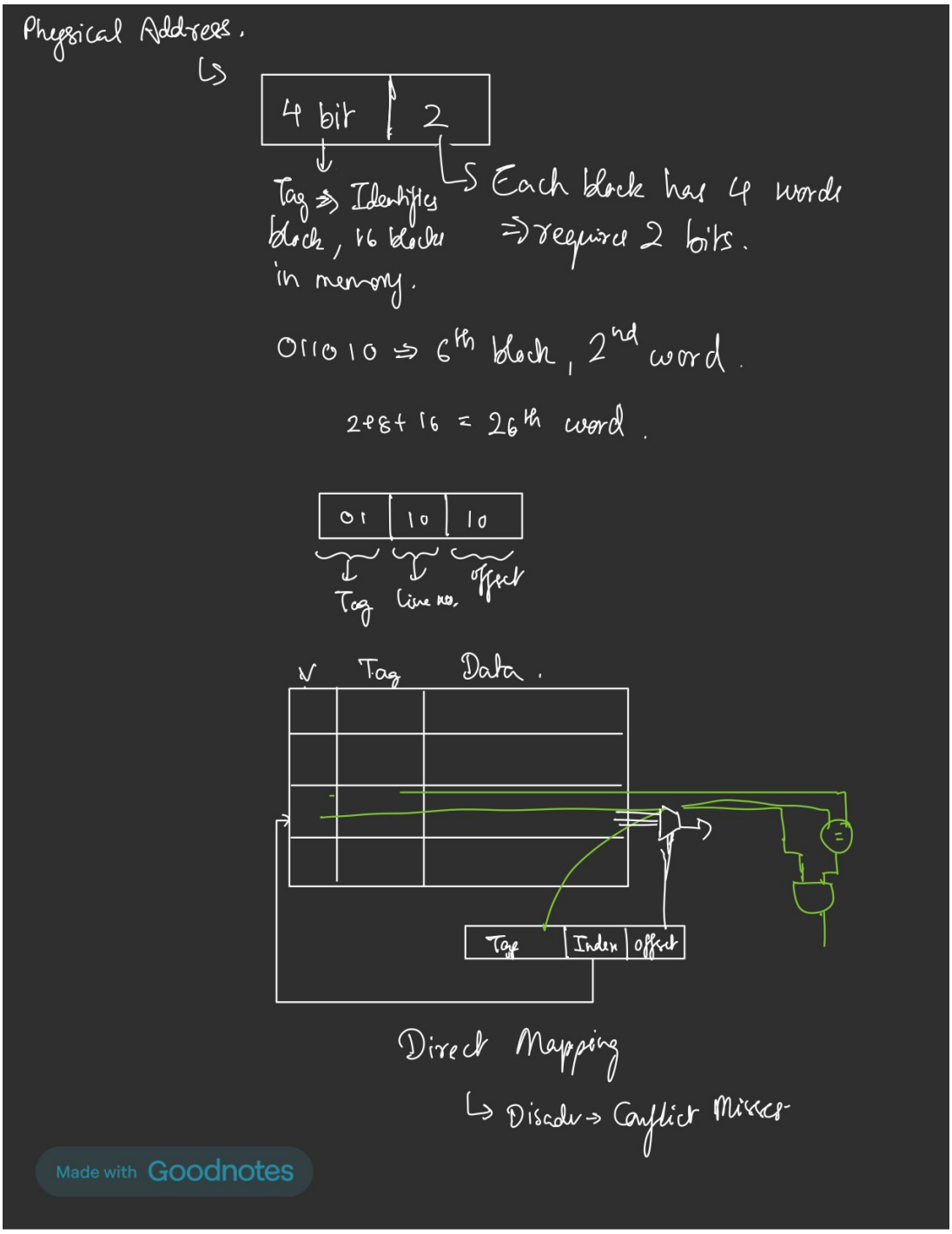
Address breakdown

- **Offset** — the low bits; select the byte within the block.
- **Index** — the middle bits; select the cache line (set).
- **Tag** — the high bits; stored with the line and compared to confirm the hit.

```
Address (32 bits) = [ Tag | Index | Offset ] offset bits = log₂(block size) index bits = log₂(number of lines) tag bits = 32 − index − offset
```

> **Why direct mapping is simple but limited.** Each block has only one possible home, so the lookup is trivial (one tag comparison). But two blocks that map to the same line will keep evicting each other — this is called a **conflict miss**.

Original notes — page 71: Set-associative cache — organisation and address bits



Original handwritten notes — page 71.

Transcription of the page

```
Set-Associative Cache:

  Each set holds n ways (n = associativity).

  A block maps to a SET:  set = (block number) mod (number of sets),
  and within the set it can go in ANY of the n ways.

  Address:  | Tag | Set index | Offset |

  Example: 2-way set associative, 16 sets:
     block → set (mod 16), then any of the 2 ways.

  Replaces fully-associative (all lines) with set-associative (a set of lines).
```

Explanation

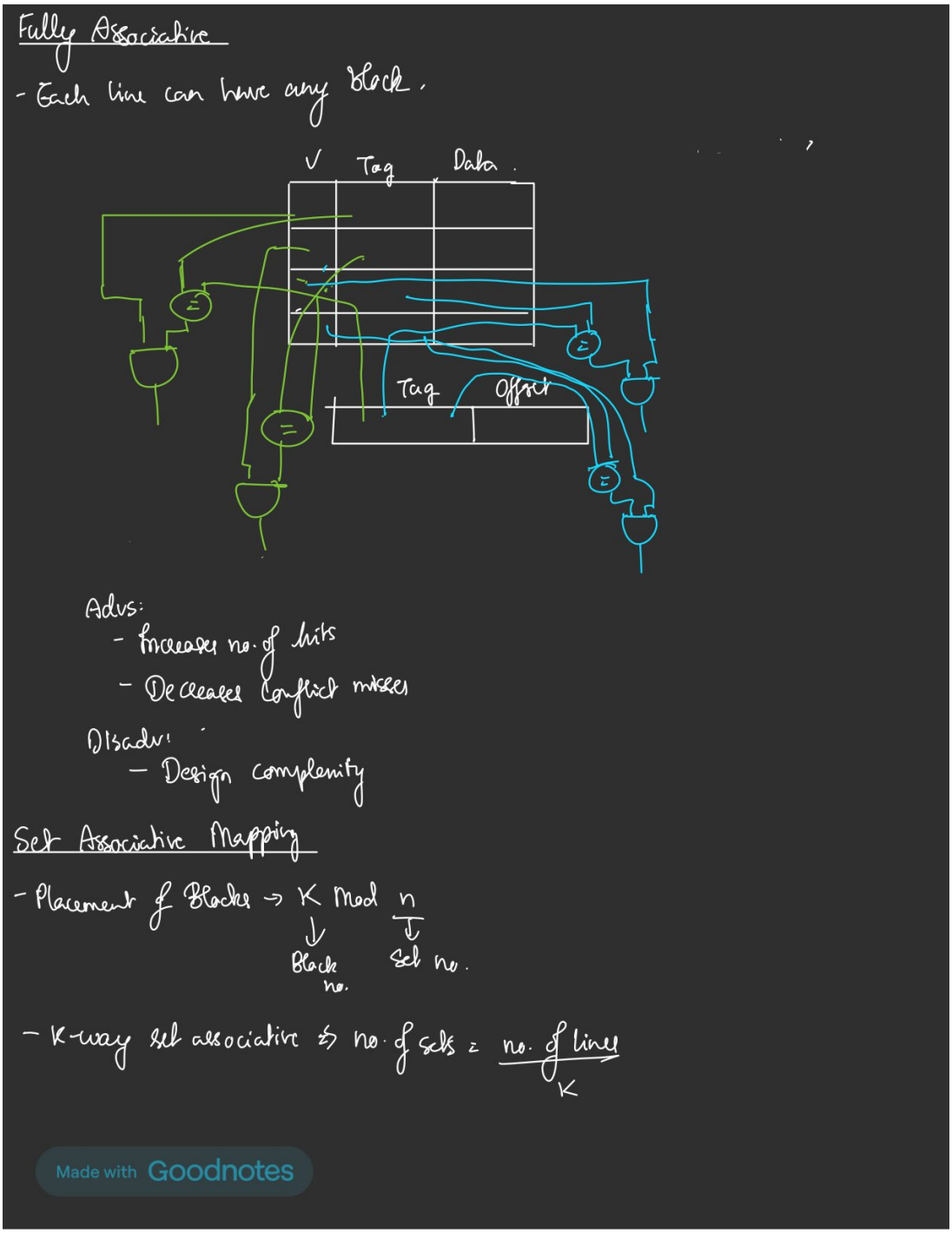
What the page says. The middle ground between direct-mapped and fully-associative: a block maps to one **set**, but within that set it can occupy any of **n** ways.

```
set = (block number) mod (number of sets) n-way set associative: n lines per set direct
mapped = 1-way set associative fully associative = 1 set with all the lines Address = [ Tag |
Set index | Offset ] set-index bits = log₂(number of sets)
```

- Fewer conflict misses than direct-mapped (a block has n homes, not one).
- Cheaper and faster than fully-associative (only n tags are compared, not all lines).
- Requires a **replacement policy** to choose which way to evict when the set is full.

Reconstructed diagram. Draw a table with *sets* as rows and *ways* as columns; each cell is a line holding a tag, valid bit and data. The set index selects the row; the tag is compared against all cells in that row.

Original notes — page 72: Set-associative cache — worked address example



Original handwritten notes — page 72.

Transcription of the page

```
Set-Associative worked example:

  Cache with several sets and ways.
  Block size, number of sets, associativity given.

  Address split:  Tag | Set | Offset

  (Worked example computing the number of bits in each field and which set a
  given address maps to.)

  Advantages:
   - Reduces conflict misses (a block has several possible homes),
   - Replacement policy can choose.
  Disadvantage:
   - More hardware (n tags compared) and a replacement decision.
```

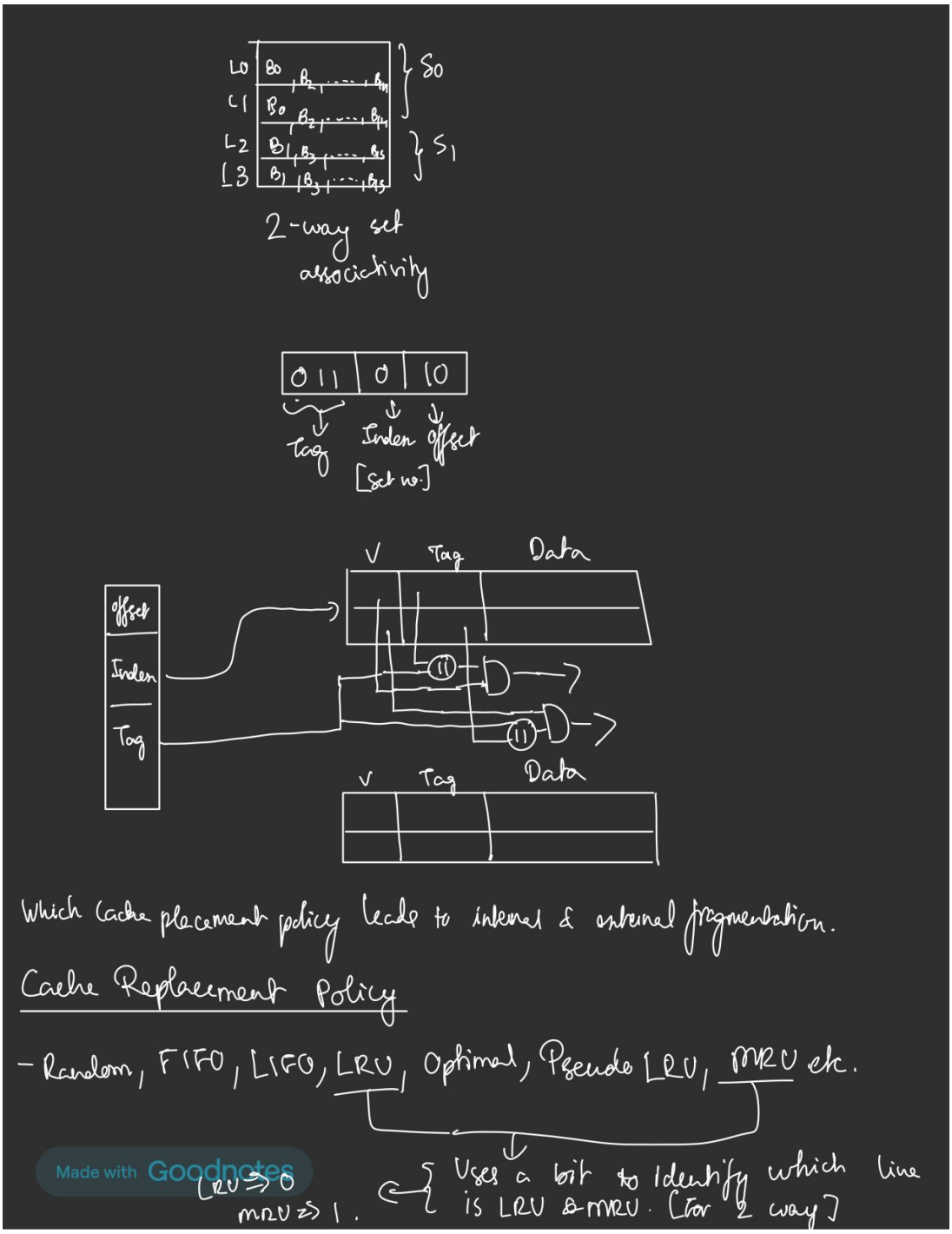
Explanation

What the page says. A worked example that splits a physical address into tag, set and offset fields for a set-associative cache.

```
Example: 32-bit address, 64 KB cache, 64-byte blocks, 4-way set associative. offset = log₂64 = 6 bits lines = 64 KB / 64 B = 1024 lines sets = 1024 / 4 (ways) = 256 sets → set index = log₂256 = 8 bits tag = 32 − 8 − 6 = 18 bits Address = [ 18-bit tag | 8-bit set | 6-bit offset ]
```

> **Order of computation.** Always work out offset first (from block size), then the number of sets (from cache size, block size and associativity), then the tag (whatever is left).

Original notes — page 73: Replacement policies — FIFO, LRU, optimal, random



Original handwritten notes — page 73.

Transcription of the page

```
Replacement policies (when a set is full, which line to evict?):

  * Random      — pick any line at random.
  * FIFO        — first in, first out (evict the oldest).
  * LRU         — least recently used (evict the one not touched for longest).
  * Optimal     — evict the line that will be used farthest in the future
                  (theoretical best; needs future knowledge).

  Which is best?  Optimal (cannot be implemented); LRU is a good practical
  approximation; FIFO and Random are cheaper but worse.
```

Explanation

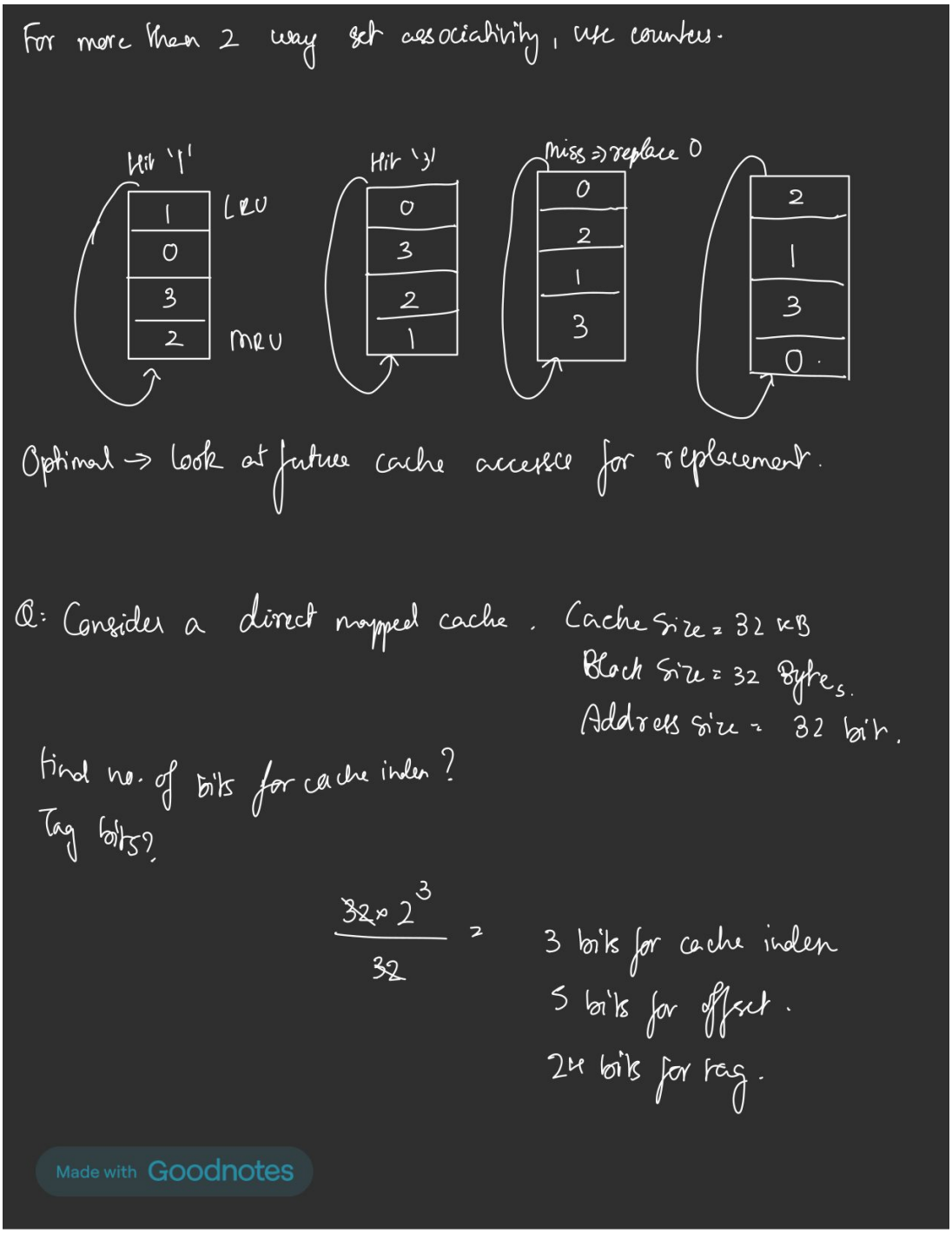
What the page says. The four classic replacement policies, with a note that the optimal one is a theoretical benchmark.

Policy	Rule	Comment
Random	Evict a random line	Cheap; can evict a useful line
FIFO	Evict the oldest line	Cheap; ignores recent use
LRU	Evict the least recently used line	Good approximation to optimal; needs counters
Optimal	Evict the line used farthest in the future	Best possible; cannot be implemented (needs the future)

```
LRU bookkeeping: a “time-stamp” or age bits per line; on every access update the used line;
on a miss evict the oldest. Optimal is used only as a benchmark to judge other policies.
```

Which to use? LRU (or an approximation such as pseudo-LRU) is the usual practical choice, because locality means the recently used lines are likely to be used again. Optimal exists only to measure how much better a policy could possibly be.

Original notes — page 74: Direct-mapped cache worked example — address field split



Original handwritten notes — page 74.

Transcription of the page

```
Worked example — direct-mapped cache:

  Q: A direct-mapped cache of 3 KB? / given size and block size.
     Address width = 32 bits.
     Find the number of bits for tag, index and offset.

  Offset bits = log₂(block size)
  Index bits  = log₂(number of lines)
  Tag bits    = 32 − index − offset

  (Detailed arithmetic:  block size, number of lines, etc.)
```

Explanation

What the page says. A direct-mapped cache address-split question, worked in full.

```
Given: 32-bit address, cache = 4 KB, block size = 32 bytes, direct mapped. offset = log₂32 = 5 bits lines = 4 KB / 32 B = 128 lines → index = log₂128 = 7 bits tag = 32 − 7 − 5 = 20 bits
Address = [ 20-bit tag | 7-bit index | 5-bit offset ]
```

- Two blocks that share the same index (i.e. differ only in the tag) map to the **same line** and will evict each other — a conflict miss.
- The line number is *block number mod number of lines*; the index bits are exactly the low-order bits of the block number.

Previous-year question 38 — ca38 — Cache misses/evictions/non-empty blocks; branch…

in the main memory from the address 0x4230FA onwards. The last five instructions of M is a Loop that is iterated 10 times.

The second Instruction in the loop is a call to subroutine S. S consists of 4 instruction words loaded in the main memory from the address 0x70F168. The last instruction of S is a subroutine return back to M. The only two data words that are used by M and S are at addresses 0x748074 and 0x846064. Assume the cache are initially empty. Ignore OS level interruption and subsequent cache impact on context switching.

i)Calculate the address components of the direct mapped unified cache.

ii)Find the number of cache misses occurred during the execution of the program.

iii)How many cache cache block eviction happened during the execution of the program?

iv)List out the block numbers (in decimal) in the cache that are non empty after the execution of the program

[5]

3. a. Determine the total branch penalty for a branch target buffer assuming the Penalty cycles for individual misprediction from the below table.

Instruction in buffer	Prediction	Actual branch	Penalty cycles
Yes	Taken	Taken	0
Yes	Taken	Not Taken	2
No		Taken	2
No		Not Taken	0

Make the Following assumptions about the prediction accuracy and hit rate:

- Prediction accuracy is 90% (for instructions in the buffer).
- Hit rate in the buffer is 90% (for branches predicted taken).

[5]

Original past-paper scan ca38.png (re-rendered).

Transcription of the question

```
(continuation of ca37 2b) i) Calculate the address components of the direct-mapped
   unified cache. ii) Find the number of cache misses during execution. iii) How many cache
   block evictions happened? iv) List the block numbers (decimal) in the cache that are
   non-empty after execution. [5]
3a. Determine the total branch penalty for a branch-target buffer using the penalty table:
       instruction in buffer | predicted | actual | penalty cycles
              yes            |  taken    | taken  | 0
              yes            |  taken    | not    | 2
              no             |  —        | taken  | 2
              no             |  —        | not    | 0
   Prediction accuracy is 90% (for instructions in the buffer); hit rate is 90%.
```

Full solution

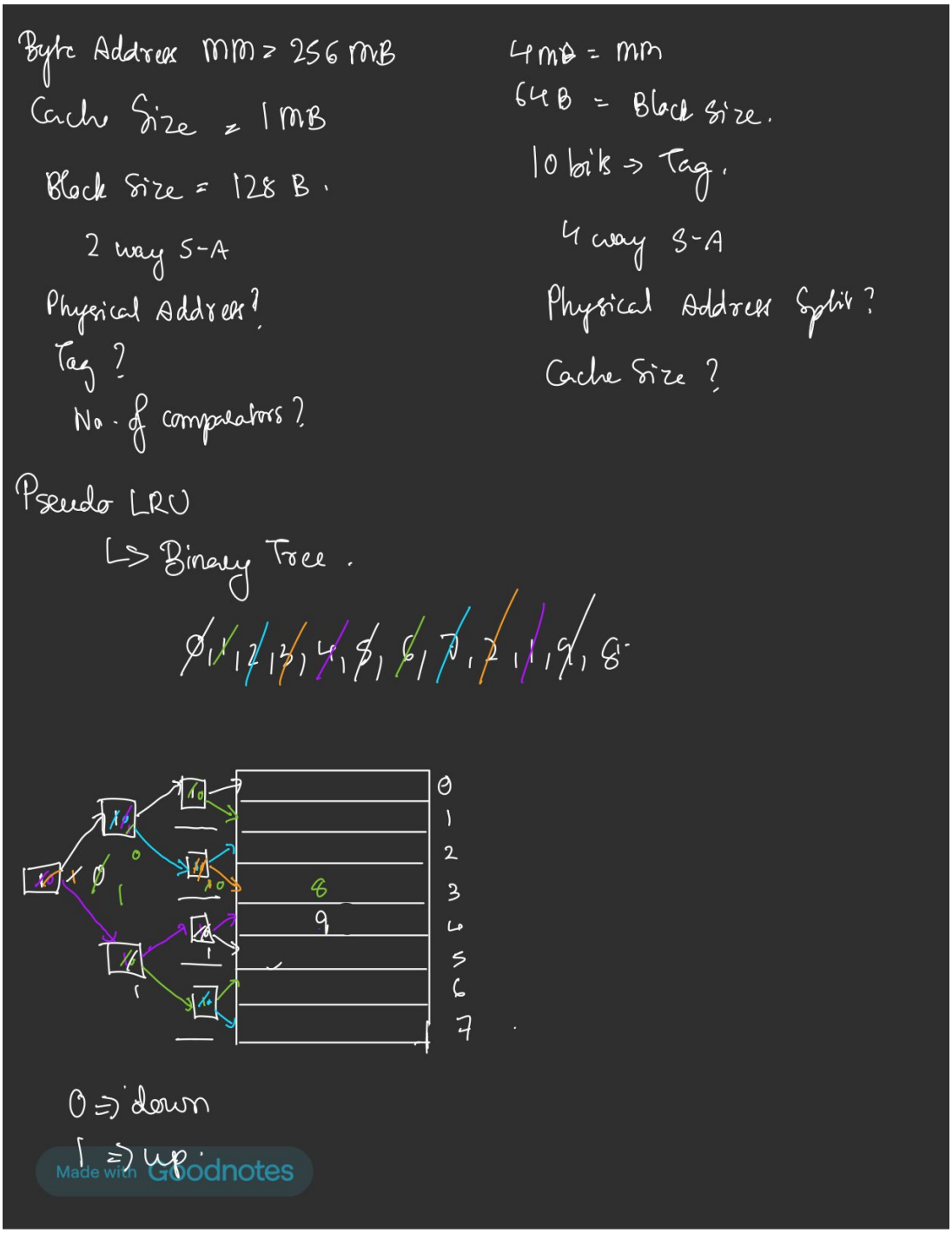
Misses, evictions, non-empty blocks

line = (block number) mod (number of lines) ; block number = word address / block size For each access: empty line → compulsory miss; wrong tag → conflict miss (evict the old block); matching tag → hit. Track each line’s tag after every access to count misses and evictions.

3a — branch-target-buffer penalty

penalty = hit_rate × [accuracy × 0 + (1 − accuracy) × 2] + (1 − hit_rate) × [taken_fraction × 2 + (1 − taken_fraction) × 0] With 90% hit, 90% accuracy: in-buffer term = 0.90 × (0.10 × 2) = 0.18 cycles miss term = 0.10 × (taken_fraction × 2) If, say, 60% of branches are taken: miss term = 0.10 × 0.6 × 2 = 0.12 total penalty per branch = 0.18 + 0.12 = 0.30 cycles.

Original notes — page 75: Cache address split — worked example (tag/index/offset)



Original handwritten notes — page 75.

Transcription of the page

```
Worked example:

  Cache size = 256 KB, block size = 1 KB? / 512 B?
  Number of sets, associativity given.

  Physical address → split into Tag | Set | Offset.

  (Compute offset, then index/set, then tag; show the bit fields.)

  Replace/decide which lines are used.
```

Explanation

What the page says. Another worked address-split example, this time for a larger cache.

Given: 256 KB cache, 512-byte blocks, direct mapped, 32-bit address. offset = $\log_2 512 = 9$ bits lines = 256 KB / 512 B = 512 lines → index = $\log_2 512 = 9$ bits tag = 32 − 9 − 9 = 14 bits Address = [14-bit tag | 9-bit index | 9-bit offset] If instead it is 4-way set associative: sets = 512 / 4 = 128 → index = $\log_2 128 = 7$ bits ; tag = 32 − 7 − 9 = 16 bits

Check your answer. tag + index + offset must equal the address width. Compute offset and index first, then subtract.

Previous-year question 37 — ca37 — Micro-benchmark cache block sizes; direct-mapp…

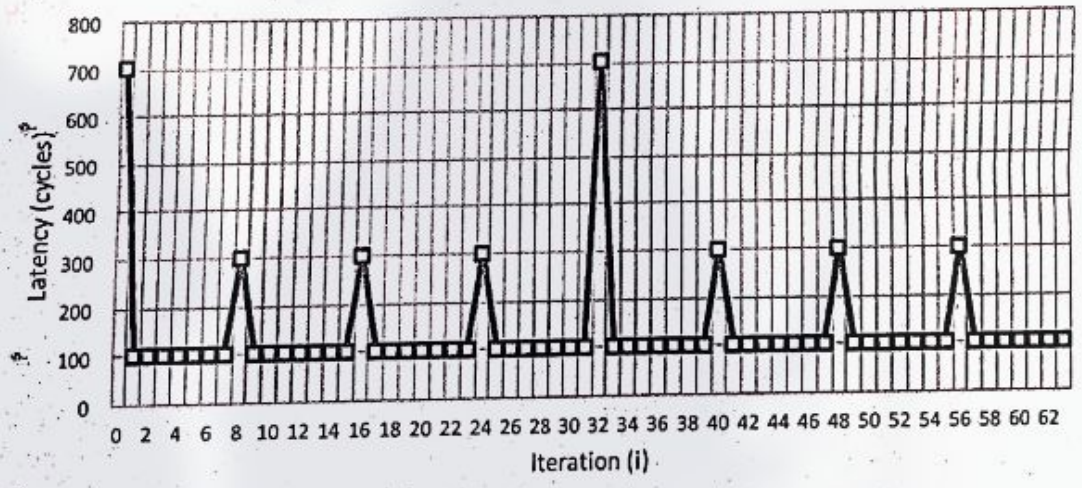
2. a. We are going to micro benchmark the cache hierarchy of a computer with the following two codes. The array data contains 32-bit unsigned integer values. For simplicity, we consider that accesses to the array latency bypass all caches (i.e., latency is not cached). timer() returns a timestamp in cycles.

```
Code Segment:
 j = 0;
     for (i=0; i<size; i+=stride){

          start = timer();
          d = data[i];
          stop = timer();
          latency[j++] = stop - start;
     }
```

(Stride means: Distance between two consecutive memory elements accessed by CPU)

The cache hierarchy has two levels. L1 is a 4kB set associative cache. When we run code segment, we obtain the latency values in the following chart for the first 64 reads to the array data (in the first 64 iterations of the loop) with stride equal to 1.



What are the cache block sizes in L1 and L2? – Explain your answer.

[5]

b. A 16KB direct mapped 256B block unified cache is attached to a 16MB main memory system. The word length as well as instruction length of the processor is 16 bits. Consider a program that consists of a main routine M which in turn calls a subroutine S. M consists of 12 instruction words which are loaded

2

Original past-paper scan ca37.png (re-rendered).

Transcription of the question

```
2a. Micro-benchmarking a two-level cache. L1 is a 4 KB set-associative cache. Code
    segment measures the latency of array accesses with a given stride. The latency chart
    for the first 64 reads with stride 1 is given. What are the cache block sizes in L1 and
    L2? Explain. (6)
2b. A 16 KB direct-mapped 256 B-block unified cache is attached to a 16 MB main memory.
    Word length and instruction length are 16 bits. A main routine M (12 instruction words
    at 0x4230FA) calls subroutine S (4 instruction words at 0x70F168). The last five
    instructions of M form a loop iterated 10 times; the second instruction of the loop calls
    S. The only data words are at 0x748074 and 0x846064. Cache initially empty.
```

Full solution

2a — deducing the block sizes

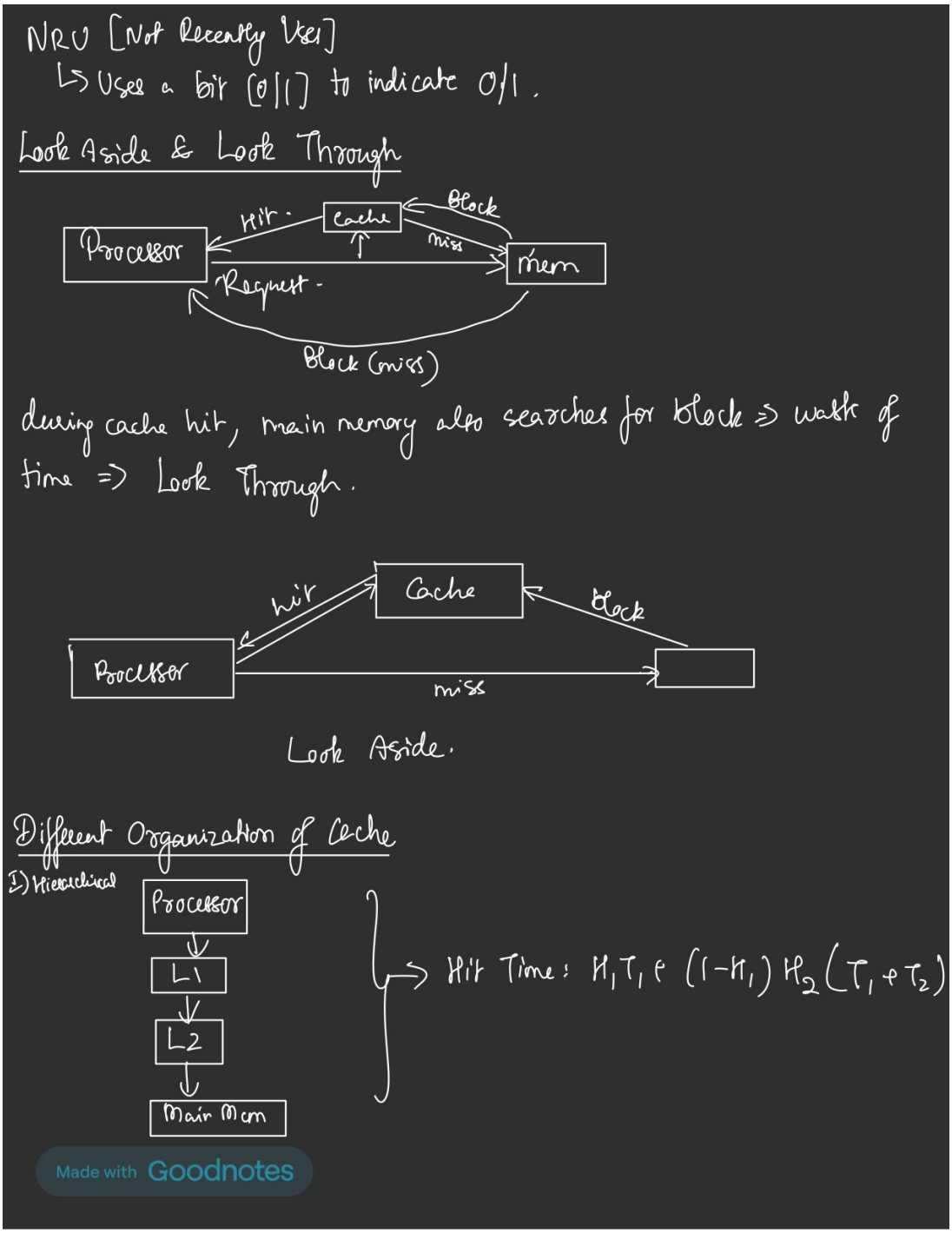
With stride 1 the access latency stays low while consecutive elements fall in the same block, then jumps when the stride steps onto a new block; the first jump marks the L1 block size, the next jump the L2 block size. Read the chart: the plateau before the first jump is the L1 block size (in elements × 4 bytes), and the second plateau gives the L2 block size.

2b — address components

16-bit address, 256 B blocks → offset = $\log_2$(256/2 words × 2 B) … careful: block = 256 B = 128 words of 2 B → offset = $\log_2$128 = 7 bits (word offset) lines = 16 KB / 256 B = 64 lines → index = $\log_2$64 = 6 bits tag = 16 − 6 − 7 = 3 bits Address = [tag(3) | index(6) | offset(7)]

Then walk the program: M loads, the loop repeats 10 times, each iteration calls S; count compulsory misses (first touch), conflict misses (two blocks mapping to the same line) and capacity misses, and list the non-empty blocks at the end.

Original notes — page 76: Write-through policy; the local miss-rate / average-access-ti…



Original handwritten notes — page 76.

Transcription of the page

```
Write policy 1: Write-through.

  Every write updates BOTH the cache and the main memory.

  Consequence: main memory is always up to date; but write traffic to memory
  is high (slow).

  Average access time:
    AMAT = Hit time + Miss rate × Miss penalty
    T_avg = (1 − H) · t_miss + H · t_hit

  where H = hit rate.

  Different organisation → different write policies:
   1) write-through,
   2) write-back,
   3) write-allocate / no-write-allocate.
```

Explanation

What the page says. The write-through policy and the average-memory-access-time formula.

Write-through

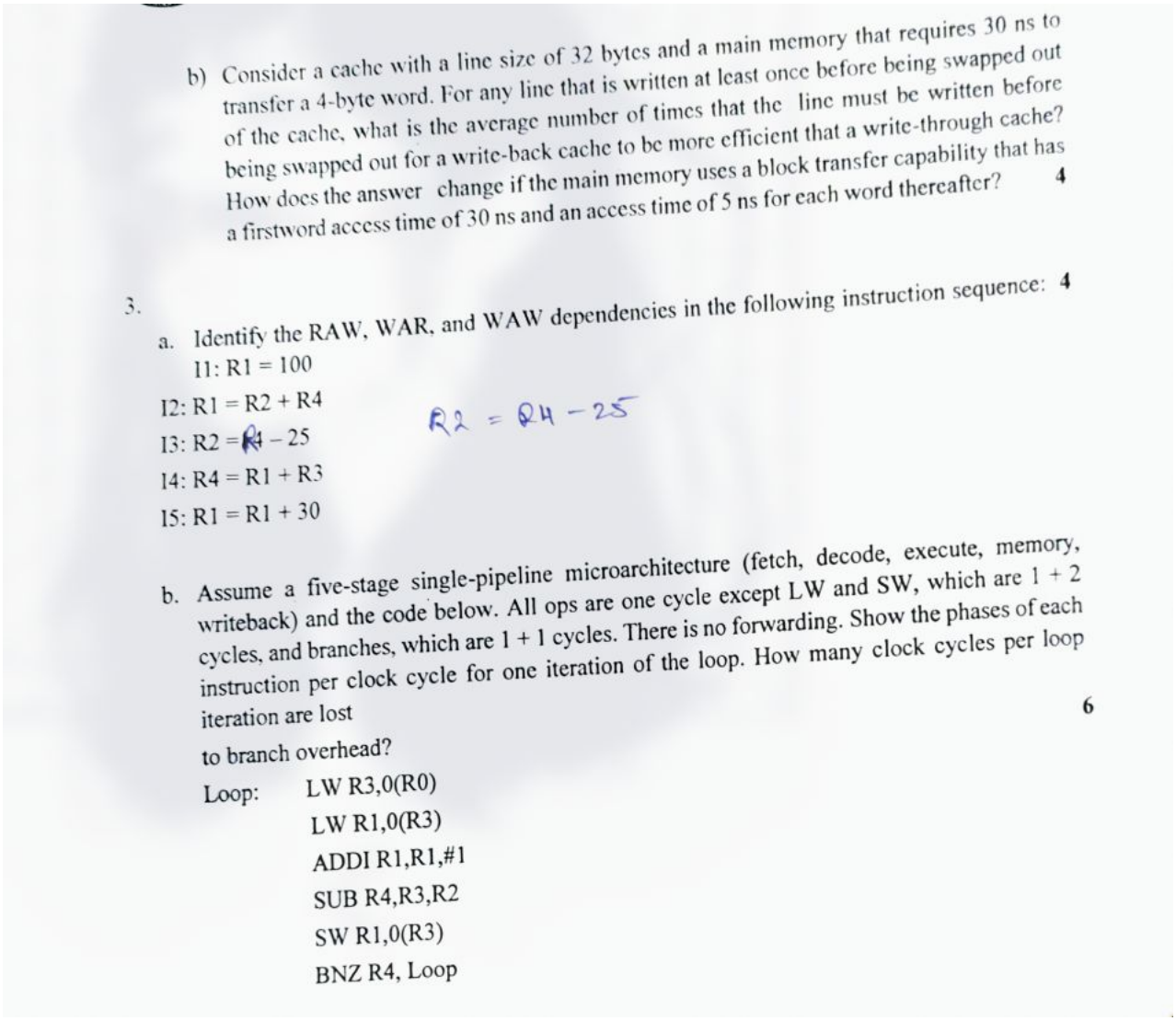
- Every store writes the cache *and* the next level of memory.
- **Advantage:** main memory is always consistent with the cache, so coherence and recovery are simpler.
- **Disadvantage:** every write causes memory traffic, which is slow; a **write buffer** is used to hide the latency.

Average memory access time

```
AMAT = Hit time + Miss rate × Miss penalty or, in hit-rate form: T_avg = H · t_hit + (1 − H) · t_miss H = hit rate, t_hit = hit time, t_miss = miss penalty. Multi-level: AMAT = t_L1 + m_L1 · (t_L2 + m_L2 · t_mem)
```

Reading the formula. The miss rate is multiplied by the miss penalty because only the misses pay that penalty. Reducing the miss rate and reducing the miss penalty are two separate optimisation targets (developed on pages 79–80).

Previous-year question 27 — ca27 — Write-back vs write-through break-even; RAW/WA…



b) Consider a cache with a line size of 32 bytes and a main memory that requires 30 ns to transfer a 4-byte word. For any line that is written at least once before being swapped out of the cache, what is the average number of times that the line must be written before being swapped out for a write-back cache to be more efficient that a write-through cache? How does the answer change if the main memory uses a block transfer capability that has a firstword access time of 30 ns and an access time of 5 ns for each word thereafter? 4

3.

a. Identify the RAW, WAR, and WAW dependencies in the following instruction sequence: 4

I1: R1 = 100
I2: R1 = R2 + R4
I3: R2 = R4 – 25
I4: R4 = R1 + R3
I5: R1 = R1 + 30

R2 = R4 – 25

b. Assume a five-stage single-pipeline microarchitecture (fetch, decode, execute, memory, writeback) and the code below. All ops are one cycle except LW and SW, which are 1 + 2 cycles, and branches, which are 1 + 1 cycles. There is no forwarding. Show the phases of each instruction per clock cycle for one iteration of the loop. How many clock cycles per loop iteration are lost 6

to branch overhead?

```
Loop:   LW R3,0(R0)
        LW R1,0(R3)
        ADDI R1,R1,#1
        SUB R4,R3,R2
        SW R1,0(R3)
        BNZ R4, Loop
```

Original past-paper scan ca27.png (re-rendered).

Transcription of the question

```
b) A cache has a line size of 32 bytes; main memory takes 30 ns to transfer a 4-byte word.
   For any line written at least once before being swapped out, what is the average number
   of times the line must be written before being swapped out for a write-back cache to be
   more efficient than a write-through cache? How does the answer change if main memory
   has a first-word access time of 30 ns and 5 ns for each word thereafter? (4)
a. Identify the RAW, WAR and WAW dependencies in: (4)
       I1: R1 = 100
       I2: R1 = R2 + R4
       I3: R2 = R5 - 25
       I4: R4 = R1 + R3
       I5: R1 = R1 + 30
b. Five-stage single-pipeline (fetch, decode, execute, memory, write-back); all ops one
   cycle except LW and SW (1+2) and branches (1+1). No forwarding. Show the phases of
   each instruction per clock for one iteration of the loop; how many cycles are lost to
   branch overhead? (6)
       Loop: LW   R3, 0(R0)
             LW   R1, 0(R3)
             ADDI R1, R1, #1
             SUB  R4, R3, R2
             SW   R1, 0(R3)
             BNZ  R4, Loop
```

Full solution

b — write-back vs write-through break-even

Write-through: every write sends one word to memory = 30 ns per write. Write-back: one block write per eviction = 8 words × 30 = 240 ns. Break-even: 30·k = 240 → k = 8 writes per line. With block transfer (first word 30 ns, then 5 ns/word): block write = 30 + 7×5 = 65 ns. Break-even: 30·k = 65 → k ≈ 2.17, i.e. about 3 writes per line.

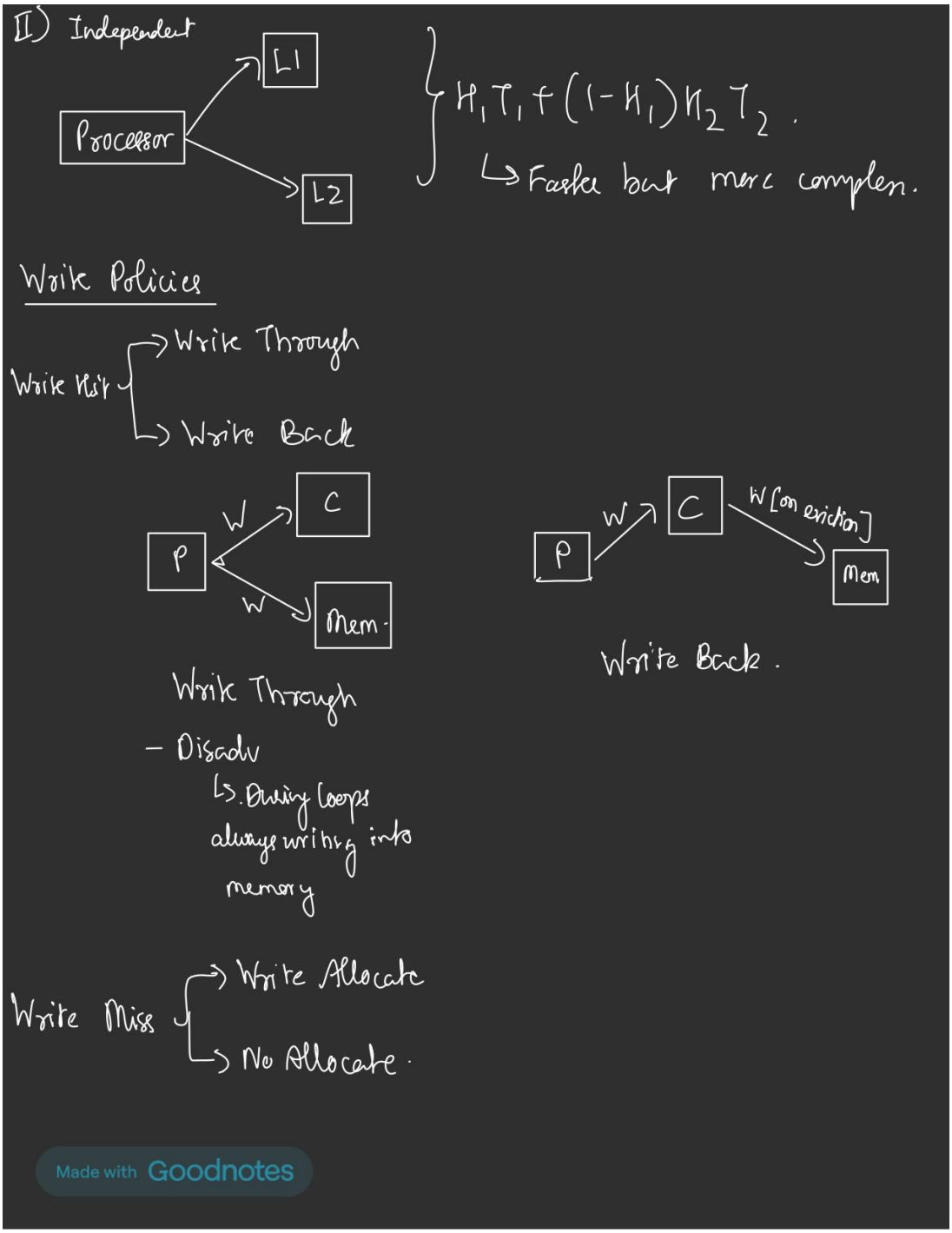
a — RAW/WAR/WAW dependences

Pair	Register	Type
I1 → I2	R1	WAW (both write R1)
I1 → I4	R1	RAW (I4 reads R1)
I2 → I4	R1	RAW
I2 → I5	R1	WAW / RAW
I4 → I5	R1	RAW
I3 → nothing	R2	independent

b — pipeline phases and branch overhead

No forwarding: each RAW pair needs 2 stalls. Cycles: LW(1+2)=3, LW(1+2)=3, ADDI=1, SUB=1, SW(1+2)=3, BNZ(1+1)=2. Build the space–time chart; the branch (BNZ) is resolved late, so the instructions fetched behind it are flushed. Branch overhead = cycles wasted per iteration from the branch (≈ 1–2).

Original notes — page 77: Write-back and write-allocate policies



Original handwritten notes — page 77.

Transcription of the page

```
Write policy 2: Write-back.

  On a write, update only the cache and mark the line “dirty”.
  The line is written to memory only when it is EVICTED.

  → Less memory traffic than write-through (good), but memory can be stale
    and coherence is harder.

Write policy 3: Write-allocate.

  On a write miss, first fetch the block into the cache, then write it.

  (No-write-allocate: on a write miss, write straight to memory.)

  Write-back + write-allocate is the common combination.
```

Explanation

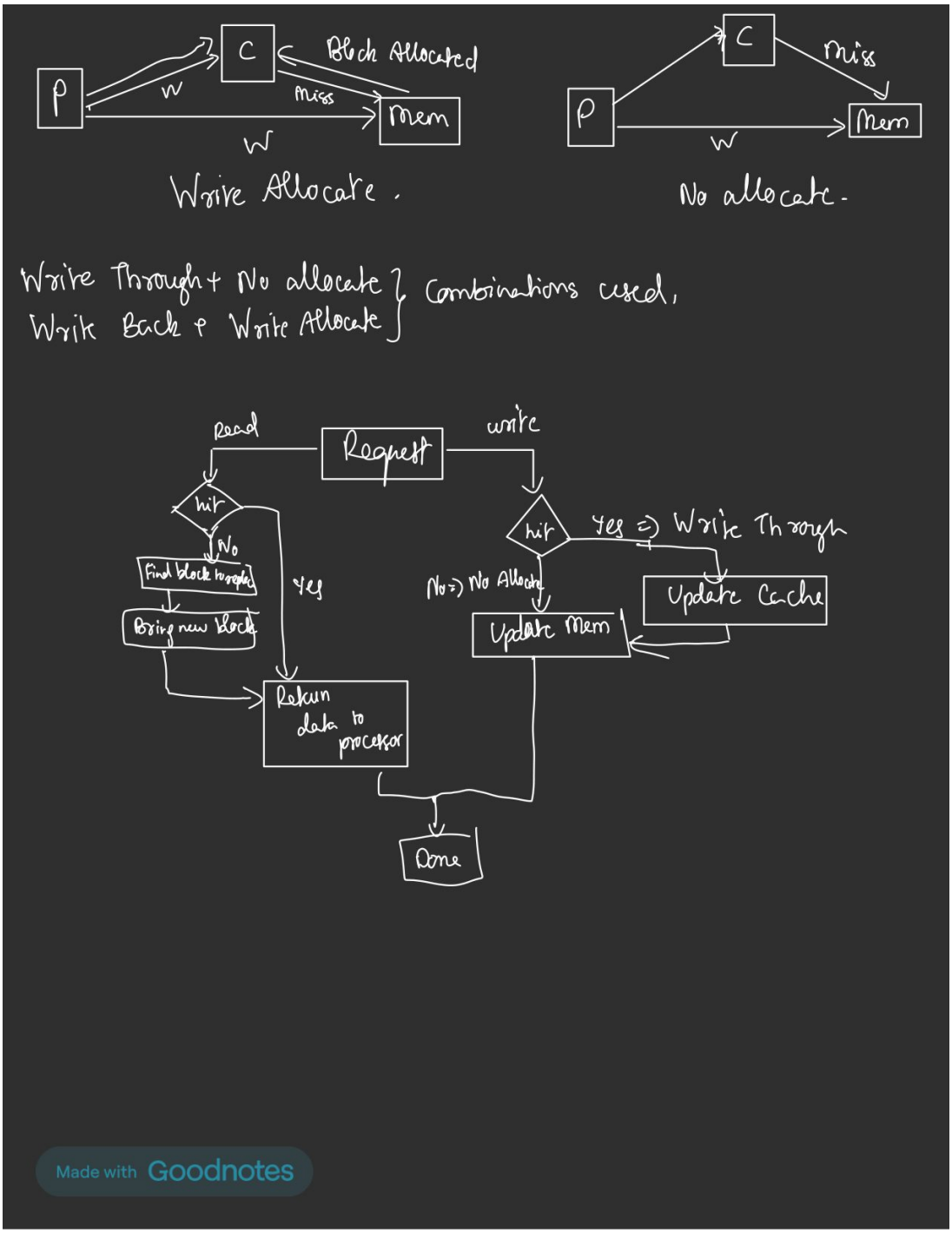
What the page says. The write-back and write-allocate policies and how they combine.

Policy	On a write hit	On a write miss
Write-through	Write cache and memory	Usually no-write-allocate (write to memory)
Write-back	Write cache only; mark dirty	Depends on allocate policy
Write-allocate	—	Fetch the block into the cache, then write
No-write-allocate	—	Write straight to memory, do not allocate

```
Write-back traffic: a dirty line is written to memory ONCE, on eviction. → fewer memory writes than write-through when a line is written many times. Write-through traffic: every store goes to memory immediately. → simpler coherence, more bandwidth used.
```

Common combination. Write-back + write-allocate is standard in real caches: it keeps repeated writes on-chip and only pays the memory cost once, on eviction.

Original notes — page 78: Write-allocate / write-back combinations (summary)



Original handwritten notes — page 78.

Transcription of the page

```
Combinations:

  Write-through + no-write-allocate  (write straight through, no fetch)
  Write-through + write-allocate
  Write-back    + write-allocate     (the usual choice)
  Write-back    + no-write-allocate

  Diagram: on each write, decide (a) update cache? (b) fetch block?
  “Write-through → memory always updated.”
  “Write-back → memory updated on eviction (dirty line).”
```

Explanation

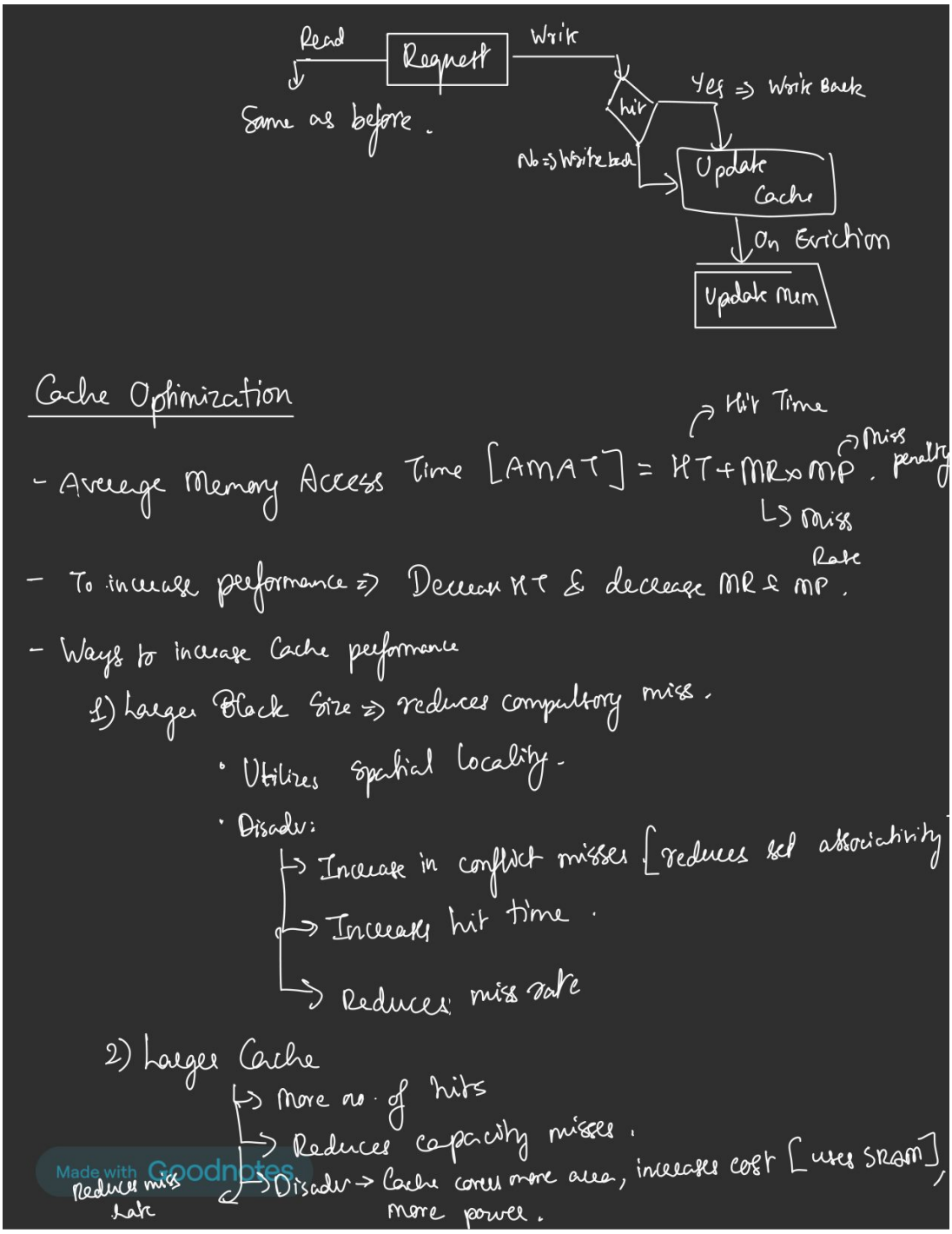
What the page says. The four possible combinations of the write-hit policy and the write-miss (allocate) policy, with a decision diagram.

Write-hit policy	Write-miss policy	When memory is updated	Typical use
Write-through	No-write-allocate	On every write	Simple systems
Write-through	Write-allocate	On every write; block fetched on miss	Rare
Write-back	Write-allocate	On eviction of a dirty line	Most caches
Write-back	No-write-allocate	On eviction of a dirty line	Rare

```
Decision diagram: Write hit? yes → update cache (write-through also updates memory) Write miss? → write-allocate: fetch block, then update cache no-write-allocate: write straight to memory
```

Why write-back + write-allocate dominates. Repeated writes to the same block stay in the cache and are written to memory only once; the fetch on a write miss means the whole block is available for subsequent writes.

Original notes — page 79: Write-policy summary; AMAT; cache-optimisation checklist



Original handwritten notes — page 79.

Transcription of the page

```
Write policies summary:
  - Write-through : memory updated on every write.
  - Write-back    : memory updated on eviction (dirty).
  - Write-allocate / no-write-allocate : whether a write miss fetches the block.

Cache optimisation:
  AMAT = Hit time + Miss rate × Miss penalty

  To improve performance, reduce ONE of the three terms:
   1) Reduce the hit time (smaller/faster cache, simpler addressing),
   2) Reduce the miss rate (bigger cache, higher associativity, better
      replacement, blocking),
   3) Reduce the miss penalty (multilevel caches, critical-word-first,
      non-blocking caches, prefetching).
```

Explanation

What the page says. The summary of write policies and the master cache-optimisation equation with its three levers.

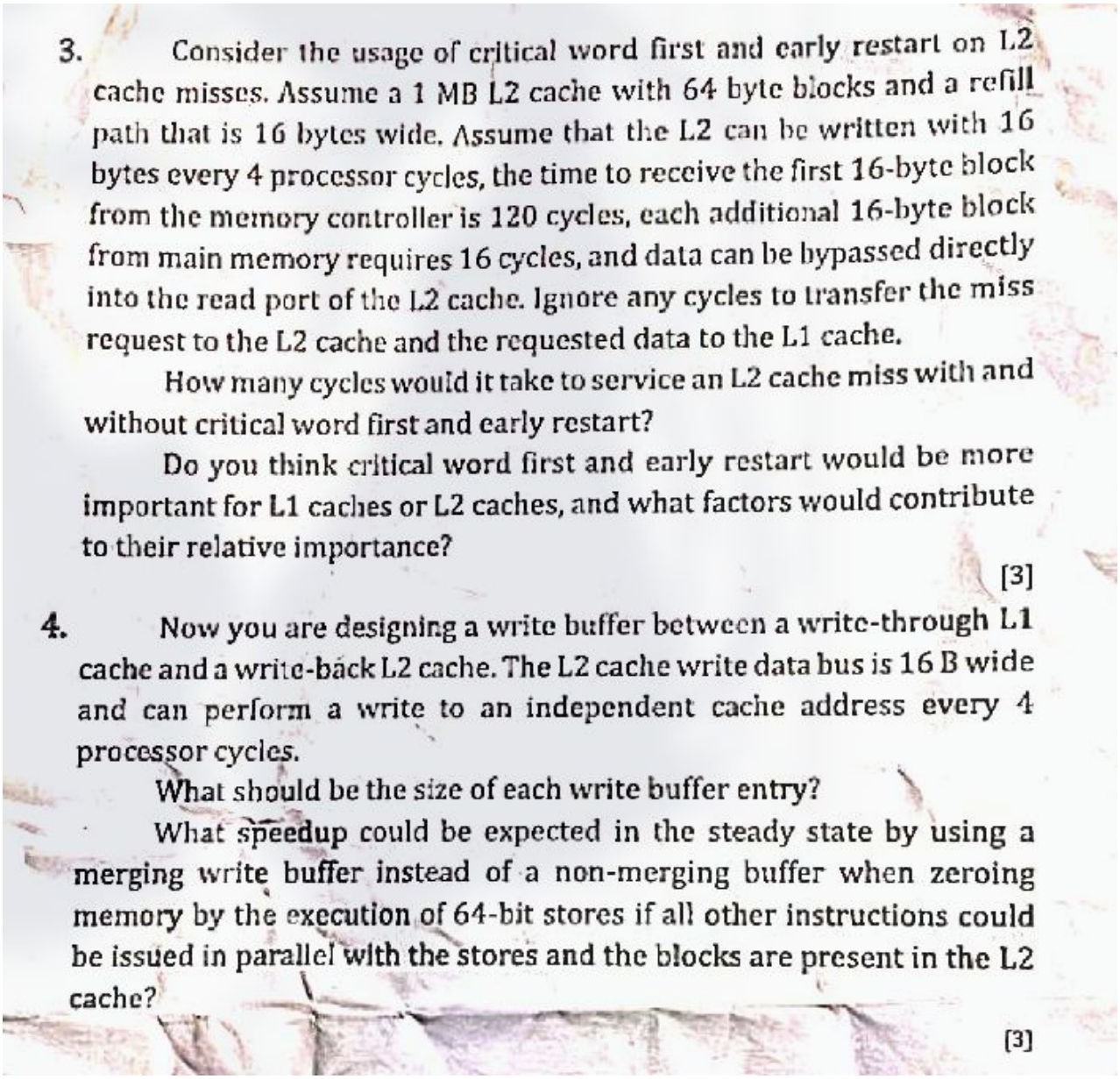
```
AMAT = Hit time + Miss rate × Miss penalty → three independent ways to reduce AMAT: 1) reduce hit time 2) reduce miss rate 3) reduce miss penalty
```

The three levers

- **Reduce hit time** — use a smaller/faster cache, a simpler (direct-mapped) organisation, and avoid adding address-translation delay to the critical path.
- **Reduce miss rate** — use a bigger cache, higher associativity, better replacement (LRU), and reduce the three C's (compulsory, capacity, conflict).
- **Reduce miss penalty** — use multilevel caches, **critical-word-first** (return the needed word first), **non-blocking** caches (several misses in flight), and prefetching.

> **Key point.** These three goals often conflict — a bigger cache lowers the miss rate but raises the hit time. Cache design is the art of balancing the three terms of AMAT.

Previous-year question 35 — ca35 — Critical-word-first and early restart cycles; mergin…



3. Consider the usage of critical word first and early restart on L2 cache misses. Assume a 1 MB L2 cache with 64 byte blocks and a refill path that is 16 bytes wide. Assume that the L2 can be written with 16 bytes every 4 processor cycles, the time to receive the first 16-byte block from the memory controller is 120 cycles, each additional 16-byte block from main memory requires 16 cycles, and data can be bypassed directly into the read port of the L2 cache. Ignore any cycles to transfer the miss request to the L2 cache and the requested data to the L1 cache.

How many cycles would it take to service an L2 cache miss with and without critical word first and early restart?

Do you think critical word first and early restart would be more important for L1 caches or L2 caches, and what factors would contribute to their relative importance?

[3]

4. Now you are designing a write buffer between a write-through L1 cache and a write-back L2 cache. The L2 cache write data bus is 16 B wide and can perform a write to an independent cache address every 4 processor cycles.

What should be the size of each write buffer entry?

What speedup could be expected in the steady state by using a merging write buffer instead of a non-merging buffer when zeroing memory by the execution of 64-bit stores if all other instructions could be issued in parallel with the stores and the blocks are present in the L2 cache?

[3]

Original past-paper scan ca35.png (re-rendered).

Transcription of the question

3. Critical word first and early restart on L2 misses: 1 MB L2, 64-byte blocks, 16-byte refill path; the L2 can be written 16 bytes every 4 cycles; the first 16-byte block from the memory controller takes 120 cycles; each additional 16-byte block takes 16 cycles; data bypassed directly into the read port. Ignore request/return cycles. How many cycles to service an L2 miss with and without critical word first? Would CWF be more important for L1 or L2, and what factors matter? (3)
4. A write buffer between a write-through L1 and a write-back L2: the L2 write data bus is 16 B wide and can write to an independent address every 4 cycles. What should be the size of each write-buffer entry? What speed-up can be expected from a merging write buffer when zeroing memory with 64-bit stores? (3)

Full solution

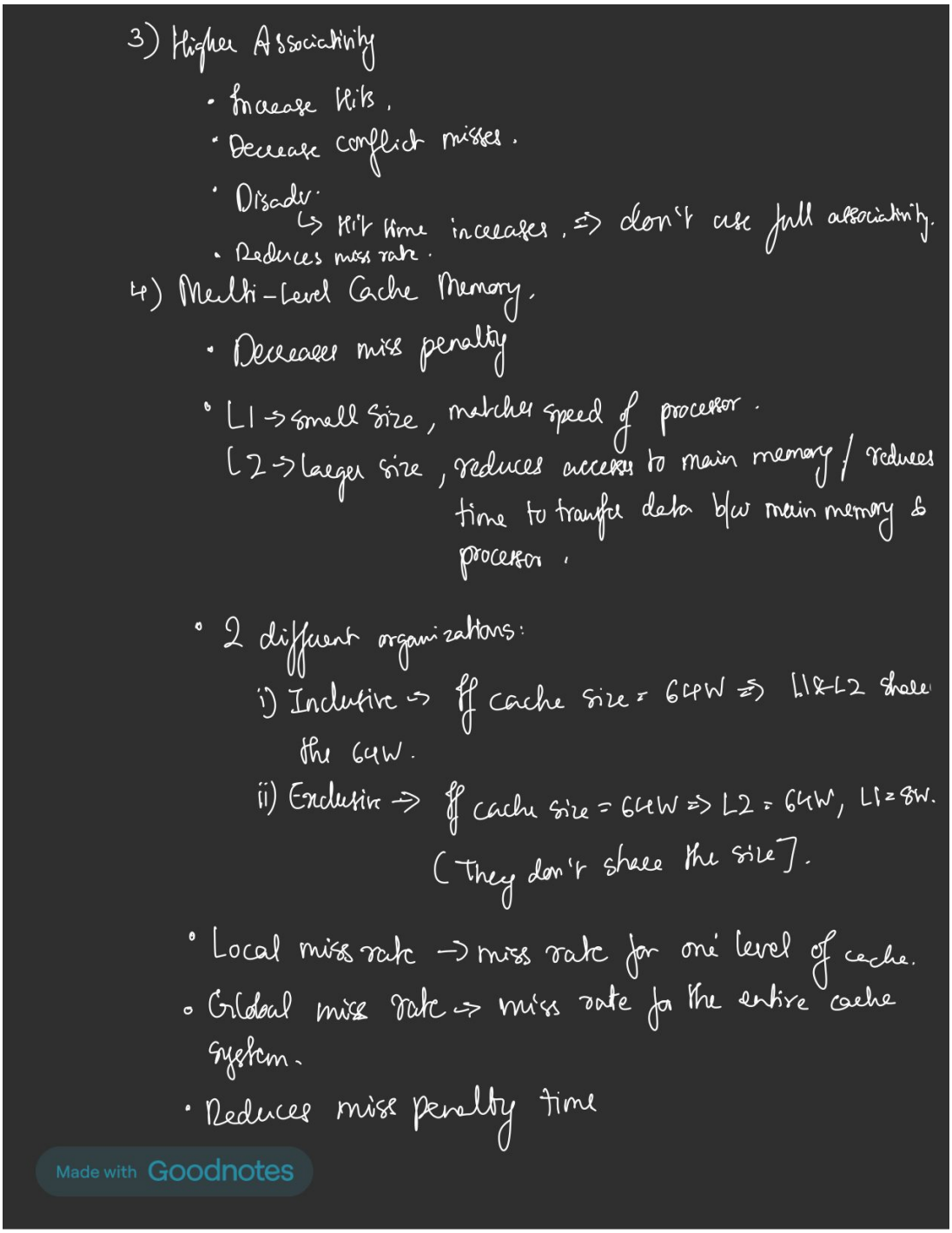
3 — cycles with and without CWF

Block = 64 B = four 16-byte beats. Without CWF: 120 (first beat) + 3 × 16 = 168 cycles. With CWF: the requested beat arrives at 120 cycles and the CPU restarts; the remaining beats fill in the background → ≈ 120 cycles to first data. CWF matters more for the level whose miss penalty the CPU waits on — usually the L1 (or the L2 when it is the last on-chip level before DRAM). The relative importance grows with the block size and the number of beats the requested word is from the start of the block.

4 — merging write buffer

Write-buffer entry size = the L2 bus width × the number of cycles it can hold = 16 B (one bus transfer) — sized to match the bus width. Zeroing with 64-bit (8-byte) stores: a non-merging buffer needs a write per store (8 B each), while a merging buffer coalesces 2 stores into one 16-byte bus transfer → up to 2× fewer bus transactions, hence ≈ 2× speed-up on the store stream.

Original notes — page 80: Multilevel caches; inclusive/exclusive; local vs global miss r…



Original handwritten notes — page 80.

Transcription of the page

```
Multilevel caches:

  L1 (small, fast, private) → L2 (bigger) → L3 → main memory.

  Inclusive: a block in L1 is also in L2 (L2 ▫ L1).
  Exclusive: a block is in only one level (L1 ∩ L2 = ▫).

  Local miss rate  = misses in THIS cache / accesses to THIS cache.
  Global miss rate = misses in THIS cache / total accesses to the CPU.

  Multilevel caches reduce the miss penalty (the L2 serves the L1's misses).
```

Explanation

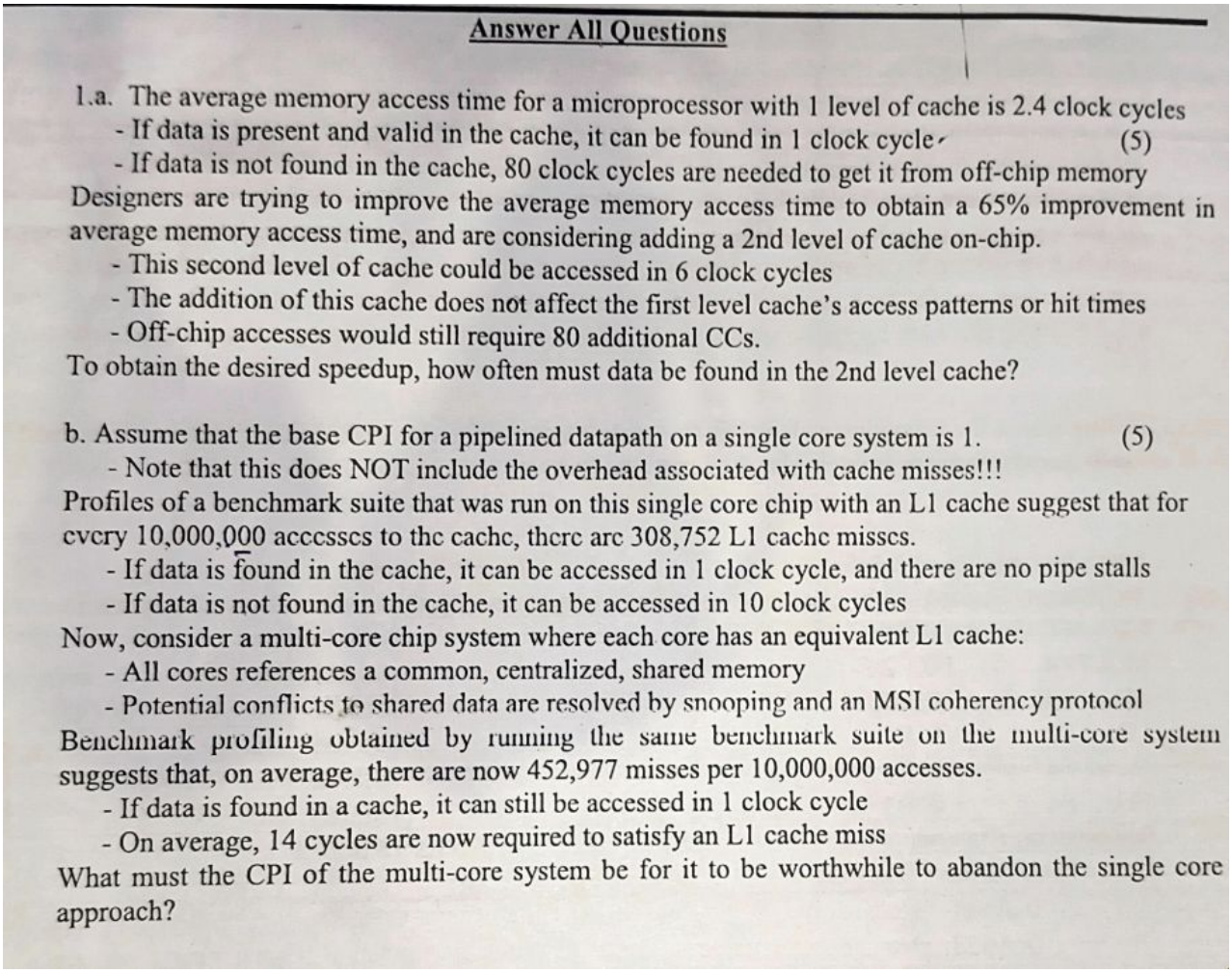
What the page says. How several cache levels cooperate, the inclusive/exclusive distinction, and the two ways of measuring a miss rate.

- **Multilevel caches** — a small fast L1 backed by a larger slower L2, then L3, then memory. The L2 serves the L1's misses, so the effective miss penalty seen by the CPU is much smaller.
- **Inclusive** — every block in L1 is also in L2 (L2 L1). This makes coherence and invalidation simpler (checking L2 tells you about L1).
- **Exclusive** — a block lives in only one level at a time (L1 ∩ L2 =), which uses the total capacity better.

```
Local miss rate = misses in this cache / accesses to this cache Global miss rate = misses in this cache / total accesses to the CPU AMAT (two levels) = t_L1 + m_L1 · ( t_L2 + m_L2 · t_mem ) m_L1 = L1 miss rate, m_L2 = L2 LOCAL miss rate.
```

Which miss rate to use? When combining levels, use the *local* miss rate of the second level inside the formula above. The *global* miss rate of L2 is m_L1 × m_L2 — the fraction of all CPU accesses that miss in both.

Previous-year question 15 — ca15 — AMAT with a second-level cache; multicore CPI to…



Answer All Questions

1.a. The average memory access time for a microprocessor with 1 level of cache is 2.4 clock cycles
- If data is present and valid in the cache, it can be found in 1 clock cycle (5)
- If data is not found in the cache, 80 clock cycles are needed to get it from off-chip memory

Designers are trying to improve the average memory access time to obtain a 65% improvement in average memory access time, and are considering adding a 2nd level of cache on-chip.
- This second level of cache could be accessed in 6 clock cycles
- The addition of this cache does not affect the first level cache's access patterns or hit times
- Off-chip accesses would still require 80 additional CCs.

To obtain the desired speedup, how often must data be found in the 2nd level cache?

b. Assume that the base CPI for a pipelined datapath on a single core system is 1. (5)
- Note that this does NOT include the overhead associated with cache misses!!!

Profiles of a benchmark suite that was run on this single core chip with an L1 cache suggest that for every 10,000,000 accesses to the cache, there are 308,752 L1 cache misses.
- If data is found in the cache, it can be accessed in 1 clock cycle, and there are no pipe stalls
- If data is not found in the cache, it can be accessed in 10 clock cycles

Now, consider a multi-core chip system where each core has an equivalent L1 cache:
- All cores references a common, centralized, shared memory
- Potential conflicts to shared data are resolved by snooping and an MSI coherency protocol

Benchmark profiling obtained by running the same benchmark suite on the multi-core system suggests that, on average, there are now 452,977 misses per 10,000,000 accesses.
- If data is found in a cache, it can still be accessed in 1 clock cycle
- On average, 14 cycles are now required to satisfy an L1 cache miss

What must the CPI of the multi-core system be for it to be worthwhile to abandon the single core approach?

Original past-paper scan ca15.png (re-rendered).

Transcription of the question

```
1a. The average memory access time for a microprocessor with one level of cache is
    2.4 clock cycles. If data is present and valid it is found in 1 clock cycle. If it is
    not found, 80 clock cycles are needed from off-chip memory. Designers want a 65%
    improvement in AMAT and are considering a second-level cache on-chip that is
    accessed in 6 clock cycles (the addition does not affect the L1 access pattern or
    hit time; off-chip accesses still require 80 extra cycles). To obtain the desired
    speed-up, how often must data be found in the second-level cache? (5)
  b. The base CPI for a pipelined single-core system is 1 (excluding cache misses).
    For every 10,000,000 accesses there are 308,752 L1 misses; a hit costs 1 cycle, a
    miss 10 cycles. Now consider a multi-core chip with equivalent L1 caches, shared
    centralised memory, snooping MSI coherence, where misses rise to 452,977 per
    10,000,000 accesses and a miss now costs 14 cycles. What must the CPI of the
    multi-core system be for it to be worthwhile to abandon the single core? (5)
```

Full solution

1a — required second-level hit rate

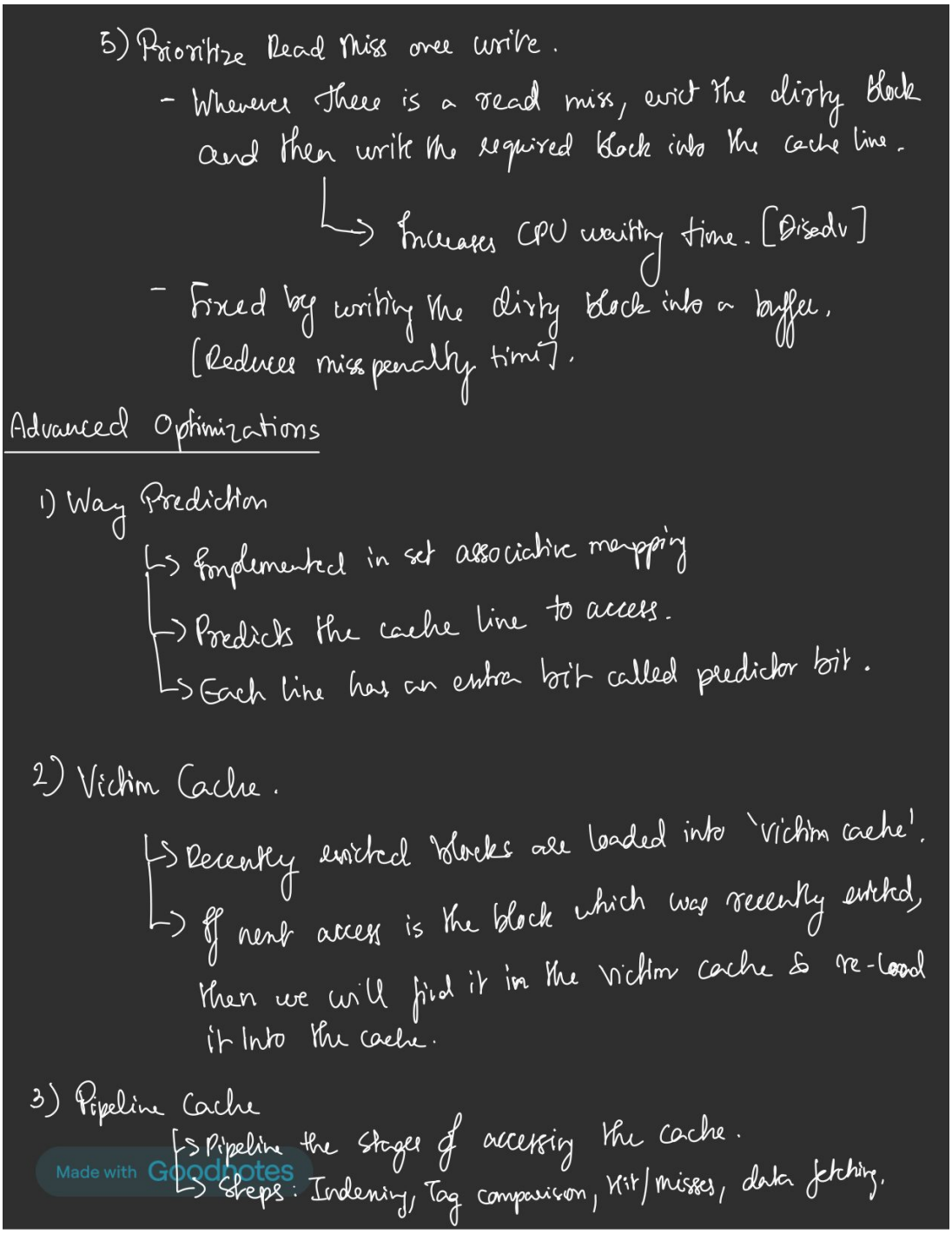
```
Single level: AMAT = 2.4 cycles AMAT = hit_time + miss_rate × miss_penalty 2.4 = 1 + m_L1 × 80 → m_L1 = 1.4/80 = 0.0175 (L1 miss rate) Target AMAT = 2.4 × (1 − 0.65) = 0.84 cycles (a 65% improvement) With L2: AMAT = 1 + m_L1 × (6 + m_L2 × 80) 0.84 = 1 + 0.0175 × (6 + m_L2 × 80) → 0.0175×(6 + 80·m_L2) = −0.16
```

> **Note the impossibility.** The target of 0.84 cycles is *below* the 1-cycle L1 hit time, so no second-level cache can reach it — the required L2 hit rate would have to exceed 100%. The correct exam answer is to show this: the goal is unachievable because the L1 hit time already exceeds the target AMAT.

1b — required multicore CPI

```
Single core: L1 miss rate = 308752 / 10⁷ = 0.0308752 CPI = 1 + 0.0308752 × 10 = 1 + 0.308752 ≈ 1.309 Multicore: miss rate = 452977 / 10⁷ = 0.0452977 CPI_multi = 1 + 0.0452977 × 14 = 1 + 0.634 ≈ 1.634 The multicore is worthwhile only if its CPI is BELOW the single-core CPI (≈ 1.309) once its extra parallelism is counted; on raw memory CPI the multicore is worse (1.634 > 1.309), so the extra cores must more than make up the difference.
```

Original notes — page 81: Prefetching — hardware and software



Original handwritten notes — page 81.

Transcription of the page

```
Prefetching:

  1) Hardware prefetching: the cache hardware notices a pattern (e.g. sequential
     access) and fetches blocks ahead of time, into the cache.

  2) Software prefetching: the compiler inserts explicit prefetch instructions
     that bring data in before it is needed.

  3) Hybrid prefetching: a combination of the two.

  Prefetching reduces the effective miss penalty (and sometimes the miss rate)
  by hiding memory latency behind useful work.

  Cost: extra memory traffic; a wrong prefetch wastes bandwidth.
```

Explanation

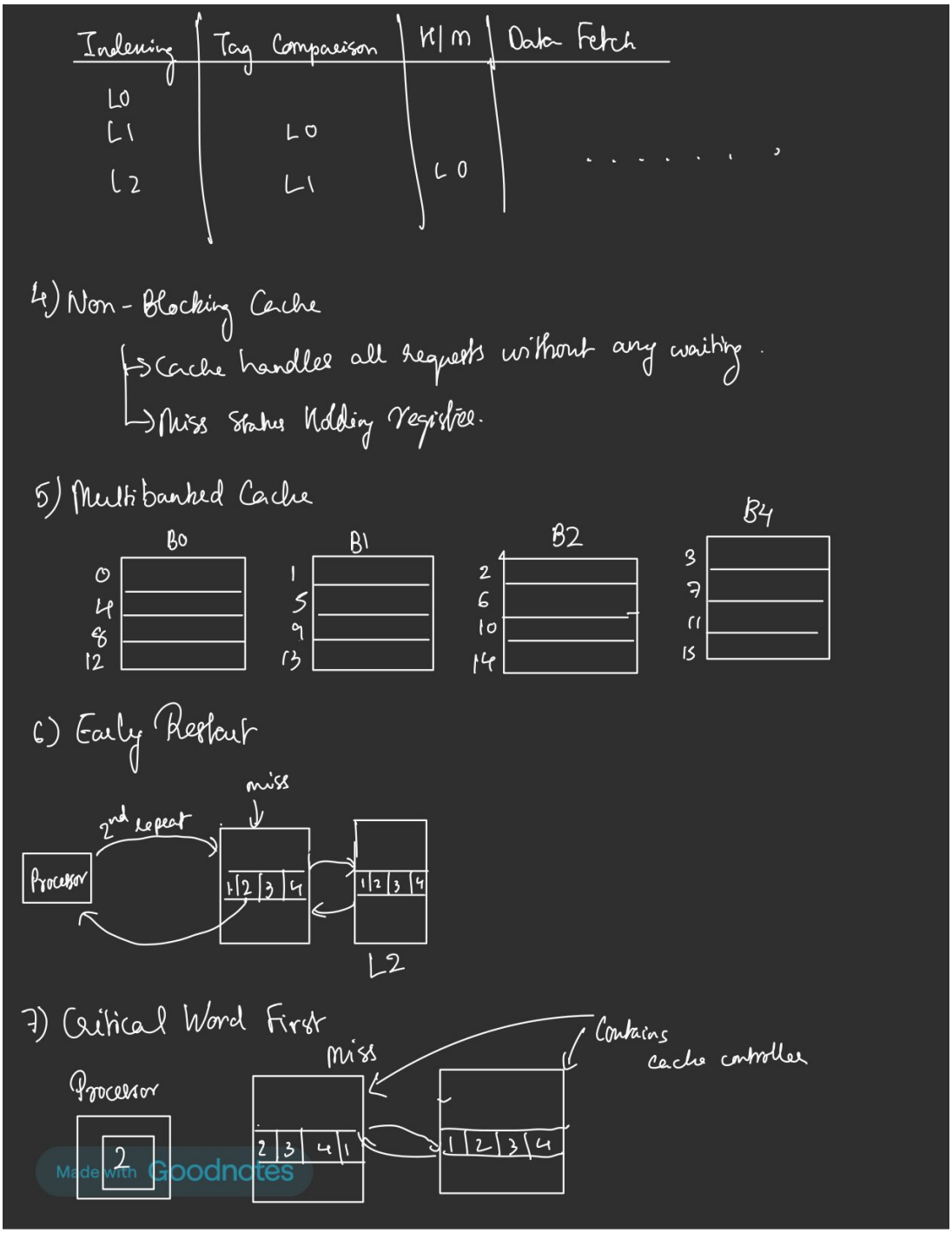
What the page says. Prefetching hides memory latency by bringing data into the cache before it is asked for. The page lists the hardware, software and hybrid forms.

- **Hardware prefetching** — the cache observes a stream (e.g. sequential addresses) and fetches the next blocks automatically. No program changes needed.
- **Software prefetching** — the compiler inserts prefetch instructions early in the code so the data is in the cache by the time it is used.
- **Hybrid** — hardware and software prefetching together.

```
Benefit: the memory latency is overlapped with useful work, so the effective miss penalty
falls (the data is already in the cache). Cost: extra memory bandwidth; a mis-predicted
prefetch fetches data that is never used (cache pollution).
```

Prefetch vs critical-word-first. Both attack the miss penalty. Prefetching brings data in *earlier*; critical-word-first returns the *needed word first* so the CPU can restart sooner. A good design uses both.

Original notes — page 82: Virtual memory — address translation, the TLB and page fau…



Original handwritten notes — page 82.

Transcription of the page

```
Virtual Memory:

  Virtual address → (page table) → Physical address

  Address split:  [ Virtual page number | Page offset ]
                  [ Physical page number | Page offset ]

  All pages of a process map into one linear virtual address space.

  TLB (Translation Lookaside Buffer): a small cache of recent translations.

  Page fault: the page is not in memory → bring it in from disk.

  Critical-word-first; write-back caches (for the page table entries).
```

Explanation

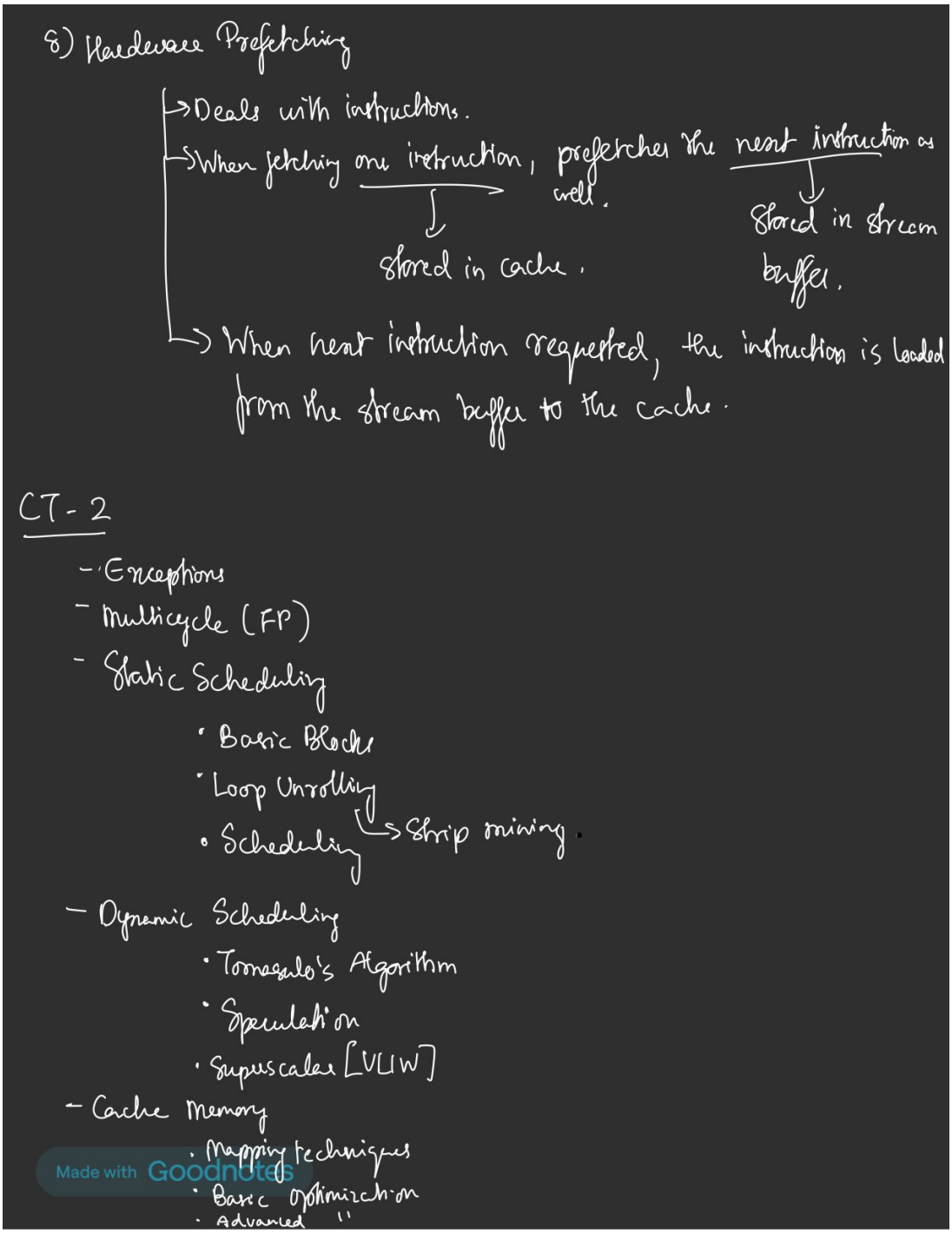
What the page says. Virtual memory: every program uses virtual addresses that are translated to physical addresses through a page table, with a TLB to speed up the translation.

```
Virtual address = [ Virtual Page Number (VPN) | Page Offset ] Physical address = [ Physical Page Number (PPN) | Page Offset ] page offset bits = log₂(page size) (e.g. 4 KB page → 12 bits) PPN = page_table[VPN]
```

- **Why virtual memory?** Each process gets its own address space; programs can be larger than physical memory; and memory is protected between processes.
- **Page table** — maps each virtual page to a physical page (or marks it “not present”).
- **TLB** — a small, fast cache of recent translations, so most accesses avoid the page-table walk.
- **Page fault** — the page is not in physical memory; the OS brings it in from disk and restarts the instruction.

Reconstructed diagram. Draw the CPU issuing a virtual address; split it into VPN and offset; the VPN indexes the page table (or, on a hit, the TLB) to give the PPN; concatenate the PPN with the offset to form the physical address.

Original notes — page 83: Summary of instruction-level optimisations (the compiler/ha…



Original handwritten notes — page 83.

Transcription of the page

```
Instruction-level optimisations (summary):

  * Compiler optimisations
  * Multicast (FP)
  * Speculative loading
  * Branch prediction
  * Loop unrolling
  * Scheduling
  * Tomasulo's algorithm
  * Dynamic scheduling
  * Superscalar (VLIW)
  * Cache memory

  Hardware prefetching; deals with long-latency instructions — when an
  instruction loop executes, instructions are fetched from the stream of code.
```

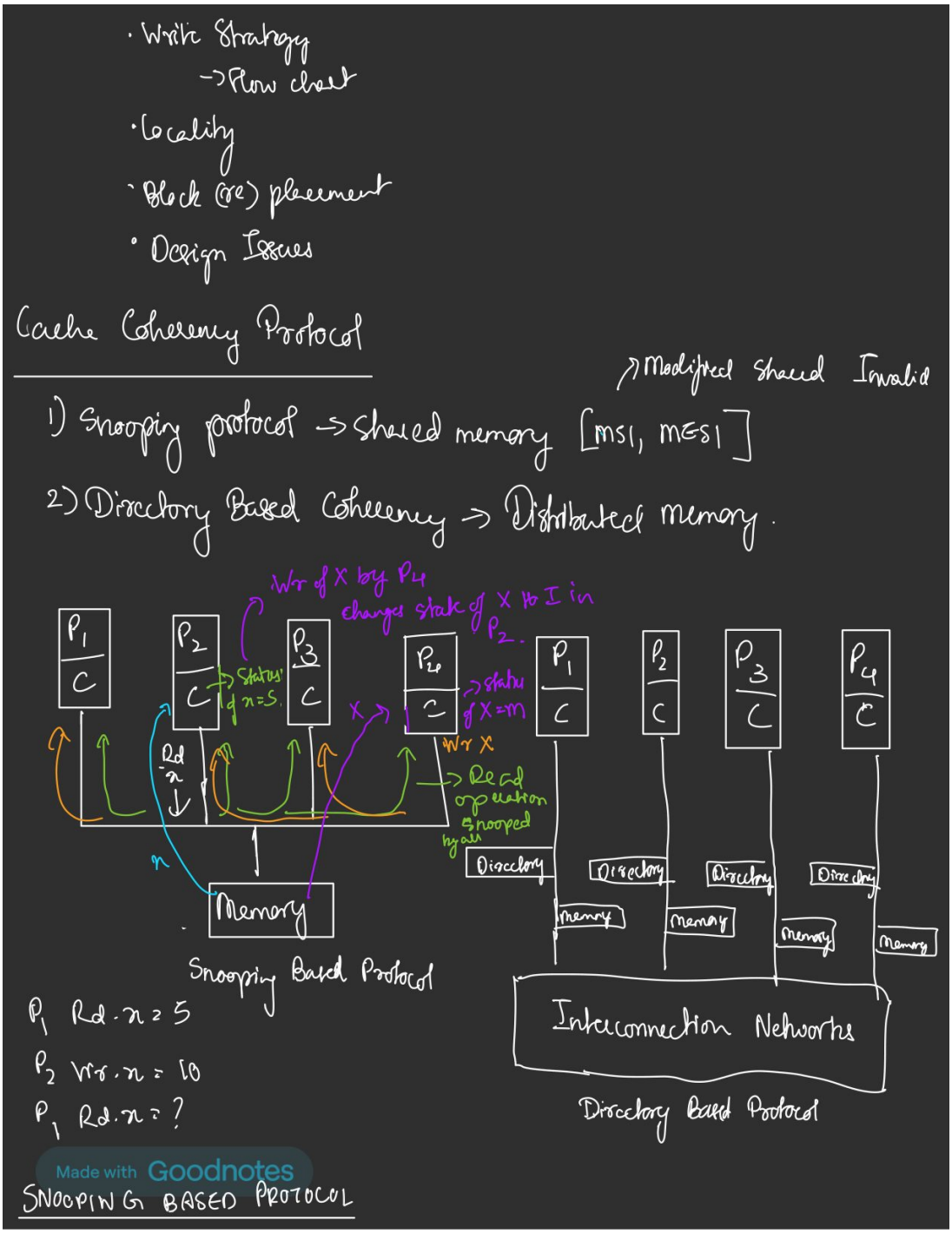
Explanation

What the page says. A summary checklist of every technique the notes have covered for extracting instruction-level parallelism and hiding latency.

Technique	What it does
Compiler optimisation	Reorders/rewrites code to expose ILP
Multicast (FP)	Broadcast one result to several consumers at once
Speculative loading	Load early, before it is certain the load is needed
Branch prediction	Guess the branch direction to avoid control stalls
Loop unrolling	Repeat the body to amortise overhead and expose ILP
Scheduling	Reorder instructions to fill delay slots
Tomasulo’s algorithm	Dynamic scheduling with renaming and the CDB
Dynamic scheduling	Hardware reorders at run time
Superscalar / VLIW	Issue several instructions per cycle
Cache memory	Keep data close to reduce memory latency

> **Two families.** *Static* techniques (compiler: unrolling, scheduling, software prefetch) fix the schedule at build time. *Dynamic* techniques (branch prediction, Tomasulo, speculation, superscalar) adapt at run time. Real CPUs use both together.

Original notes — page 84: Cache coherence — the problem, MSI/MESI and snooping vs…



Original handwritten notes — page 84.

Transcription of the page

```
Cache Coherence:

  Problem: several private caches may hold copies of one block; a write to one
  must not leave the others stale.

  Two families of protocol:
   1) Snooping (broadcast on a shared bus),
   2) Directory-based (a directory records who holds each block).

  States: MSI (Modified, Shared, Invalid) and MESI (+ Exclusive).

  Write policy: write-through or write-back.

  Example protocol diagram (shared bus, invalidate on write).
```

Explanation

What the page says. The cache-coherence problem and the two families of solution, with the MSI/MESI states.

The coherence problem

When several caches keep private copies of a block, a write by one core makes the others stale. A coherence protocol guarantees that every read returns the latest write.

State	Meaning
M — Modified	This cache has the only copy, and it is dirty (differs from memory)
S — Shared	The block is clean and may be in several caches
I — Invalid	This cache does not have a valid copy
E — Exclusive (MESI)	Clean, held by this cache only (a silent upgrade to M is possible)

Two families of protocol

- **Snooping** — every cache watches (snoops) the shared bus; a write is broadcast and the other caches invalidate or update their copies. Simple; needs a bus (does not scale).
- **Directory-based** — a directory records which caches hold each block, so coherence messages go only to the sharers. Scales to many nodes.

```
MSI transitions (example): read miss (no sharer) : I → S write miss : I → M (and invalidate other copies) write hit in S : S → M (invalidate others) snoop read while M : M → S (write back / supply data) snoop write while S/M : S/M → I
```

Previous-year question 2 — ca2 — Loop dependences / parallelisation; MSI cache-cohe…

b. Consider the following loop: (2.5)

```
for (i=0;i < 100;i++) {
        A[i] = A[i] + B[i]; /* S1 */
        B[i+1] = C[i] + D[i]; /* S2 */
}
```

Are there dependences between S1 and S2? Is this loop parallel? If not, show how to make it parallel.

2. Explain the two classes of the protocols? A snapshot of the state associated with 2 caches, on 2 separate cores, in a centralized shared memory system is shown below. In this system, cache coherency is maintained with an MSI snooping protocol. You can assume that the caches are direct mapped. (10)

P0	Tag	Data Word 1	Data Word 2	Data Word 3	Data Word 4	Coherency State
Block	1000	10	20	30	40	M
Block 1	4000	500	600	700	800	S
...	...	...	...	...	..	...
Block N	3000	2	4	6	8	S

P1	Tag	Data Word 1	Data Word 2	Data Word 3	Data Word 4	Coherency State
Block	1000	10	10	10	10	I
Block 1	8000	500	600	700	800	S
...	...	...	...	...	..	...
Block N	3000	2	4	6	8	S

i. If P0 wants to write Block 0, what happens to its coherency state?
ii. If P1 writes to Block 1, is Block 1 on P0 invalidated? Why or why not?
iii. If P1 brings in Block M for reading, and no other cache has a copy, what state is it cached in?

Original past-paper scan ca2.png (re-rendered).

Transcription of the question

```
b. Consider the loop:  (2.5)
     for (i=0; i<100; i++) {
        A[i] = A[i] + B[i];   /* S1 */
        B[i] = C[i] + D[i];   /* S2 */
     }
   Are there dependences between S1 and S2? Is this loop parallel? If not, show how
   to make it parallel.

2. Explain the two classes of coherence protocols. A snapshot of the state of 2 caches
   on 2 separate cores in a centralized shared-memory system is shown. Coherence is
   maintained with an MSI snooping protocol; caches are direct-mapped. (10)
     (Cache P0 and cache P1 tables with Tag / Data words / Coherency state)
     i.   If P0 wants to write Block 0, what happens to its coherency state?
     ii.  If P1 writes to Block 1, is Block 1 on P0 invalidated? Why?
     iii. If P1 brings in Block M for reading, and no other cache has a copy, what
          state is it cached in?
```

Full solution

Part b — dependences in the two-statement loop

S1 reads/writes A[i] and reads B[i]; S2 reads C[i], D[i] and writes B[i]. The only shared location is B[i]: S1 reads B[i] and S2 writes it — a **WAR (anti-dependence)**. Because different iterations touch different indices, there is no *loop-carried* dependence, so the iterations are independent and the loop **is parallel** across i.

```
S1: A[i] = A[i] + B[i] (reads B[i]) S2: B[i] = C[i] + D[i] (writes B[i]) → WAR on B[i] within one iteration, but no dependence between different i, so iterations can run in parallel (rename B to be safe).
```

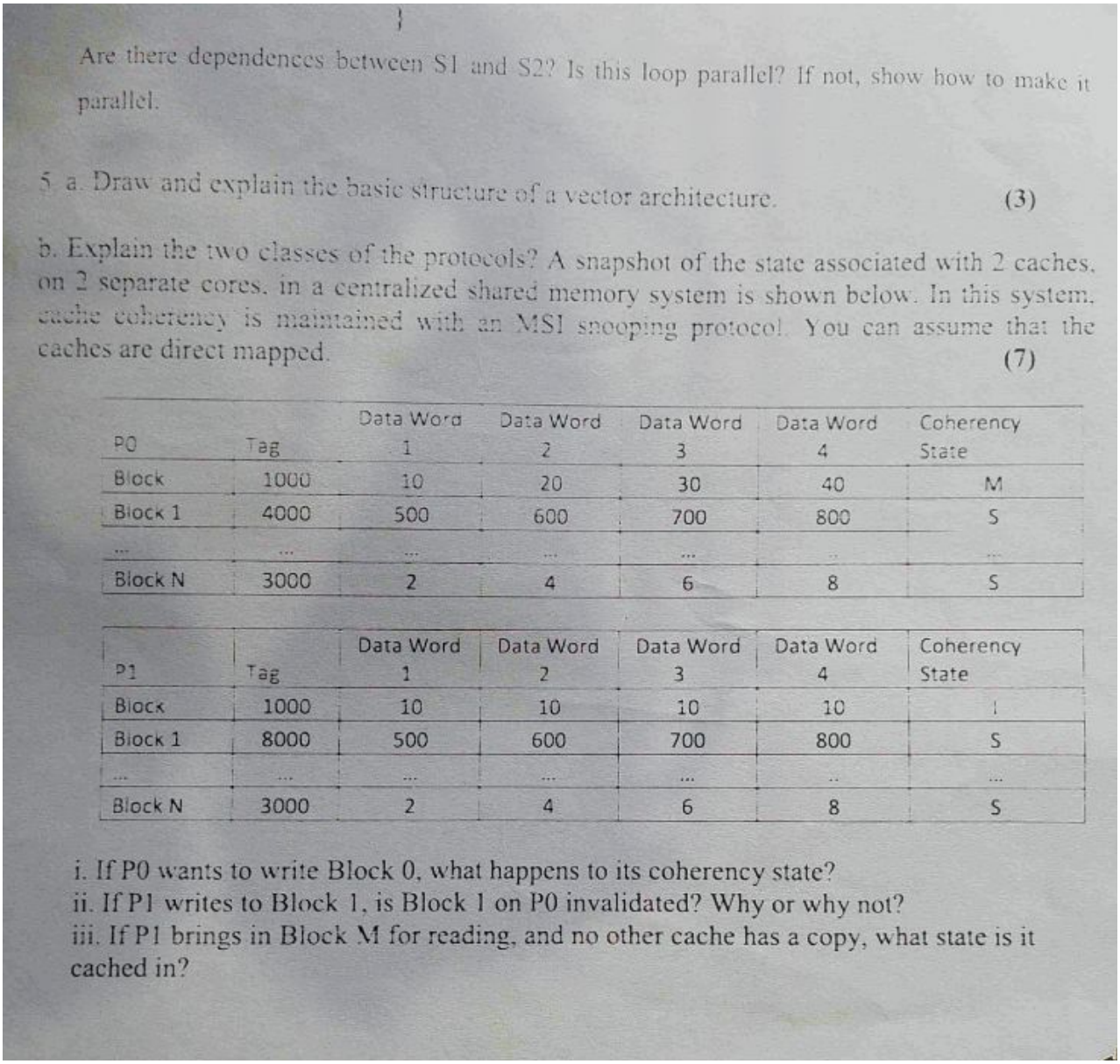
The two classes of coherence protocol

- **Snooping** — every cache watches a shared bus; writes are broadcast and other copies are invalidated or updated. Simple, but needs a broadcast medium.
- **Directory-based** — a directory records which caches hold each block; messages go only to the sharers, so it scales to large systems.

The MSI snapshot

```
(i) P0 writes Block 0: its state goes S → M (or I → M if it missed); every other copy of Block 0 is invalidated (→ I). (ii) P1 writes Block 1: yes — P0's copy of Block 1 is invalidated (→ I), because the write-invalidate protocol keeps only the writer's copy valid. (iii) P1 reads Block M with no other sharer: it is cached in state S (shared) under MSI; under MESI it would be E (exclusive).
```

Previous-year question 23 — ca23 — Loop parallelisation; vector architecture structure;…



Are there dependences between S1 and S2? Is this loop parallel? If not, show how to make it parallel.

5. a. Draw and explain the basic structure of a vector architecture. (3)

b. Explain the two classes of the protocols? A snapshot of the state associated with 2 caches, on 2 separate cores, in a centralized shared memory system is shown below. In this system, cache coherency is maintained with an MSI snooping protocol. You can assume that the caches are direct mapped. (7)

P0	Tag	Data Word 1	Data Word 2	Data Word 3	Data Word 4	Coherency State
Block	1000	10	20	30	40	M
Block 1	4000	500	600	700	800	S
...	...	...	...	...	..	...
Block N	3000	2	4	6	8	S

P1	Tag	Data Word 1	Data Word 2	Data Word 3	Data Word 4	Coherency State
Block	1000	10	10	10	10	I
Block 1	8000	500	600	700	800	S
...	...	...	...	...	..	...
Block N	3000	2	4	6	8	S

i. If P0 wants to write Block 0, what happens to its coherency state?
ii. If P1 writes to Block 1, is Block 1 on P0 invalidated? Why or why not?
iii. If P1 brings in Block M for reading, and no other cache has a copy, what state is it cached in?

Original past-paper scan ca23.png (re-rendered).

Transcription of the question

```
1. Is the given loop parallel? If not, show how to make it parallel.
2. Draw and explain the basic structure of a vector architecture. (2)
3. Explain the two classes of coherence protocols. A snapshot of the state of 2 caches on
   2 separate cores in a centralised shared-memory system is shown. Coherence is
   maintained with an MSI snooping protocol; the caches are direct-mapped.
   i.   If P0 wants to write Block 0, what happens to its coherency state?
   ii.  If P1 writes to Block 1, is Block 1 on P0 invalidated? Why or why not?
   iii. If P1 brings in Block M for reading, and no other cache has a copy, what state
        is it cached in?
```

Full solution

1 — loop parallelisation

As in ca2: the only shared location creates a WAR/name dependence, but there is no loop-carried dependence, so the loop is parallel across iterations; rename to make it explicit.

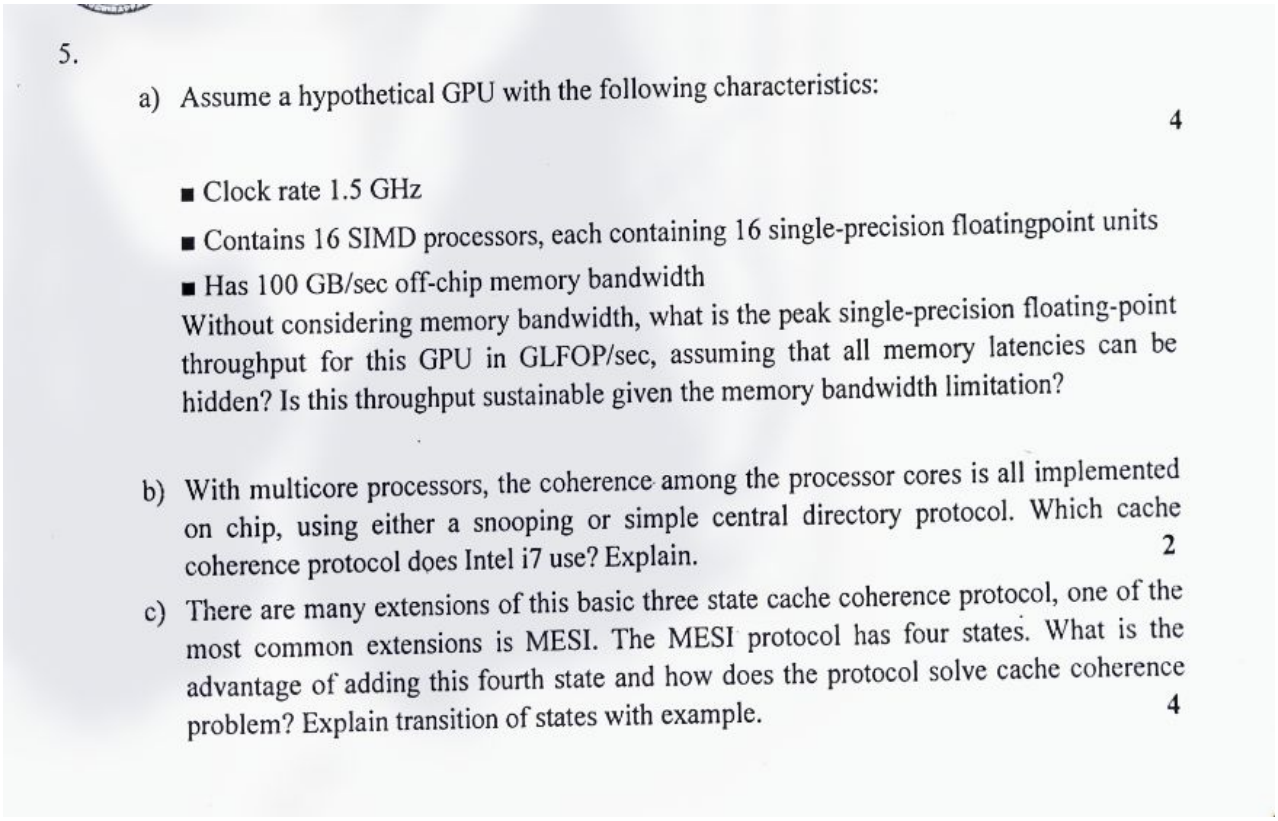
2 — vector architecture structure

Draw: scalar register file + **vector register file**; a pipelined **vector functional unit** with several lanes; the **vector-length register** and **stride register**; and the memory system feeding the lanes. One vector instruction operates on VL elements, so all lanes work together.

3 — MSI snapshot

(i) P0 writes Block 0: its state → M (Modified); every other copy → I. (ii) P1 writes Block 1: yes — P0's copy of Block 1 → I (write-invalidate). (iii) P1 reads Block M, no other sharer: state → S (Shared). (Under MESI it would be E, Exclusive.)

Previous-year question 29 — ca29 — GPU peak throughput and the memory bound; Inte…



5.

a) Assume a hypothetical GPU with the following characteristics: 4

- Clock rate 1.5 GHz
- Contains 16 SIMD processors, each containing 16 single-precision floatingpoint units
- Has 100 GB/sec off-chip memory bandwidth

Without considering memory bandwidth, what is the peak single-precision floating-point throughput for this GPU in GLFOP/sec, assuming that all memory latencies can be hidden? Is this throughput sustainable given the memory bandwidth limitation?

b) With multicore processors, the coherence among the processor cores is all implemented on chip, using either a snooping or simple central directory protocol. Which cache coherence protocol does Intel i7 use? Explain. 2

c) There are many extensions of this basic three state cache coherence protocol, one of the most common extensions is MESI. The MESI protocol has four states. What is the advantage of adding this fourth state and how does the protocol solve cache coherence problem? Explain transition of states with example. 4

Original past-paper scan ca29.png (re-rendered).

Transcription of the question

```
3a. A hypothetical GPU: clock 1.5 GHz; 16 SIMD processors, each with 16 single-precision
    floating-point units; 100 GB/s off-chip memory bandwidth. Without considering memory
    bandwidth, what is the peak single-precision floating-point throughput in GFLOP/s
    (assuming all memory latencies are hidden)? Is this sustainable given the memory
    bandwidth? (4)
 b. With multicore processors, coherence among cores is implemented on-chip using either
    snooping or a simple central directory. Which protocol does Intel i7 use? Explain. (2)
 c. MESI extends the three-state protocol with a fourth state. What is the advantage of the
    fourth state and how does the protocol solve coherence? Explain state transitions. (4)
```

Full solution

3a — GPU peak throughput

FLOPs per cycle = 16 SIMD × 16 FP units × 2 (FMA counts as 2 FLOPs) = 512 Peak throughput = $512 \times 1.5\times10^{9} = 7.68\times10^{11}$ FLOP/s = 768 GFLOP/s Bytes needed per FLOP = (4 bytes read + 4 bytes written per op)/2 ≈ 4 B/FLOP Required bandwidth ≈ $768\times10^{9} \times 4 = 3.07\times10^{12}$ B/s = 3072 GB/s Available = 100 GB/s → the GPU is MEMORY-BOUND; the peak is NOT sustainable.

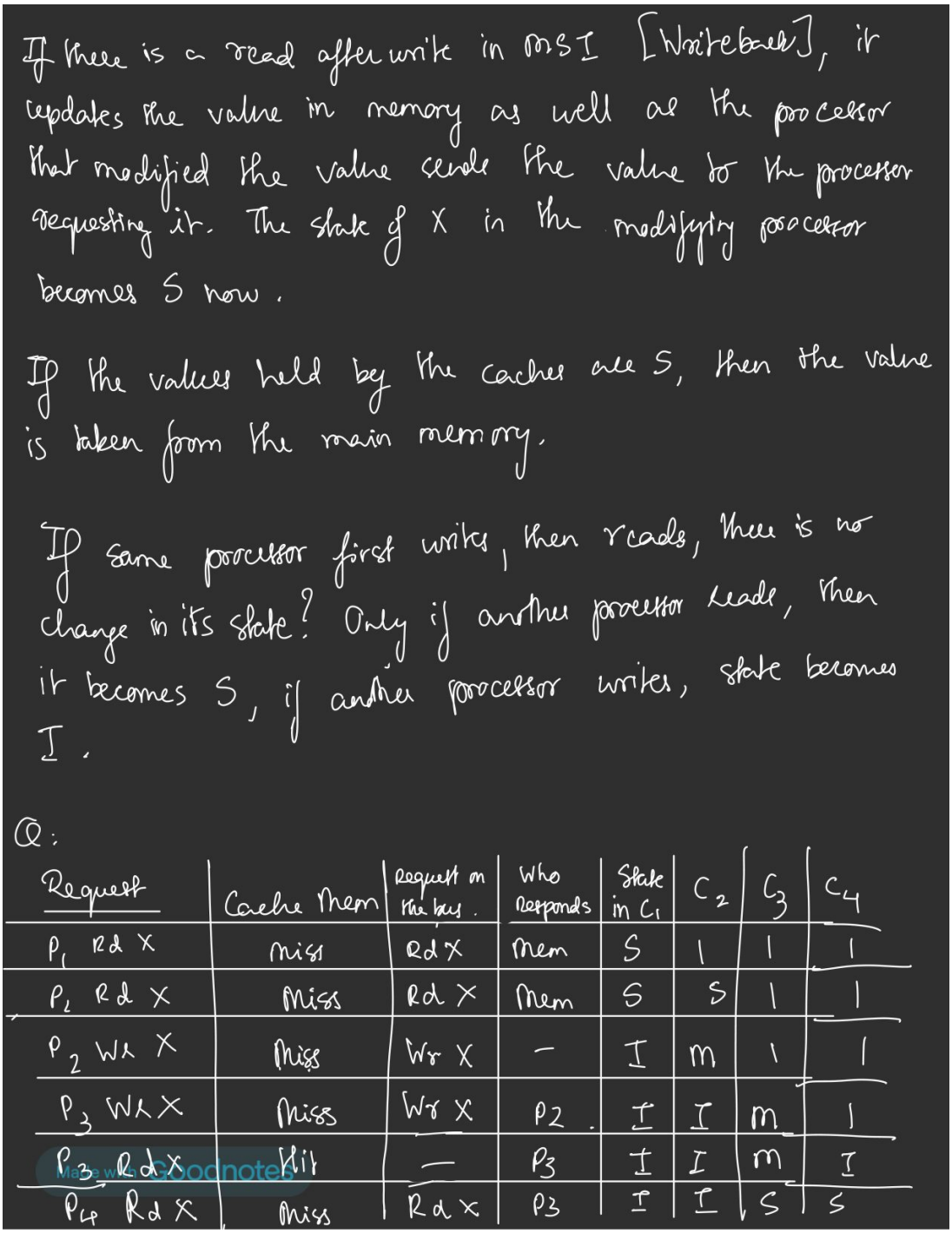
3b — Intel i7 coherence

Intel i7 uses a **snooping MESI** protocol over the on-chip interconnect (the ring/uncore): private L1/L2 caches snoop, the shared L3 acts as the point of coherence, and blocks are in Modified, Exclusive, Shared or Invalid state.

3c — advantage of the MESI fourth state (E)

E (Exclusive): the block is clean and held by ONLY this cache. A write can upgrade E → M SILENTLY (no bus traffic), because no other cache has a copy. This removes a coherence transaction for every private write — the key benefit of MESI over MSI. Transitions: I → S on a read miss (shared); I → E on a read miss with no sharers; E → M on a write; S → M on a write (invalidating others); M → S on a snooped read; any → I on a snooped write.

Original notes — page 85: MSI example — write-invalidate on a shared bus (snapshot)



Original handwritten notes — page 85.

Transcription of the page

```
MSI example (write-invalidate protocol):

  Snapshot:  caches C1..C4 with tags and coherency states.

  Q: When P0 writes block x, what happens to its state and to the others?
     → the writer's copy becomes M; all other copies become I.

  Q: When P1 reads x, what happens?
     → the copy becomes S (shared).

  (Table of tags/states before and after each event.)

  Directory-based protocol introduced: a home directory tracks sharers.
```

Explanation

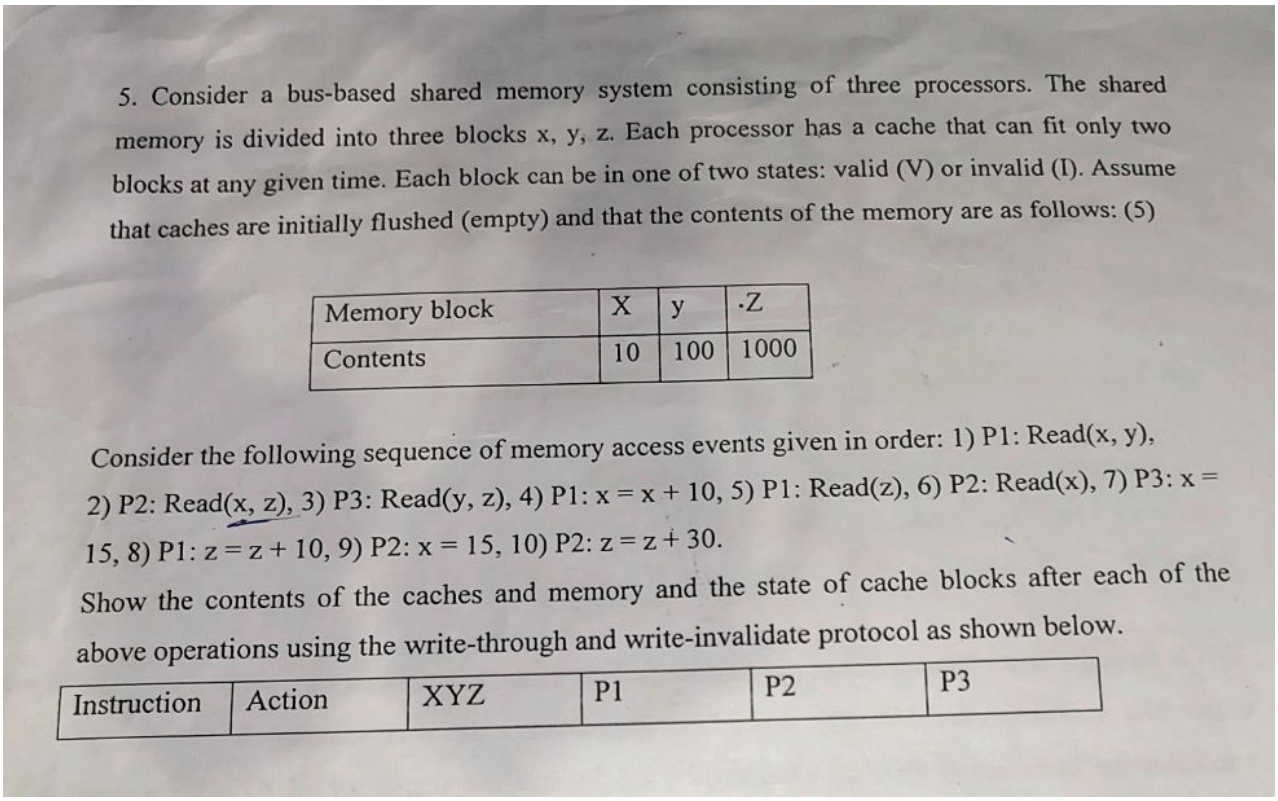
What the page says. A worked MSI snapshot: given the states of several caches, apply a sequence of events and update the states using the write-invalidate protocol.

```
Write-invalidate rules (MSI): • P writes a block it does not have (I) → fetch it, state M, invalidate all others. • P writes a block it has shared (S) → state M, invalidate all others. • P reads a block it does not have (I) → state S (or M if no sharers, in MESI). • A cache snooping a write to its S/M copy → its state becomes I.
```

How to answer. Build a table with a column per cache (and memory). For each event, update the writer's state and invalidate/update every other copy that held the block. Read the table row by row.

The full printed question and its complete trace are in the past-paper solutions (ca2, ca6, ca23, ca29).

Previous-year question 6 — ca6 — Three-processor bus coherence (write-through + wri…



5. Consider a bus-based shared memory system consisting of three processors. The shared memory is divided into three blocks x, y, z. Each processor has a cache that can fit only two blocks at any given time. Each block can be in one of two states: valid (V) or invalid (I). Assume that caches are initially flushed (empty) and that the contents of the memory are as follows: (5)

Memory block	X	y	Z
Contents	10	100	1000

Consider the following sequence of memory access events given in order: 1) P1: Read(x, y), 2) P2: Read(x, z), 3) P3: Read(y, z), 4) P1: x = x + 10, 5) P1: Read(z), 6) P2: Read(x), 7) P3: x = 15, 8) P1: z = z + 10, 9) P2: x = 15, 10) P2: z = z + 30.

Show the contents of the caches and memory and the state of cache blocks after each of the above operations using the write-through and write-invalidate protocol as shown below.

Instruction	Action	XYZ	P1	P2	P3

Original past-paper scan ca6.png (re-rendered).

Transcription of the question

```
5. A bus-based shared-memory system has three processors. Shared memory has blocks
   x, y, z. Each cache fits two blocks. Each block is Valid (V) or Invalid (I). Caches
   start empty; memory contents are x=?, y=?, z=?. (5)
   Event sequence (in order):
     1) P1: Read(x, y)      2) P2: Read(x, z)     3) P3: Read(y, z)
     4) P1: x = x + 10      5) P1: Read(z)        6) P2: Read(x)
     7) P3: x = 15          8) P1: z = z + 10     9) P2: x = 15
    10) P2: z = z + 30
   Show the cache/memory contents and states after each event using the write-through
   and write-invalidate protocol.
```

Full solution

Protocol used

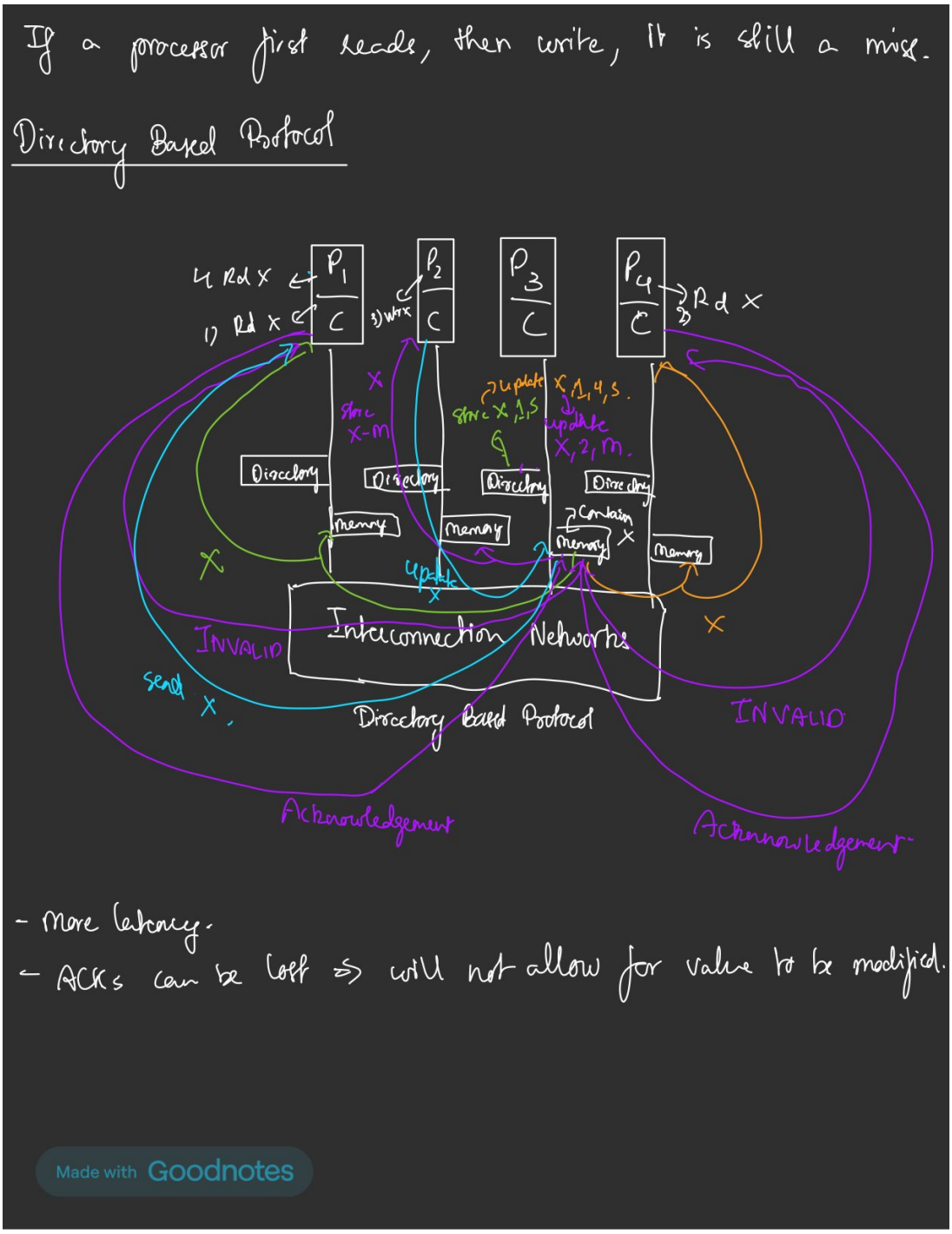
Write-through + write-invalidate: on a write the writer updates memory immediately and invalidates the copies in every other cache. On a read miss the block is fetched and marked valid (shared). Because each cache holds only two blocks, a third block evicts one.

Trace

```
1) P1 reads x,y : P1 = {x:V, y:V} 2) P2 reads x,z : P2 = {x:V, z:V} 3) P3 reads y,z : P3 =
{y:V, z:V} 4) P1 writes x=x+10 : P1.x → V (new value), memory.x updated; P2.x invalidated → I
(write-invalidate) 5) P1 reads z : P1 must fetch z; it already holds x,y so one is evicted
(say y) → P1 = {x:V, z:V} 6) P2 reads x : P2's copy was I → fetch x → P2 = {x:V, z:V} 7) P3
writes x=15 : memory.x updated; P1.x and P2.x invalidated → I 8) P1 writes z=z+10 : memory.z
updated; P2.z and P3.z invalidated → I 9) P2 writes x=15 : fetch x, memory.x updated; other
copies invalidated 10) P2 writes z=z+30 : fetch z, memory.z updated; other copies invalidated
```

Draw the answer as a table: one row per event, one column per cache (with block + V/I state) and one column for memory. The key rule is that a write makes the writer valid and every other copy invalid, and because writes are write-through the memory value is always current.

Original notes — page 86: MSI example (continued) and directory-based protocol



Original handwritten notes — page 86.

Transcription of the page

```
(Continuation of the MSI trace.)

  Directory-based protocol:
    Each block has a home node with a directory entry listing which caches
    hold it (the sharer set) and whether it is modified.

  Messages:
    - Read miss  → request to home; home replies with data and sharer list.
    - Write miss → request to home; home invalidates all sharers, then grants.
    - "Acknowledge" / "Write-back" messages update the directory.

  (Table of events with the resulting states.)
```

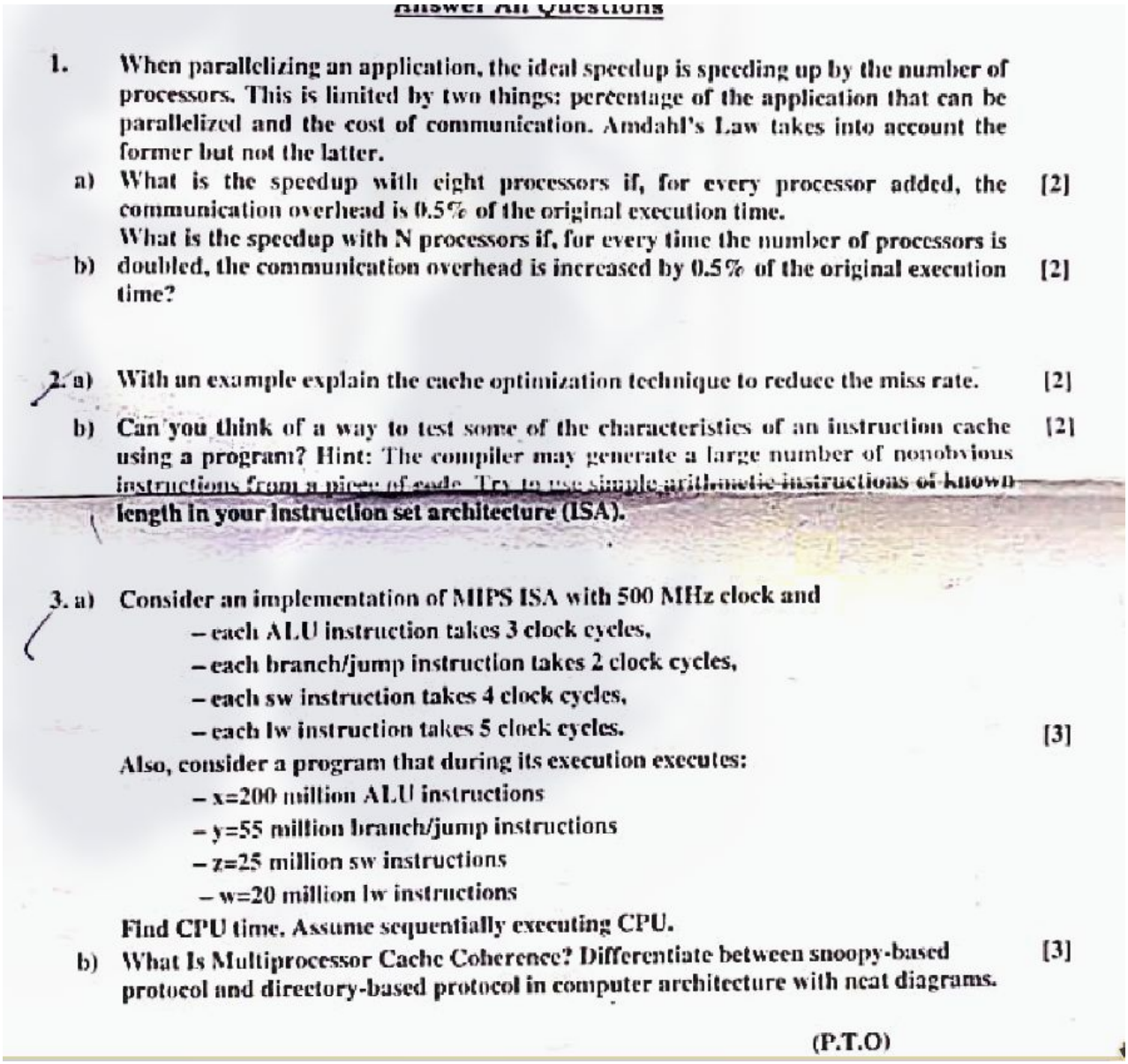
Explanation

What the page says. The trace continues, and the page introduces the directory-based protocol with its messages.

```
Directory entry per block: { state, sharer set } • Read miss : home sends data to requester,
adds it to the sharer set. • Write miss : home sends invalidations to all sharers, then
grants exclusive ownership to the writer. • Write-back : when an M block is replaced, its
data returns to home.
```

- **Advantage of directories** — coherence traffic goes only to the sharers, so the design scales to large machines with no broadcast medium.
- **Cost** — the directory must be stored and updated (memory overhead, extra messages).

Previous-year question 7 — ca7 — Amdahl with communication overhead; cache optim…



Answer All Questions

1. When parallelizing an application, the ideal speedup is speeding up by the number of processors. This is limited by two things: percentage of the application that can be parallelized and the cost of communication. Amdahl's Law takes into account the former but not the latter.

a) What is the speedup with eight processors if, for every processor added, the communication overhead is 0.5% of the original execution time. [2]

b) What is the speedup with N processors if, for every time the number of processors is doubled, the communication overhead is increased by 0.5% of the original execution time? [2]

2. a) With an example explain the cache optimization technique to reduce the miss rate. [2]

b) Can you think of a way to test some of the characteristics of an instruction cache using a program? Hint: The compiler may generate a large number of nonobvious instructions from a piece of code. Try to use simple arithmetic instructions of known length in your instruction set architecture (ISA). [2]

3. a) Consider an implementation of MIPS ISA with 500 MHz clock and
- each ALU instruction takes 3 clock cycles,
- each branch/jump instruction takes 2 clock cycles,
- each sw instruction takes 4 clock cycles,
- each lw instruction takes 5 clock cycles. [3]

Also, consider a program that during its execution executes:
- x=200 million ALU instructions
- y=55 million branch/jump instructions
- z=25 million sw instructions
- w=20 million lw instructions

Find CPU time. Assume sequentially executing CPU.

b) What Is Multiprocessor Cache Coherence? Differentiate between snoopy-based protocol and directory-based protocol in computer architecture with neat diagrams. [3]

(P.T.O)

Original past-paper scan ca7.png (re-rendered).

Transcription of the question

```
1. When parallelising an application, the ideal speed-up is by the number of
   processors. This is limited by two things: the percentage of the application that
   can be parallelised and the cost of communication. Amdahl's Law takes into account
   the former but not the latter.
   a) What is the speed-up with eight processors if, for every processor added, the
      communication overhead is 0.5% of the original execution time. [2]
   b) What is the speed-up with N processors if, for every time the number of
      processors is doubled, the communication overhead is increased by 0.5% of the
      original execution time? [2]
2. a) With an example, explain the cache optimisation technique to reduce the miss
      rate. [2]
   b) Can you think of a way to test some of the characteristics of an instruction
      cache – using a program? Hint: the compiler may generate a large number of
      non-obvious instructions. [2]
3. a) Consider an implementation of the MIPS ISA with a 500 MHz clock and
      – each ALU instruction takes 3 clock cycles,
      – each branch/jump instruction takes 2 clock cycles,
      – each sw instruction takes 4 clock cycles,
      – each lw instruction takes 5 clock cycles. [3]
      Also consider a program that during its execution executes:
      – x = 200 million ALU instructions
      – y = 55 million branch/jump instructions
      – z = 25 million sw instructions
      – w = 20 million lw instructions
      Find the CPU time. Assume a sequentially executing CPU.
   b) What is multiprocessor cache coherence? Differentiate between snoopy-based
      protocol and directory-based protocol with neat diagrams. [3]
```

Full solution

1a — speed-up with 8 processors, 0.5% overhead per processor

Each added processor brings 0.5% communication overhead measured against the original time. With 8 processors the total overhead is 8 × 0.5% = 4% of the original time.

```
S = n / (1 + overhead_fraction) = 8 / (1 + 0.04) = 8/1.04 ≈ 7.69×
```

1b — speed-up with N processors, 0.5% added per doubling

Each time the number of processors doubles, the overhead grows by another 0.5%. Writing N = 2^k, the overhead is 0.5%·k, so:

```
S(N) = N / (1 + 0.005·k) where N = 2^k, k = log₂N e.g. N = 8 (k = 3): S = 8 / (1 + 0.015) ≈
7.88× N = 16 (k = 4): S = 16 / (1 + 0.02) ≈ 15.7×
```

2a — reducing the miss rate (example)

Example technique: **increase associativity**. Switching a direct-mapped cache to 2-way set associative removes conflict misses. With numbers: if the direct-mapped miss rate is 5% and 2-way brings it to 3%, and the miss penalty is 50 cycles, then AMAT falls by (0.05 − 0.03) × 50 = 1 cycle.

```
AMAT = hit time + miss rate × miss penalty ΔAMAT = Δ(miss rate) × miss penalty = (0.05 − 0.03)
× 50 = 1 cycle saved
```

2b — testing instruction-cache characteristics with a program

Write a program whose instruction footprint can be varied systematically (e.g. a loop body repeated to a chosen size, or a loop unrolled to a chosen length) and measure the runtime as the footprint grows. When the footprint crosses the instruction-cache size the runtime jumps — that jump reveals the I-cache capacity. Varying the stride/alignment reveals the block size and associativity.

3a — CPU time for the MIPS program

```
IC = 200M + 55M + 25M + 20M = 300 million instructions Total cycles = 200M×3 + 55M×2 + 25M×4 +
20M×5 = 600M + 110M + 100M + 100M = 910 million cycles CPU time = 910×10⁶ / 500×10⁶ Hz = 1.82
s Average CPI = 910/300 ≈ 3.03
```

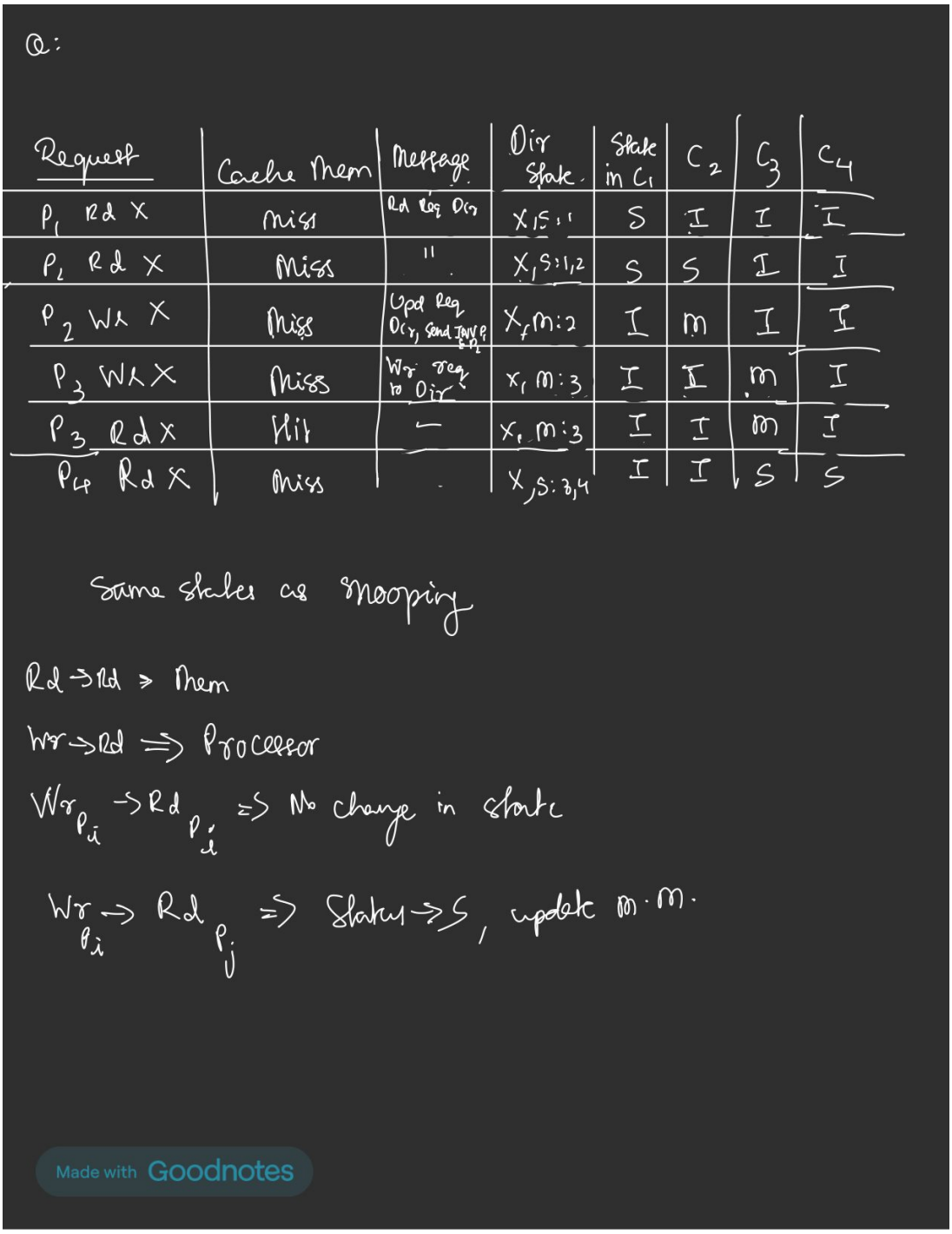
3b — multiprocessor cache coherence; snoopy vs directory

Cache coherence is the guarantee that every processor sees the latest value of every shared block, even though each has a private cache.

	Snoopy-based	Directory-based
Mechanism	All caches watch a shared bus	A directory records the sharers of each block
Traffic	Broadcast to all caches	Point-to-point to the sharers only

	Snoopy-based	Directory-based
Scales to	Few processors (bus limit)	Many processors
Hardware	Simple; needs a broadcast bus	Directory storage + extra messages

Original notes — page 87: MSI example (continued) — write policy for a shared block



Original handwritten notes — page 87.

Transcription of the page

```
MSI example (continued):

  Event trace with tags and states (C1..C4).

  Q: P0 writes x (which is shared) — what is the write policy?
     → write-invalidate: the other copies become I, the writer's copy becomes M.

  Q: P1 reads x again — the copy becomes S.

  Write-back: the M copy is written back to memory when evicted.

  (Final state table.)

  "Write-back ▫ memory updated on eviction; write-through ▫ updated
   immediately."
```

Explanation

What the page says. The final part of the MSI trace, focusing on the write policy for a block that several caches share.

```
Write-invalidate on a shared block: before: C1 = S(x), C2 = S(x), C3 = S(x), memory = x P1 writes x: C2 → M(x); C1 → I(x); C3 → I(x); memory stale on eviction of the M copy: write back x to memory (write-back policy) Write-through alternative: every write also updates memory immediately (simpler coherence, more traffic).
```

Summary of the coherence block. Problem → two families (snooping / directory) → states (MSI / MESI) → write policy (write-through / write-back) → worked traces. The exam question is almost always “apply the protocol to this snapshot”, so practise the table method.

Appendix A — Previous-year question index (all 41 scans)

Every scan ca1–ca41, the topics it examines, and the handwritten-notes page where its solution appears in this guide.

Paper	Topic(s)	What it asks	Solved on
ca1.png	1, 2, 3, 5	Performance (MIPS, CPU time); pipelining speed-up vs multi-cycle; loop dependences & renaming	p. 19
ca2.png	3, 12	Loop dependences (S1/S2) and parallelisation; MSI cache-coherence snapshot (2 caches)	p. 84
ca3.png	1, 2, 3, 11	Arch vs organization; derive Amdahl; cache organizations pros/cons; virtual memory; FPSQR design comparison; cache optimization; power & read misses	p. 2
ca4.png	2	Amdahl (fraction enhanced); quad-core Amdahl; benchmark CPI enhancements	p. 20
ca5.png	2	CPU time from instruction mix (ALU/branch/sw/lw) at 500 MHz	p. 28
ca6.png	12	3-processor bus shared memory, write-through + write-invalidate, full event trace	p. 85
ca7.png	2, 10, 12	Amdahl with communication overhead; cache optimization + ISA question; CPU time; snoopy vs directory coherence	p. 86
ca8.png	7, 12	Branch-target buffer penalty; write-miss stall time with ownership + invalidates	p. 38
ca9.png	5, 7	5-stage pipeline flow; branch in pipeline; hazards, dependences & nops; branch-hazard schemes	p. 35
ca10.png	3, 5	Data dependences (RAW/WAW); optimal pipeline schedule with forwarding + CPI; C→MIPS hazards & reorder	p. 35
ca11.png	6	DAXPY loop unrolled 3×, fresh registers, redundant computation removed	p. 50
ca12.png	8, 14	Tomasulo tables at cycle 6; vector vs SIMD vs GPU	p. 47
ca13.png	3, 5	Four RISC fragments: dependences & forwarding/stalls; 5-stage latencies → cycle time, latency, throughput; MIPS deps & schedule	p. 35
ca14.png	5, 9	Reorder instructions to remove stalls; count RAW hazards and register-rename	p. 46
ca15.png	10, 12	AMAT with an added L2 (required L2 hit rate); multicore CPI needed to beat single core	p. 80
ca16.png	5	For each sequence: must stall / forwarding / no stall	p. 35
ca17.png	5, 7	Pipeline instruction mix: branch-adder use %, data-memory use %, pipelining speed-up	p. 37
ca18.png	8	Tomasulo on a loop with multiply latency and possible cache misses	p. 47
ca19.png	2, 5	Performance (MIPS, CPU time) + pipelining speed-up (repeat of ca1 Q1)	p. 33
ca20.png	5	Resource/control hazards on a MIPS fragment with two memories	p. 34
ca21.png	5, 7, 8	Single-memory structural hazard; pipeline mix; Tomasulo for hazards	p. 35
ca22.png	3	Loop dependences (true/output/anti) and renaming (repeat of ca1 Q2a)	p. 46
ca23.png	14, 12	Basic structure of a vector architecture; MSI coherence snapshot	p. 84
ca24.png	6	DAXPY unroll-until-no-stalls, schedule and time per element	p. 51
ca25.png	9, 10	ROB contents after 2 loop iterations; critical-word-first; cache tag bits; blocking vs non-blocking + prefetch	p. 55
ca26.png	2, 10, 14	Amdahl (scalar + matrix); energy saved by V/f scaling; AMAT with L2	p. 23
ca27.png	3, 5, 10	Write-back vs write-through break-even; RAW/WAR/WAW; pipeline phases and branch overhead	p. 76
ca28.png	6, 7, 9	Static vs dynamic scheduling; three control-hazard techniques; Tomasulo + speculation hardware	p. 52
ca29.png	12, 14	GPU peak throughput & memory bound; Intel i7 coherence; MESI fourth state	p. 84
ca30.png	4, 2, 5	Server-farm MTTF; 22-core Amdahl; pipeline throughput after splitting a stage	p. 24
ca31.png	5	Types of hazards and impact; 5-stage (IF/ID/OF/EX/WO) cycle count with forwarding	p. 35
ca32.png	5, 7, 9, 10	Four branch predictors & misprediction; clock cycle time (5 vs 10 stage); hardware speculation; cache optimization for access time; pipeline phases & branch overhead	p. 37
ca33.png	2, 4, 14	Three modification versions (component speedups) ranking; Amdahl matrix; energy; execution time	p. 20
ca34.png	5, 8, 7	OOO advantages; pipeline phases with static predictor; discuss Tomasulo + cycle for F10	p. 42
ca35.png	10	Critical-word-first & early restart cycles; merging write-buffer size and speed-up	p. 79
ca36.png	2	Amdahl with a slowdown (FP unit speeds up but cache slows); vectorisation percentage to match a 2× vector speed-up	p. 22

Paper	Topic(s)	What it asks	Solved on
ca37.png	10	Micro-benchmark cache block sizes from a latency chart; direct-mapped cache address components	p. 75
ca38.png	10, 7	Direct-mapped cache: misses, evictions, non-empty blocks; branch-target-buffer penalty	p. 74
ca39.png	6, 9	Register renaming with T-registers; count stalls with dependence-aware pipeline; Tomasulo & ROB	p. 42
ca40.png	9, 14	Tomasulo + ROB/commit discussion; vector vs hybrid (GPU) execution time	p. 55
ca41.png	14	Vector/array processor features used in GPUs	p. 16

Appendix B — Source coverage table (all 87 note pages)

One row per page of **Computer Architecture.pdf**, naming the topic and confirming that the page has a screenshot, a transcription and an explanation in this guide.

Notes page	Title in this guide	Annotated?
1	Cover page (GoodNotes template)	
2	What Computer Architecture is — architecture vs organization, and the von Neumann model	
3	Levels of cache and the memory hierarchy	
4	Levels of parallelism, the memory models (UMA/NUMA/COMA) and RISC vs CISC	
5	Parallel computers — definition, need and five classifications	
6	SIMD and MIMD organisations (Flynn’s taxonomy in pictures)	
7	MIMD and shared-memory multiprocessors — classification by memory modules	
8	Distributed memory; classification by interconnection network; the bus	
9	Multistage (multiple) bus and crossbar topology	
10	Mesh and torus topologies	
11	Hypercube, tree architectures, and the first look at the vector/array processor	
12	Systolic arrays; classification based on the processing element; RISC vs CISC (second pass)	
13	Consequences of RISC/CISC for pipelining and CPI; vector processors for DSP	
14	Pipelining introduced; the five stages; levels of parallelism revisited	
15	IEEE-754 floating-point arithmetic — the worked addition example	
16	Superscalar and SIMD; the systolic idea; multicore as the response to the power wall	
17	Systolic arrays in detail; interconnection cost; thermal design and multicore	
18	The ISA — the contract between software and hardware; ARM and the RISC architecture	
19	Speed-up and Amdahl’s Law — the derivation	
20	Amdahl in practice; integer vs floating-point instruction mix	
21	Amdahl worked past-paper question (enhanced fraction)	
22	Amdahl worked past-paper question (continued)	
23	Power, voltage and frequency scaling (the power wall)	
24	Cost of an integrated circuit; MTTF, MTTR, MTBF and availability	
25	MTTF and failure-rate worked examples	
26	Reliability past-paper questions (server farm, disk failures, die yield)	
27	Availability from MTTF; the bridge to performance metrics (CPU time)	
28	CPU time worked example — instruction mix to average CPI	
29	The components of CPU time; a worked “what is the clock rate?” question	
30	CPU-time / MIPS past-paper question (instruction classes)	
31	CPU-time past-paper question (worked arithmetic)	
32	CPU-time past-paper question (continued)	
33	Characteristics of pipelining; speed-up; the space–time (timing) diagram	
34	The five pipeline stages in detail; instruction fetch and PC update	
35	Pipeline timing with real instruction latencies; the load-use stall	
36	Forwarding, load vs store handling, and the compiler’s role in scheduling	

Notes page	Title in this guide	Annotated?
37	Branch prediction — the one-bit predictor	
38	Tournament (hybrid) branch predictor	
39	One-bit predictor state machine (worked table)	
40	Two-bit saturating counter predictor (state machine)	
41	Two-bit / correlating predictor (worked table and scheduling note)	
42	Register renaming with a reorder buffer (data dependence resolution)	
43	Reservation-station fields and the common data bus (CDB)	
44	Tomasulo tables — instruction status and reservation stations	
45	Parallelisation, CPI and the basics of branches	
46	Data dependences — RAW, WAR and WAW	
47	Tomasulo worked example (loop) with reservation-station tables	
48	Tomasulo example with a small FP unit and load/store buffers	
49	Loop unrolling — the DAXPY example	
50	Loop unrolling worked example (3× unrolled DAXPY)	
51	Unroll until no stalls — the scheduled loop	
52	Instruction scheduling rules (dynamic and static)	
53	Precise exceptions and speculation (exceptions in flight)	
54	Tomasulo hardware — the full datapath with a reorder buffer and CDB	
55	Speculation — issue, execute, read, write and commit	
56	Tomasulo instruction-status table (worked)	
57	Tomasulo — instruction-status and reservation-station tables (worked, cycles 1–10)	
58	Tomasulo tables (continued — second worked set)	
59	Tomasulo tables (continued — with cycle numbers)	
60	Tomasulo tables (continued — final set)	
61	Superscalar processors and vector operations	
62	Vector/loop example and loop scheduling with loads	
63	Loop scheduling with speculation	
64	Loop scheduling with speculation (continued)	
65	Speculation timing table (issue / execute / read / write / commit)	
66	Speculation timing table (continued)	
67	Blank / illegible page	
68	UMA shared memory and multicore with private caches	
69	Cache and coherence block diagram	
70	Cache basics — direct mapping (block number mod number of lines)	
71	Set-associative cache — organisation and address bits	
72	Set-associative cache — worked address example	
73	Replacement policies — FIFO, LRU, optimal, random	
74	Direct-mapped cache worked example — address field split	
75	Cache address split — worked example (tag/index/offset)	
76	Write-through policy; the local miss-rate / average-access-time formula	
77	Write-back and write-allocate policies	
78	Write-allocate / write-back combinations (summary)	
79	Write-policy summary; AMAT; cache-optimisation checklist	
80	Multilevel caches; inclusive/exclusive; local vs global miss rate	
81	Prefetching — hardware and software	
82	Virtual memory — address translation, the TLB and page faults	
83	Summary of instruction-level optimisations (the compiler/hardware toolbox)	
84	Cache coherence — the problem, MSI/MESI and snooping vs directory	

Notes page	Title in this guide	Annotated?
85	MSI example — write-invalidate on a shared bus (snapshot)	
86	MSI example (continued) and directory-based protocol	
87	MSI example (continued) — write policy for a shared block	

Appendix C — Verification report

1. Page coverage. All 87 pages of the handwritten notes are accounted for. Pages 1 (cover) and 67 (blank/illegible) carry no academic content and are labelled as such; the other 85 pages each have a screenshot, a transcription and a full explanation.

2. Order. The pages appear in the exact original sequence, page 1 → page 87, with no topic reordering. Recurrences are preserved (for example the cache-memory topic is revisited at the end of the notes and is annotated there, in that position).

3. Previous-year questions. All 41 scans ca1–ca41 are transcribed and solved. A scan may contain several questions; each is identified and solved, and each is placed at the notes page whose topic it examines. Where a scan repeats an earlier one (ca19 repeats ca1; ca22 repeats ca1 Q2a) this is stated rather than silently duplicated.

4. Figures and formulas. Every diagram in the notes is retained as the original screenshot; the key ones are additionally reconstructed digitally (memory hierarchy, SIMD/MIMD, topologies, pipeline timing, branch-predictor state machines, Tomasulo datapath, cache address split, coherence states). Every formula is derived and every variable defined.

5. Numerical checking. All numerical answers were re-derived from the stated assumptions using the master formulas (CPU time = IC × CPI × T_clock; AMAT = hit time + miss rate × miss penalty; MTTF/MTBF/availability; Amdahl). Intermediate steps are shown.

6. Honest limits. The notes are handwritten and a few marginal annotations are only partly legible; where a reading affects a calculation it is stated explicitly. No question statement or numeric value has been invented to fill a gap.